THE 1-DAY PREPARATION MANUAL

Compiler Verification Software Engineering Intern

Campus Online Assessment & Interviews

Theory · Worked examples · Candidate-reported questions · Practice with answers

Role snapshot (from the campus notice)

Team & role	Software Engineering Intern, **compiler verification** team (GPU and embedded compilers)
Internship stipend	INR 100,000 per month
Work location	Pune & Bengaluru
Assessment type & platform	Mix of **Aptitude** and **Technical** (online) on **HackerRank**
Skills to be assessed	**C++/C, Python, ML exposure, Compiler**
Eligibility	Final-year B.Tech / M.Tech in CS, Electrical, Electronics or IT (winter intern & full-time roles)

What is inside

Part I	Role decoder, *what candidates actually reported*, and a 24-hour battle plan
Part II	C, C++, Python and the coding-round toolkit (DSA)
Part III	Compiler theory: lexing, parsing, IR/SSA, optimisation, code generation, linking
Part IV	LLVM IR, FileCheck/lit, the NVIDIA toolchain (NVVM, PTX, SASS), CUDA, parallel programming
Part V	Compiler verification methodology (fuzzing, differential/metamorphic testing, reduction) and a Python automation lab
Part VI	ML exposure, OS/computer-architecture/number systems, aptitude & puzzles
Part VII	Question bank (reported vs. practice), mock OA, interview flashcards, cheat sheets

Unofficial study material. Not affiliated with, endorsed by, or written by NVIDIA. The role text and assessment details are taken from the campus notice supplied by the student; *interview and OA details are anecdotes from public candidate reports (links given in Chapter 2) and may differ for your drive.* Always follow the instructions in your official invitation. Compiled October 2026 for the 2026–27 campus cycle.

How to use this manual

The one-paragraph version

You have about one day. Do **not** read this front to back. Read Part I (35 minutes), then follow the hour-by-hour plan in Chapter 3. Every section carries a priority mark: ★★★ = must master, ★★ = should know, ★ = skim if time remains. The three-star material alone is a complete, passable preparation.

Conventions used in the book

Element	Meaning
★★★ ★★ ★	Priority for your OA/interview (three stars = highest)
Green box	Must-know facts and definitions
Red box	Traps: the options and mistakes that examiners love
Blue box	Exam tips and shortcuts
Orange box	*Reported by candidates*: a question or topic that a real candidate reported publicly (with link)
Teal box	Worked examples
Code on grey background	Code in the book; for C, C++ and Python it was *compiled and executed while this document was built*
Code on yellow background	**Real captured output** of the code above it (GCC 13, Clang/LLVM 18, Python 3.13 on Linux x86-64) — never typed by hand
`Q`*n.m* / **Answer:**	Practice questions with answers directly below; cover the answer and try first

Honest limits of this material

- **Previous-year questions:** NVIDIA does not publish its OA. What exists are candidate write-ups (GeeksforGeeks, Glassdoor, Jointaro, LeetCode Discuss, placement blogs). Chapter 2 lists what they report and Chapter **??** turns it into practice. Anything in this book that is *not* labelled "reported" is my own practice material written to cover the same topics; it is *not* a claim about what will appear.
- Formats change by team, location and year. Confirm duration, sections, language list and negative marking from your invitation.
- Specific GPU architecture numbers (registers, shared-memory sizes, sm_XX features) change with every generation. Where a number matters, it is stated for a named architecture; do not memorise them as universal.

Contents

How to use this manual **1**

I Orientation 7

1 Role Decoder: What You Are Really Being Hired For **8**

1.1 The job in plain words ★★★ . . . 8
1.2 Job-description line to syllabus map ★★★ . . . 8
1.3 Where the compiler stack sits ★★★ . . . 9
1.4 A typical week (inferred from public job postings) ★★ . . . 9
1.5 The bug zoo ★★★ . . . 10

2 Exam Intelligence: What Candidates Actually Reported **11**

2.1 Reports specific to compiler roles ★★★ . . . 11
2.2 Reports on the online assessment format ★★★ . . . 12
2.3 Job-posting evidence ★★ . . . 13
2.4 What the evidence says ★★★ . . . 13
2.5 Your probable OA, reconstructed ★★★ . . . 14
2.6 Interview style, as reported ★★ . . . 14
2.7 Source list . . . 14

3 The 24-Hour Battle Plan and OA Tactics **16**

3.1 Choose your budget ★★★ . . . 16
3.2 Hour-by-hour plan (P3, adjust to your OA time) ★★★ . . . 16
3.3 Tactics inside the OA ★★★ . . . 17
3.4 HackerRank mechanics ★★★ . . . 18
3.5 Test-day checklist ★★ . . . 19

II Programming Foundations 20

4 C Programming: Output Traps, Memory and UB **21**

4.1 Memory layout and storage classes ★★★ . . . 21
4.2 Types, sizes, promotions and conversions ★★★ . . . 22
4.3 Pointers and arrays ★★★ . . . 23
4.4 Undefined, unspecified and implementation-defined behaviour ★★★ . . . 25
4.5 The preprocessor ★★★ . . . 27
4.6 Bit manipulation ★★★ . . . 28
4.7 Structs, unions, padding, bit-fields ★★ . . . 29
4.8 Dynamic memory ★★★ . . . 30
4.9 Functions, call-by-value and the calling convention ★★★ . . . 31
4.10 Qualifiers: `volatile`, `restrict`, `inline`, `static` ★★ . . . 32
4.11 Standard-library essentials and pitfalls ★★ . . . 33
4.12 Reimplement the library: strlen, strcmp, memcpy, memmove, atoi ★★★ . . . 34
4.13 Find the bugs: the “bug-hunt” round ★★★ . . . 34
4.14 Output-prediction drills (22 questions) ★★★ . . . 36

5 C++: Object Model, Move Semantics and Resource Management **41**
5.1 What C++ adds to C (one-minute map) ★★ . . . 41
5.2 Object lifetime: which constructors and destructors run? ★★★ . . . 41
5.3 Polymorphism: virtual functions and the vtable ★★★ . . . 43
5.4 Rule of 0/3/5, copy vs. move, and why `noexcept` matters ★★★ . . . 44
5.5 RAII and smart pointers ★★★ . . . 46
5.6 Operator overloading and a "write a class" answer ★★★ . . . 47
5.7 Diamond inheritance and virtual bases ★★ . . . 47
5.8 Templates ★★★ . . . 48
5.9 The STL: containers, complexities, iterator invalidation ★★★ . . . 49
5.10 Lambdas, `auto`, casts and const-correctness ★★ . . . 50
5.11 Exceptions and stack unwinding ★★ . . . 51
5.12 Undefined behaviour, ODR and initialisation order in C++ ★★★ . . . 51
5.13 C++ output-prediction drills ★★★ . . . 52

6 Python: Data Model, Gotchas and Automation **54**
6.1 The data model: names, objects, mutability ★★★ . . . 54
6.2 Numbers, strings and containers ★★★ . . . 56
6.3 Iterators, generators and the standard toolbox ★★★ . . . 57
6.4 Functions, decorators, classes ★★ . . . 57
6.5 Concurrency and performance facts ★★ . . . 59
6.6 Python for verification automation: the standard-library tour ★★★ . . . 59
6.7 Compiler-flavoured Python: `ast`, `dis` ★★ . . . 60
6.8 Python drills (more) ★★★ . . . 61

7 The Coding Round: Patterns, Templates, Stress Testing **62**
7.1 Constraints decide the algorithm ★★★ . . . 62
7.2 From problem signal to technique ★★★ . . . 62
7.3 Reusable C++ templates (compiled and tested) ★★★ . . . 63
7.4 Stress testing: the habit that finds your own bugs ★★★ . . . 65
7.5 Solved problems that candidates reported ★★★ . . . 65

III Compiler Theory **73**

8 Compilers: Big Picture and Lexical Analysis **74**
8.1 What a compiler is, and the vocabulary ★★★ . . . 74
8.2 The real C toolchain, stage by stage ★★★ . . . 75
8.3 Lexical analysis ★★★ . . . 75
8.4 Regular expressions and finite automata ★★★ . . . 76
8.5 Practice ★★★ . . . 78

9 Syntax Analysis: Grammars and Parsing **79**
9.1 Context-free grammars ★★★ . . . 79
9.2 Top-down parsing and LL(1) ★★★ . . . 80
9.3 Bottom-up parsing: LR family ★★★ . . . 82
9.4 Parser generators and what real compilers do ★★ . . . 85
9.5 Formal-language facts that appear in MCQs ★★ . . . 85
9.6 Practice ★★★ . . . 85

10 Semantic Analysis, Intermediate Code, SSA, Runtime **87**
10.1 Syntax-directed definitions and translation ★★ . . . 87
10.2 Types, scopes and symbol tables ★★ . . . 87
10.3 Intermediate representations ★★★ . . . 88
10.4 Static single assignment (SSA) ★★★ . . . 89
10.5 Runtime environments ★★★ . . . 90
10.6 Lab: a complete mini compiler in 250 lines ★★ . . . 91
10.7 Practice ★★★ . . . 93

11 Optimisation, Code Generation and Linking **95**
11.1 What an optimiser is allowed to do ★★★ . . . 95
11.2 Local optimisations (within a basic block) ★★★ . . . 95
11.3 Data-flow analysis ★★★ . . . 96
11.4 Loop optimisations and dependence analysis ★★★ . . . 98
11.5 Interprocedural optimisation ★★ . . . 101
11.6 Code generation ★★★ . . . 101
11.7 Linking and loading ★★ . . . 102
11.8 Practice ★★★ . . . 103

IV LLVM, the NVIDIA Toolchain and GPUs **104**

12 LLVM Crash Course: IR, Passes, Back Ends **105**
12.1 Architecture and the tools ★★★ . . . 105
12.2 IR in one page ★★★ . . . 106
12.3 Memory addressing, flags, poison ★★★ . . . 107
12.4 The pass manager and the passes you should recognise ★★★ . . . 109
12.5 Vectorisation in IR ★★ . . . 110
12.6 The back end: from IR to machine code ★★ . . . 110
12.7 MLIR, Triton and the AI-compiler ecosystem ★★ . . . 110
12.8 GCC in one paragraph ★ . . . 111
12.9 Practice ★★★ . . . 111

13 Testing Compilers the LLVM Way: lit and FileCheck **112**
13.1 The test pyramid for a compiler ★★ . . . 112
13.2 lit: the test driver ★★ . . . 112
13.3 FileCheck: match the output, not the accident ★★ . . . 113
13.4 Writing tests that survive refactoring ★★ . . . 115
13.5 Practice ★★ . . . 116

14 NVIDIA's Stack: nvcc, NVVM, PTX and SASS **117**
14.1 The CUDA compilation pipeline ★★★ . . . 117
14.2 NVVM IR: LLVM IR with GPU conventions ★★ . . . 118
14.3 PTX: the virtual ISA ★★★ . . . 119
14.4 More kernels, more PTX ★★★ . . . 121
14.5 A generated coverage matrix: how the back end lowers each IR operation ★★ . . . 122
14.6 SASS: the machine code ★★ . . . 123
14.7 How a team verifies a GPU compiler (the job, concretely) ★★★ . . . 124
14.8 Embedded targets, cross-compilation and functional safety ★★ . . . 124
14.9 Practice ★★★ . . . 125

15 CUDA and GPU Architecture **126**
15.1 The mental model ★★★ . . . 126
15.2 Execution: warps, divergence, barriers ★★★ . . . 127
15.3 Memory hierarchy and access patterns ★★★ . . . 128
15.4 Occupancy and the roofline ★★ . . . 130
15.5 Run CUDA without a GPU: a 60-line emulator ★★ . . . 131
15.6 Race detection with ThreadSanitizer ★★ . . . 134
15.7 The CUDA bug catalogue ★★ . . . 134
15.8 Host-side API cheat sheet ★★ . . . 135
15.9 Recent hardware features: the one-line versions ★ . . . 136
15.10 Practice questions ★★ . . . 137

16 Parallel Programming: Threads, Atomics, OpenMP, Patterns **139**
16.1 Vocabulary and speed-up laws ★★ . . . 139
16.2 Data races, mutexes and atomics ★★★ . . . 140
16.3 Deadlock, livelock, starvation ★★ . . . 141
16.4 Condition variables and producer–consumer ★★ . . . 142
16.5 The memory model: why `seq_cst` is the default ★★ . . . 142
16.6 False sharing and cache effects ★ . . . 144
16.7 OpenMP ★★ . . . 144
16.8 Parallel patterns and their GPU forms ★★ . . . 145
16.9 Python concurrency (you will use it in the automation lab) ★★★ . . . 146
16.10 Practice questions ★★ . . . 146

V Compiler Verification and the Automation Lab **148**

17 Compiler Verification: Oracles, Differential Testing, Reduction **149**
17.1 What "verifying a compiler" means ★★★ . . . 149
17.2 The oracle problem ★★★ . . . 149
17.3 Differential testing in practice ★★★ . . . 150
17.4 Generating test programs ★★★ . . . 152
17.5 Metamorphic testing and EMI ★★ . . . 153
17.6 Test-case reduction ★★★ . . . 154
17.7 Isolating the culprit: bisection and triage ★★ . . . 155
17.8 Formal and semi-formal methods in one page ★★ . . . 156
17.9 Test adequacy: coverage and mutation ★★ . . . 156
17.10 A test plan for a GPU and embedded compiler ★★★ . . . 156
17.11 Flaky tests and infrastructure hygiene ★★ . . . 157
17.12 Writing the bug report ★★ . . . 158
17.13 Practice questions ★★★ . . . 158

18 The Automation Lab: A Mini Verification Framework in Python **161**
18.1 What we build ★★★ . . . 161
18.2 Step 1: a random program generator ★★★ . . . 162
18.3 Step 2: differential testing against a reference interpreter ★★★ . . . 163
18.4 Step 3: seeded bugs, so that the method can be measured ★★★ . . . 164
18.5 Step 4: metamorphic variants (no reference needed) ★★ . . . 165
18.6 Step 5: automatic test-case reduction ★★★ . . . 165

18.7 Step 6: the mutation study ★★★ . . . 167
18.8 Step 7: from reproducer to regression test ★★★ . . . 168
18.9 Step 8: a mini lit ★★★ . . . 171
18.10 Step 9: reporting and triage ★★ . . . 174
18.11 From the lab to NVIDIA's problem ★★★ . . . 175
18.12 Exercises ★★ . . . 176
18.13 Practice questions ★★★ . . . 176

VI Machine Learning, CS Fundamentals and Aptitude 177

19 Machine Learning Exposure: Basics, Numerics, ML Compilers 178
19.1 The ML map in one page ★★★ . . . 178
19.2 Learning is optimisation ★★★ . . . 179
19.3 Generalisation: overfitting, bias–variance, regularisation ★★★ . . . 180
19.4 Metrics ★★★ . . . 181
19.5 Two classic unsupervised algorithms, from scratch ★★ . . . 181
19.6 Neural networks in 40 lines ★★★ . . . 182
19.7 Numerics every GPU and compiler person must know ★★★ . . . 184
19.8 Compilers *for* machine learning ★★★ . . . 185
19.9 Machine learning *in* and *for* compilers ★★ . . . 187
19.10 Practice questions ★★★ . . . 187

20 CS Fundamentals in a Hurry: Numbers, Logic, Architecture, OS 189
20.1 Number representation and IEEE-754 ★★★ . . . 189
20.2 Digital logic in ten minutes ★★ . . . 190
20.3 Computer architecture ★★★ . . . 191
20.4 Operating systems ★★★ . . . 193
20.5 Databases in one page ★ . . . 195
20.6 Computer networks in one page ★ . . . 197
20.7 A shell-and-tools survival kit ★★ . . . 198
20.8 Drills, with answers computed ★★★ . . . 199
20.9 Practice questions ★★ . . . 199

21 Aptitude and Puzzles: The Non-Technical Half of the Online Assessment 201
21.1 How to run the aptitude round ★★★ . . . 201
21.2 Quantitative formula sheet, every row verified ★★★ . . . 202
21.3 Calculation shortcuts, and the proof that they work ★★★ . . . 203
21.4 Logical reasoning: seven question types ★★ . . . 204
21.5 Classic puzzles, solved by search ★★ . . . 206
21.6 Probability puzzles by enumeration and simulation ★★ . . . 208
21.7 Logic puzzles verified by model checking ★★ . . . 208
21.8 Verbal ability, if it appears ★ . . . 210
21.9 Mock test: 30 questions in 40 minutes ★★★ . . . 210

Part I
Orientation

1. Role Decoder: What You Are Really Being Hired For

In this chapter

Read time: 15 min. After this chapter you can explain, in two sentences, what the compiler-verification team does, which line of the job description maps to which topic, and why the OA mixes C/C++, Python, ML and compilers.

1.1 The job in plain words ★★★

A compiler verification engineer does **not** write the compiler. They write the *programs and tools that try to break it*. Reading the role description line by line:

- **"Implement programs or GPU code in PTX, CUDA, C/C++, or a GPU assembly language to verify NVIDIA GPU / embedded compilers."** You write *test programs* whose purpose is to expose a wrong result, a crash or a rejected-valid/accepted-invalid program in a compiler. PTX is NVIDIA's virtual assembly language (Chapter 14); CUDA is the C++ dialect for GPUs (Chapter 15).
- **"Design and implement tools, libraries, and/or automation to support . . . compiler development and verification."** Test generators, runners, result parsers, triage and reduction scripts, dashboards. This is where **Python** lives (Chapters 6 and 18).
- **"Collaborate with compiler and hardware teams to create scalable verification infrastructure for NVIDIA embedded platforms, ensuring robustness and reliability across compiler releases."** Every nightly or release build of the compiler must be re-tested automatically on many targets (GPU architectures, embedded boards). Regression testing, CI and bug triage (Chapter 17).

Verification vs. validation vs. testing (vocabulary you should use)

Testing runs programs and checks results. **Verification** asks "did we build the compiler *right*" (does it conform to the language/ISA specification?). **Validation** asks "did we build the *right* compiler" (does it serve users' needs?). In industry usage, "compiler verification" mostly means: systematic testing plus tooling, sometimes plus formal methods, aimed at proving the compiler's output preserves the meaning of the source program.

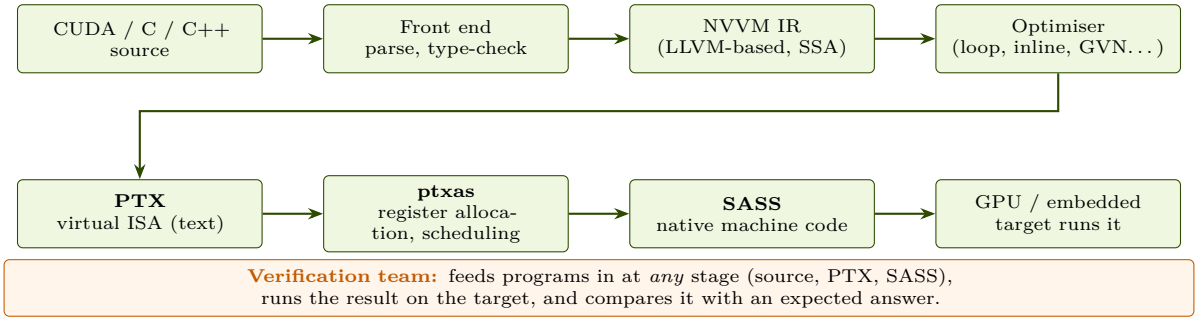
1.2 Job-description line to syllabus map ★★★

JD / assessment line	What it implies you must be able to do	Chapters
PTX, CUDA, C/C++, GPU assembly	Read/write simple PTX and CUDA kernels; know the GPU execution model; recognise SASS	14, 15, 4, 5
Verify GPU / embedded compilers	Compiler testing theory: oracles, differential and metamorphic testing, fuzzing, reduction; cross-compilation basics	17, 14
Tools, libraries, automation	Python scripting, subprocess, parsing, concurrency, report generation	6, 18
Scalable verification infrastructure	CI, sharding, timeouts, flaky-test handling, triage and bisection	17
Strong academic background	DSA, OS, computer architecture, discrete maths	7, 20
Understanding of programming languages *and their specification*	Undefined/unspecified behaviour, sequence points, ODR; grammars and parsing	4, 5, 9
Strong C/C++ & Python	Output prediction, memory model, STL, data model	4–7
Stand out: compilers & machine learning	Phases, IR/SSA, optimisations, LLVM; ML basics and ML compilers	8–12, 19
Stand out: parallel programming, automation	CUDA/OpenMP/threads; automation lab	15, 16, 18
OA: Aptitude	Quant, logical reasoning, probability, puzzles	21

JD / assessment line	What it implies you must be able to do	Chapters
OA: ML exposure	Supervised learning, metrics, neural-network basics, ML compilers	19

1.3 Where the compiler stack sits ★★★

A GPU compiler is a *pipeline*. Each arrow below is a place where a bug can hide, and the verification team attacks every stage:



Two ideas to keep in your head for the whole day:

1. **PTX is a virtual ISA; SASS is the real machine code.** The same PTX text can be assembled (by `ptxas`) for different GPU generations, or JIT-compiled by the driver at run time. Writing tests directly in PTX lets the team test the back end without any front-end noise.
2. **"Embedded" compilers** target NVIDIA's Arm-based SoC platforms and the small processors inside GPUs. NVIDIA publicly states that its GPU microcontrollers moved from the proprietary Falcon design to RISC-V (Tom's Hardware). Embedded verification adds cross-compilation, simulators/boards, code-size limits and sometimes **functional-safety** rules such as ISO 26262 (one senior compiler-verification posting asks for it; Chapter 17).

Do not confuse the two "verification" jobs

NVIDIA also hires *chip design-verification* interns (SystemVerilog/UVM, FIFO-depth maths, FSMs, muxes). Several Glassdoor reports that you will find when you search "NVIDIA verification intern interview" are **that** job (for example the 2020 and 2025 Santa Clara reports with FIFO-depth questions). Your role is *software* compiler verification: C/C++, Python, PTX/CUDA, compilers. Ignore RTL/UVM material unless your invitation says otherwise.

1.4 A typical week (inferred from public job postings) ★★

1. **Read the spec.** A new PTX instruction or a C++ feature lands. You read the ISA/standard text and list its corner cases.
2. **Write tests.** Hand-written directed tests for each corner case, plus *generated* tests: parameterised PTX templates, random program generators (Csmith-style), combinatorial sweeps over types, modifiers and rounding modes.
3. **Run at scale.** A Python runner compiles each test at several optimisation levels/targets, runs it (on a GPU, a simulator or an embedded board), enforces timeouts, and stores results.
4. **Compare.** Results are compared with an *oracle*: another compiler, a CPU reference, the unoptimised build, or a self-checking assertion.
5. **Triage.** Failures are de-duplicated, minimised (delta-debugging), bisected to the compiler change that introduced them, and filed with a small reproducer.
6. **Close the loop.** The reproducer becomes a permanent regression test; metrics (pass rate, crash rate, coverage) are reported to compiler and hardware teams.

Five habits interviewers want to hear

1. **The specification is the law.** Know the difference between undefined, unspecified and implementation-defined behaviour; a "failing" test that relies on UB is a bad test, not a compiler bug.
2. **Tests must be deterministic and self-checking** (fixed seeds, no uninitialised reads, no data races, tolerance for floating-point).
3. **Minimise before you report** (a 3-line reproducer beats a 3000-line one).
4. **Automate everything repeatable** (if you did it twice, script it).
5. **Measure** (coverage, pass rate, time-to-triage); claims without numbers are weak.

1.5 The bug zoo ★★★

Failure class	What it looks like
ICE / crash	Compiler aborts or segfaults ("internal compiler error"), even on valid input
Hang / timeout	Compiler or compiled program never terminates (often a pass in an infinite loop)
Miscompilation	Compiles fine, produces the *wrong result*; usually the most severe class (silent wrong code)
Accepts invalid / rejects valid	Front-end bug: a program that violates the spec compiles, or a legal program is refused
Wrong diagnostics	Missing or misleading error/warning message
Performance regression	Optimised code slower (or larger) than the previous release or than a competitor
Debug-info / ABI bug	Debugger shows wrong values; calling convention or struct layout mismatch between modules
Non-determinism	Different output on different runs of the same build (data race, uninitialised memory, iteration order)

Exam tip – yaad rakho

If an MCQ says "which of the following is the most dangerous compiler bug", the expected answer is almost always *miscompilation (silent wrong code)*.

2. Exam Intelligence: What Candidates Actually Reported

In this chapter

Read time: 20 min. This chapter collects what public candidate reports say about NVIDIA's compiler-intern and software-intern OA/interviews, then turns it into a study-weight table. It is the evidence behind the priorities in the rest of the book.

How this evidence was gathered – read this first

- The reports were found with web search (GeeksforGeeks, Jointaro, Glassdoor, LeetCode Discuss, placement blogs, job boards). The research sandbox could *not* open those sites directly (network policy), so the details below come from **search-engine summaries of the pages**, not from the pages themselves. Open the numbered links in Section 2.7 to read the originals.
- Reddit threads could not be retrieved at all (the search tool does not index Reddit); if you have time, search *"NVIDIA intern OA"* on r/developersIndia and r/Btechtards yourself.
- All reports are anonymous, small-sample anecdotes spread over 2016–2026, for different teams and cities. **Nothing here is official NVIDIA information.** Where two summaries of the same source disagreed, this chapter says so instead of picking one.

2.1 Reports specific to compiler roles ★★★

Source	Role / place / year	Reported by candidates: compiler-role interviews and OAs
[S1] GfG	Compiler intern (6 months); page updated Jul 2025	**R1 online test:** 21 MCQs on C/C++, programming, data structures and *compiler output*, plus one coding question; "fairly straightforward if C/C++ and DS fundamentals are solid"; about 25 candidates reached interviews. **R2 live "bug hunt":** a ∼250-line C program (binary tree → doubly linked list in spiral order); 5–10 minutes to find bugs. The candidate spotted missing overflow/underflow checks in the stack code but **missed a call-by-value bug**; follow-up questions on memory allocation.
[S2] GfG	LLVM compiler intern, Fall 2025	Four rounds. OA on HackerEarth (core CS fundamentals). Technical rounds: C++ output and recurrences; a chessboard DSA problem (king from $(1,1)$ to $(8,8)$: all paths and the shortest); how virtual memory maps to physical memory; analysing a code snippet for variable values and memory locations; basic compiler optimisations. Managerial round: motivation, project challenges, **phases of a compiler, code-generation stages, optimisation stages**.
[S3] GfG	LLVM compiler intern (6M, spring)	OA on C/C++, data structures and compiler design; technical rounds on code output and cycle detection in a linked list.
[S4] Jointaro / Glassdoor	Compiler intern, Pittsburgh, Nov 2024	Round 1 (senior manager): topological sort, which turned into **automatic differentiation and data-flow analysis**; "why do compilers do loop unrolling?"; mostly theoretical. Round 2: **pointer analysis, dependence analysis**, architectural considerations behind optimisations.
[S5] Jointaro / Glassdoor	Compiler intern, Austin, Apr 2025	45-minute virtual round: projects (C++ and GPU), then a short C++ class: **which constructors and destructors run at each step, and what memory-management problems exist**. Verbal reasoning; no live coding.
[S6] Jointaro / Glassdoor	Compiler intern, Shanghai, Mar 2024	OA plus technical interview. "Program **matrix multiplication** and describe **all** applicable optimisations (they mean all)"; "program **graph colouring**".
[S7] Glassdoor	Compiler intern, Bengaluru, Nov 2023 (accepted; applied via university)	DSA-centred: a **2-D knapsack** DP problem, stack operations, and compiler-related questions. Process took about two months.
[S8] Glassdoor	Compiler engineer (full-time)	"**Loop unrolling**"; given a concrete IR example: how would the compiler perform the *most aggressive optimisation*, and what are the trade-offs?

Source	Role / place / year	(continued)
[S9] LeetCode Discuss	Compiler engineer, Pune, Sep 2021 (reject)	Only a fragment was visible: first round cancelled, coding interviewers gave no hints. (Open the thread for details.)
[S10] ProgramHelp	NVIDIA intern virtual onsite (**low confidence**)	Reported topics: constant substitution, loop unrolling, function inlining, code reordering, the *motivation* for unrolling and inlining, and the LLVM IR `getelementptr` instruction with the `inbounds` flag. A second search could not re-confirm this; treat as plausible topics, not as fact.

2.2 Reports on the online assessment format ★★★

Source	Role / place / year	Reported by candidates: OA format (software roles; mostly India)
[S11] GfG	System software intern, on-campus drive, Mar 2025	HackerRank, about 60 minutes, proctored in college labs: roughly **28 MCQs + 2 coding problems** on aptitude, data structures, algorithms and OS. About 700–800 applicants, about 500 shortlisted for the OA.
[S12] GfG	System software intern, on-campus 2024	HackerRank, 50 minutes: aptitude, maths and C MCQs. Another summary of the same site says ∼25 MCQs on C/C++, OS, DSA and probability, and that the follow-up interview *reused questions from the OA*. One summary mentions **negative marking**; others say none (varies by drive).
[S13] GfG / LeetCode	SDE intern (6 months), on-campus, Feb 2024	OA: 2 CP/DSA problems + aptitude, "medium to hard"; result in 2–3 days. Interview 1: a **Trie** problem and a DP problem (partition an array to minimise the sum difference), both LeetCode-hard-like.
[S14] Glassdoor (Pune)	System software, Pune, 2025	60-minute HackerRank with **2 coding questions plus C++/Python, Linux and analytical questions**.
[S15] Jointaro	Software intern, New Delhi, Oct 2025	Topics listed: **COA, OS, DSA, puzzles, DBMS**; online coding assessments followed by several technical interviews.
[S16] Jointaro / Glassdoor	Hardware interns, India, 2021–2025	Digital-design MCQs then two HackerRank C/C++ problems in one hour (camera-proctored, easy–medium; one was "find the missing element in an array"). Another: **15 questions in 50 minutes**, including a probability question about three vehicles. Another: 60 minutes of digital electronics, aptitude, puzzles and computer architecture.
[S17] Jointaro (older)	Bengaluru CAD/ASIC and systems software, 2016–2020	OA with MCQs and two C questions; a HackerRank round on **bit-shifting of characters**; an interview round on "C only: memory allocation and bits".
[S18] University / blog pages	General NVIDIA SWE	75-minute HackerRank screen with at least two DSA problems plus MCQs; another source: 3 coding questions in 90 minutes; "format depends on the team": one team reportedly **C only**, another 7 questions in 50 minutes, another 1 question in 25 minutes. Placement blogs quote 2–3 coding questions, 15–20 technical MCQs and 10–15 aptitude questions, clear rate 10–15% (SEO-style pages; weakest evidence).
[S19] Glassdoor	Software (Yokneam), Nov 2025	Take-home HackerRank with three coding problems; reported items: palindrome check, **shuffle a playlist so same-artist songs are not adjacent**; verbal: a data structure with **O(1) init / get / set / reset**.
[S20] GfG (old)	On-campus, c. 2018	40-question written test, no negative marking (C/C++ output, OS, aptitude); technical: pair-sum in C (first $O(n^2)$, then optimised); final round: C++ and puzzles.
[S21] GfG (tag index)	On-campus, c. 2022	Test covered aptitude, C fundamentals, OOP, code snippets, programming, and "**AI or statistics**" – the only hint of ML-flavoured MCQs.
[S22] LeetCode Discuss	SWE, off-campus 2023	A hard **pointer-output** question; "**is the compiler little- or big-endian?**"; where each variable is stored (static / local / global / dynamic); templates and inline functions.
[S23] GfG	Bengaluru infrastructure team, on-campus	Written test: **C++ concepts, virtual functions, abstract classes, output programs, extern variables, compilation errors**.

Source	Role / place / year	(continued)
[S24] Glassdoor (C++ roles)	C++ developer / first rounds	Write a C++ class with **operator overloading and a copy constructor**; deep vs. shallow copy; stack vs. heap; **RAII**.
[S25] Exponent	SWE	**memcpy / memmove from scratch**; design an **LRU cache**.

2.3 Job-posting evidence ★★

- **Intern posting** (Built In, "Compiler Verification Intern – Fall 2025", [J1]): build verification infrastructure for NVIDIA embedded platforms; requires strong C/C++ & Python; stand-out: compilers & ML projects, parallel programming or automation. This is the same text as your role description.
- **Pune listing** ([J4], a third-party notice for "Compiler Verification Intern – Fall 2026, Pune") says B.Tech/M.Tech CS, Electrical, Electronics or IT students may apply and that duration/stipend are not stated in the posting itself.
- **Senior postings** ([J2], [J3]): additionally list *machine learning, formal verification and fuzzing* to automate compiler verification, scripting in Python/shell/Perl, compiler front-end or LLVM experience as a plus, and (in one posting only) ISO 26262 functional-safety verification.
- **Related compiler postings** ([J6]): "LLVM and MLIR compiler engineer" expects loop, inter-procedural and global optimisations and strong C++.

2.4 What the evidence says ★★★

Topic	Reports mentioning it (of 25)	What to take from it
C/C++ output & memory (ctor/dtor, pointers, endianness, storage, call-by-value, virtual)	≈9	The most consistent theme. Practise *predicting output* of tricky snippets and *finding bugs* in a page of C.
DSA coding (trees, linked lists, DP, graphs, Trie, heap, bit tricks)	≈12	Expect 1–3 problems of easy–medium level; for the interview, medium–hard in the style of LeetCode.
Compiler optimisation *concepts* (loop unrolling, inlining, data-flow, pointer/dependence analysis, graph colouring, matrix-multiply optimisations)	≈6	Interview favourite. Know *why* each optimisation helps, when it hurts, and how the compiler decides.
Compiler basics (phases, code generation, optimisation stages)	≈3	Easy marks in MCQs and the managerial round.
OS / computer architecture (virtual memory, scheduling, caches)	≈5	Appears in OA MCQs and in compiler interviews (virtual→physical memory).
Aptitude, probability, puzzles	≈6	Light but present; do not skip the probability and permutation basics.
ML / statistics	≈1	Your OA table lists "ML exposure", so expect a few conceptual MCQs; breadth over depth.
GPU/CUDA/PTX specifics	few	Not reported for *intern* OAs, but the JD names PTX/CUDA; interviewers may ask conceptual questions (warps, divergence, memory spaces).

Study weights this book uses (derived from the reports above)

1. **C/C++ output prediction, memory, UB** (largest, because it is simultaneously OA, interview and the job).
2. **DSA coding** with fast, bug-free C++/Python templates.
3. **Compiler theory**: phases, parsing tables, IR/SSA, data-flow, *loop optimisations*, register allocation (graph colouring), linking.
4. **Aptitude & probability** (cheap marks; short formulas).
5. **OS/COA essentials** (virtual memory, scheduling, caches, number representation).
6. **ML basics** (metrics, bias–variance, neural-net basics, ML compilers).
7. **LLVM/PTX/CUDA concepts** (differentiators in the interview; know the vocabulary).
8. **Verification methodology** (differential testing, fuzzing, reduction): the *job itself*; interviewers love candidates who can speak about it.

2.5 Your probable OA, reconstructed ★★★

Putting the reports together, the most probable shape (to be confirmed by your invitation) is a single HackerRank session of roughly 60–90 minutes:

Section	Likely size	Content
Aptitude	10–15 MCQs	Percentages, ratios, time–work, probability, permutations, series, logical reasoning, a puzzle or two
Technical MCQs	15–25 MCQs	C/C++ output and concepts; Python snippets; data structures; OS/COA; compiler theory (parsing, optimisation, phases); a few ML questions
Coding	1–3 problems	Array/string/bit manipulation, hashing, sorting, linked list/tree, DP; usually C/C++ or Python (check the language list; one report says a team accepted C only)

Time budget for a 75-minute paper (adapt to yours)

Spend about 12 minutes on aptitude, 25 minutes on technical MCQs (30 seconds per MCQ you know; skip anything slower than 2 minutes), and 35 minutes on coding (easy problem first). Reserve the last 3 minutes to re-check edge cases and remove debug prints. Details in Chapter 3.

2.6 Interview style, as reported ★★

- **Format:** one or two technical rounds (30–60 minutes), then a managerial/HR round; some compiler loops are *entirely verbal* (no coding).
- **Code reading beats code writing:** "find the bugs in this C program", "which constructors run", "what is the output" (S1, S3, S5).
- **Theory with trade-offs:** "why unroll loops?", "what is the downside of aggressive optimisation?" (S4, S8). Always answer in the pattern *benefit → cost → when the compiler decides to do it.*
- **Projects matter:** Austin and Pune reports start with projects (C++/GPU/compilers). Prepare a 2-minute story per project (Chapter **??**).
- **The OA is reused:** one report says interview questions repeated OA questions; keep a list of every OA question you remember afterwards.

2.7 Source list

Candidate reports

S1 https://www.geeksforgeeks.org/interview-experiences/nvidia-interview-experience-for-compiler-intern-6-months/

S2 https://www.geeksforgeeks.org/interview-experiences/nvidia-llvm-compiler-engineer-internship-fall-2025/

S3 https://www.geeksforgeeks.org/interview-experiences/nvidia-interview-experience-llvm-compiler-engineer-6m-spring-internship/

S4 https://www.jointaro.com/interviews/companies/nvidia/experiences/compiler-engineer-intern-pittsburgh-pa-november-1-2024-no-offer-neutral-cb716c3f/

S5 https://www.jointaro.com/interviews/companies/nvidia/experiences/compiler-engineer-intern-austin-tx-april-1-2025-no-offer-positive-1674ddef/

S6 https://www.jointaro.com/interviews/companies/nvidia/experiences/compiler-engineer-intern-shanghai-shanghai-march-1-2024-no-offer-neutral-199e988d/

S7 https://www.glassdoor.co.in/Interview/NVIDIA-Compiler-Engineer-Intern-Interview-Questions-EI_IE7633.0,6_KO7,31.htm

S8 https://www.glassdoor.com/Interview/NVIDIA-Compiler-Engineer-Interview-Questions-EI_IE7633.0,6_KO7,24.htm

S9 https://leetcode.com/discuss/interview-experience/1517813/nvidia-compiler-engineer-pune-india-sep-2021-reject/

S10 https://programhelp.net/en/vo/nvidia-intern-real-interview-experience/

S11 https://www.geeksforgeeks.org/interview-experiences/nvidia-system-software-engineer-intern-on-campus-drive-experience/

S12 https://www.geeksforgeeks.org/interview-experiences/nvidia-interview-experience-for-system-software-engineer-internshipon-campus/

S13 https://www.geeksforgeeks.org/interview-experiences/nvidia-interview-experience-for-sde-internship-on-campus-2024/ and https://leetcode.com/discuss/interview-experience/4730338/

S14 https://static.glassdoor.com.ar/Interview/NVIDIA-Interview-Questions-E7633_P51.htm (Glassdoor listing page)

S15 https://www.jointaro.com/interviews/companies/nvidia/experiences/software-engineer-intern-new-delhi-delhi-october-1-2025-no-offer-positive-9bdac977/

S16 https://www.jointaro.com/interviews/companies/nvidia/experiences/hardware-intern-india-october-1-2024-no-offer-positive-22ba4975/, https://www.jointaro.com/interviews/companies/nvidia/experiences/hardware-internship-india-january-8-2025-no-offer-positive-bc6c3d7a/, https://www.jointaro.com/interviews/companies/nvidia/experiences/hardware-internship-india-march-17-2021-no-offer-positive-36f589b3/

S17 https://www.jointaro.com/interviews/companies/nvidia/work-experiences/software-systems-engineer-bengaluru-february-5-2020-4-1a44b6c8/, https://www.jointaro.com/interviews/companies/nvidia/work-experiences/cad-engineer-asic-bengaluru-november-29-2016-1-3da7b8ff/

S18 https://career.online.ou.edu/blog/2025/02/25/get-a-job-at-nvidia-interview-process-and-top-questions/, https://www.linkjob.ai/interview-questions/nvidia-hackerrank-test/, https://www.lodely.com/companies/nvidia/online-assessment, https://placementpapers.app/nvidia/

S19 https://www.glassdoor.ie/Interview/NVIDIA-Interview-E7633-RVW101986297.htm, https://www.glassdoor.sg/Interview/NVIDIA-Interview-E7633-RVW104894219.htm

S20 https://www.geeksforgeeks.org/?p=228100

S21 https://geeksforgeeks.org/tag/nvidia

S22 https://leetcode.com/discuss/post/4032849/nvidia-interview-off-campus-2023

S23 https://www.geeksforgeeks.org/?p=132922

S24 https://www.glassdoor.sg/Interview/NVIDIA-Interview-E7633-RVW15149051.htm, https://www.glassdoor.co.in/Interview/NVIDIA-Interview-E7633-RVW23765350.htm, https://www.glassdoor.co.uk/Interview/NVIDIA-Interview-E7633-RVW9395333.htm

S25 https://www.tryexponent.com/experiences/nvidia-software-engineer-interview-90647e

Job postings and context

J1 https://builtin.com/job/compiler-verification-intern-fall-2025/6273014

J2 https://jointaro.com/jobs/nvidia/compiler-verification-software-developer

J3 https://builtinbengaluru.in/job/senior-compiler-verification-software-developer/3285594

J4 https://campusreality.in/notification/nvidia-compiler-verification-intern-fall-2026-pune/

J5 https://www.nvidia.com/en-eu/about-nvidia/careers/university-recruiting-new/

J6 https://frontlinesmedia.in/nvidia-is-hiring-llvm-and-mlir-compiler-engineer/

3. The 24-Hour Battle Plan and OA Tactics

In this chapter

Read time: 15 min. A time-boxed schedule (with a 6-hour and a 12-hour cut), exam-day tactics, and HackerRank mechanics with templates that were compiled and run while building this book.

3.1 Choose your budget ★★★

Budget	Study hours	What to do
P1: panic	6 h	Only three-star sections. C/C++ output traps (Ch. 4, 5), compiler theory essentials (Ch. 8–11: phases, FIRST/FOLLOW, LR basics, optimisations, data-flow), two timed coding problems (Ch. 7), aptitude formula sheet (Ch. 21), one 45-minute mini-mock.
P2: normal	12 h	P1 *plus*: Python gotchas (Ch. 6), OS/COA essentials (Ch. 20), ML quick pass (Ch. 19), LLVM/PTX/CUDA vocabulary (Ch. 12–15), verification vocabulary (Ch. 17), full mock OA.
P3: full	∼14 h	Everything below, including the automation lab (Ch. 18), interview flashcards (Ch. **??**) and a second set of timed coding problems.

3.2 Hour-by-hour plan (P3, adjust to your OA time) ★★★

Hours are counted from the moment you start. About 14 hours of focused work plus meals/breaks plus 7 hours of sleep fits inside 24 hours; if the OA is in the morning, put the sleep *before* the last two blocks and do those after waking.

Time (H+)	Block	Chapters	Done when ...
0:00–0:30	Orientation	1, 2, 3	You can state the OA shape, the priorities, and today's schedule. Install/open a local C++ and Python environment.
0:30–2:30	**C and C++** (output traps, UB, memory, OOP, STL)	4, 5	≥80% on the output-prediction drills; every miss logged with the rule behind it.
2:30–2:45	Break		Water, walk, no phone.
2:45–4:15	**Compilers I**: phases, lexing, parsing (FIRST/FOLLOW, LL(1), LR/SLR)	8, 9	You can compute FIRST/FOLLOW, an LL(1) table and an LR(0) automaton on paper.
4:15–5:45	**Compilers II**: semantic analysis, IR/SSA, runtime, **optimisation**, data-flow, code generation, linking	10, 11	You can run reaching-definitions by hand, explain six loop optimisations, and colour a small interference graph.
5:45–6:15	Meal		
6:15–7:30	**LLVM, PTX, CUDA**: IR reading, FileCheck, vecadd PTX line by line, warps/divergence/memory spaces	12–16	You can read the vecadd PTX aloud and explain every instruction.
7:30–8:30	**Verification methodology + automation lab**	17, 18	You can explain differential testing, EMI, fuzzing, delta debugging; you ran the lab scripts.

Time (H+)	Block	Chapters	Done when ...
8:30–9:15	Python data model and gotchas	6	100% on the Python drills.
9:15–9:45	Dinner break		
9:45–10:30	ML quick pass (metrics, bias–variance, neural nets, ML compilers)	19	You can compute precision/recall/F1 and explain backprop in four sentences.
10:30–11:30	**Aptitude** (formulas + 30 timed questions)	21	30 questions in 40 minutes, ≥75% correct.
	— Sleep 7 hours (do not skip; recall and focus beat one more chapter) —		
+0:00–1:30	**Coding practice** (timed)	7	Two reported problems solved in ≤25 min each in C++ *and* one in Python.
+1:30–2:15	OS, computer architecture, number systems	20	Virtual memory, scheduling, caches, IEEE-754 reviewed.
+2:15–3:30	**Mock OA** (full length, strict timing)	22	Score logged; every error classified (concept / careless / time).
+3:30–4:00	Interview flashcards, project stories	23	Two-minute story for each project; ten rapid-fire answers out loud.
+4:00–4:20	Final 20 minutes: "Top-40 card"	App. **??**	Re-read, then stop. Check logistics.

How to study so that it sticks – active recall

1. **Cover the answer, attempt, then read.** For every `Q`*n.m* write your answer first (even a guess).
2. **Keep an error log** (one line per miss: topic, wrong belief, correct rule). Re-read the log, not the chapters, in the last hour.
3. **Type, do not just read.** Compile every output-prediction snippet you got wrong; the compiler is the examiner you can interrogate.
4. **50/10 rhythm:** 50 minutes focus, 10 minutes off. Meals and sleep are part of the plan.

3.3 Tactics inside the OA ★★★

1. **Spend the first 2 minutes reading the whole paper.** Note section sizes, any section-level time limits, whether you can go back, and the language list for coding questions.
2. **Attack order:** fastest marks first. Usually aptitude and short MCQs, then the *easier* coding problem, then harder MCQs, then the other coding problem. If sections are individually timed and locked, follow the lock order instead.
3. **MCQ discipline:** at most 90 seconds per MCQ; mark and move. For code-output MCQs: first look for *traps* (UB, integer promotion, operator precedence, off-by-one, uninitialised variable, missing `break`, macro side effects, `sizeof` of a pointer) before computing anything.
4. **Coding: get partial credit early.** HackerRank scores by test case. Submit a correct brute force quickly, then optimise. Read the constraints first (Chapter 7, complexity table).
5. **Never leave the last 3 minutes unused:** re-run samples, check edge cases ($n = 0$, $n = 1$, all equal, negatives, maximum size), delete debug output.

Negative marking: when is guessing correct?

Suppose a correct answer gives $+1$ and a wrong answer gives $-p$, with k equally likely options after elimination. A blind guess among k options has expected score

$$E = \frac{1}{k} \cdot 1 - \frac{k-1}{k}\, p.$$

$E > 0 \iff p < \frac{1}{k-1}$. For 4 options and the common penalty $p = \frac{1}{4}$: blind guess $E = \frac{1}{4} - \frac{3}{4} \cdot \frac{1}{4} = +\frac{1}{16} > 0$; after eliminating one option ($k = 3$) $E = \frac{1}{3} - \frac{2}{3} \cdot \frac{1}{4} = +\frac{1}{6}$. **Rule:** with no penalty always guess; with $p \le \frac{1}{3}$ guessing is never harmful in expectation; with a harsher penalty guess only after eliminating options. (Reports differ: some NVIDIA drives had negative marking, some did not – [S12], [S20]. Check your instructions.)

3.4 HackerRank mechanics ★★★

- **Languages:** pick the one in which you are fastest *and* least likely to overflow. C++ for speed/STL; Python for string/array manipulation and big integers. Check the exact language versions in the editor drop-down (they change).
- **Run vs. Submit:** *Run Code* tests your custom input; *Submit* runs all hidden tests. Many OAs allow limited submissions per question; sample-passing is not full marks.
- **Errors:** *Wrong Answer* (logic or format), *Runtime Error* (segfault, exception, `exit code != 0`, array out of range, divide by zero), *Time Limit Exceeded* (complexity), *Compilation Error* (wrong language version or a missing header).
- **Proctoring:** reports mention camera-proctored tests (S11, S16, S18). Close other tabs/apps, keep ID ready, avoid leaving full-screen, use a charged laptop and stable network.

3.4.1 Fast I/O templates (compiled and run)

C++ template: fast input, 64-bit sum

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);   // do not mix C stdio and iostream afterwards
    cin.tie(nullptr);              // do not flush cout before every cin
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    long long sum = accumulate(a.begin(), a.end(), 0LL);   // 0LL, not 0: avoids int accumulation
    cout << sum << "\n";           // "\n" is faster than endl (no flush)
    return 0;
}
```

```
6
```

Python template: read everything at once

```python
import sys

def main():
    data = sys.stdin.read().split()          # fastest way to read many numbers
    n = int(data[0])
    a = list(map(int, data[1:1 + n]))
    print(sum(a))

main()
```

```
6
```

3.4.2 The silent killers

Integer overflow: the classic wrong answer

```cpp
#include <stdio.h>
int main(void) {
    int a = 100000;
    int  bad  = a * a;              /* signed overflow: undefined behaviour (typically wraps) */
    long long good = (long long)a * a;   /* widen BEFORE multiplying */
    printf("%d %lld\n", bad, good);
    return 0;
}
```

```
1410065408 10000000000
```

Pitfall checklist

- $10^5 \times 10^5$ overflows 32-bit `int`; sums of 10^5 numbers of size 10^9 need 64 bits. Cast *before* multiplying: `(long long)a*b`.
- C++: `vector<int> v(n)` then reading `n` numbers is fine, but `v.size()-1` on an empty vector underflows (`size_t` is unsigned).
- Python: recursion limit is about 1000 by default; use iteration or `sys.setrecursionlimit`. Avoid `input()` in loops of 10^5 lines.
- Output format: trailing spaces are usually tolerated, but a missing newline or extra text is not. Print only what is asked.
- Floating point: print with fixed precision (`cout<<fixed<<setprecision(6)`) when requested; never compare doubles with `==`.

3.5 Test-day checklist ★★

☐ Invitation email re-read: date/time zone, duration, sections, languages, negative marking, proctoring rules.

☐ Laptop charged and plugged in; browser updated; camera/mic permissions tested in the HackerRank system check; stable network (mobile hotspot as a backup).

☐ Government ID and college ID on the desk; quiet room; water; paper and pen for rough work (confirm that paper is allowed).

☐ A local C++/Python environment ready only for *warm-up* (not during the test unless allowed).

☐ Sleep $\geq$7 hours, a light meal, and a 30-minute warm-up of three easy problems before the test starts.

Part II
Programming Foundations

4. C Programming: Output Traps, Memory and UB

In this chapter

Priority ★★★. Time: 75 min (C) + drills. C questions are the most consistently reported topic (S1, S3, S12, S17, S20, S22): *predict the output*, *find the bug*, *where does this variable live*, *is the machine little-endian*. And they are the job: a compiler verifier must write C that has **no undefined behaviour** so that a wrong result can only be the compiler's fault. All outputs in yellow boxes were produced by compiling and running the code (GCC 13, x86-64 Linux, `-O0` unless stated).

4.1 Memory layout and storage classes ★★★

Where things live in a C program

Segment	Holds	Notes
`.text`	machine code	read-only, executable
`.rodata`	string literals, `const` globals	read-only; writing is undefined behaviour (usually SIGSEGV)
`.data`	initialised globals and `statics`	copied from the executable image
`.bss`	zero-initialised (or uninitialised) globals/statics	occupies no space in the file; zeroed by the loader
heap	`malloc`/`calloc`/`realloc` memory	grows upward; you must `free` it; fragmentation possible
stack	locals, parameters, return addresses	grows downward on x86-64; automatic lifetime; limited (about 8 MB default on Linux)

The linker's symbol table shows the section class of each global. In `nm` output: `T` = text, `D`/`d` = initialised data (global/static), `B` = bss, `R` = read-only data, `U` = undefined (resolved at link time).

```c
#include <stdlib.h>
int g_init = 5;                 /* initialised global      -> .data (D) */
int g_zero;                     /* uninitialised global    -> .bss  (B) */
const int g_const = 9;          /* const global            -> .rodata (R) */
static int g_static = 7;        /* file-local initialised  -> .data (d) */
const char *msg = "hi";         /* pointer in .data (D); the literal "hi" is in .rodata */
int main(void) {
    int local = 1;              /* stack */
    int *h = malloc(4);         /* h itself on the stack; the 4 bytes on the heap */
    free(h);
    return local + g_init;
}
```

```
$ gcc -c mem.c -o mem.o && nm mem.o
```

```
                 U free
0000000000000000 R g_const
0000000000000000 D g_init
0000000000000004 d g_static
0000000000000000 B g_zero
0000000000000000 T main
                 U malloc
0000000000000000 D msg
```

Class	Scope	Lifetime	Linkage	Default init
`auto` (default for locals)	block	the block execution	none	indeterminate (garbage)
`register`	block	the block	none	indeterminate; cannot take address in C
`static` (local)	block	whole program	none	zero
`static` (global/function)	file	whole program	**internal**	zero
`extern` / plain global	file or program	whole program	**external**	zero

Q4.1 What does this print?

```c
#include <stdio.h>
void counter(void) {
    static int n = 0;      /* initialised once, persists across calls */
    int m = 0;             /* re-created at every call */
    n++; m++;
    printf("%d,%d ", n, m);
}
int main(void) {
    for (int i = 0; i < 3; i++) counter();
    printf("\n");
    return 0;
}
```

Answer: captured output

```
1,1 2,1 3,1
```

The `static` local keeps its value (1, 2, 3) while the automatic `m` restarts at 1 each time.

4.2 Types, sizes, promotions and conversions ★★★

4.2.1 Sizes (LP64: Linux/macOS 64-bit) – know these cold

sizeof on x86-64 Linux (LP64)

```c
#include <stdio.h>
#include <stdint.h>
int main(void) {
    printf("char=%zu short=%zu int=%zu long=%zu long long=%zu\n",
           sizeof(char), sizeof(short), sizeof(int), sizeof(long), sizeof(long long));
    printf("float=%zu double=%zu long double=%zu pointer=%zu size_t=%zu\n",
           sizeof(float), sizeof(double), sizeof(long double), sizeof(void *), sizeof(size_t));
    printf("int32_t=%zu int64_t=%zu\n", sizeof(int32_t), sizeof(int64_t));
    return 0;
}
```

```
char=1 short=2 int=4 long=8 long long=8
float=4 double=8 long double=16 pointer=8 size_t=8
int32_t=4 int64_t=8
```

Platform differences

On 64-bit **Windows** (LLP64) `long` is 4 bytes; on 32-bit systems pointers are 4 bytes. If a question does not name the platform, assume the common LP64 sizes above and say so. `sizeof` yields `size_t` (unsigned): print it with `%zu`.

4.2.2 Integer promotions and the usual arithmetic conversions

Must know

1. Any operand narrower than `int` (`char`, `short`, `bool`) is first promoted to `int`.
2. If both operands then have the same signedness, the narrower converts to the wider.
3. If the unsigned operand has rank $\geq$ the signed one, the **signed operand becomes unsigned**. (So $-1 < 1u$ is false: -1 becomes 4294967295.)
4. Integer division truncates toward zero; `a%b` has the sign of `a` (C99 and later).

Signed/unsigned traps

```c
#include <stdio.h>
int main(void) {
    unsigned int u = 1;
    int i = -1;
    printf("i < u            : %d\n", i < u);          /* -1 converted to unsigned: huge */
    printf("sizeof(int) > -1 : %d\n", sizeof(int) > -1);/* -1 converted to size_t */
    unsigned char c = 200;
    printf("c + c            : %d\n", c + c);          /* promoted to int: 400, not 144 */
    printf("(unsigned char)(c + c): %d\n", (unsigned char)(c + c));
    char s = (char)200;                                 /* implementation-defined: usually -56 */
    printf("(char)200        : %d\n", s);
    printf("-7/2=%d  -7%%2=%d  7/-2=%d  7%%-2=%d\n", -7/2, -7%2, 7/-2, 7%-2);
    printf("sizeof('a')=%zu  sizeof(1 ? 1 : 2.0)=%zu\n", sizeof('a'), sizeof(1 ? 1 : 2.0));
    return 0;
}
```

```
i < u            : 0
sizeof(int) > -1 : 0
c + c            : 400
(unsigned char)(c + c): 144
(char)200        : -56
-7/2=-3  -7%2=-1  7/-2=-3  7%-2=1
sizeof('a')=4  sizeof(1 ? 1 : 2.0)=8
```

C vs. C++ difference worth a mark

A character constant such as `'a'` has type `int` in C (so `sizeof('a')` is 4) but type `char` in C++ (so 1). The same one-line program prints 4 compiled as C and 1 compiled as C++ (verified while building this book).

4.3 Pointers and arrays ★★★

Rules that decide most pointer MCQs

- `a[i]` is defined as `*(a+i)`; therefore `i[a]` is legal and equal. Pointer arithmetic is **scaled by the pointee size**: `p+1` advances `sizeof(*p)` bytes.
- An array name *decays* to a pointer to its first element in most expressions, **except** as the operand of `sizeof`, `&`, and a string-literal initialiser.
- A function parameter declared `int a[10]` is really `int *a`: `sizeof a` inside the function is the pointer size.
- `p-q` (same array) has type `ptrdiff_t` and counts *elements*. Comparing/subtracting pointers into different objects is undefined.
- Precedence: postfix `++ -- () []` > prefix `++ -- * & sizeof (cast)` > `* / %` ... Hence `*p++` is `*(p++)`, while `(*p)++` increments the pointee.

Array vs pointer, decay, and *p++ variants

```c
#include <stdio.h>
void f(int a[10]) { printf("inside f: sizeof a = %zu\n", sizeof a); }   /* a is int* */
int main(void) {
    int arr[10] = {0};
    printf("sizeof arr = %zu, elements = %zu\n", sizeof arr, sizeof arr / sizeof arr[0]);
    f(arr);
    printf("sizeof(arr+1) = %zu\n", sizeof(arr + 1));          /* decayed to pointer */
```

```
    int a[] = {10, 20, 30};
    int *p = a;
    int x = *p++;           /* x = 10, then p -> a[1] */
    printf("*p++ : x=%d, now *p=%d\n", x, *p);
    int y = (*p)++;         /* y = 20, a[1] becomes 21 */
    printf("(*p)++ : y=%d, a[1]=%d\n", y, a[1]);
    int z = ***p;           /* p -> a[2], z = 30 */
    printf("***p : z=%d\n", z);
    printf("2[a] = %d, &a[2]-&a[0] = %td\n", 2[a], &a[2] - &a[0]);
    return 0;
}
```

```
sizeof arr = 40, elements = 10
inside f: sizeof a = 8
sizeof(arr+1) = 8
*p++ : x=10, now *p=20
(*p)++ : y=20, a[1]=21
***p : z=30
2[a] = 30, &a[2]-&a[0] = 2
```

2-D arrays and pointer-to-array

```
#include <stdio.h>
int main(void) {
    int m[2][3] = {{1, 2, 3}, {4, 5, 6}};
    printf("sizeof m=%zu  sizeof m[0]=%zu  sizeof *m=%zu\n", sizeof m, sizeof m[0], sizeof *m);
    printf("*(*(m+1)+2)=%d  **m=%d  *(m[0]+1)=%d\n", *(*(m + 1) + 2), **m, *(m[0] + 1));
    int (*pa)[3] = m;               /* pointer to an array of 3 ints */
    printf("pa[1][2]=%d  (pa+1) advances %td bytes\n", pa[1][2], (char *)(pa + 1) - (char *)pa);
    int *ap[2] = {m[0], m[1]};      /* array of 2 pointers */
    printf("ap[1][0]=%d  sizeof ap=%zu\n", ap[1][0], sizeof ap);
    return 0;
}
```

```
sizeof m=24  sizeof m[0]=12  sizeof *m=12
*(*(m+1)+2)=6  **m=1  *(m[0]+1)=2
pa[1][2]=6  (pa+1) advances 12 bytes
ap[1][0]=4  sizeof ap=16
```

Strings: array vs. pointer to literal

`char s1[] = "abc";` creates a *modifiable copy* (4 bytes including the NUL). `char *s2 = "abc";` points to a literal in read-only memory: `s2[0]='X';` is **undefined** (typically a crash). `sizeof s1` is 4, `sizeof s2` is 8.

Writing to a string literal crashes (SIGSEGV = signal 11)

```
#include <stdio.h>
int main(void) {
    char *s2 = "abc";
    printf("before\n");
    fflush(stdout);
    s2[0] = 'X';          /* undefined behaviour: literal is in .rodata */
    printf("after\n");
    return 0;
}
```

```
before
[killed by signal 11]
```

4.3.1 The const reading rule and function pointers

Read declarations **right to left**: `const int *p` = "p is a pointer to const int" (cannot change `*p`); `int *const p` = "p is a const pointer to int" (cannot change `p`); `const int *const p`: neither.

Modifying through a pointer-to-const does not compile

```
int main(void) {
    int v = 1;
    const int *p = &v;
    *p = 2;          /* error: assignment of read-only location */
    return 0;
```

```
}
```

```
prog.c: In function 'main':
prog.c:4:8: error: assignment of read-only location '*p'
    4 |     *p = 2;          /* error: assignment of read-only location */
      |        ^
```

Function pointers and qsort (comparator must not overflow)

```c
#include <stdio.h>
#include <stdlib.h>
static int cmp_asc(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);      /* NOT x - y: that can overflow for large values */
}
int apply(int (*op)(int, int), int a, int b) { return op(a, b); }
static int add(int a, int b) { return a + b; }
static int mul(int a, int b) { return a * b; }
int main(void) {
    int v[] = {5, -2, 9, 0, 3};
    qsort(v, 5, sizeof v[0], cmp_asc);
    for (int i = 0; i < 5; i++) printf("%d ", v[i]);
    printf("\n%d %d\n", apply(add, 3, 4), apply(mul, 3, 4));
    int (*table[2])(int, int) = {add, mul};    /* array of function pointers */
    printf("%d\n", table[1](5, 6));
    return 0;
}
```

```
-2 0 3 5 9
7 12
30
```

4.4 Undefined, unspecified and implementation-defined behaviour ★★★

This is the single most important idea for a compiler verifier.

The three kinds of "unportable" behaviour

Undefined behaviour (UB): the standard imposes *no requirements*. The program may crash, print anything, appear to work, or be optimised into something surprising **because compilers assume UB never happens**.

Unspecified behaviour: several outcomes are allowed, none required to be documented (e.g. evaluation order of function arguments).

Implementation-defined behaviour: the implementation must document its choice (e.g. `sizeof(int)`, signedness of plain `char`, right shift of a negative number).

Common undefined behaviour in C	Why a compiler may exploit it
Signed integer overflow (`INT_MAX+1`)	Lets the optimiser assume `x+1>x` is always true, widen loop counters, etc.
Shift by a negative amount or by $\geq$ the width; left-shifting a 1 into the sign bit	Hardware shifts differ; compilers fold freely
Division by zero; `INT_MIN / -1`	Traps on x86 (SIGFPE); folded arbitrarily otherwise
Out-of-bounds array access, null or dangling dereference, use-after-free, double free	Memory is assumed to be well-formed; optimisers reorder/eliminate accesses
Reading an uninitialised automatic variable (in most cases)	Value may differ at each use
Modifying a variable twice between sequence points (`i = i++ + ++i`)	Order of side effects is not fixed
Strict-aliasing violation (reading a `float` through an `int*`)	Loads/stores of different types are assumed not to alias

Common undefined behaviour in C	Why a compiler may exploit it
Modifying a string literal or a `const` object	Placed in read-only memory
Data race on non-atomic objects (C11/C++11)	Memory model assumes race freedom
`printf` with a wrong format specifier / wrong argument count	Varargs ABI mismatch

4.4.1 Live demonstration: three compilers, three answers

ub1.c: signed overflow hidden in x + 1 > x

```c
#include <stdio.h>
#include <limits.h>
int f(int x) { return x + 1 > x; }      /* true for all x --- unless x + 1 overflows (UB) */
int main(void) { printf("%d\n", f(INT_MAX)); return 0; }
```

```
$ for cc in gcc clang; do for o in -O0 -O2; do echo "$cc $o -> $($cc $o -w ub1.c -o ub1 && ./ub1)"; done; done
```

```
gcc -O0 -> 1
gcc -O2 -> 1
clang -O0 -> 0
clang -O2 -> 1
```

What that table means

The same *valid-looking* program prints 1 or 0 depending on compiler and optimisation level (GCC folds the comparison even at `-O0`; Clang `-O0` actually wraps). Neither compiler is wrong, because the program has undefined behaviour. **If a verification test contains UB, a mismatch between two compilers proves nothing.** This is why random-program generators such as Csmith are designed around UB avoidance (Chapter 17).

ub2.c: unspecified evaluation order of function arguments

```c
#include <stdio.h>
int main(void) {
    int j = 1;
    printf("%d %d\n", j++, j++);   /* modifies j twice without sequencing: UB (and order is unspecified) */
    return 0;
}
```

```
$ for cc in gcc clang; do $cc -w ub2.c -o ub2 && echo "$cc: $(./ub2)"; done
```

```
gcc: 2 1
clang: 1 2
```

GCC evaluates the arguments right-to-left here and Clang left-to-right. A correct program must not depend on either order.

Tools that find UB

`gcc/clang -fsanitize=undefined` (UBSan: signed overflow, bad shifts, null/misaligned access), `-fsanitize=address` (ASan: heap/stack buffer overflow, use-after-free, leaks), `-fsanitize=thread`, `-fsanitize=memory` (Clang), `-Wall -Wextra`, and `valgrind`. Example, UBSan output for `INT_MAX + 1` on a volatile variable:

```
$ gcc -fsanitize=undefined ubsan.c -o ubsan && ./ubsan 2>&1 | sed 's/^[^:]*:/prog.c:/'
```

```
prog.c:3:48: runtime error: signed integer overflow: 2147483647 + 1 cannot be represented in type 'int'
-2147483648
```

4.4.2 Well-defined ways to do what people try with UB

Type punning: union (legal in C) or memcpy (legal in C and C++)

```c
#include <stdio.h>
#include <string.h>
#include <stdint.h>
int main(void) {
    float f = 1.0f;
    union { float f; uint32_t u; } pun = { .f = f };
    uint32_t viaMemcpy;
    memcpy(&viaMemcpy, &f, sizeof f);
    printf("1.0f bits: union=0x%08x memcpy=0x%08x\n", pun.u, viaMemcpy);
    /* uint32_t bad = *(uint32_t *)&f;   <- strict-aliasing violation: undefined */
    unsigned x = 1u << 31;                    /* fine: unsigned */
    printf("1u<<31 = %u\n", x);
    printf("1.0/0 = %f, -1.0/0 = %f\n", 1.0 / 0, -1.0 / 0);   /* IEEE: inf, not UB on IEEE targets */
    return 0;
}
```

```
1.0f bits: union=0x3f800000 memcpy=0x3f800000
1u<<31 = 2147483648
1.0/0 = inf, -1.0/0 = -inf
```

4.5 The preprocessor ★★★

Macro traps: precedence and double evaluation

```c
#include <stdio.h>
#define SQ_BAD(x)   x * x
#define SQ_OK(x)    ((x) * (x))
#define MAX(a, b)   ((a) > (b) ? (a) : (b))
#define STR(x)      #x                  /* stringize */
#define CAT(a, b)   a##b                /* token pasting */
int main(void) {
    printf("SQ_BAD(2+3)   = %d   (expands to 2+3 * 2+3)\n", SQ_BAD(2 + 3));
    printf("SQ_OK(2+3)    = %d\n", SQ_OK(2 + 3));
    printf("100/SQ_BAD(5) = %d   (expands to 100/5 * 5)\n", 100 / SQ_BAD(5));
    int a = 3, b = 7;
    int m = MAX(a++, b++);                /* b++ is evaluated twice when b is chosen */
    printf("MAX(a++,b++)  = %d, a=%d, b=%d\n", m, a, b);
    printf("%s, %d\n", STR(hello world), CAT(1, 2) + 1);
    printf("line %d in %s\n", __LINE__, "prog.c");
    return 0;
}
```

```
SQ_BAD(2+3)   = 11   (expands to 2+3 * 2+3)
SQ_OK(2+3)    = 25
100/SQ_BAD(5) = 100   (expands to 100/5 * 5)
MAX(a++,b++)  = 8, a=4, b=9
hello world, 13
line 15 in prog.c
```

Preprocessor facts

- Macros are **textual substitution before parsing**: no types, no scope, no evaluation. Parenthesise every parameter and the whole body.
- `#x` stringizes the argument; `a##b` pastes tokens; `__VA_ARGS__` for variadic macros; `__LINE__`, `__FILE__`, `__func__` (the last is a C99 identifier, not a macro).
- Multi-statement macros: `do { ... } while (0)` so they behave as one statement (works with `if/else`).
- Include guards (`#ifndef X / #define X / #endif`) or `#pragma once` prevent double inclusion; the preprocessor's output is what the compiler sees (`gcc -E`).
- Prefer `static inline` functions (C) or `constexpr` functions (C++): typed, debuggable, no double evaluation.

4.6 Bit manipulation ★★★

Bit tricks appear in OAs (S17: "bit-shifting operations on characters"), in coding questions, and in every compiler (strength reduction, instruction selection).

Goal	Expression (unsigned x, bit index k)
Set / clear / toggle / test bit k	`x | (1u<<k)` `x & ~(1u<<k)` `x ^ (1u<<k)` `(x>>k) & 1u`
Lowest set bit (isolate)	`x & -x` (two's complement: $-x = \sim x + 1$)
Clear lowest set bit	`x & (x-1)` (Kernighan's popcount loops on this)
Is a power of two?	`x && !(x & (x-1))`
Low k bits mask / extract bits $[i..j]$	`(1u<<k)-1` `(x>>i) & ((1u<<(j-i+1))-1)`
Swap without a temporary	`a^=b; b^=a; a^=b;` (only for distinct objects; do not write it as one expression)
Unique element when all others appear twice	XOR all elements ($a \oplus a = 0$, $a \oplus 0 = a$)
Count trailing / leading zeros, popcount (GCC/Clang)	`__builtin_ctz(x)`, `__builtin_clz(x)`, `__builtin_popcount(x)`
Round up to the next power of two (32-bit)	`x--; x|=x>>1; x|=x>>2; x|=x>>4; x|=x>>8; x|=x>>16; x++;`

Bit tricks on x = 0b10110100 (180)

```c
#include <stdio.h>
#include <stdint.h>
static int popcount_kernighan(unsigned x) { int c = 0; while (x) { x &= x - 1; c++; } return c; }
int main(void) {
    unsigned x = 0xB4;   /* 1011 0100 */
    printf("x=%u  lowest set bit: x&-x = %u  clear it: x&(x-1) = %u\n", x, x & -x, x & (x - 1));
    printf("popcount=%d (Kernighan %d)  ctz=%d  clz(32-bit)=%d\n",
           __builtin_popcount(x), popcount_kernighan(x), __builtin_ctz(x), __builtin_clz(x));
    printf("is_pow2(64)=%d is_pow2(65)=%d\n", (64 && !(64 & 63)), (65 && !(65 & 64)));
    int a = 6, b = 9;
    a ^= b; b ^= a; a ^= b;
    printf("after xor-swap: a=%d b=%d\n", a, b);
    int arr[] = {4, 7, 4, 9, 7};          /* 9 is the unique one */
    int u = 0; for (int i = 0; i < 5; i++) u ^= arr[i];
    printf("unique = %d\n", u);
    printf("-x == ~x+1 ? %d\n", -(int)x == ~(int)x + 1);
    printf("-16 >> 2 = %d (arithmetic shift on this ABI; implementation-defined)\n", -16 >> 2);
    printf("~5=%d  5<<2=%d  5&3=%d 5|3=%d 5^3=%d\n", ~5, 5 << 2, 5 & 3, 5 | 3, 5 ^ 3);
    return 0;
}
```

```
x=180  lowest set bit: x&-x = 4  clear it: x&(x-1) = 176
popcount=4 (Kernighan 4)  ctz=2  clz(32-bit)=24
is_pow2(64)=1 is_pow2(65)=0
after xor-swap: a=9 b=6
unique = 9
-x == ~x+1 ? 1
-16 >> 2 = -4 (arithmetic shift on this ABI; implementation-defined)
~5=-6  5<<2=20  5&3=1 5|3=7 5^3=6
```

Shifting characters (the S17-style trap)

Char shifts: promotion to int, truncation on store, and 8-bit rotate

```c
#include <stdio.h>
int main(void) {
    unsigned char c = 0x81;                 /* 1000 0001 */
    printf("c << 1            = 0x%x  (promoted to int, NOT truncated)\n", c << 1);
    unsigned char d = c << 1;               /* truncated when stored back */
    printf("(unsigned char)   = 0x%x\n", d);
    signed char s = (signed char)0x81;      /* -127 */
    printf("s >> 1            = %d (sign-extended; implementation-defined)\n", s >> 1);
    unsigned char r = (unsigned char)((c << 1) | (c >> 7));   /* rotate left by 1 */
    printf("rotl8(0x81, 1)    = 0x%x\n", r);
```

```
    return 0;
}
```

```
c << 1            = 0x102  (promoted to int, NOT truncated)
(unsigned char)   = 0x2
s >> 1            = -64 (sign-extended; implementation-defined)
rotl8(0x81, 1)    = 0x3
```

Endianness check (reported in S22)

Is this machine little-endian?

```c
#include <stdio.h>
#include <stdint.h>
int main(void) {
    uint32_t v = 0x01020304;
    unsigned char *p = (unsigned char *)&v;     /* char access is always allowed */
    printf("first byte in memory: 0x%02x -> %s-endian\n", p[0], p[0] == 0x04 ? "little" : "big");
    return 0;
}
```

```
first byte in memory: 0x04 -> little-endian
```

4.7 Structs, unions, padding, bit-fields ★★

Layout rules

Each member is placed at an offset that is a multiple of its *alignment* (for scalars: its size). The struct's alignment is the largest member alignment, and its total size is rounded up to a multiple of that. **Declare members from largest to smallest to minimise padding.** A `union`'s size is its largest member's size (rounded up to its alignment); all members share the same bytes. Bit-field ordering and packing are implementation-defined.

Padding, offsetof, unions, bit-fields

```c
#include <stdio.h>
#include <stddef.h>
struct A { char c; int i; char d; };                    /* 1 + 3 pad + 4 + 1 + 3 pad */
struct B { int i; char c; char d; };                    /* 4 + 1 + 1 + 2 pad */
struct __attribute__((packed)) P { char c; int i; char d; };   /* GCC/Clang extension */
struct C { char c; double d; char e; };                 /* double needs 8-byte alignment */
union  U { int i; char c[8]; double d; };
struct BF { unsigned a : 3; unsigned b : 5; unsigned c : 8; };
int main(void) {
    printf("A=%zu (offsets %zu %zu %zu)\n", sizeof(struct A), offsetof(struct A, c), offsetof(struct A, i),
     offsetof(struct A, d));
    printf("B=%zu (offsets %zu %zu %zu)\n", sizeof(struct B), offsetof(struct B, i), offsetof(struct B, c),
     offsetof(struct B, d));
    printf("P=%zu  C=%zu  U=%zu  BF=%zu\n", sizeof(struct P), sizeof(struct C), sizeof(union U), sizeof(struct BF)
     );
    return 0;
}
```

```
A=12 (offsets 0 4 8)
B=8 (offsets 0 4 5)
P=6  C=24  U=8  BF=4
```

Why this matters to a compiler team

Struct layout is part of the **ABI**: if two compilers disagree on padding or bit-field placement, code compiled by one cannot call code compiled by the other. ABI/layout tests are a classic verification area.

4.8 Dynamic memory ★★★

Must know

- `malloc(n)`: uninitialised bytes; `calloc(k,n)`: zeroed, checks `k*n` overflow; `realloc(p,n)`: may move the block; on failure returns NULL *and the old block is still valid*, so never write `p = realloc(p, n)` (leak on failure); `free(NULL)` is a no-op; `free` twice is undefined.
- Idiom: `T *p = malloc(count * sizeof *p);` (type-safe: survives a change of `T`). Always test for NULL.
- **Memory leak** = allocated block unreachable and never freed. **Dangling pointer** = pointer to freed/expired storage. **Buffer overflow** = access past the allocation (the root of most security bugs).
- Allocators (glibc `malloc`) manage free lists in the heap (`brk`) and use `mmap` for large blocks; fragmentation and alignment (16 bytes on x86-64) matter.

2-D array on the heap, safe realloc, then correct cleanup

```c
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    int rows = 3, cols = 4;
    int **m = malloc(rows * sizeof *m);
    if (!m) return 1;
    for (int i = 0; i < rows; i++) {
        m[i] = calloc(cols, sizeof **m);          /* zero-initialised row */
        if (!m[i]) return 1;
        for (int j = 0; j < cols; j++) m[i][j] = i * j;
    }
    for (int i = 0; i < rows; i++) { for (int j = 0; j < cols; j++) printf("%d ", m[i][j]); printf("\n"); }
    for (int i = 0; i < rows; i++) free(m[i]);   /* free rows first ... */
    free(m);                                     /* ... then the pointer array */

    int *v = malloc(2 * sizeof *v), cap = 2, n = 0;
    for (int x = 1; x <= 5; x++) {
        if (n == cap) {                          /* amortised O(1) append: double the capacity */
            int *tmp = realloc(v, 2 * cap * sizeof *v);
            if (!tmp) { free(v); return 1; }     /* old block still valid if realloc failed */
            v = tmp; cap *= 2;
        }
        v[n++] = x * x;
    }
    printf("n=%d cap=%d last=%d\n", n, cap, v[n - 1]);
    free(v);
    return 0;
}
```

```
0 0 0 0
0 1 2 3
0 2 4 6
n=5 cap=8 last=25
```

Sanitizers turn silent memory bugs into precise reports. The three classic bugs, each compiled with `-fsanitize=address` (addresses and process ids removed from the output):

```
$ gcc -g -fsanitize=address X.c -o X && ./X     # for X in oob, uaf, leak
```

```
[oob]
ERROR: AddressSanitizer: heap-buffer-overflow
SUMMARY: AddressSanitizer: heap-buffer-overflow oob.c:2 in main
[uaf]
ERROR: AddressSanitizer: heap-use-after-free
SUMMARY: AddressSanitizer: heap-use-after-free uaf.c:2 in main
[leak]
ERROR: LeakSanitizer: detected memory leaks
SUMMARY: AddressSanitizer: 16 byte(s) leaked in 1 allocation(s).
```

4.9 Functions, call-by-value and the calling convention ★★★

C passes **everything by value**: the callee gets a copy of each argument. "Pass by reference" means passing a pointer *by value*. This is exactly the bug the S1 candidate missed in a 250-line program.

Call-by-value: why the head of a linked list is not updated

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int val; struct Node *next; } Node;

void push_bad(Node *head, int v) {          /* head is a COPY of the caller's pointer */
    Node *n = malloc(sizeof *n);
    n->val = v; n->next = head;
    head = n;                               /* updates only the local copy; n leaks */
}
void push_good(Node **head, int v) {        /* pointer to the caller's pointer */
    Node *n = malloc(sizeof *n);
    n->val = v; n->next = *head;
    *head = n;
}
void print_list(const Node *h) { for (; h; h = h->next) printf("%d ", h->val); printf("<end>\n"); }
void swap_val(int a, int b) { int t = a; a = b; b = t; }          /* no effect on the caller */
void swap_ptr(int *a, int *b) { int t = *a; *a = *b; *b = t; }
int main(void) {
    Node *head = NULL;
    push_bad(head, 1); push_bad(head, 2);
    print_list(head);                       /* list is still empty */
    push_good(&head, 1); push_good(&head, 2);
    print_list(head);
    int x = 1, y = 2;
    swap_val(x, y); printf("after swap_val: %d %d\n", x, y);
    swap_ptr(&x, &y); printf("after swap_ptr: %d %d\n", x, y);
    return 0;
}
```

```
<end>
2 1 <end>
after swap_val: 1 2
after swap_ptr: 2 1
```

4.9.1 Recursion, stack frames and the System V AMD64 calling convention

x86-64 System V ABI (Linux/macOS) in six lines

- Integer/pointer arguments: `rdi, rsi, rdx, rcx, r8, r9`, then the stack (right to left). Floating-point: `xmm0--xmm7`. Return value: `rax` (or `rdx:rax`, `xmm0`).
- **Callee-saved:** `rbx, rbp, r12--r15` (a function that uses them must restore them). **Caller-saved:** everything else (`rax, rcx, rdx, rsi, rdi, r8--r11, xmm`).
- The stack pointer must be 16-byte aligned at a `call`; the 128 bytes below `rsp` (the "red zone") may be used by leaf functions without adjusting `rsp`.
- A **stack frame** holds the return address, saved registers, locals and spilled temporaries; deep recursion exhausts the (about 8 MB) stack: stack overflow.

```c
int add3(int a, int b, int c) { return a + b + c; }
```

```
$ gcc -O1 -S -masm=intel add3.c
```

```
add3:
    add     edi, esi
    lea     eax, [rdi+rdx]
    ret
```

The three arguments arrive in `edi, esi, edx` (the 32-bit halves of `rdi, rsi, rdx`) and the result leaves in `eax`: the compiler turned `a+b+c` into one `add` and one `lea` (a 3-operand add in disguise).

4.9.2 Variadic functions

stdarg: default argument promotions

```c
#include <stdio.h>
#include <stdarg.h>
int sum(int n, ...) {
    va_list ap; va_start(ap, n);
    int s = 0;
    for (int i = 0; i < n; i++) s += va_arg(ap, int);   /* char/short arrive as int */
    va_end(ap);
    return s;
}
double avg(int n, ...) {
    va_list ap; va_start(ap, n);
    double s = 0;
    for (int i = 0; i < n; i++) s += va_arg(ap, double); /* float arrives as double: never va_arg(ap, float) */
    va_end(ap);
    return s / n;
}
int main(void) {
    printf("%d %.2f\n", sum(4, 1, 2, 3, 4), avg(3, 1.0, 2.0f, 6.0));
    return 0;
}
```

```
10 3.00
```

4.10 Qualifiers: `volatile`, `restrict`, `inline`, `static` ★★

- **`volatile`**: every access in the source must be performed (no caching in a register, no elimination). Use for memory-mapped I/O and variables changed by signal handlers. It is *not* atomic and gives no inter-thread ordering.
- **`restrict`** (C99): a promise that, during the pointer's lifetime, the object it points to is accessed only through it (or pointers derived from it). It lets the compiler vectorise; if the promise is false the behaviour is undefined.
- **`inline`**: a hint plus linkage rules. In C99/C11 a plain `inline` function needs one external definition; `static inline` avoids that problem.
- **`static`** on a function or global gives it internal linkage (invisible to other translation units).

The effect of `volatile` is visible in the generated code:

```c
int flag;                /* plain */
volatile int vflag;      /* volatile */
void wait_plain(void)    { while (!flag)  { } }
void wait_volatile(void) { while (!vflag) { } }
```

```
$ gcc -O2 -S -masm=intel vol.c
```

```
wait_plain:
    mov    eax, DWORD PTR flag[rip]
    test    eax, eax
    jne    .L1
.L3:
    jmp    .L3
.L1:
    ret
wait_volatile:
.L6:
    mov    eax, DWORD PTR vflag[rip]
    test    eax, eax
    je    .L6
    ret
```

Read that assembly

`wait_plain` loads `flag` **once**, and if it was 0 jumps to `.L3:  jmp .L3`, an infinite loop: the compiler proved nothing in the loop changes `flag`. `wait_volatile` re-loads `vflag` on every iteration. This is a legal optimisation, not a compiler bug; correct code needs `volatile` (or, for threads, `atomic`).

4.11 Standard-library essentials and pitfalls ★★

printf format cheat sheet (every specifier here is a classic MCQ)

```c
#include <stdio.h>
int main(void) {
    printf("[%5d] [%-5d] [%05d] [%+d] [%*d]\n", 42, 42, 42, 42, 6, 42);
    printf("[%x] [%X] [%#x] [%o] [%#o]\n", 255, 255, 255, 8, 8);
    printf("[%e] [%g] [%g] [%.3f] [%10.2f] [%-10.2f]\n", 1234.5678, 1234.5678, 0.00001234, 3.14159, 3.14159,
     3.14159);
    printf("[%c] [%s] [%.3s] [%10s] [%-10s] [%%]\n", 'x', "hello", "hello", "hi", "hi");
    printf("[%zu] [%lld] [%u] [%hhd] [%ld]\n", sizeof(int), 1LL << 40, 4000000000u, (signed char)200, -5L);
    int n = printf("abc\n");                       /* printf returns the number of characters written */
    printf("returned %d\n", n);
    return 0;
}
```

```
[   42] [42   ] [00042] [+42] [    42]
[ff] [FF] [0xff] [10] [010]
[1.234568e+03] [1234.57] [1.234e-05] [3.142] [      3.14] [3.14      ]
[x] [hello] [hel] [        hi] [hi        ] [%]
[4] [1099511627776] [4000000000] [-56] [-5]
abc
returned 4
```

scanf family and string-function pitfalls

```c
#include <stdio.h>
#include <string.h>
int main(void) {
    int a = 0, b = 0;
    int n = sscanf("12 abc", "%d %d", &a, &b);          /* returns the number of items converted */
    printf("sscanf returned %d, a=%d\n", n, a);
    char w[8];
    sscanf("hello world", "%7s", w);                    /* always give %s a width: no bound = overflow */
    printf("[%s]\n", w);

    char buf[4];
    strncpy(buf, "abcdef", sizeof buf);                 /* source too long: NOT null-terminated! */
    buf[sizeof buf - 1] = '\0';                         /* so terminate it yourself */
    printf("%s\n", buf);

    printf("%d %d %d\n", strcmp("a", "b") < 0, strcmp("b", "a") > 0, strcmp("a", "a"));
    char s[] = "a,b,,c"; char *t = strtok(s, ",");      /* strtok modifies its input, merges delimiters, is not re-
     entrant */
    while (t) { printf("%s|", t); t = strtok(NULL, ","); }
    printf("\n");

    char m[] = "abcdef";
    memmove(m + 2, m, 4);                               /* overlapping copy: only memmove is safe */
    printf("%s\n", m);
    return 0;
}
```

```
sscanf returned 1, a=12
[hello]
abc
1 1 0
a|b|c|
ababcd
```

Library traps examiners like

`gets` was removed in C11 (use `fgets`, which keeps the newline). `strcpy/strcat/sprintf` do not check bounds. `strlen` is $O(n)$, so a loop `for(i=0;i<strlen(s);i++)` is quadratic. `memset` sets *bytes* (`memset(a,1,n)` on ints writes 0x01010101). `rand()%n` is biased and not secure. `atoi` cannot report errors (use `strtol`). `printf("%s", NULL)` is undefined.

4.12 Reimplement the library: strlen, strcmp, memcpy, memmove, atoi ★★★

Reported in S25 (“memcpy / memmove from scratch”) and a staple of C interviews. The key insight for `memmove`: if the destination starts *after* the source and the ranges overlap, copy **backwards**.

Library functions from scratch, with tests

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <limits.h>
#include <assert.h>

size_t my_strlen(const char *s) { const char *p = s; while (*p) p++; return (size_t)(p - s); }
char *my_strcpy(char *d, const char *s) { char *r = d; while ((*d++ = *s++)) ; return r; }
int my_strcmp(const char *a, const char *b) {
    while (*a && *a == *b) { a++; b++; }
    return (unsigned char)*a - (unsigned char)*b;            /* compare as unsigned char, like the standard */
}
void *my_memcpy(void *d, const void *s, size_t n) {          /* regions must NOT overlap */
    unsigned char *dp = d; const unsigned char *sp = s;
    while (n--) *dp++ = *sp++;
    return d;
}
void *my_memmove(void *d, const void *s, size_t n) {
    unsigned char *dp = d; const unsigned char *sp = s;
    if (dp == sp || n == 0) return d;
    if (dp < sp) { while (n--) *dp++ = *sp++; }              /* forward copy is safe */
    else { dp += n; sp += n; while (n--) *--dp = *--sp; }    /* dest after src: copy backwards */
    return d;
}
int my_atoi(const char *s) {
    while (isspace((unsigned char)*s)) s++;
    int sign = 1;
    if (*s == '+' || *s == '-') { if (*s == '-') sign = -1; s++; }
    long long v = 0;
    while (isdigit((unsigned char)*s)) {
        v = v * 10 + (*s - '0');
        if (sign * v > INT_MAX) return INT_MAX;               /* clamp instead of overflowing */
        if (sign * v < INT_MIN) return INT_MIN;
        s++;
    }
    return (int)(sign * v);
}
int main(void) {
    char a[16];
    assert(my_strlen("hello") == 5 && my_strlen("") == 0);
    assert(strcmp(my_strcpy(a, "copy"), "copy") == 0);
    assert(my_strcmp("abc", "abd") < 0 && my_strcmp("b", "a") > 0 && my_strcmp("x", "x") == 0);
    char f[] = "abcdef"; my_memmove(f + 2, f, 4);  printf("memmove forward overlap : %s\n", f);  /* ababcd */
    char g[] = "abcdef"; my_memmove(g, g + 2, 4);  printf("memmove backward overlap: %s\n", g);  /* cdefef */
    printf("atoi: %d %d %d %d\n", my_atoi("  -42abc"), my_atoi("+7"), my_atoi("99999999999"), my_atoi("
     -99999999999"));
    char h[8]; my_memcpy(h, "xyz", 4); printf("memcpy: %s\n", h);
    puts("all asserts passed");
    return 0;
}
```

```
memmove forward overlap : ababcd
memmove backward overlap: cdefef
atoi: -42 7 2147483647 -2147483648
memcpy: xyz
all asserts passed
```

4.13 Find the bugs: the “bug-hunt” round ★★★

The S1 interview gave a candidate a 250-line C program and 5–10 minutes to find bugs. The program below is deliberately wrong in at least ten ways. **Try to list them before reading the answers.**

```c
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <string.h>

#define CAP 4
typedef struct { int data[CAP]; int top; } Stack;

void push(Stack *s, int v) { s->data[s->top++] = v; }
int  pop(Stack *s)         { return s->data[--s->top]; }

typedef struct Node { char *name; struct Node *next; } Node;

void add_name(Node *head, const char *name) {
    Node *n = malloc(sizeof(Node));
    n->name = malloc(strlen(name));
    strcpy(n->name, name);
    n->next = head;
    head = n;
}

int sum(const int *a, int n) {
    int s;
    for (int i = 0; i <= n; i++) s += a[i];
    return s;
}

int main(void) {
    Stack st = { .top = 0 };
    for (int i = 0; i < 6; i++) push(&st, i);
    const char *msg = "done";
    if (msg == "done") puts("equal");
    int *buf = malloc(10 * sizeof(int));
    free(buf);
    buf[0] = 1;
    printf("%d %d\n", sum(buf, 3), pop(&st));
    return 0;
}
```

Answer key (line numbers refer to the listing)

1. **L8 `push`:** no capacity check: buffer overflow when `top == CAP` (L29 pushes 6 items into 4 slots).
2. **L9 `pop`:** no empty check: reads `data[-1]` on an empty stack (underflow).
3. **L13–L19 `add_name`:** `head` is a copy; the assignment on L18 is lost (needs `Node **head`), so the new node leaks and the list never grows. Nothing ever frees `n` or `n->name`.
4. **L14–L15:** `malloc` results are never checked for NULL.
5. **L15–L16:** `malloc(strlen(name))` misses the `+1` for the terminating NUL; `strcpy` then writes one byte past the block.
6. **L22:** `s` is uninitialised (should be 0).
7. **L23:** off-by-one: `i <= n` reads `a[n]`.
8. **L31:** `msg == "done"` compares pointers, not contents (use `strcmp`); result is unspecified.
9. **L32–L34:** `malloc`'d memory is never initialised, and **L34 writes through a freed pointer** (use-after-free); L35 then passes the freed pointer to `sum`.
10. **L28–L29 / L35:** the caller trusts `push` blindly; `pop` is called after an overflow already corrupted `top` (it is stored right after `data`).

What a compiler and a sanitizer can find automatically (real output):

```
$ gcc -O2 -Wall -Wextra -c bug.c
```

```
line 8: warning: iteration 4 invokes undefined behavior [-Waggressive-loop-optimizations]
line 9: warning: array subscript 5 is above array bounds of 'int[4]' [-Warray-bounds=]
line 16: warning: '__builtin___strcpy_chk' writing one too many bytes into a region of a size that depends on '
     strlen' [-Wstringop-overflow=]
line 22: warning: 's' is used uninitialized [-Wuninitialized]
line 23: warning: 's' may be used uninitialized [-Wmaybe-uninitialized]
line 23: warning: pointer 'buf' may be used after 'free' [-Wuse-after-free]
line 31: warning: comparison with string literal results in unspecified behavior [-Waddress]
line 34: warning: pointer 'buf' used after 'free' [-Wuse-after-free]
line 35: warning: pointer 'buf' used after 'free' [-Wuse-after-free]
```

```
$ gcc -g -fsanitize=address,undefined bug.c -o bug && ./bug
```

```
bug.c:8:37: runtime error: index 4 out of bounds for type 'int [4]'
ERROR: AddressSanitizer: heap-use-after-free
SUMMARY: AddressSanitizer: heap-use-after-free bug.c:34 in main
```

Bug-hunt method

Read in this order: (1) **signatures** (call-by-value? missing `const`? wrong return type), (2) **every allocation** (size, NULL check, matching `free`, one owner), (3) **every loop bound** (`<` vs. `<=`, empty input, one element), (4) **every comparison** (`==` on strings/floats, signed vs. unsigned), (5) **uninitialised or stale values**, (6) **overflow** of counters and indices. Say the check out loud: interviewers score the method as much as the findings.

4.14 Output-prediction drills (22 questions) ★★★

Try each before looking at the captured output. All behaviours are well-defined unless stated.

Q4.2 What is printed?

```c
#include <stdio.h>
int main(void) { printf("%d\n", printf("Hi")); return 0; }
```

Answer: captured output

```
Hi2
```

Why: the inner `printf` prints `Hi` and returns the number of characters written (2), which the outer one prints.

Q4.3 What are `x`, `y`, `z`?

```c
#include <stdio.h>
int main(void) { int x = 5; int y = x++ + 10; int z = ++x + 10; printf("%d %d %d\n", x, y, z); return 0; }
```

Answer: captured output

```
7 15 17
```

Why: `x++` yields 5 then increments ($y = 15$, $x = 6$); `++x` increments first ($x = 7$, $z = 17$).

Q4.4 Precedence of `&` versus `==`.

```c
#include <stdio.h>
int main(void) { int a = 2, b = 3; printf("%d %d\n", a & b == 3, (a & b) == 2); return 0; }
```

Answer: captured output

```
0 1
```

Why: `==` binds tighter than `&`: `a & (b==3)` $= 2$ `&` $1 = 0$. With parentheses `(2&3)==2` is 1. (The precedence order bitwise-vs-equality is a C historical wart.)

Q4.5 Switch without `break`.

```c
#include <stdio.h>
int main(void) {
    switch (2) { case 1: printf("1"); case 2: printf("2"); case 3: printf("3"); break; default: printf("d"); }
    printf("\n"); return 0;
}
```

Answer: captured output

```
23
```

Why: execution enters at `case 2` and *falls through* until the `break`.

Q4.6 Comma operator and assignment.

```c
#include <stdio.h>
int main(void) { int x = (1, 2, 3); int y; y = 1, 2; printf("%d %d\n", x, y); return 0; }
```

Answer: captured output

```
3 1
```

Why: the comma operator yields its last operand (x = 3). In `y = 1, 2` the assignment has higher precedence than the comma, so y = 1 and the 2 is discarded.

Q4.7 Short-circuit evaluation.

```c
#include <stdio.h>
int main(void) {
    int i = 0, j = 0;
    if (i++ || i++) { }      /* first operand 0 -> evaluate the second */
    if (j++ && j++) { }      /* first operand 0 -> second NOT evaluated */
    printf("%d %d\n", i, j); return 0;
}
```

Answer: captured output

```
2 1
```

Why: `||` evaluates the right side only if the left is false; `&&` only if the left is true. There is a sequence point after the left operand, so this is well defined.

Q4.8 Floating-point equality.

```c
#include <stdio.h>
int main(void) {
    float f = 0.1f; double d = 0.1;
    printf("%d %d %d\n", f == d, 0.1 + 0.2 == 0.3, (float)0.1 == f);
    return 0;
}
```

Answer: captured output

```
0 0 1
```

Why: `0.1f` is the nearest *float* to 0.1, which differs from the nearest *double*, so `f == d` is false; `0.1+0.2` is not exactly `0.3` in binary; converting the double to float gives the same float, so the last is true.

Q4.9 Character arithmetic.

```c
#include <stdio.h>
int main(void) { char c = 'A'; c += 2; printf("%c %d %d\n", c, c, 'a' - 'A'); return 0; }
```

Answer: captured output

```
C 67 32
```

Why: chars are small integers (ASCII 'A' = 65). The difference between lower and upper case letters is 32 (a single bit, 0x20).

Q4.10 Partial array initialisation and `sizeof`.

```c
#include <stdio.h>
int main(void) { int a[5] = {1, 2}; int s = 0; for (int i = 0; i < 5; i++) s += a[i]; printf("%d %zu\n", s, sizeof
      a); return 0; }
```

Answer: captured output

```
3 20
```

Why: the remaining elements are zero-initialised; `sizeof a` is 5 × 4 bytes.

Q4.11 Strings, `sizeof`, `strlen`.

```c
#include <stdio.h>
#include <string.h>
int main(void) {
    char s[] = "hello"; char *p = s; p[0] = 'J';
    char t[] = "ab\0cd";
    printf("%s %zu %zu %zu | %zu %zu\n", s, sizeof s, sizeof p, strlen(s), strlen(t), sizeof t);
    return 0;
}
```

Answer: captured output

```
Jello 6 8 5 | 2 6
```

Why: `sizeof s` counts the NUL (6); `sizeof p` is a pointer (8); `strlen` stops at the first NUL, so `"ab\0cd"` has length 2 but occupies 6 bytes.

Q4.12 Call-by-value.

```c
#include <stdio.h>
void inc(int x) { x++; }
void inc2(int *x) { (*x)++; }
int main(void) { int a = 1; inc(a); inc2(&a); printf("%d\n", a); return 0; }
```

Answer: captured output

```
2
```

Why: `inc` changes a copy; `inc2` changes the original.

Q4.13 Recursion order.

```c
#include <stdio.h>
void f(int n) { if (n == 0) return; printf("%d ", n); f(n - 1); printf("%d ", n); }
int main(void) { f(3); printf("\n"); return 0; }
```

Answer: captured output

```
3 2 1 1 2 3
```

Why: the first `printf` runs on the way down (3 2 1), the second on the way back up (1 2 3). A frame per call, so depth $= n$.

Q4.14 Struct padding.

```c
#include <stdio.h>
struct S { char a; int b; char c; };
int main(void) { printf("%zu\n", sizeof(struct S)); return 0; }
```

Answer: captured output

```
12
```

Why: `a` at 0, 3 bytes padding, `b` at 4, `c` at 8, 3 bytes tail padding so the size is a multiple of 4. Reordering to `int b; char a, c;` gives 8.

Q4.15 Reading an `int` array through a `char*`.

```c
#include <stdio.h>
int main(void) { int a[4] = {1, 2, 3, 4}; char *c = (char *)a; printf("%d %d\n", *(c + 4), *(int *)(c + 8));
    return 0; }
```

Answer: captured output

```
2 3
```

Why: on a little-endian machine the first byte of `a[1]` is 2; `c+8` is the start of `a[2]`. (Character-type access is allowed by the aliasing rules.)

Q4.16 Static storage duration is zero-initialised.

```c
#include <stdio.h>
static int s;
int main(void) { static int t; printf("%d %d\n", s, t); return 0; }
```

Answer: captured output

```
0 0
```

Why: objects with static storage duration are zeroed before the program starts, unlike automatic variables.

Q4.17 Nested conditional operator.

```
#include <stdio.h>
int main(void) { int a = 5; printf("%d\n", a > 3 ? a < 10 ? 1 : 2 : 3); return 0; }
```

Answer: captured output

```
1
```

Why: `?:` is right-associative: `a>3 ? (a<10 ? 1 : 2) : 3`.

Q4.18 Unsigned wrap-around.

```
#include <stdio.h>
int main(void) { unsigned int u = 0; u--; printf("%u %d\n", u, u == 4294967295u); return 0; }
```

Answer: captured output

```
4294967295 1
```

Why: unsigned arithmetic is defined modulo 2^{32} (unlike signed overflow, which is undefined).

Q4.19 Integer vs. floating division and casts.

```
#include <stdio.h>
int main(void) { printf("%d %f %d\n", 5 / 2, 5 / 2.0, (int)2.9 + (int)-2.9); return 0; }
```

Answer: captured output

```
2 2.500000 0
```

Why: integer division truncates; `(int)` truncates toward zero, so $2 + (-2) = 0$.

Q4.20 Octal and hexadecimal literals.

```
#include <stdio.h>
int main(void) { printf("%d %d %d\n", 010, 0x1F, 010 + 0x10); return 0; }
```

Answer: captured output

```
8 31 24
```

Why: a leading 0 means octal ($010 = 8$); `0x1F` $= 31$.

Q4.21 Swapping pointers.

```
#include <stdio.h>
void swap_ptr(int **a, int **b) { int *t = *a; *a = *b; *b = t; }
int main(void) { int x = 1, y = 2; int *p = &x, *q = &y; swap_ptr(&p, &q); printf("%d %d %d %d\n", *p, *q, x, y);
    return 0; }
```

Answer: captured output

```
2 1 1 2
```

Why: only the pointers were swapped; `x` and `y` keep their values.

Q4.22 `sizeof` does not evaluate its operand.

```
#include <stdio.h>
int main(void) { int i = 5; printf("%zu %d\n", sizeof(i++), i); return 0; }
```

Answer: captured output

```
4 5
```

Why: `sizeof` is resolved at compile time from the type; `i++` is never executed.

Q4.23 A stray semicolon.

```
#include <stdio.h>
int main(void) { int i; for (i = 0; i < 3; i++); printf("%d\n", i); return 0; }
```

Answer: captured output

```
3
```

Why: the `;` is the loop body (empty); `printf` runs once after the loop, with `i == 3`.

Q4.24 Signed vs. unsigned comparison inside a loop (the classic hang).

```c
#include <stdio.h>
int main(void) {
    unsigned n = 3, count = 0;
    for (int i = -1; i < n; i++) count++;     /* i is converted to unsigned for the comparison */
    printf("%u\n", count);
    return 0;
}
```

Answer: captured output

```
0
```

Why: `-1` converts to 4294967295 which is *not* less than 3, so the loop body never runs. The usual symptom of mixing `int` and `size_t`.

5. C++: Object Model, Move Semantics and Resource Management

In this chapter

Priority ★★★. Time: 90 min across both C++ chapters. Reported C++ topics: constructors/destructors and memory management (S5), virtual functions and abstract classes (S23), copy constructor and operator overloading, deep vs. shallow copy, RAII (S24), templates and inline functions (S22). As in the C chapter, every output below is real (g++ 13, `-std=c++20`, `-O0`).

5.1 What C++ adds to C (one-minute map) ★★

Feature	Key point
Classes, access control	`struct` = default *public*; `class` = default *private*; otherwise identical
References `T&`	Alias, must be bound at initialisation, cannot be reseated, cannot be null
`new`/`delete`	`new` = allocate + *construct*; `delete` = *destruct* + free; pair `new[]` with `delete[]`
Overloading, default args	Overloads resolved at compile time; **default arguments are bound statically**
Templates	Compile-time generics; instantiated per type; the basis of the STL
RAII	Resources are owned by objects; destructors release them, even during exception unwinding
`nullptr`, `enum class`, `auto`, `constexpr`	Modern replacements for `NULL`, plain `enum`, verbose types, and macros

5.2 Object lifetime: which constructors and destructors run? ★★★

S5 reports: *"Here is a short C++ class; which constructors and destructors run at each step, and what memory-management problems exist?"* Practise on this tracer class.

```cpp
#include <iostream>
#include <string>
#include <utility>
struct T {
    std::string n;
    T(std::string s) : n(std::move(s)) { std::cout << "ctor " << n << "\n"; }
    T(const T& o) : n(o.n + "'") { std::cout << "copy " << o.n << " -> " << n << "\n"; }
    T(T&& o) noexcept : n(o.n + "&&") { std::cout << "move " << o.n << " -> " << n << "\n"; }
    ~T() { std::cout << "dtor " << n << "\n"; }
};
T make()  { T t("local"); return t; }        // named return value: NRVO may apply
T make2() { return T("temp"); }              // prvalue: guaranteed elision since C++17
void by_value(T x) { std::cout << "in by_value(" << x.n << ")\n"; }
int main() {
    std::cout << "--- A\n"; T a("a");
    std::cout << "--- B\n"; T b = a;               // copy
    std::cout << "--- C\n"; T c = std::move(a);    // move (a is left valid but unspecified)
    std::cout << "--- D\n"; by_value(b);           // parameter is copy-constructed from b
    std::cout << "--- E\n"; T d = make();
    std::cout << "--- F\n"; T e = make2();
    std::cout << "--- end\n";
}
```

```
$ g++ -std=c++20 life.cpp && ./a.out
```

```
--- A
ctor a
--- B
copy a -> a'
--- C
```

```
move a -> a&&
--- D
copy a' -> a''
in by_value(a'')
dtor a''
--- E
ctor local
--- F
ctor temp
--- end
dtor temp
dtor local
dtor a&&
dtor a'
dtor a
```

Reading the trace

1. **A** constructs `a`. **B** calls the *copy* constructor (name becomes `a'`). **C** calls the *move* constructor (`a&&`).
2. **D**: pass-by-value *copy-constructs the parameter* (`a''`); it is destroyed when `by_value` returns. Pass by `const T&` to avoid the copy.
3. **E, F:** *no* copy or move is printed. `make2` returns a prvalue: since C++17 the object is constructed directly in the caller (*guaranteed copy elision*). `make` returns a named local: compilers *may* elide (NRVO; GCC does even at `-O0`), but the standard does not require it.
4. **End of main:** locals are destroyed in **reverse order of construction**: `e (temp), d (local), c (a&&), b (a'), a`.

For contrast, forbid elision (allowed to be disabled for returns of named locals; in C++14 mode it also shows the conceptual moves of the prvalue):

```
$ g++ -std=c++14 -fno-elide-constructors life.cpp && ./a.out   # sections E and F only
```

```
--- E
ctor local
move local -> local&&
dtor local
move local&& -> local&&&&
dtor local&&
--- F
ctor temp
move temp -> temp&&
dtor temp
move temp&& -> temp&&&&
dtor temp&&
--- end
```

5.2.1 Construction and destruction order with inheritance and members

The order rules (memorise)

1. Construction: **base classes** (in declaration order; virtual bases first), then **data members in the order they are declared in the class** (*not* the order in the constructor's initialiser list), then the constructor body.
2. Destruction: exactly the reverse: destructor body, members in reverse declaration order, then base classes in reverse.
3. While a base-class constructor runs, the object's dynamic type is *the base*: **a virtual call inside a constructor or destructor does not reach the derived override.**
4. Deleting a derived object through a base pointer is undefined behaviour unless the base destructor is `virtual`.

Order of construction/destruction; virtual call in a constructor

```
#include <iostream>
struct Base {
    Base()  { std::cout << "Base() "; init(); }               // virtual call during construction
    virtual void init() { std::cout << "Base::init "; }
    virtual ~Base() { std::cout << "~Base "; }
};
```

```
struct Member {
    const char* n;
    Member(const char* n) : n(n) { std::cout << "Member(" << n << ") "; }
    ~Member() { std::cout << "~Member(" << n << ") "; }
};
struct Derived : Base {
    Member m1{"m1"}; Member m2{"m2"};
    Derived() { std::cout << "Derived() "; }
    void init() override { std::cout << "Derived::init "; }
    ~Derived() override { std::cout << "~Derived "; }
};
struct P { Member a, b; P() : b("b"), a("a") {} };      // initialiser list order is IGNORED: a then b
int main() {
    { Derived d; }              std::cout << "\n";
    Base* p = new Derived; delete p;  std::cout << "\n";   // OK because ~Base is virtual
    { P p2; }                    std::cout << "\n";
}
```

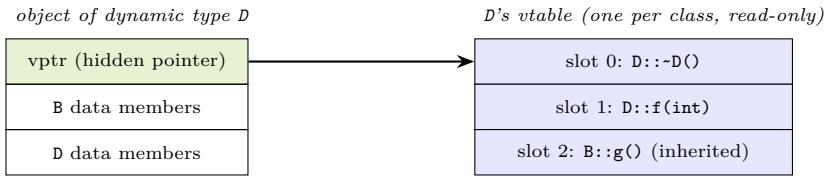
Answer: captured output

```
Base() Base::init Member(m1) Member(m2) Derived() ~Derived ~Member(m2) ~Member(m1) ~Base
Base() Base::init Member(m1) Member(m2) Derived() ~Derived ~Member(m2) ~Member(m1) ~Base
Member(a) Member(b) ~Member(b) ~Member(a)
```

What goes wrong without `virtual ~Base()`

`delete p` (with `p` of type `Base*` pointing to a `Derived`) would call only `Base::~Base`: the derived part and its members (`m1`, `m2`) are never destroyed (leak) and the behaviour is formally undefined. **Rule: any class meant to be a polymorphic base gets a virtual destructor** (or a protected non-virtual one).

5.3 Polymorphism: virtual functions and the vtable ★★★



`p->f()`: load `p->vptr`, load slot 1, **indirect call**. Cost: one extra load + an unpredictable-branch risk; cannot inline unless devirtualised.

Must know

- A class with at least one virtual function gets a hidden **vptr** in every object (8 bytes on x86-64) and one **vtable** per class. `sizeof` grows accordingly.
- `override` makes the compiler check that you really override; `final` forbids further overriding. A *pure virtual* function (`= 0`) makes the class **abstract** (cannot be instantiated); it may still have a body.
- Overloading is resolved at compile time using the *static* type; overriding is resolved at run time using the *dynamic* type; **default arguments use the static type**.
- Object slicing: copying a `Derived` into a `Base` *object* (not reference/pointer) discards the derived part and the dynamic type.
- Multiple inheritance can duplicate a shared base (the "diamond"); `virtual` inheritance gives one shared sub-object.

Static default args, slicing, empty-base optimisation, vptr size

```
#include <iostream>
struct B { virtual void f(int x = 1) { std::cout << "B::f " << x << "\n"; } };
struct D : B { void f(int x = 2) override { std::cout << "D::f " << x << "\n"; } };
struct E1 {};                          // empty class
struct E2 { virtual void v() {} };     // polymorphic
struct E3 : E1 { int i; };             // empty base optimisation
int main() {
    B* p = new D;  p->f();             // dynamic type picks D::f, static type supplies default 1
    D d;  B& r = d;  r.f();            // same
    B b = D();     b.f();              // slicing: b is a plain B
    std::cout << sizeof(E1) << " " << sizeof(E2) << " " << sizeof(E3) << "\n";
```

```
    delete p;
}
```

Answer: captured output

```
D::f 1
D::f 1
B::f 1
1 8 4
```

Why: the first two print `D::f 1` (virtual dispatch to `D::f`, but the default argument comes from `B`'s declaration); slicing gives `B::f 1`. An empty class occupies 1 byte (so distinct objects have distinct addresses), a vptr makes it 8, and an empty *base* costs nothing.

5.3.1 Overload resolution (a favourite C++ MCQ)

Order of preference: **exact match** > **promotion** (`char` or `short` to `int`; `float` to `double`) > **standard conversion** (`int` to `double`; `int` to `char`) > user-defined conversion > ellipsis. Two equally good candidates ⇒ *ambiguous* (a compile error).

Which overload is called?

```
#include <iostream>
void f(int)         { std::cout << "int, "; }
void f(long)        { std::cout << "long, "; }
void f(double)      { std::cout << "double, "; }
void f(const char*) { std::cout << "const char*, "; }
void g(char) { std::cout << "g(char), "; }
void g(int)  { std::cout << "g(int), "; }
int main() {
    f(1); f(1L); f(1.0f); f(nullptr); f('a'); f(0);
    g('a'); g(97);
    std::cout << "\n";
}
```

Answer: captured output

```
int, long, double, const char*, int, int, g(char), g(int),
```

Why: `1.0f` is *promoted* to `double` (better than converting to `int`/`long`); `'a'` is promoted to `int`; `nullptr` can only become `const char*`. Now a call that *looks* fine but is ambiguous:

g(3.5): double converts equally well to char and to int

```
void g(char) {}
void g(int)  {}
int main() { g(3.5); }
```

```
prog.cpp: In function 'int main()':
prog.cpp:3:15: error: call of overloaded 'g(double)' is ambiguous
    3 | int main() { g(3.5); }
      |              ~^~~~
prog.cpp:1:6: note: candidate: 'void g(char)'
... (5 more lines)
```

5.4 Rule of 0/3/5, copy vs. move, and why `noexcept` matters ★★★

The rules

- **Rule of 3:** if a class needs a custom destructor, copy constructor or copy assignment (it owns a resource), it almost certainly needs all three.
- **Rule of 5:** add the **move constructor** and **move assignment**. **Rule of 0:** prefer members that already manage themselves (`std::vector`, `std::string`, `std::unique_ptr`) so you write none of the five.
- **Deep copy** duplicates the owned resource; the compiler-generated (*shallow*) copy copies the pointer, causing a double `delete`.
- `std::move(x)` is just a cast to `T&&`; it moves nothing by itself. The move constructor does the stealing and must leave the source valid.
- Containers like `std::vector` move elements on reallocation *only if the move constructor is* ***`noexcept`***

(otherwise they copy, to keep the strong exception guarantee).

A resource-owning class with copy-and-swap assignment

```
#include <iostream>
#include <algorithm>
#include <utility>
class Buffer {
    size_t n_; int* p_;
public:
    explicit Buffer(size_t n) : n_(n), p_(n ? new int[n]() : nullptr) { std::cout << "ctor(" << n_ << ") "; }
    Buffer(const Buffer& o) : n_(o.n_), p_(o.n_ ? new int[o.n_] : nullptr) {      // deep copy
        std::copy(o.p_, o.p_ + n_, p_); std::cout << "copy "; }
    Buffer(Buffer&& o) noexcept : n_(o.n_), p_(o.p_) { o.n_ = 0; o.p_ = nullptr; std::cout << "move "; }
    Buffer& operator=(Buffer o) noexcept { swap(o); std::cout << "assign "; return *this; }   // by value: handles
      copy AND move
    ~Buffer() { delete[] p_; std::cout << "dtor "; }
    void swap(Buffer& o) noexcept { std::swap(n_, o.n_); std::swap(p_, o.p_); }
    size_t size() const { return n_; }
};
int main() {
    Buffer a(3);            // ctor(3)
    Buffer b = a;           // copy
    Buffer c = std::move(a);// move; a is now empty
    b = c;                  // copy into parameter, swap, parameter dies
    c = Buffer(5);          // temporary, swap, old c data dies
    std::cout << "\nsizes: " << a.size() << b.size() << c.size() << "\n";
}   // c, b, a destroyed
```

Answer: captured output

```
ctor(3) copy move copy assign dtor ctor(5) assign dtor
sizes: 035
dtor dtor dtor
```

Q5.1 Why does this vector copy its elements on growth although a move constructor exists?

vector reallocation: move vs copy (and capacity growth)

```
#include <iostream>
#include <vector>
struct X1 { X1(){} X1(const X1&){ std::cout<<"copy "; } X1(X1&&)          { std::cout<<"move "; } };
struct X2 { X2(){} X2(const X2&){ std::cout<<"copy "; } X2(X2&&) noexcept { std::cout<<"move "; } };
template<class T> void run(const char* name) {
    std::cout << name << ": ";
    std::vector<T> v;
    for (int i = 0; i < 4; i++) { v.push_back(T()); std::cout << "| "; }
    std::cout << "\n";
}
int main() {
    run<X1>("move not noexcept");
    run<X2>("move noexcept    ");
    std::vector<int> v; size_t last = 0;
    for (int i = 0; i < 20; i++) { v.push_back(i); if (v.capacity() != last) { last = v.capacity(); std::cout <<
     last << " "; } }
    std::cout << "(capacity growth in libstdc++: doubles)\n";
}
```

Answer: captured output

```
move not noexcept: move | move copy | move copy copy | move |
move noexcept    : move | move move | move move move | move |
1 2 4 8 16 32 (capacity growth in libstdc++: doubles)
```

Why: `push_back` may reallocate (copy/move all old elements). To keep the *strong* exception guarantee it uses `std::move_if_noexcept`: if the move constructor can throw, it copies. Mark move operations `noexcept`. (Capacity growth is implementation-defined: libstdc++ and libc++ double; MSVC grows by 1.5×.)

5.5 RAII and smart pointers ★★★

RAII = Resource Acquisition Is Initialisation

Acquire the resource in a constructor; release it in the destructor. Because destructors of fully constructed locals run on *every* exit path (return, `break`, exception), leaks become structurally impossible. Examples: `std::unique_ptr`, `std::shared_ptr`, `std::lock_guard`, `std::fstream`, `std::vector`.

Type	Semantics
`unique_ptr<T>`	Sole ownership, move-only, zero overhead over a raw pointer; custom deleters supported; first choice
`shared_ptr<T>`	Shared ownership via a **control block** (strong count, weak count, deleter); atomic reference counts; `make_shared` allocates object+block together
`weak_ptr<T>`	Non-owning observer of a `shared_ptr`; `lock()` gives a `shared_ptr` or empty; **breaks ownership cycles**
raw pointer / reference	Non-owning view; never `delete` through it

shared_ptr cycle leaks; weak_ptr back-reference fixes it

```
#include <iostream>
#include <memory>
struct N {
    std::shared_ptr<N> next;   // owning
    std::weak_ptr<N>   back;   // non-owning
    const char* n;
    N(const char* n) : n(n) {}
    ~N() { std::cout << "~" << n << " "; }
};
int main() {
    {   auto a = std::make_shared<N>("a1"), b = std::make_shared<N>("b1");
        a->next = b; b->next = a;                          // cycle: counts never reach 0
        std::cout << "cycle: use_count(a)=" << a.use_count() << "\n"; }
    std::cout << "[after cycle scope: no destructor ran -> memory leak]\n";
    {   auto a = std::make_shared<N>("a2"), b = std::make_shared<N>("b2");
        a->next = b; b->back = a;                          // weak back edge
        std::cout << "weak: use_count(a)=" << a.use_count() << " use_count(b)=" << b.use_count() << "\n"; }
    std::cout << "[after weak scope]\n";
    auto u = std::make_unique<int>(7);
    auto v = std::move(u);                                  // unique_ptr is move-only
    std::cout << (u == nullptr) << " " << *v << "\n";
}
```

Answer: captured output

```
cycle: use_count(a)=2
[after cycle scope: no destructor ran -> memory leak]
weak: use_count(a)=1 use_count(b)=2
~a2 ~b2 [after weak scope]
1 7
```

Copying a unique_ptr does not compile (by design)

```
#include <memory>
int main() { std::unique_ptr<int> a(new int(1)); std::unique_ptr<int> b = a; }
```

```
prog.cpp: In function 'int main()':
prog.cpp:2:75: error: use of deleted function 'std::unique_ptr<_Tp, _Dp>::unique_ptr(const std::unique_ptr<_Tp,
     _Dp>&) [with _Tp = int; _Dp = std::default_delete<int>]'
    2 | int main() { std::unique_ptr<int> a(new int(1)); std::unique_ptr<int> b = a; }
      |                                                                           ^
... (5 more lines)
```

Smart-pointer pitfalls

- Never build two `shared_ptr`s from the same raw pointer (two control blocks ⇒ double free); copy the `shared_ptr` or use `enable_shared_from_this`.
- `new[]` must be matched with `delete[]` (use `unique_ptr<T[]>` or `std::vector`); mixing `malloc/delete` or `new/free` is undefined.
- Prefer `make_unique`/`make_shared`: single allocation for the latter, and exception-safe argument evaluation.
- Passing `const shared_ptr<T>&` costs nothing; passing by value does an atomic increment.

5.6 Operator overloading and a "write a class" answer ★★★

S24 reports: *"Write a C++ class with operator overloading and a copy constructor."* A model answer touches: constructors, rule of 3/5 if it owns memory, `const`-correctness, member vs. non-member operators, and returning the right type from each operator.

Vec2: value type with overloaded operators

```
#include <iostream>
#include <stdexcept>
class Vec2 {
    double x_, y_;
public:
    Vec2(double x = 0, double y = 0) : x_(x), y_(y) {}             // default + converting ctor
    Vec2(const Vec2&) = default;                                     // rule of 0: members are trivially copyable
    double operator[](int i) const { if (i < 0 || i > 1) throw std::out_of_range("Vec2"); return i ? y_ : x_; }
    double& operator[](int i)      { if (i < 0 || i > 1) throw std::out_of_range("Vec2"); return i ? y_ : x_; }
    Vec2& operator+=(const Vec2& o) { x_ += o.x_; y_ += o.y_; return *this; }   // member: modifies *this
    Vec2& operator++()    { ++x_; ++y_; return *this; }              // prefix: returns the NEW value by reference
    Vec2  operator++(int) { Vec2 old = *this; ++*this; return old; } // postfix: dummy int, returns the OLD value
     by value
    friend Vec2 operator+(Vec2 a, const Vec2& b) { return a += b; }  // non-member: symmetric conversions
    friend bool operator==(const Vec2& a, const Vec2& b) { return a.x_ == b.x_ && a.y_ == b.y_; }
    friend std::ostream& operator<<(std::ostream& os, const Vec2& v) { return os << "(" << v.x_ << "," << v.y_ <<
     ")"; }
};
int main() {
    Vec2 a(1, 2), b(3, 4);
    Vec2 c = a + b;                       // copy-initialisation from the returned temporary
    Vec2 d(c);                            // copy constructor
    std::cout << c << " " << (c == d) << " " << c[1] << "\n";
    std::cout << c++ << " " << c << " " << ++c << "\n";
    try { c[2]; } catch (const std::out_of_range& e) { std::cout << "caught " << e.what() << "\n"; }
}
```

Answer: captured output

```
(4,6) 1 6
(4,6) (5,7) (6,8)
caught Vec2
```

Operator-overloading rules

- Cannot be overloaded: `::`, `.`, `.*`, `?:`, `sizeof`. Cannot invent new operators. At least one operand must be a user-defined type.
- `=`, `[]`, `()`, `->` must be *member* functions; `<<`/`>>` for streams must be non-members (the left operand is the stream).
- Prefix `++` returns `T&`; postfix `++(int)` returns `T` by value (and is slower for heavy types: prefer `++it`).
- Keep the usual semantics: `+` returns a new value, `+=` modifies and returns `*this`; if you define `==`, define `!=` (automatic in C++20) and keep them consistent.

5.7 Diamond inheritance and virtual bases ★★

Two copies of A vs. one shared A

```
#include <iostream>
struct A  { A()  { std::cout << "A "; } };
struct B1 : A { B1() { std::cout << "B "; } };
struct C1 : A { C1() { std::cout << "C "; } };
struct D1 : B1, C1 { D1() { std::cout << "D | "; } };      // contains TWO A sub-objects
struct VB : virtual A { VB() { std::cout << "B "; } };
struct VC : virtual A { VC() { std::cout << "C "; } };
struct VD : VB, VC { VD() { std::cout << "D"; } };         // ONE shared A, built first by the most-derived class
int main() { D1 d1; VD vd; std::cout << "\n"; std::cout << sizeof(D1) << " " << sizeof(VD) << "\n"; }
```

Answer: captured output

```
A B A C D | A B C D
2 16
```

Why: without `virtual`, `D1` has two `A` bases (ambiguous member access, double construction); with virtual inheritance the *most derived* class constructs the single `A` first. Virtual bases cost extra pointers (hence the bigger `VD`).

5.8 Templates ★★★

Must know

- Templates are **compile-time generics**: the compiler instantiates a separate function/class for each distinct set of template arguments. Consequences: header-only definitions, longer compile times, possible code bloat, and errors reported at instantiation.
- **Specialisation:** *full* (`template<>` followed by the specialised declaration) for one exact type; *partial* (classes only) for a family (e.g. `T*`). Function templates cannot be partially specialised: use overloading.
- **Deduction** takes the argument types: `max(1, 2.5)` fails to deduce one `T`; write `max<double>(1, 2.5)`.
- **SFINAE** ("substitution failure is not an error"), `if constexpr` (C++17) and **concepts** (C++20) select code by type properties. `typename` marks a dependent type name.
- **Variadic templates + fold expressions** (C++17): `(args + ...)`. **CRTP**: a class derives from `Base<Derived>` to get static (zero-cost) polymorphism.

Specialisation, variadic/fold, if constexpr, SFINAE, constexpr

```
#include <iostream>
#include <string>
#include <type_traits>
template <typename T> T maxof(T a, T b) { return a > b ? a : b; }
template <> const char* maxof<const char*>(const char* a, const char* b) {      // full specialisation
    return std::string(a) > std::string(b) ? a : b; }                            // (a > b on raw pointers would compare ADDRESSES)
template <typename T> struct Describe { static const char* name() { return "other"; } };
template <typename T> struct Describe<T*> { static const char* name() { return "pointer"; } };   // partial
template <> struct Describe<int> { static const char* name() { return "int"; } };                // full
template <typename... Ts> auto fold_sum(Ts... ts) { return (ts + ...); }                          // C++17 fold
template <typename T> const char* kind(T) {
    if constexpr (std::is_integral_v<T>) return "integral";          // discarded branch is not instantiated
    else if constexpr (std::is_floating_point_v<T>) return "floating";
    else return "other";
}
template <typename T> std::enable_if_t<std::is_integral_v<T>, T> half(T v) { return v / 2; }    // SFINAE: only integers
template <int N> struct Fact { static constexpr long long value = N * Fact<N - 1>::value; };   // template metaprogramming
template <> struct Fact<0> { static constexpr long long value = 1; };
constexpr long long fact(int n) { return n <= 1 ? 1 : n * fact(n - 1); }                        // constexpr function
static_assert(fact(5) == 120 && Fact<5>::value == 120, "evaluated at compile time");
template <class D> struct Shape { double area() const { return static_cast<const D*>(this)->area_impl(); } };   // CRTP
struct Sq : Shape<Sq> { double s; explicit Sq(double s) : s(s) {} double area_impl() const { return s * s; } };
int main() {
    std::cout << maxof(3, 7) << " " << maxof(2.5, 1.5) << " " << maxof<const char*>("pear", "apple") << "\n";
    std::cout << Describe<double>::name() << " " << Describe<char*>::name() << " " << Describe<int>::name() << "\n";
    std::cout << fold_sum(1, 2, 3, 4) << " " << fold_sum(1.5, 2.5) << " " << kind(7) << " " << kind(2.0) << " " << kind("x") << "\n";
```

```
    std::cout << half(9) << " " << Fact<10>::value << " " << Sq(3).area() << "\n";
}
```

Answer: captured output

```
7 2.5 pear
other pointer int
10 4 integral floating other
4 3628800 9
```

max(1, 2.5): conflicting deduction

```
template <typename T> T maxof(T a, T b) { return a > b ? a : b; }
int main() { return (int)maxof(1, 2.5); }
```

```
prog.cpp: In function 'int main()':
prog.cpp:2:31: error: no matching function for call to 'maxof(int, double)'
    2 | int main() { return (int)maxof(1, 2.5); }
      |                          ~~~~~^~~~~~~~
... (7 more lines)
```

Templates vs. macros vs. `inline` (S22 asked about templates and inline functions)

Macro: text substitution, no types, double evaluation. **`inline` function:** ordinary typed function whose body the compiler *may* inline; `inline` also relaxes the one-definition rule so the definition can live in a header. **Template:** a family of functions/classes generated per type. Inlining is a compiler decision; the keyword is only a hint.

5.9 The STL: containers, complexities, iterator invalidation ★★★

Container	Access / find	Insert / erase (and notes)
`vector`	$O(1)$ index; $O(n)$ find	push_back amortised $O(1)$; middle $O(n)$; contiguous (cache-friendly)
`deque`	$O(1)$ index	push/pop at both ends $O(1)$; middle $O(n)$; not contiguous
`list` / `forward_list`	$O(n)$	$O(1)$ at a known iterator; node-based, poor locality; splice $O(1)$
`array<T,N>`	$O(1)$	fixed size, no heap
`set` / `map` (+multi)	$O(\log n)$	$O(\log n)$; red-black tree; sorted iteration; `map[]` **inserts** a default value
`unordered_set/map`	average $O(1)$, worst $O(n)$	hash table; rehash invalidates iterators; needs `hash` and `==`
`priority_queue`	top $O(1)$	push/pop $O(\log n)$; max-heap by default (`greater<>` gives min-heap)
`stack`, `queue`		adaptors over `deque`/`vector`
`bitset<N>`	$O(1)$ bit access	fixed size; fast bulk bit ops

Algorithm	Complexity / contract
`sort`	$O(n \log n)$ (introsort); needs random access and a **strict weak ordering** (`<`, never `<=`: that is UB and can crash)
`stable_sort`, `partial_sort`, `nth_element`	$O(n \log n)$ stable; $O(n \log k)$; $O(n)$ average
`lower_bound`, `upper_bound`, `binary_search`	$O(\log n)$ on a *sorted* range; first $\geq$ / first $>$ the key
`unique`, `remove`, `remove_if`	**do not shrink** the container: they return the new logical end; use the *erase–remove idiom*
`accumulate`, `iota`, `next_permutation`, `reverse`, `rotate`	$O(n)$; `next_permutation` yields all $n!$ orders if you start sorted

Iterator invalidation (classic MCQ)

vector: `push_back` invalidates *all* iterators/references if it reallocates (capacity exceeded), otherwise only `end()`; `insert/erase` invalidate everything at/after the position. **deque:** insert in the middle invalidates all; at the ends it invalidates iterators but not references. **list/set/map:** only iterators to erased elements. **unordered containers:** a rehash invalidates all iterators (references/pointers to elements stay valid). Erasing while looping: `it = c.erase(it)` (it returns the next iterator), never `c.erase(it); ++it`.

STL idioms: erase-remove, bounds, map[], heap, string search

```cpp
#include <algorithm>
#include <functional>
#include <iostream>
#include <map>
#include <numeric>
#include <queue>
#include <string>
#include <vector>
using namespace std;
int main() {
    vector<int> v{5, 2, 8, 2, 9, 1, 5};
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());                    // sort + unique + erase = distinct values
    for (int x : v) cout << x << ' ';  cout << "| ";
    cout << *lower_bound(v.begin(), v.end(), 5) << " " << *upper_bound(v.begin(), v.end(), 5) << " | ";
    v.erase(remove_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; }), v.end());   // erase-remove idiom
    for (int x : v) cout << x << ' ';  cout << "\n";
    map<string, int> m;  m["a"]++;  m["b"] += 2;
    cout << m.size() << " " << m["zzz"] << " " << m.size() << " " << m.count("q") << " " << m.size() << "\n";   //
      operator[] inserts
    priority_queue<int, vector<int>, greater<int>> pq;               // min-heap
    for (int x : {5, 1, 4, 2, 3}) pq.push(x);
    while (!pq.empty()) { cout << pq.top() << ' '; pq.pop(); }
    cout << "| ";
    vector<int> w(5); iota(w.begin(), w.end(), 1);                   // 1 2 3 4 5
    cout << accumulate(w.begin(), w.end(), 0) << " " << accumulate(w.begin(), w.end(), 1, multiplies<int>()) << "
     | ";
    string s = "compiler verification";
    cout << s.find("ver") << " " << (s.find("xyz") == string::npos) << " " << s.substr(0, 8) << "\n";
    vector<int> p{1, 2, 3}; int cnt = 0;
    do { cnt++; } while (next_permutation(p.begin(), p.end()));
    cout << cnt << " permutations\n";
}
```

Answer: captured output

```
1 2 5 8 9 | 5 8 | 1 5 9
2 0 3 0 3
1 2 3 4 5 | 15 120 | 9 1 compiler
6 permutations
```

Why (map line): `m.size()` is 2; `m["zzz"]` silently inserts a zero entry (size 3); `count` does not insert. Use `find` or `at` (throws) to look up without inserting.

5.10 Lambdas, auto, casts and const-correctness ★★

Lambda captures; auto deduction

```cpp
#include <iostream>
#include <type_traits>
#include <vector>
#include <algorithm>
int main() {
    int x = 10;
    auto by_val = [x]() mutable { x++; return x; };     // copy captured at creation; mutable lets us modify the
     copy
    auto by_ref = [&x]() { x++; return x; };            // reference: sees and changes the original
    std::cout << by_val() << " " << by_val() << " " << x << " " << by_ref() << " " << x << "\n";
    const int& r = x;
    auto a = r;                                          // auto drops const and reference -> int
    auto& b = r;                                         // keeps const -> const int&
    std::cout << std::is_same_v<decltype(a), int> << std::is_same_v<decltype(b), const int&> << "\n";
```

```
    auto l = {1, 2, 3};                                  // std::initializer_list<int>
    std::cout << std::is_same_v<decltype(l), std::initializer_list<int>> << "\n";
    std::vector<std::pair<int,int>> pts{{3, 1}, {1, 2}, {2, 0}};
    std::sort(pts.begin(), pts.end(), [](const auto& p, const auto& q) { return p.second < q.second; });   //
     generic lambda as comparator
    for (auto& [a1, b1] : pts) std::cout << a1 << ':' << b1 << ' ';                              // structured
     bindings
    std::cout << "\n";
}
```

Answer: captured output

```
11 12 10 11 11
11
1
2:0 3:1 1:2
```

Casts and const-correctness

- `static_cast`: well-defined conversions between related types (numeric, up/down a hierarchy without checking). `dynamic_cast`: checked down/cross-casts of *polymorphic* types (needs RTTI; returns `nullptr` or throws `bad_cast`). `const_cast`: adds/removes `const`; *modifying* a truly const object through it is UB. `reinterpret_cast`: reinterpret bits (pointer↔integer); legal only in narrow cases. C-style `(T)x` tries all of them: avoid.
- A `const` member function promises not to change observable state; `mutable` members (caches, mutexes) are the exception. Overload on `const` to give `const` and non-`const` access (see `Vec2::operator[]`).
- `explicit` stops implicit conversions through single-argument constructors and conversion operators. Brace initialisation forbids narrowing: `int a{3.5};` does not compile.
- **Value categories:** lvalue (has identity, can take its address), prvalue (a pure temporary), xvalue (an expiring object, e.g. `std::move(x)`); `T&&` binds to rvalues; in a template, `T&&` is a *forwarding reference*, and `std::forward<T>` preserves the category ("perfect forwarding").

5.11 Exceptions and stack unwinding ★★

Destructors run during unwinding, in reverse order

```
#include <iostream>
#include <stdexcept>
struct G { const char* n; G(const char* n) : n(n) { std::cout << "+" << n << " "; } ~G() { std::cout << "-" << n
     << " "; } };
void f() { G a("a"); throw std::runtime_error("boom"); }
int main() {
    try { G b("b"); f(); std::cout << "not reached "; }
    catch (const std::exception& e) { std::cout << "caught " << e.what() << "\n"; }
}
```

Answer: captured output

```
+b +a -a -b caught boom
```

Must know

Throw by value, catch by (const) reference. Handlers are tried in order, so put *derived* types before bases. **Destructors must not throw** (they are `noexcept` by default; throwing during unwinding calls `std::terminate`). Exception-safety levels: *basic* (no leaks, valid state), *strong* (commit-or-rollback), *no-throw*. Exceptions cost nothing on the non-throwing path in modern ABIs (table-based unwinding) but are slow when thrown; GPU device code (CUDA) does not support them.

5.12 Undefined behaviour, ODR and initialisation order in C++ ★★★

C++ undefined behaviour	Typical real-world symptom
Signed overflow, out-of-range shifts, divide by zero	Surprising optimisation (see Chapter 4); SIGFPE
Out-of-bounds `v[i]` (no check), `front()` of an empty container	Silent corruption; use `at()` or `-D_GLIBCXX_ASSERTIONS` to trap
Using an invalidated iterator, dangling reference, reference to a destroyed temporary	Garbage values, crashes far from the bug
`delete` of a base pointer without virtual destructor; `delete` vs. `delete[]` mismatch	Leaks, heap corruption
Reading an inactive union member (type punning other than via `memcpy`/`bit_cast`)	Compiler may assume the other member is untouched
Data race on a non-atomic object; modifying a `const` object; violating `restrict`/aliasing rules	Impossible-looking behaviour at `-O2`
ODR violation: two different definitions of the same entity in different translation units	Link succeeds; wrong function is silently picked
Missing return in a non-void function (C++)	Falls off the end: UB (unlike `main`)

Static initialisation order fiasco

Within one translation unit globals are initialised in definition order; **across translation units the order is unspecified**. A global in `a.cpp` that uses a global from `b.cpp` in its constructor may see it uninitialised. Fix: a function-local `static` (the "construct on first use" idiom), initialised thread-safely on first call. Function-local statics are initialised once, the first time control passes through them.

Initialisation order within one translation unit

```cpp
#include <iostream>
struct S { S(const char* n) { std::cout << "init " << n << "\n"; } };
S a("a");  static S b("b");
void f() { static S c("c"); }                 // initialised once, on first call
int main() { std::cout << "main start\n"; f(); f(); S d("d"); std::cout << "main end\n"; }
S z("z");                                      // defined later in the file, still before main
```

Answer: captured output

```
init a
init b
init z
main start
init c
init d
main end
```

Why: namespace-scope objects are constructed before `main` in definition order (`a, b, z`); `c` is constructed at the first call of `f`; `d` is an automatic.

5.13 C++ output-prediction drills ★★★

Q5.2 Does `std::move` move?

```cpp
#include <iostream>
#include <string>
#include <utility>
int main() {
    std::string a = "hello";
    std::string b = std::move(a);                    // moves: a is valid but unspecified (empty in practice)
    const std::string c = "world";
```

```
    std::string d = std::move(c);                    // c is const: T&& cannot bind -> COPY constructor is used
    std::cout << "[" << a << "] [" << b << "] [" << c << "] [" << d << "]\n";
}
```

Answer: captured output

```
[] [hello] [world] [world]
```

Why: `std::move` is only a cast. A moved-from `string` is typically empty. A `const` object cannot be moved from, so the copy constructor runs and `c` keeps its value.

Q5.3 Narrowing in brace initialisation.

int a{3.5}; is ill-formed

```
int main() { int a{3.5}; (void)a; }
```

```
(no output)
```

Why: brace-init forbids narrowing conversions (double to int); `int a = 3.5;` would compile (with truncation) and `int a(3.5)` too.

Q5.4 `sizeof` of common standard types (libstdc++, x86-64).

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>
#include <map>
#include <functional>
int main() {
    std::cout << "string=" << sizeof(std::string) << " vector=" << sizeof(std::vector<int>)
              << " unique_ptr=" << sizeof(std::unique_ptr<int>) << " shared_ptr=" << sizeof(std::shared_ptr<int>)
              << " map=" << sizeof(std::map<int,int>) << " function=" << sizeof(std::function<void()>) << "\n";
}
```

Answer: captured output (implementation-specific!)

```
string=32 vector=24 unique_ptr=8 shared_ptr=16 map=48 function=32
```

Why: `vector` = 3 pointers (begin, end, end-of-capacity); `unique_ptr` = 1 pointer; `shared_ptr` = object pointer + control-block pointer; `string` embeds a 15-char small buffer. Do not memorise these numbers: say "implementation-defined" and explain the layout.

Q5.5 Order of `<<` evaluation and `i++`.

```
#include <iostream>
int main() {
    int i = 1;
    std::cout << i++ << " " << i << " " << ++i << "\n";   // C++17: left-to-right for the operands of <<
}
```

Answer: captured output

```
1 2 3
```

Why: since C++17 the operands of `<<` are sequenced left to right, so this is well defined. Before C++17 it was unspecified; the *function-argument* version `f(i++, i)` is still unspecified.

6. Python: Data Model, Gotchas and Automation

In this chapter

Priority ★★★. Time: 60 min. Python is half the job (tools, runners, parsers) and is named in the OA skills list. Expect snippet-output MCQs on mutability, scoping, comprehensions and class behaviour, plus a coding problem. Outputs below are real (CPython 3.13); behaviour that depends on the interpreter version is flagged.

6.1 The data model: names, objects, mutability ★★★

Four sentences that answer half of all Python MCQs

1. Variables are **names bound to objects**; assignment never copies: `b = a` makes a second name for the same object.
2. **Mutable** types (`list, dict, set, bytearray`, most user classes) change in place; **immutable** types (`int, float, str, tuple, frozenset, bytes`) never do; operations create new objects.
3. `==` compares *values* (`__eq__`); `is` compares *identity*. Use `is` only for `None`/singletons.
4. Function arguments are passed "by object reference": the callee gets a new name for the caller's object; rebinding it does not affect the caller, mutating it does.

Q6.1 Identity, equality and the small-integer cache.

```
a = int("257"); b = int("257")
print(a == b, a is b)            # equal values, distinct objects
c = int("5"); d = int("5")
print(c == d, c is d)            # CPython caches the small ints -5..256 (an implementation detail)
l1, l2 = [], []
print(l1 == l2, l1 is l2)
x = None
print(x is None, x == None)      # 'is None' is the idiom
```

Answer: captured output

```
True False
True True
True False
True True
```

Why: never rely on `is` for numbers or strings: whether two equal values share an object is an interpreter optimisation, not language semantics.

Q6.2 The mutable default-argument trap.

```
def add(x, acc=[]):               # the default list is created ONCE, when def runs
    acc.append(x)
    return acc

def add_fixed(x, acc=None):
    if acc is None:
        acc = []
    acc.append(x)
    return acc

print(add(1), add(2), add(3))
print(add_fixed(1), add_fixed(2), add_fixed(3))
```

Answer: captured output

```
[1, 2, 3] [1, 2, 3] [1, 2, 3]
[1] [2] [3]
```

Why: all three calls mutate the same list object; the three prints show it after all calls finished. Use `None` as the sentinel.

Q6.3 Aliasing, shallow and deep copies.

```
import copy
g = [[0] * 3] * 3                 # three references to ONE inner list
g[0][0] = 1
print(g)
h = [[0] * 3 for _ in range(3)]   # three distinct lists
h[0][0] = 1
print(h)
a = [[1, 2], [3]]
shallow, deep = a.copy(), copy.deepcopy(a)
a[0].append(99)
print(shallow, deep)              # shallow copy shares the inner lists
```

Answer: captured output

```
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
[[1, 2, 99], [3]] [[1, 2], [3]]
```

Q6.4 `+=` versus `+`, and a tuple that raises but still changes.

```
a = [1, 2]; b = a; a += [3]       # += on a list extends IN PLACE
print(b)
a = [1, 2]; b = a; a = a + [3]    # + builds a NEW list and rebinds a
print(b)
t = ([1], 2)
try:
    t[0] += [3]                   # list.__iadd__ mutates the list, THEN tuple item assignment fails
except TypeError as e:
    print("TypeError:", e)
print(t)
```

Answer: captured output

```
[1, 2, 3]
[1, 2]
TypeError: 'tuple' object does not support item assignment
([1, 3], 2)
```

Q6.5 Late-binding closures.

```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])          # i is looked up when the lambda RUNS, after the loop ended (i == 2)
gs = [lambda i=i: i for i in range(3)]   # default argument captures the value at definition time
print([g() for g in gs])
```

Answer: captured output

```
[2, 2, 2]
[0, 1, 2]
```

Q6.6 Scope: LEGB, `global`, `nonlocal`.

```
x = 10
def f():
    print("f sees", x)            # no assignment to x in f: reads the global
def g():
    try:
        print(x)                 # x is assigned below, so x is LOCAL in all of g
        x = 5
    except UnboundLocalError:
        print("UnboundLocalError")
def h():
    global x
    x += 1
def counter():
    n = 0
    def inc():
        nonlocal n               # rebind the enclosing function's variable
        n += 1
        return n
    return inc
f(); g(); h(); print(x)
```

```
c = counter(); print(c(), c(), c())
```

Answer: captured output

```
f sees 10
UnboundLocalError
11
1 2 3
```

Why: Python decides a name's scope at compile time per function: any assignment inside makes it local unless declared `global`/`nonlocal`. Lookup order: **L**ocal, **E**nclosing, **G**lobal, **B**uilt-in.

6.2 Numbers, strings and containers ★★★

Expression	C / C++	Python	Reason
$-7/2$	-3	-3.5 (`-7 // 2` $= -4$)	Python `/` is true division; `//` **floors**
$-7 \bmod 2$	-1	1	Python's result takes the sign of the *divisor*
Integer size	32/64-bit, overflow	arbitrary precision	big ints never overflow (but are slower)
`round(2.5)`	(library dependent)	2	banker's rounding: ties go to even

```
print(-7 // 2, -7 % 2, divmod(-7, 2), 7 // -2, 7 % -2)
print(round(2.5), round(3.5), round(-2.5), int(-2.9), 0.1 + 0.2, 0.1 + 0.2 == 0.3)
print(2 ** 100, 3 / 2, 3 // 2, 3.0 // 2, -3 // 2.0, 2 ** -1)
print(True + True, True == 1, isinstance(True, int), 1 < 2 < 3, 1 < 3 < 2)
print(0 or "a", "" or [] or None, 1 and 2, 0 and 2, not [])   # and/or return one of their OPERANDS
```

Answer: captured output

```
-4 1 (-4, 1) -4 -1
2 4 -2 -2 0.30000000000000004 False
1267650600228229401496703205376 1.5 1 1.0 -2.0 0.5
2 True True True False
a None 2 0 True
```

Strings: slicing, immutability, f-strings

```
s = "compiler"
print(s[::-1], s[1:4], s[-3:], s[::2], repr(s[10:]), s.upper(), s.find("p"), s.count("e"))
print(",".join(["a", "b"]), "a b  c".split(), "a,b,,c".split(","), "  x ".strip() + "|")
print(f"{3.14159:.2f} {42:05d} {255:#x} {'x':>4} {1234567:,}")
try:
    s[0] = "C"
except TypeError as e:
    print("TypeError:", e)
```

Answer: captured output

```
relipmoc omp ler cmie '' COMPILER 3 1
a,b ['a', 'b', 'c'] ['a', 'b', '', 'c'] x|
3.14 00042 0xff    x 1,234,567
TypeError: 'str' object does not support item assignment
```

Containers: dict, set, Counter, deque, defaultdict

```
from collections import Counter, defaultdict, deque
c = Counter("mississippi")
print(c.most_common(2), c["z"])
d = defaultdict(list); d["a"].append(1); d["a"].append(2); print(dict(d))
dq = deque([1, 2, 3]); dq.appendleft(0); dq.rotate(1); print(dq)
st = {1, 2, 3}
print(st & {2, 3, 4}, st | {4}, st - {1}, st ^ {3, 4})
try:
    {[1]: 2}
```

```
except TypeError as e:
    print("TypeError:", e)
words = ["bb", "a", "ccc", "dd"]
print(sorted(words, key=len), sorted(words, key=lambda w: (-len(w), w)))   # sort is stable
r = words.sort()
print(r, words)                                                            # list.sort() returns None
```

Answer: captured output

```
[('i', 4), ('s', 4)] 0
{'a': [1, 2]}
deque([3, 0, 1, 2])
{2, 3} {1, 2, 3, 4} {2, 3} {1, 2, 4}
TypeError: unhashable type: 'list'
['a', 'bb', 'dd', 'ccc'] ['ccc', 'bb', 'dd', 'a']
None ['a', 'bb', 'ccc', 'dd']
```

Complexity cheat-sheet (CPython)

`list`: index $O(1)$, append amortised $O(1)$, `insert(0,x)`/`pop(0)`/`x in list` $O(n)$. `deque`: both ends $O(1)$. `dict`/`set`: average $O(1)$ get/set/`in`. `str += str` in a loop is $O(n^2)$ in the worst case; build a list and `"".join`. `sorted` is Timsort, $O(n \log n)$, stable. `heapq` is a min-heap (negate for max-heap); `bisect` does binary search on sorted lists.

6.3 Iterators, generators and the standard toolbox ★★★

```
from itertools import islice, accumulate, groupby, permutations, combinations, product
g = (x * x for x in range(4))
print(list(g), list(g))                    # a generator is exhausted after one pass
def fib():
    a, b = 0, 1
    while True:
        yield a                            # suspends here, keeps local state
        a, b = b, a + b
print(list(islice(fib(), 10)))
print(list(accumulate([1, 2, 3, 4])), list(combinations("abc", 2)), len(list(permutations(range(4)))))
print(list(product("ab", repeat=2)), list(zip([1, 2, 3], "ab")), [k for k, _ in groupby("aabbbc")])
print(list(map(lambda x: x * 2, [1, 2])), [x for x in range(10) if x % 3 == 0], {x: x * x for x in range(3)})
```

Answer: captured output

```
[0, 1, 4, 9] []
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
[1, 3, 6, 10] [('a', 'b'), ('a', 'c'), ('b', 'c')] 24
[('a', 'a'), ('a', 'b'), ('b', 'a'), ('b', 'b')] [(1, 'a'), (2, 'b')] ['a', 'b', 'c']
[2, 4] [0, 3, 6, 9] {0: 0, 1: 1, 2: 4}
```

Must know

`map`, `filter`, `zip`, `range`, `enumerate` return lazy iterators in Python 3. `zip` stops at the shortest input. A comprehension has its own scope (the loop variable does not leak in Python 3). `for/else`: the `else` block runs if the loop was *not* ended by `break`. `try/else`: runs if no exception occurred.

6.4 Functions, decorators, classes ★★

Decorators, lru_cache, class attributes, MRO

```
import functools

def traced(fn):
    @functools.wraps(fn)                   # keeps fn.__name__ and docstring
    def wrapper(*args, **kwargs):
        print("calling", fn.__name__, args)
        return fn(*args, **kwargs)
    return wrapper

@traced
```

```
def add(a, b): return a + b
print(add(1, 2), add.__name__)

@functools.lru_cache(maxsize=None)
def fibm(n): return n if n < 2 else fibm(n - 1) + fibm(n - 2)
print(fibm(80), fibm.cache_info())

class A:
    items = []                              # CLASS attribute: shared by all instances
    def __init__(self): self.n = 0         # instance attribute
class B(A): pass
class C(A): pass
class D(B, C): pass
a1, a2 = A(), A(); a1.items.append(1)
print(a2.items, [k.__name__ for k in D.__mro__])   # C3 linearisation: D, B, C, A, object

class P:
    def __init__(self, x): self.x = x
    def __eq__(self, o): return self.x == o.x
try:
    {P(1)}                                  # defining __eq__ without __hash__ makes instances unhashable
except TypeError as e:
    print("TypeError:", e)

class Temp:
    def __init__(self): self._c = 0
    @property
    def f(self): return self._c * 9 / 5 + 32
    @f.setter
    def f(self, v): self._c = (v - 32) * 5 / 9
t = Temp(); t.f = 212; print(round(t._c), t.f)
```

Answer: captured output

```
calling add (1, 2)
3 add
23416728348467685 CacheInfo(hits=78, misses=81, maxsize=None, currsize=81)
[1] ['D', 'B', 'C', 'A', 'object']
TypeError: unhashable type: 'P'
100 212.0
```

6.4.1 Exceptions and context managers

```
def f():
    try:
        return "try"
    finally:
        print("finally runs before returning |", end=" ")
print(f())
for i in range(3):
    try:
        if i == 1: continue
        print("body", i, end=" ")
    finally:
        print("fin", i, end=" | ")
print()
try:
    try:
        1 / 0
    except ZeroDivisionError as e:
        raise ValueError("bad input") from e
except ValueError as e:
    print(repr(e), "<- cause:", repr(e.__cause__))
import contextlib
@contextlib.contextmanager
def tag(name):
    print(f"<{name}>", end=""); yield; print(f"</{name}>", end="")
with tag("a"), tag("b"): print("x", end="")
print()
```

Answer: captured output

```
finally runs before returning | try
body 0 fin 0 | fin 1 | body 2 fin 2 |
ValueError('bad input') <- cause: ZeroDivisionError('division by zero')
<a><b>x</b></a>
```

6.5 Concurrency and performance facts ★★

- **GIL** (global interpreter lock): in standard CPython only one thread executes Python bytecode at a time. Threads help I/O-bound work (waiting on disk/network/subprocesses); for CPU-bound work use `multiprocessing` / `ProcessPoolExecutor` or native extensions (NumPy releases the GIL). CPython 3.13 ships an *experimental* free-threaded build (PEP 703); do not assume it.
- **Memory:** reference counting plus a cyclic garbage collector; objects are freed when the count hits zero. `del x` removes a name, not necessarily the object.
- **`asyncio`:** cooperative concurrency on one thread (`async def`/`await`); good for many sockets, useless for CPU work.
- **Speed tips for contests:** read all input with `sys.stdin`, prefer comprehensions and built-ins, avoid `list.insert(0)` and repeated string concatenation, use local variables in hot loops, `sys.setrecursionlimit` (and think about iteration instead) for deep recursion.

6.6 Python for verification automation: the standard-library tour ★★★

This is the toolkit behind "design and implement tools, libraries and automation" (Chapter 18 assembles it into a harness).

subprocess with timeout, regex log parsing, JSON, parallel map, diff, hashing

```
import subprocess, sys, re, json, difflib, hashlib
import concurrent.futures as cf

# 1. run a command: capture output, enforce a timeout, inspect the exit status (never shell=True with untrusted
    text)
r = subprocess.run([sys.executable, "-c", "print('hi'); raise SystemExit(3)"], capture_output=True, text=True,
    timeout=5)
print("rc =", r.returncode, "stdout =", repr(r.stdout))
try:
    subprocess.run([sys.executable, "-c", "import time; time.sleep(5)"], timeout=0.3)
except subprocess.TimeoutExpired as e:
    print("timed out after", e.timeout, "s")

# 2. parse a test log with named regex groups and emit JSON
log = "test_add: FAIL (exit=1) 0.52s\ntest_mul: PASS (exit=0) 0.11s\n"
pat = re.compile(r"(?P<name>\w+): (?P<res>PASS|FAIL) \(exit=(?P<rc>\d+)\) (?P<t>[\d.]+)s")
rows = [m.groupdict() for m in pat.finditer(log)]
print(json.dumps(rows))
print("failed:", [r["name"] for r in rows if r["res"] == "FAIL"])

# 3. run jobs in parallel (threads are fine when the work is waiting on subprocesses)
with cf.ThreadPoolExecutor(max_workers=4) as ex:
    print(list(ex.map(lambda n: n * n, range(6))))

# 4. compare expected vs. actual output
print("\n".join(difflib.unified_diff(["a", "b", "c"], ["a", "x", "c"], "expected", "actual", lineterm="")))

# 5. fingerprint an output (e.g. to deduplicate crashes or compare runs)
print(hashlib.sha256(b"abc").hexdigest()[:16])
```

Answer: captured output

```
rc = 3 stdout = 'hi\n'
timed out after 0.3 s
[{"name": "test_add", "res": "FAIL", "rc": "1", "t": "0.52"}, {"name": "test_mul", "res": "PASS", "rc": "0", "t":
    "0.11"}]
failed: ['test_add']
[0, 1, 4, 9, 16, 25]
--- expected
+++ actual
```

```
@@ -1,3 +1,3 @@
 a
-b
+x
 c
ba7816bf8f01cfea
```

Module	Use in a compiler-verification script
`subprocess`	invoke compilers/runners; `run(..., capture_output=True, text=True, timeout=T, check=False)`
`argparse`, `pathlib`, `shutil`, `tempfile`	CLI tools, path handling, scratch dirs per test (`TemporaryDirectory`)
`re`, `json`, `csv`	parse logs, emit machine-readable reports
`concurrent.futures`, `multiprocessing`	shard thousands of tests across cores
`random` (with `seed`)	reproducible test generation; always print the seed
`hashlib`, `difflib`	output fingerprints, readable diffs
`logging`	levelled logs with timestamps instead of `print`
`unittest` / `pytest`	structure the tests; `pytest.mark.parametrize` for the type × modifier sweeps typical of ISA tests
`ast`, `dis`, `tokenize`	parse/inspect Python itself (next section)

6.7 Compiler-flavoured Python: `ast`, `dis` ★★

Python is itself a compiler to bytecode. Seeing its AST and bytecode makes the "front end → IR → interpreter" pipeline concrete.

Abstract syntax tree and bytecode of a small expression (CPython 3.13)

```
import ast, dis
tree = ast.parse("x = 1 + 2 * 3")
print(ast.dump(tree.body[0], indent=1))
print(ast.literal_eval("[1, 2, {'a': (3, 4)}]"))      # safe evaluation of literals; never use eval() on untrusted
      text
dis.dis(compile("a + b * c", "<expr>", "eval"))
```

Captured output

```
Assign(
 targets=[
  Name(id='x', ctx=Store())],
 value=BinOp(
  left=Constant(value=1),
  op=Add(),
  right=BinOp(
   left=Constant(value=2),
   op=Mult(),
   right=Constant(value=3))))
[1, 2, {'a': (3, 4)}]
  0           RESUME                   0

  1           LOAD_NAME                0 (a)
              LOAD_NAME                1 (b)
              LOAD_NAME                2 (c)
              BINARY_OP                5 (*)
              BINARY_OP                0 (+)
              RETURN_VALUE
```

Reading it: `*` sits *below* `+` in the tree because multiplication binds tighter: the parser encodes precedence in the tree shape. The bytecode is a postfix (stack-machine) form: load `a`, load `b`, load `c`, multiply, add, return. Chapter 10 builds the same pipeline by hand.

6.8 Python drills (more) ★★★

Q6.7 What does this print?

```
x = [1, 2, 3]
y = x
y.append(4)
z = x[:]
z.append(5)
def f(lst):
    lst = lst + [9]
    lst.append(8)
f(x)
print(x, y is x, z is x, len(z))
```

Answer: captured output

```
[1, 2, 3, 4] True False 5
```

Why: `y` aliases `x`; `z` is a shallow copy. Inside `f`, `lst + [9]` creates a new list and rebinds the *local* name, so the appended 8 lands in the new list and `x` is unchanged by the call.

Q6.8 Class attribute versus instance attribute.

```
class C:
    n = 0
    def bump(self): self.n += 1          # reads class attr, then creates an INSTANCE attr
a, b = C(), C()
a.bump(); a.bump(); b.bump()
print(a.n, b.n, C.n)
```

Answer: captured output

```
2 1 0
```

Q6.9 Comprehension and generator scoping/laziness.

```
funcs = []
for i in range(3):
    funcs.append(lambda x, i=i: x + i)
print([f(10) for f in funcs])
lst = [1, 2, 3]
gen = (v * 2 for v in lst)
lst.append(4)                            # the generator reads lst lazily, at iteration time
print(list(gen))
print(sum(1 for _ in range(10)), max([3, 9, 2], key=lambda v: -v), sorted("bca", reverse=True))
```

Answer: captured output

```
[10, 11, 12]
[2, 4, 6, 8]
10 2 ['c', 'b', 'a']
```

Q6.10 String and integer edge cases.

```
print("abc" * 2, "a" + "b" * 2, "abc"[::-1] == "cba", "a" < "B", ord("a"), chr(100), "5" + "5", int("5") + int("5"
    ))
print(bool("False"), bool(""), bool(0.0), bool([0]), None == False, 5 // 2 * 2.0, 2 ** 3 ** 2, -2 ** 2)
print(10 ** 18 + 1 == 1000000000000000001, 7 / 2 == 3.5, int(7 / 2), 0xFF, 0b101, 0o17, 1e3)
```

Answer: captured output

```
abcabc abb True False 97 d 55 10
True False False True False 4.0 512 -4
True True 3 255 5 15 1000.0
```

Why: `**` is right-associative ($2^{(3^2)} = 512$) and binds tighter than unary minus ($-2^2 = -4$). Non-empty strings are truthy even if they spell "False". Uppercase letters sort before lowercase in ASCII.

7. The Coding Round: Patterns, Templates, Stress Testing

In this chapter

Priority ★★★. Time: 90 min (templates 30 + timed problems 60). Every report mentions 1–3 coding problems (S1–S3, S6, S7, S11, S13, S18, S19). This chapter gives (i) how to choose an algorithm from the constraints, (ii) a pattern table, (iii) tested C++ and Python templates, (iv) a **stress-testing** habit (brute force vs. fast solution) that is also the core skill of a verification engineer. Section 7.5 solves the specific problems that candidates reported.

7.1 Constraints decide the algorithm ★★★

A modern judge does about 10^8 simple operations per second in C++ and about 10^7 in Python. Read the limits first:

Input size n	Target complexity	Typical technique
$n \le 10$	$O(n!)$	permutations, brute force, backtracking
$n \le 20$–25	$O(2^n \cdot n)$	bitmask DP, subsets, meet-in-the-middle
$n \le 500$	$O(n^3)$	Floyd–Warshall, interval DP, 3 nested loops
$n \le 5{,}000$	$O(n^2)$	2-D DP (LCS, edit distance, knapsack with small capacity)
$n \le 10^5$	$O(n \log n)$	sorting, heap, binary search, segment tree/Fenwick, Dijkstra, DSU
$n \le 10^6$–10^7	$O(n)$	prefix sums, two pointers, hashing, sieve, monotonic stack/queue
$n \le 10^{18}$	$O(\log n)$ or $O(\sqrt{n})$	binary search on the answer, fast exponentiation, number theory

7.2 From problem signal to technique ★★★

If the problem says ...	Think ...
sorted array, "find", "minimum x such that"	binary search (also *on the answer* with a monotone predicate)
pair/triple with a sum, contiguous subarray with a property	two pointers, sliding window, prefix sums + hash map
"number of subarrays with sum/xor $= k$"	prefix sum/xor + hash map of counts
"kth largest", "top k", merge k sorted	heap (priority queue)
"next greater element", "largest rectangle", matching brackets	monotonic stack / stack
dependencies, "order of tasks", "can finish all"	topological sort (Kahn) / cycle detection
shortest path: unit weights / non-negative weights	BFS / Dijkstra
connected components, "are a and b connected", Kruskal	union–find (DSU)
prefix lookups, autocomplete, word dictionary	trie
"count the ways", "min/max cost", overlapping subproblems	dynamic programming (state, transition, base case, order)

If the problem says ...	Think ...
subset choices with a capacity	knapsack (0/1: iterate capacity *downward*; unbounded: upward)
"maximise the count" with an exchange argument	greedy after sorting (prove with an exchange argument)
"no two adjacent equal", schedule with cooldown	max-heap greedy
bits, XOR, "appears once", parity	XOR tricks, bitmasks (Chapter 4)
design: LRU, min-stack, O(1) operations	hash map + doubly linked list; auxiliary stacks; timestamps

7.3 Reusable C++ templates (compiled and tested) ★★★

Binary search on the answer, DSU, Dijkstra, grid BFS

```cpp
#include <bits/stdc++.h>
using namespace std;

// smallest x in [lo, hi] with pred(x) true; pred must be monotone (F...FT...T) and pred(hi) true
template <class P> long long first_true(long long lo, long long hi, P pred) {
    while (lo < hi) {
        long long mid = lo + (hi - lo) / 2;       // not (lo+hi)/2: avoids overflow
        if (pred(mid)) hi = mid; else lo = mid + 1;
    }
    return lo;
}

struct DSU {                                      // union-find with path compression + union by size
    vector<int> p, sz;
    explicit DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};

vector<long long> dijkstra(const vector<vector<pair<int, int>>>& g, int src) {
    const long long INF = LLONG_MAX / 4;
    vector<long long> dist(g.size(), INF);
    priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<>> pq;
    dist[src] = 0; pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;                // stale queue entry
        for (auto [v, w] : g[u])
            if (dist[u] + w < dist[v]) { dist[v] = dist[u] + w; pq.push({dist[v], v}); }
    }
    return dist;
}

int bfs_grid(const vector<string>& g, int sr, int sc, int tr, int tc) {   // '#' = wall, 4 directions
    int R = g.size(), C = g[0].size();
    vector<vector<int>> d(R, vector<int>(C, -1));
    queue<pair<int, int>> q; q.push({sr, sc}); d[sr][sc] = 0;
    const int dr[] = {1, -1, 0, 0}, dc[] = {0, 0, 1, -1};
    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();
        if (r == tr && c == tc) return d[r][c];
        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr >= 0 && nr < R && nc >= 0 && nc < C && g[nr][nc] != '#' && d[nr][nc] < 0) {
                d[nr][nc] = d[r][c] + 1; q.push({nr, nc});
            }
        }
```

```
    }
    return -1;
}

int main() {
    cout << "isqrt-ceil(50) = " << first_true(0, 50, [](long long x) { return x * x >= 50; }) << "\n";
    DSU d(6); d.unite(0, 1); d.unite(1, 2); d.unite(3, 4);
    set<int> roots; for (int i = 0; i < 6; i++) roots.insert(d.find(i));
    cout << "components = " << roots.size() << ", 0~2: " << (d.find(0) == d.find(2)) << ", 0~3: " << (d.find(0) ==
      d.find(3)) << "\n";
    vector<vector<pair<int, int>>> g(5);
    auto add = [&](int a, int b, int w) { g[a].push_back({b, w}); g[b].push_back({a, w}); };
    add(0, 1, 4); add(0, 2, 1); add(2, 1, 2); add(1, 3, 1); add(2, 3, 5); // node 4 unreachable
    auto dist = dijkstra(g, 0);
    cout << "dist: "; for (int i = 0; i < 4; i++) cout << dist[i] << ' '; cout << (dist[4] > 1e17 ? "inf" : "?")
     << "\n";
    vector<string> grid = {"..#.", "..#.", "....", "#..."};
    cout << "grid shortest = " << bfs_grid(grid, 0, 0, 0, 3) << "\n";
}
```

Answer: captured output

```
isqrt-ceil(50) = 8
components = 3, 0~2: 1, 0~3: 0
dist: 0 3 1 4 inf
grid shortest = 7
```

Python toolbox: bisect, heapq, deque, prefix sums, sliding window, Counter

```
import bisect, heapq
from collections import deque, Counter
from itertools import accumulate

a = [1, 3, 3, 5, 8]
print(bisect.bisect_left(a, 3), bisect.bisect_right(a, 3), bisect.bisect_left(a, 4))   # lower_bound, upper_bound
h = []
for x in [5, 1, 4]: heapq.heappush(h, x)
print(heapq.heappop(h), heapq.nlargest(2, [7, 2, 9, 4]), heapq.nsmallest(2, [7, 2, 9, 4]))
pre = [0] + list(accumulate([2, 4, 6, 8]))                # sum(l..r) = pre[r+1] - pre[l]
print(pre, pre[3] - pre[1])

def longest_unique(s):                                    # sliding window: longest substring without repeats
    last, left, best = {}, 0, 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] >= left: left = last[ch] + 1
        last[ch] = right
        best = max(best, right - left + 1)
    return best
print(longest_unique("abcabcbb"), longest_unique("pwwkew"))

def max_window(nums, k):                                  # monotonic deque: max of every window of size k
    dq, out = deque(), []
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] <= x: dq.pop()
        dq.append(i)
        if dq[0] <= i - k: dq.popleft()
        if i >= k - 1: out.append(nums[dq[0]])
    return out
print(max_window([1, 3, -1, -3, 5, 3, 6, 7], 3))
print(Counter("abracadabra").most_common(1))
```

Answer: captured output

```
1 3 3
1 [9, 7] [2, 4]
[0, 2, 6, 12, 20] 10
3 3
[3, 3, 5, 5, 6, 7]
[('a', 5)]
```

7.4 Stress testing: the habit that finds your own bugs ★★★

Write the *simple, obviously correct* brute force first, then compare it with the clever solution on thousands of random small inputs. This is **differential testing** (Chapter 17) applied to your own code, and a verification engineer's reflex.

Stress test: greedy 'maximize greatness' vs. brute force over all permutations

```cpp
#include <bits/stdc++.h>
using namespace std;

int fast(vector<int> a) {                       // sort, then match each value with the smallest strictly smaller
     target
    sort(a.begin(), a.end());
    int j = 0;
    for (int x : a) if (x > a[j]) j++;
    return j;
}
int brute(const vector<int>& a) {               // try every permutation
    vector<int> p = a; sort(p.begin(), p.end());
    int best = 0;
    do {
        int c = 0;
        for (size_t i = 0; i < a.size(); i++) c += p[i] > a[i];
        best = max(best, c);
    } while (next_permutation(p.begin(), p.end()));
    return best;
}
int main() {
    mt19937 rng(12345);                          // fixed seed: failures are reproducible
    for (int it = 0; it < 3000; it++) {
        int n = 1 + rng() % 7;
        vector<int> a(n);
        for (int& x : a) x = rng() % 5;          // small values => many duplicates (the interesting case)
        if (fast(a) != brute(a)) {
            cout << "MISMATCH on:"; for (int x : a) cout << ' ' << x; cout << "\n";
            return 1;
        }
    }
    cout << "3000 random tests: fast == brute\n";
    cout << fast({1, 3, 5, 2, 1, 3, 1}) << " " << fast({1, 2, 3, 4}) << "\n";
}
```

Answer: captured output

```
3000 random tests: fast == brute
4 3
```

A 6-step routine for every coding problem (use it on the OA)

1. Restate the problem; list **constraints** and decide the target complexity.
2. Work the **sample by hand**; invent one tiny edge case ($n = 1$, duplicates, negatives, empty).
3. Write the **brute force** (correct but slow) and submit it if it passes some tests (partial credit).
4. Spot the bottleneck; pick the technique from the table above.
5. Code carefully with 64-bit arithmetic where needed.
6. Stress-test against the brute force, then re-check overflow, off-by-one and I/O format.

7.5 Solved problems that candidates reported ★★★

Each problem below was mentioned in a public NVIDIA interview/OA report (source tag in the orange box); the code is complete, compiled and *asserted* during the build. Whether the exact problem appears in your test is unknowable; the *techniques* recur.

7.5.1 DP: minimum difference of two subset sums (S13)

S13: "partition an array to minimise the sum difference"

Split an array into two subsets so that $|\text{sum}_1 - \text{sum}_2|$ is minimal. *Idea:* subset-sum reachability up to $\lfloor S/2 \rfloor$ (0/1 knapsack, iterate sums downward); the answer is $S - 2s^*$ for the largest reachable $s^* \le S/2$. Time $O(nS)$.

```cpp
#include <bits/stdc++.h>
using namespace std;
int min_diff(const vector<int>& a) {
    int S = accumulate(a.begin(), a.end(), 0);
    vector<char> dp(S / 2 + 1, 0); dp[0] = 1;
    for (int x : a)
        for (int s = S / 2; s >= x; --s) dp[s] |= dp[s - x];     // downward: each item used at most once
    for (int s = S / 2; s >= 0; --s) if (dp[s]) return S - 2 * s;
    return S;
}
int main() {
    assert(min_diff({1, 6, 11, 5}) == 1);
    assert(min_diff({3, 1, 4, 2, 2, 1}) == 1);
    assert(min_diff({1, 2, 7}) == 4);
    cout << min_diff({1, 6, 11, 5}) << " " << min_diff({3, 1, 4, 2, 2, 1}) << " " << min_diff({1, 2, 7}) << "\n";
}
```

Answer: captured output

```
1 1 4
```

7.5.2 DP: "2-D knapsack" (S7) and classic 0/1 knapsack

S7: a "2-D knapsack" DP in a Bengaluru compiler-intern interview

The report does not give the statement. The standard *two-constraint* knapsack is "Ones and Zeroes": pick the largest subset of binary strings using at most m zeros and n ones. State `dp[i][j]` = best count within budgets (i, j); iterate both budgets downward.

```python
def find_max_form(strs, m, n):
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for s in strs:
        z, o = s.count("0"), s.count("1")
        for i in range(m, z - 1, -1):          # both dimensions downward: 0/1 (each string at most once)
            for j in range(n, o - 1, -1):
                dp[i][j] = max(dp[i][j], dp[i - z][j - o] + 1)
    return dp[m][n]

def knapsack(weights, values, W):
    dp = [0] * (W + 1)
    for w, v in zip(weights, values):
        for c in range(W, w - 1, -1):          # downward for 0/1; upward would allow unlimited copies
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[W]

assert find_max_form(["10", "0001", "111001", "1", "0"], 5, 3) == 4
assert find_max_form(["10", "0", "1"], 1, 1) == 2
assert knapsack([10, 20, 30], [60, 100, 120], 50) == 220
print(find_max_form(["10", "0001", "111001", "1", "0"], 5, 3), find_max_form(["10", "0", "1"], 1, 1), knapsack
    ([10, 20, 30], [60, 100, 120], 50))
```

Answer: captured output

```
4 2 220
```

7.5.3 Topological sort and cycle detection (S4)

S4: "topological sort", then automatic differentiation and data-flow analysis

Kahn's algorithm: repeatedly remove nodes of in-degree 0. If fewer than n nodes are output, the graph has a cycle. Why a compiler cares: instruction scheduling orders a dependence DAG; data-flow analyses iterate in reverse post-order; reverse-mode automatic differentiation walks the computation DAG in reverse topological order.

```
#include <bits/stdc++.h>
using namespace std;
// returns a topological order; sets cyclic=true if none exists. O(V+E)
vector<int> topo(int n, const vector<pair<int, int>>& edges, bool& cyclic) {
    vector<vector<int>> g(n); vector<int> indeg(n, 0);
    for (auto [u, v] : edges) { g[u].push_back(v); indeg[v]++; }
    queue<int> q;
    for (int i = 0; i < n; i++) if (!indeg[i]) q.push(i);
    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
    }
    cyclic = (int)order.size() != n;
    return order;
}
int main() {
    bool cyc;
    vector<pair<int,int>> e = {{5,2},{5,0},{4,0},{4,1},{2,3},{3,1}};
    auto ord = topo(6, e, cyc);
    vector<int> pos(6); for (int i = 0; i < 6; i++) pos[ord[i]] = i;
    bool valid = true; for (auto [u, v] : e) valid &= pos[u] < pos[v];     // every edge goes forward
    cout << "order:"; for (int x : ord) cout << ' ' << x; cout << " | valid=" << valid << " cyclic=" << cyc << "\n
    ";
    topo(3, {{0,1},{1,2},{2,0}}, cyc);
    cout << "3-cycle: cyclic=" << cyc << "\n";
}
```

Answer: captured output

```
order: 4 5 2 0 3 1 | valid=1 cyclic=0
3-cycle: cyclic=1
```

7.5.4 Trie (S13)

S13: a Trie problem "similar to a LeetCode hard"

Insert/search/prefix-count in $O(|word|)$. Store a *pass count* per node to answer "how many words start with this prefix" in $O(|prefix|)$.

```
#include <bits/stdc++.h>
using namespace std;
struct Trie {
    struct Node { int next[26]; int pass = 0; bool end = false; Node() { fill(next, next + 26, -1); } };
    vector<Node> t{Node()};                                    // t[0] is the root; children are indices
    void insert(const string& s) {
        int u = 0;
        for (char ch : s) {
            int i = ch - 'a';
            if (t[u].next[i] < 0) { int id = t.size(); t.emplace_back(); t[u].next[i] = id; }
            u = t[u].next[i]; t[u].pass++;
        }
        t[u].end = true;
    }
    int walk(const string& s) const { int u = 0; for (char ch : s) { u = t[u].next[ch - 'a']; if (u < 0) return
     -1; } return u; }
    bool search(const string& s) const { int u = walk(s); return u >= 0 && t[u].end; }
    int count_prefix(const string& p) const { int u = walk(p); return u < 0 ? 0 : t[u].pass; }
};
int main() {
    Trie tr;
    for (string w : {"apple", "app", "apply", "ape", "bat"}) tr.insert(w);
```

```
    assert(tr.search("app") && !tr.search("appl") && tr.count_prefix("ap") == 4 && tr.count_prefix("ba") == 1 &&
     tr.count_prefix("z") == 0);
    cout << tr.search("app") << tr.search("appl") << " " << tr.count_prefix("ap") << " " << tr.count_prefix("ba")
     << " " << tr.count_prefix("z") << "\n";
}
```

Answer: captured output

```
10 4 1 0
```

7.5.5 Prefix XOR + hashing: subarrays with XOR equal to k (S16 "bitwise XOR subarray")

```
#include <bits/stdc++.h>
using namespace std;
long long count_xor_subarrays(const vector<int>& a, int k) {
    unordered_map<int, long long> freq; freq[0] = 1;           // empty prefix
    int pre = 0; long long ans = 0;
    for (int x : a) {
        pre ^= x;                                              // pre = XOR of a[0..i]
        auto it = freq.find(pre ^ k);                          // earlier prefix p with p ^ pre == k
        if (it != freq.end()) ans += it->second;
        freq[pre]++;
    }
    return ans;
}
long long brute(const vector<int>& a, int k) {
    long long c = 0;
    for (size_t i = 0; i < a.size(); i++) { int x = 0; for (size_t j = i; j < a.size(); j++) { x ^= a[j]; c += (x
     == k); } }
    return c;
}
int main() {
    assert(count_xor_subarrays({4, 2, 2, 6, 4}, 6) == 4);
    mt19937 rng(7);
    for (int it = 0; it < 2000; it++) {
        int n = 1 + rng() % 12; vector<int> a(n); for (int& x : a) x = rng() % 8; int k = rng() % 8;
        assert(count_xor_subarrays(a, k) == brute(a, k));
    }
    cout << count_xor_subarrays({4, 2, 2, 6, 4}, 6) << " (2000 random checks vs brute force passed)\n";
}
```

Answer: captured output

```
4 (2000 random checks vs brute force passed)
```

7.5.6 Greedy: Maximize Greatness of an Array (S11–S12 OA problem)

Statement (LeetCode 2592): rearrange `nums` into a permutation `perm` maximising the number of indices with `perm[i] > nums[i]`. The solution and its brute-force stress test are in the stress test earlier in this chapter: *sort, then for each value greedily beat the smallest not-yet-beaten target*; $O(n \log n)$.

7.5.7 Reorganise a string / shuffle a playlist (S19)

S19: "shuffle a playlist so that songs by the same artist are not played back to back"

Same problem as "Reorganize String": always emit the most frequent remaining item that is not the one just used; impossible when the top frequency exceeds $\lceil n/2 \rceil$.

```
import heapq
from collections import Counter

def reorganize(items):
    heap = [(-c, x) for x, c in Counter(items).items()]
    heapq.heapify(heap)
    out, held = [], None                        # 'held' = the item used last step (cannot be reused immediately)
    while heap:
        c, x = heapq.heappop(heap)
        out.append(x)
        if held and held[0] < 0: heapq.heappush(heap, held)
```

```
        held = (c + 1, x)                       # one fewer copy remaining
    return out if len(out) == len(items) else []

def ok(seq): return all(a != b for a, b in zip(seq, seq[1:]))
assert reorganize(list("aab")) == list("aba")
assert reorganize(list("aaab")) == []
for s in ["aabbcc", "aaabbc", "abcdefg", "aabbbccc"]:
    r = reorganize(list(s)); assert len(r) == len(s) and ok(r)
print("".join(reorganize(list("aab"))), reorganize(list("aaab")), "".join(reorganize(list("aaabbc"))))
```

Answer: captured output

```
aba [] ababac
```

7.5.8 Design: O(1) init / get / set / reset (S19) and LRU cache (S25)

S19: "a data structure with O(1) init, get, set and reset"; S25: "LRU cache"

Reset array: keep a global *version* counter; each cell stores its value and the version at which it was written. A cell is valid only if its version equals the current one, so `fill(v)` is just "bump the version, remember the default". **LRU:** hash map from key to list node + doubly linked list ordered by recency: all operations $O(1)$.

```
#include <bits/stdc++.h>
using namespace std;
class ResetArray {
    vector<int> val, ver; int cur = 0, defv;
public:
    ResetArray(int n, int d) : val(n), ver(n, 0), defv(d) {}
    int  get(int i) const { return ver[i] == cur ? val[i] : defv; }
    void set(int i, int v) { val[i] = v; ver[i] = cur; }
    void fill(int d) { cur++; defv = d; }                       // O(1): older cells become stale
};
class LRU {
    int cap; list<pair<int, int>> items;                        // front = most recently used
    unordered_map<int, list<pair<int, int>>::iterator> pos;
public:
    explicit LRU(int c) : cap(c) {}
    int get(int k) {
        auto it = pos.find(k); if (it == pos.end()) return -1;
        items.splice(items.begin(), items, it->second);          // move node to front in O(1)
        return it->second->second;
    }
    void put(int k, int v) {
        auto it = pos.find(k);
        if (it != pos.end()) { it->second->second = v; items.splice(items.begin(), items, it->second); return; }
        if ((int)items.size() == cap) { pos.erase(items.back().first); items.pop_back(); }   // evict least
     recently used
        items.emplace_front(k, v); pos[k] = items.begin();
    }
};
int main() {
    ResetArray a(5, 0);
    a.set(2, 7);
    cout << a.get(2) << a.get(3) << " "; a.fill(9); cout << a.get(2) << a.get(0) << " "; a.set(2, 1); cout << a.
     get(2) << "\n";
    LRU c(2);
    c.put(1, 1); c.put(2, 2);
    cout << c.get(1) << " "; c.put(3, 3);                        // evicts key 2
    cout << c.get(2) << " "; c.put(4, 4);                        // evicts key 1
    cout << c.get(1) << " " << c.get(3) << " " << c.get(4) << "\n";
}
```

Answer: captured output

```
70 99 1
1 -1 -1 3 4
```

7.5.9 Trees and lists: spiral order → doubly linked list (S1), cycle detection (S3)

S1: "convert a binary tree into a doubly linked list in spiral order"; S3: "cycle detection in a linked list"; S4–S5: pointers

Spiral (zig-zag) level order alternates direction per level; then relink the visited nodes with `left=prev`, `right=next`. Floyd's tortoise–hare detects a cycle in $O(n)$ time and $O(1)$ space; resetting one pointer to the head and moving both by one step finds the cycle's entry.

```
#include <bits/stdc++.h>
using namespace std;
struct Node { int val; Node *left, *right; Node(int v) : val(v), left(nullptr), right(nullptr) {} };

Node* to_dll_spiral(Node* root) {
    if (!root) return nullptr;
    vector<Node*> order; deque<Node*> level{root}; bool rtl = false;
    while (!level.empty()) {
        vector<Node*> cur(level.begin(), level.end());
        if (rtl) reverse(cur.begin(), cur.end());
        order.insert(order.end(), cur.begin(), cur.end());
        deque<Node*> nxt;
        for (Node* n : level) { if (n->left) nxt.push_back(n->left); if (n->right) nxt.push_back(n->right); }
        level.swap(nxt); rtl = !rtl;
    }
    for (size_t i = 0; i < order.size(); i++) {                  // reuse left/right as prev/next
        order[i]->left  = i ? order[i - 1] : nullptr;
        order[i]->right = i + 1 < order.size() ? order[i + 1] : nullptr;
    }
    return order[0];
}
struct L { int v; L* next; };
L* cycle_start(L* head) {                                        // Floyd
    L *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) {
            slow = head;
            while (slow != fast) { slow = slow->next; fast = fast->next; }
            return slow;
        }
    }
    return nullptr;
}
int main() {
    Node* r = new Node(1);
    r->left = new Node(2); r->right = new Node(3);
    r->left->left = new Node(4); r->left->right = new Node(5); r->right->left = new Node(6); r->right->right = new
      Node(7);
    Node* h = to_dll_spiral(r);
    cout << "forward: "; Node* t = h; for (; t->right; t = t->right) cout << t->val << ' '; cout << t->val;
    cout << " | backward: "; for (; t; t = t->left) cout << t->val << ' '; cout << "\n";
    L a{1, nullptr}, b{2, nullptr}, c{3, nullptr}, d{4, nullptr}, e{5, nullptr};
    a.next = &b; b.next = &c; c.next = &d; d.next = &e; e.next = &c;      // 5 -> 3 closes the cycle
    L* s = cycle_start(&a);
    cout << "cycle starts at " << (s ? s->v : -1);
    L x{1, nullptr}, y{2, nullptr}; x.next = &y;                          // a separate, acyclic list
    cout << "; acyclic list -> " << (cycle_start(&x) ? "cycle" : "none");
    e.next = nullptr;                                                      // cut the back edge
    cout << "; after cutting the cycle -> " << (cycle_start(&a) ? "cycle" : "none") << "\n";
}
```

Answer: captured output

```
forward: 1 3 2 4 5 6 7 | backward: 7 6 5 4 2 3 1
cycle starts at 3; acyclic list -> none; after cutting the cycle -> none
```

7.5.10 King paths on a chessboard (S2)

S2: "a king at (1,1) must reach (8,8): find all possible paths and the shortest"

Interpretations: (a) *monotone* paths using the moves right, up and diagonal-up-right: counted by DP (the **Delannoy number** $D(7,7) = 48639$); (b) shortest path with all 8 king moves: BFS, equal to the Chebyshev distance 7. Clarify the move set with the interviewer first: with all 8 moves and revisits allowed, the number of paths is infinite.

```
from collections import deque

def count_monotone(n=8):
    dp = [[0] * n for _ in range(n)]
    dp[0][0] = 1
    for r in range(n):
        for c in range(n):
            if r or c:
                dp[r][c] = (dp[r-1][c] if r else 0) + (dp[r][c-1] if c else 0) + (dp[r-1][c-1] if r and c else 0)
    return dp[n-1][n-1]

def king_shortest(n=8, blocked=()):
    dist = {(0, 0): 0}; q = deque([(0, 0)])
    while q:
        r, c = q.popleft()
        if (r, c) == (n-1, n-1): return dist[(r, c)]
        for dr in (-1, 0, 1):
            for dc in (-1, 0, 1):
                nr, nc = r + dr, c + dc
                if (dr or dc) and 0 <= nr < n and 0 <= nc < n and (nr, nc) not in blocked and (nr, nc) not in dist:

                    dist[(nr, nc)] = dist[(r, c)] + 1; q.append((nr, nc))
    return -1

assert count_monotone(8) == 48639 and king_shortest(8) == 7
print(count_monotone(8), king_shortest(8), king_shortest(8, blocked={(1, 1), (2, 2), (3, 3)}))
```

Answer: captured output

```
48639 7 8
```

7.5.11 Graph colouring (S6) and its compiler connection

S6: "program graph colouring" (Shanghai compiler intern)

This is the model of **register allocation**: variables are vertices, an edge joins two variables that are live at the same time, and each of the k machine registers is a colour (Chapter 11). Deciding k-colourability is NP-complete, so compilers use heuristics (Chaitin–Briggs). Interview answer: a greedy colouring in some order, plus exact backtracking for small graphs.

```
def greedy_coloring(adj):                          # colours vertices in order of decreasing degree (Welsh-Powell)

    color = {}
    for v in sorted(adj, key=lambda x: -len(adj[x])):
        used = {color[u] for u in adj[v] if u in color}
        c = 0
        while c in used: c += 1
        color[v] = c
    return color

def can_color(adj, k):                             # exact backtracking: is a proper k-colouring possible?
    verts = sorted(adj, key=lambda x: -len(adj[x])); color = {}
    def go(i):
        if i == len(verts): return True
        v = verts[i]
        for c in range(k):
            if all(color.get(u) != c for u in adj[v]):
                color[v] = c
                if go(i + 1): return True
                del color[v]
        return False
    return go(0)
```

```
def chromatic(adj):
    k = 1
    while not can_color(adj, k): k += 1
    return k

c5 = {i: [(i - 1) % 5, (i + 1) % 5] for i in range(5)}                # odd cycle
k4 = {i: [j for j in range(4) if j != i] for i in range(4)}            # complete graph
grid = {0: [1, 3], 1: [0, 2], 2: [1, 3], 3: [2, 0]}                   # even cycle (bipartite)
assert chromatic(c5) == 3 and chromatic(k4) == 4 and chromatic(grid) == 2
print("C5:", chromatic(c5), " K4:", chromatic(k4), " C4:", chromatic(grid), " greedy(C5) uses", len(set(
    greedy_coloring(c5).values())))
```

Answer: captured output

```
C5: 3  K4: 4  C4: 2  greedy(C5) uses 3
```

7.5.12 Pair sum, anagram, palindrome, missing number: the warm-ups (S16, S20)

```
from collections import Counter

def pair_sum(a, target):                    # hash map: O(n) time; the O(n^2) double loop is the 'first attempt' (
    S20)
    seen = {}
    for i, x in enumerate(a):
        if target - x in seen: return seen[target - x], i
        seen[x] = i
    return None

def pair_sum_sorted(a, target):             # two pointers on a sorted array: O(n) time, O(1) space
    lo, hi = 0, len(a) - 1
    while lo < hi:
        s = a[lo] + a[hi]
        if s == target: return lo, hi
        lo, hi = (lo + 1, hi) if s < target else (lo, hi - 1)
    return None

def is_anagram(a, b): return Counter(a) == Counter(b)
def is_palindrome(s):
    t = [ch.lower() for ch in s if ch.isalnum()]
    return t == t[::-1]
def missing(a, n):                          # numbers 1..n with one missing: XOR (or n(n+1)/2 - sum)
    x = 0
    for i in range(1, n + 1): x ^= i
    for v in a: x ^= v
    return x

assert pair_sum([2, 7, 11, 15], 9) == (0, 1) and pair_sum_sorted([1, 3, 4, 6, 9], 10) == (0, 4)
assert is_anagram("listen", "silent") and not is_anagram("abc", "abd")
assert is_palindrome("A man, a plan, a canal: Panama") and not is_palindrome("compiler")
assert missing([1, 2, 4, 5], 5) == 3
print(pair_sum([2, 7, 11, 15], 9), pair_sum_sorted([1, 3, 4, 6, 9], 10), is_anagram("listen", "silent"),
    is_palindrome("A man, a plan, a canal: Panama"), missing([1, 2, 4, 5], 5))
```

Answer: captured output

```
(0, 1) (0, 4) True True 3
```

7.5.13 Matrix multiplication "and all applicable optimisations" (S6)

Writing the three nested loops is trivial; the interview question is the *list of optimisations* and *why*. That list — loop interchange, tiling/blocking, register blocking, vectorisation, unrolling, threading, transposing, Strassen — is in Chapter 11, with a measured experiment.

Part III
Compiler Theory

8. Compilers: Big Picture and Lexical Analysis

In this chapter

Priority ★★★. Time: 60 min (this chapter) of the 3 hours for Part III. "Phases of a compiler" is asked in managerial rounds (S2) and OA MCQs; lexing and finite automata are easy, mechanical marks. Where a worked example has a numeric answer (DFA states, token counts), a script in the book *computes* it so you can trust it.

8.1 What a compiler is, and the vocabulary ★★★

Definitions you must be able to state in one sentence each

Compiler translates a program in a source language into an equivalent program in a target language (machine code, bytecode, another high-level language) *before* it runs.

Interpreter executes the source (or an IR) directly; slower per operation but flexible. **JIT** compilers compile hot code at run time (HotSpot, V8, the NVIDIA driver's PTX JIT). **AOT** = ahead of time.

Cross-compiler runs on one machine (host) and emits code for another (target): essential for embedded/Arm/RISC-V work. **Transpiler**: source to source.

Assembler / linker / loader assembler: assembly → object file; linker: combines object files and libraries, resolving symbols and relocations → executable; loader: maps the executable into memory and starts it.

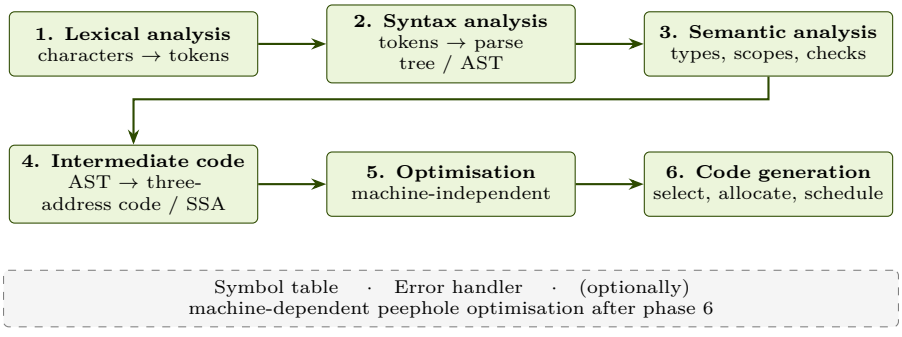
Bootstrapping writing a compiler in the language it compiles (a "self-hosting" compiler), starting from a smaller one.

Front end / middle end / back end front end = language dependent (lex, parse, semantic analysis, produce IR); middle end = IR optimisations (target independent); back end = target dependent (instruction selection, register allocation, scheduling, emission).

Why the three-part design? The $M \times N$ argument

With a common IR, supporting M languages on N targets needs M front ends + N back ends instead of $M \cdot N$ compilers. That is exactly the LLVM design (Chapter 12) and NVIDIA's CUDA pipeline: many front ends → NVVM/LLVM IR → PTX → per-architecture SASS.

8.1.1 The phases



Phase	Output	Example for `a = b + 60`
Lexical	token stream	`id(a) = id(b) + num(60)`
Syntax	parse tree / AST	`=(a, +(b, 60))`
Semantic	annotated AST	`60` converted to float if `b` is float (inserted cast)
Intermediate code	TAC	`t1 = b + 60.0; a = t1`
Optimisation	better IR	constant folding, CSE, loop opts
Code generation	target code	`movss xmm0,[b]; addss xmm0,[c60]; movss [a],xmm0`

Analysis vs. synthesis: phases 1–3 *analyse* the source (front end); 4–6 *synthesise* the target. **Phase vs. pass:** a pass is one read of the program/IR; several phases may be grouped into one pass (a one-pass compiler is possible for simple languages; optimisers are many passes).

Frequently asked: "Which phase detects . . . ?"

Illegal character or malformed number: **lexical**. Missing semicolon, unbalanced parenthesis: **syntax**. Undeclared variable, type mismatch, wrong argument count, `break` outside a loop: **semantic**. Unreachable code, division by a constant zero: usually **semantic/optimisation** warnings. Out-of-bounds index and null dereference: **run time** (compilers cannot decide them in general).

8.2 The real C toolchain, stage by stage ★★★

Everything below is real output for this four-line program.

```c
#include <stdio.h>
#define SQUARE(x) ((x) * (x))
int square_plus_one(int n) { return SQUARE(n) + 1; }
int main(void) { printf("%d\n", square_plus_one(7)); return 0; }
```

1. Preprocessing (`gcc -E`): includes and macros expanded; the compiler proper never sees the macro.

```
$ gcc -E hello.c | tail -3
```

```
# 3 "hello.c"
int square_plus_one(int n) { return ((n) * (n)) + 1; }
int main(void) { printf("%d\n", square_plus_one(7)); return 0; }
```

2. Compilation to assembly (`gcc -S`): front end + optimiser + back end. At `-O0` every variable lives in memory (stack); that is why `-O0` code is slow but debuggable.

```
$ gcc -S -O0 -masm=intel hello.c  # function square_plus_one
```

```
square_plus_one:
    push    rbp
    mov     rbp, rsp
    mov     DWORD PTR -4[rbp], edi
    mov     eax, DWORD PTR -4[rbp]
    imul    eax, eax
    add     eax, 1
    pop     rbp
    ret
```

3. Assembly to an object file (`gcc -c`), then **4. linking**: the object contains machine code and *unresolved symbols*; the linker binds `printf` to the C library.

```
$ gcc -c hello.c && nm hello.o && gcc hello.o -o hello && ./hello
```

```
0000000000000016 T main
                 U printf
0000000000000000 T square_plus_one
50
```

`T` = defined in the text section, `U` = undefined here (provided by another object/library). The five steps (preprocess, compile, assemble, link, load) are what "`gcc hello.c`" hides.

8.3 Lexical analysis ★★★

Token, lexeme, pattern

A **token** is a (name, attribute) pair handed to the parser (`id`, `num`, `if`, `+`). A **lexeme** is the actual character sequence (`count`, `60`). A **pattern** is the rule (a regular expression) describing a token's lexemes. The scanner skips whitespace and comments and enters identifiers into the symbol table.

Scanner rules examiners test

- **Longest match (maximal munch):** take the longest prefix that matches any pattern. So `x+++y` scans as `x ++ + y`, and `ifx` is one identifier, not `if` then `x`.
- **Priority:** if two patterns match the same longest lexeme (`if` matches keyword and identifier), the rule listed first wins: put keywords before identifiers.
- Regular expressions describe tokens because finite automata run in a single left-to-right pass with $O(1)$ work per character. Lexical errors: a character that starts no token; unterminated string/comment; malformed number.
- Tools: **Lex/Flex** generate a DFA-driven scanner from a rule file; hand-written scanners use `switch` on the first character. Input is read through a two-buffer scheme with sentinels so end-of-buffer checks cost one comparison.

A maximal-munch tokenizer, and the token-counting MCQ

```
import re
TOKEN = re.compile(r'''
    (?P<ws>\s+)
  | (?P<str>"(?:\\.|[^"\\])*")
  | (?P<num>\d+(?:\.\d+)?)
  | (?P<kw>(?:if|else|while|return|int)\b)       # BEFORE id, so keywords win ties
  | (?P<id>[A-Za-z_]\w*)
  | (?P<op>\+\+|--|<=|>=|==|!=|&&|\|\||->|[-+*/%<>=!&|^~?:;,.(){}\[\]])   # multi-char operators first
''', re.X)

def tokens(src):
    pos = 0
    while pos < len(src):
        m = TOKEN.match(src, pos)
        if not m: raise SyntaxError(f"lexical error at {pos}: {src[pos]!r}")
        pos = m.end()
        if m.lastgroup != "ws": yield (m.lastgroup, m.group())

for src in ['printf("%d", &i);', 'x = y+++z;', 'if (a<=b) return 1;', 'ifx = 3']:
    toks = list(tokens(src))
    print(len(toks), [t for _, t in toks])
```

Answer: captured output

```
8 ['printf', '(', '"%d"', ',', '&', 'i', ')', ';']
7 ['x', '=', 'y', '++', '+', 'z', ';']
9 ['if', '(', 'a', '<=', 'b', ')', 'return', '1', ';']
3 ['ifx', '=', '3']
```

Why: `printf("%d", &i);` has 8 tokens (`printf ( "%d" , & i ) ;`); `x = y+++z;` has 7 because `+++` is munched as `++` then `+`; `ifx` is an identifier.

8.4 Regular expressions and finite automata ★★★

Regex operator (precedence high → low)	Meaning
`*` (Kleene star), `+`, `?`	zero or more, one or more, zero or one
concatenation `ab`	a followed by b
union `a|b`	either

The equivalence cycle (and the sizes that blow up)

Regex → **NFA** (Thompson's construction: at most $2|r|$ states, linear) → **DFA** (subset construction: worst case 2^n states) → **minimal DFA** (Hopcroft, $O(n \log n)$; unique up to renaming by Myhill–Nerode). A DFA has exactly one transition per symbol per state; an NFA may have none, several, or ε-moves. They recognise the *same* class: the regular languages. **Subset construction:** DFA state = set of NFA states; start = ε-closure of the NFA start; $\delta(S, a) = \varepsilon\text{-closure}(\text{move}(S, a))$; accepting if the set contains an NFA accepting

state.

Worked example: $(a|b)^*abb$ — the textbook NFA (11 states) → DFA → minimal DFA

The script builds the Thompson-style NFA from the Dragon Book (states 0–10, accepting state 10), runs the subset construction, then minimises by partition refinement.

Subset construction and DFA minimisation for (a|b)*abb

```
eps = {0: [1, 7], 1: [2, 4], 3: [6], 5: [6], 6: [1, 7]}                    # epsilon moves
sym = {(2, 'a'): 3, (4, 'b'): 5, (7, 'a'): 8, (8, 'b'): 9, (9, 'b'): 10}   # symbol moves
start, accepting = 0, {10}

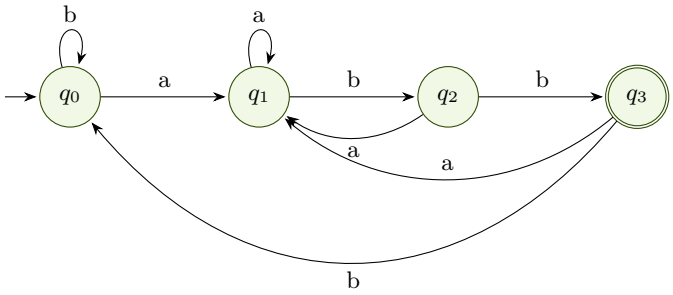
def closure(S):
    stack, out = list(S), set(S)
    while stack:
        for t in eps.get(stack.pop(), []):
            if t not in out: out.add(t); stack.append(t)
    return frozenset(out)
def move(S, a): return {sym[(s, a)] for s in S if (s, a) in sym}

A = closure({start}); names = {A: 'A'}; order = [A]; delta = {}
for S in order:                                  # 'order' grows while we iterate: BFS over DFA states
    for a in 'ab':
        T = closure(move(S, a))
        if T not in names: names[T] = chr(65 + len(names)); order.append(T)
        delta[(names[S], a)] = names[T]
for S in order:
    print(names[S], sorted(S), "accepting" if S & accepting else "")
print("DFA states:", len(order))
print({k: v for k, v in sorted(delta.items())})

# minimise: start with {accepting} / {non-accepting}, split until every block behaves uniformly
acc = {names[S] for S in order if S & accepting}
parts = [acc, set(names.values()) - acc]
changed = True
while changed:
    changed = False
    for P in list(parts):
        for a in 'ab':
            groups = {}
            for s in P:
                tgt = next(i for i, Q in enumerate(parts) if delta[(s, a)] in Q)
                groups.setdefault(tgt, set()).add(s)
            if len(groups) > 1:
                parts.remove(P); parts.extend(groups.values()); changed = True; break
        if changed: break
print("minimal DFA blocks:", sorted(sorted(P) for P in parts), "->", len(parts), "states")
```

Captured output

```
A [0, 1, 2, 4, 7]
B [1, 2, 3, 4, 6, 7, 8]
C [1, 2, 4, 5, 6, 7]
D [1, 2, 4, 5, 6, 7, 9]
E [1, 2, 4, 5, 6, 7, 10] accepting
DFA states: 5
{('A', 'a'): 'B', ('A', 'b'): 'C', ('B', 'a'): 'B', ('B', 'b'): 'D', ('C', 'a'): 'B', ('C', 'b'): 'C', ('D', 'a'):
      'B', ('D', 'b'): 'E', ('E', 'a'): 'B', ('E', 'b'): 'C'}
minimal DFA blocks: [['A', 'C'], ['B'], ['D'], ['E']] -> 4 states
```



Reading the result: the subset construction produces 5 DFA states A–E; states A and C behave identically (same transitions into the same blocks, both non-accepting), so the minimal DFA has 4 states: $q_0 = \{A, C\}$, $q_1 = B$, $q_2 = D$, $q_3 = E$. The state records "how much of `abb` has been matched as a suffix".

8.4.1 Counting minimal DFA states (the standard MCQ family)

Language over $\{a, b\}$	**Minimal DFA states**	**Why**
all strings ending in `ab`	3	remember the longest matching suffix: ε, `a`, `ab`
all strings ending in `abb`	4	as above, one more
strings with an even number of a's	2	parity
strings whose length is a multiple of 3	3	residue mod 3
the k-th symbol from the end is `a`	2^k	a DFA must remember the last k symbols (NFA needs $k+1$): the classic NFA→DFA blow-up
strings containing `aa`	3	seen nothing / last was a / found

Regular vs. not

A finite automaton has *finite* memory. $\{a^n b^n\}$, balanced parentheses, $\{ww^R\}$, and "number of a's equals number of b's" are **not regular** (pumping lemma). Lexers therefore cannot match nested constructs: that is the parser's job (Chapter 9).

8.5 Practice ★★★

Q8.1 How many tokens are in `for (i = 0; i < n; i++) sum += a[i];`?
Answer: **20** tokens: `for`, `(`, `i`, `=`, `0`, `;`, `i`, `<`, `n`, `;`, `i`, `++`, `)`, `sum`, `+=`, `a`, `[`, `i`, `]`, `;`. (`++` and `+=` are single tokens by maximal munch.)

Q8.2 Which phase reports "`error: 'x' undeclared`"? Which reports "`expected ';' before 'return'`"?
Answer: Undeclared identifier: **semantic analysis** (uses the symbol table). Missing semicolon: **syntax analysis**.

Q8.3 A minimal DFA for strings over $\{0, 1\}$ that are binary numbers divisible by 3 has how many states?
Answer: **3**: track the remainder mod 3. Reading bit b maps remainder r to $(2r + b)$ mod 3; the accepting state is remainder 0.

Q8.4 True or false: every NFA with n states has an equivalent DFA with at most 2^n states, and some need exactly that many.
Answer: True: subset construction gives the upper bound, and the "n-th symbol from the end" languages show it is tight.

Q8.5 Why must a lexer put keyword rules before the identifier rule?
Answer: Both match `while` with the same length; the tie is broken by rule order. (An alternative is to match every identifier and look it up in a keyword table.)

9. Syntax Analysis: Grammars and Parsing

In this chapter

Priority ★★★. Time: 90 min. Parsing is the most computational part of compiler theory and the standard source of numerical MCQs (FIRST/FOLLOW sets, LL(1) check, number of LR(0) states, conflict type, class of a grammar). Every table and trace in this chapter was *generated by a script* (Appendix **??**) so the numbers are right.

9.1 Context-free grammars ★★★

Vocabulary

A CFG is $G = (N, T, P, S)$: non-terminals, terminals, productions $A \to \alpha$, start symbol. A **derivation** $\Rightarrow$ rewrites a non-terminal; it is **leftmost** (always expand the leftmost non-terminal; top-down parsers find this) or **rightmost** (bottom-up parsers find it *in reverse*). A **sentential form** is any string derivable from S; a **sentence** has only terminals. A **parse tree** shows the derivation structure; the **AST** is the parse tree with syntactic noise (parentheses, chain rules) removed.

9.1.1 Ambiguity, precedence and associativity

A grammar is **ambiguous** if some sentence has two parse trees (equivalently two leftmost derivations). $E \to E + E \mid E * E \mid$ `id` is ambiguous: `id+id*id` can group as `(id+id)*id` or `id+(id*id)`. The fix encodes precedence by *layering* and associativity by the *direction of recursion*:

Ambiguous	**Unambiguous (left-assoc. $+$, $*$; $*$ binds tighter)**
$E \to E + E \mid E * E \mid (E) \mid$ `id`	$E \to E + T \mid T$; $T \to T * F \mid F$; $F \to (E) \mid$ `id`

Rules of thumb

- **Left recursion** ($E \to E + T$) makes an operator *left*-associative; **right recursion** ($E \to T \hat{} E$) makes it *right*-associative (exponentiation, assignment, `?:`).
- Operators lower in the grammar (deeper non-terminals) bind *tighter*.
- Ambiguity is undecidable in general; there is no algorithm that always removes it. The **dangling else** (`if E then S | if E then S else S`) is the classic: the usual fix attaches `else` to the nearest `if` via "matched/unmatched" statements, or the parser generator resolves the shift/reduce conflict in favour of *shift*.

9.1.2 Grammar transformations a top-down parser needs

Problem	**Transformation**	**Example**
Immediate left recursion	$A \to A\alpha \mid \beta \implies A \to \beta A',\ A' \to \alpha A' \mid \varepsilon$	$E \to E + T \mid T$ becomes $E \to TE',\ E' \to +TE' \mid \varepsilon$
Indirect left recursion	order the non-terminals $A_1..A_n$; substitute earlier ones into later ones, then remove immediate recursion	$S \to Aa,\ A \to Sb \mid c$: substitute S in A
Common prefix	left factoring: $A \to \alpha\beta_1 \mid \alpha\beta_2 \implies A \to \alpha A',\ A' \to \beta_1 \mid \beta_2$	$S \to iEtS \mid iEtSeS$ becomes $S \to iEtSS'$, $S' \to eS \mid \varepsilon$

These transformations preserve the language (not the tree shape; semantic actions need care). Left recursion is fine for *bottom-up* parsers: it uses less stack there.

9.2 Top-down parsing and LL(1) ★★★

9.2.1 Recursive descent: one function per non-terminal

Must know

Turn the left-recursive rule into a loop (EBNF): $E \to T\,\{(+|-)\,T\}$, $T \to F\,\{(*|/)\,F\}$, $F \to num \mid (E) \mid -F$. Each function consumes tokens and returns a value (or an AST node). **Left-associativity falls out of the loop.** A *right*-recursive rule for $-$ would compute $10 - (4 - 3)$ instead of $(10 - 4) - 3$.

Recursive-descent evaluator (and the right-recursion associativity bug)

```
import re

def tokenize(s): return re.findall(r"\d+|[-+*/()]", s)

class Parser:
    def __init__(self, text): self.t, self.i = tokenize(text), 0
    def peek(self): return self.t[self.i] if self.i < len(self.t) else None
    def eat(self, expect=None):
        tok = self.peek()
        if tok is None or (expect and tok != expect): raise SyntaxError(f"expected {expect!r}, got {tok!r}")
        self.i += 1; return tok
    def expr(self):                                   # E -> T { (+|-) T }      (left-associative via the loop)
        v = self.term()
        while self.peek() in ("+", "-"):
            op = self.eat(); r = self.term()
            v = v + r if op == "+" else v - r
        return v
    def term(self):                                   # T -> F { (*|/) F }
        v = self.factor()
        while self.peek() in ("*", "/"):
            op = self.eat(); r = self.factor()
            v = v * r if op == "*" else v / r
        return v
    def factor(self):                                 # F -> num | ( E ) | - F
        tok = self.peek()
        if tok == "(": self.eat("("); v = self.expr(); self.eat(")"); return v
        if tok == "-": self.eat("-"); return -self.factor()
        return int(self.eat())
    def expr_right_recursive(self):                   # WRONG on purpose: E -> T [ - E ]  (right-associative minus)

        v = self.term()
        if self.peek() == "-": self.eat(); return v - self.expr_right_recursive()
        return v

for src in ["2+3*(4-1)", "10-4-3", "2*3+4*5", "-(2+3)*2", "100/5/2"]:
    print(f"{src:12} = {Parser(src).expr()}")
print("10-4-3 with right recursion =", Parser("10-4-3").expr_right_recursive(), "(wrong: should be 3)")
```

Answer: captured output

```
2+3*(4-1)    = 11
10-4-3       = 3
2*3+4*5      = 26
-(2+3)*2     = -10
100/5/2      = 10.0
10-4-3 with right recursion = 9 (wrong: should be 3)
```

9.2.2 FIRST and FOLLOW

Definitions and rules

FIRST(α): set of terminals that can begin a string derived from α (plus ε if $\alpha \Rightarrow^* \varepsilon$). **FOLLOW($A$)**: set of terminals that can appear immediately to the right of A in some sentential form ($\$ \in$ FOLLOW(start)); **FOLLOW never contains ε.**

1. FIRST(terminal) $=$ {terminal}. For $X \to Y_1 Y_2 \ldots Y_k$: add FIRST(Y_1)\{ε}; if $\varepsilon \in$ FIRST(Y_1) add FIRST(Y_2)\{ε}, and so on; if all Y_i can vanish add ε.
2. FOLLOW: for $A \to \alpha B \beta$ add FIRST(β)\{ε} to FOLLOW(B); if $\varepsilon \in$ FIRST(β) (or β is empty) add FOLLOW(A) to FOLLOW(B). Repeat until nothing changes.

LL(1) condition

A grammar is **LL(1)** iff for every non-terminal A with alternatives $\alpha_1 \mid \alpha_2 \mid \cdots$: (i) $\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \emptyset$ for $i \neq j$; (ii) at most one alternative derives ε; (iii) if $\alpha_i \Rightarrow^* \varepsilon$ then $\text{FIRST}(\alpha_j) \cap \text{FOLLOW}(A) = \emptyset$ for $j \neq i$. Consequences: **no LL(1) grammar is left-recursive or ambiguous**, and the parse table has at most one entry per cell. The first L: scan Left-to-right; the second L: Leftmost derivation; (1): one token of lookahead.

The expression grammar made LL(1) by removing left recursion:

FIRST, FOLLOW and the LL(1) table for the transformed expression grammar

```
from lrtool import *
LL1 = """E -> T E'
E' -> + T E' | eps
T -> F T'
T' -> * F T' | eps
F -> ( E ) | id"""
g = parse_grammar(LL1)
show_grammar(g)
first, follow, _ = first_follow(g, "E")
for A in g: print("  FIRST(%s) = %-14s FOLLOW(%s) = %s" % (A, sorted(first[A]), A, sorted(follow[A])))
print_ll1_table(g, "E")
```

Captured output

```
  (1) E -> T E'
  (2) E' -> + T E'
  (3) E' -> eps
  (4) T -> F T'
  (5) T' -> * F T'
  (6) T' -> eps
  (7) F -> ( E )
  (8) F -> id
  FIRST(E) = ['(', 'id']    FOLLOW(E) = ['$', ')']
  FIRST(E') = ['+', 'eps']   FOLLOW(E') = ['$', ')']
  FIRST(T) = ['(', 'id']    FOLLOW(T) = ['$', ')', '+']
  FIRST(T') = ['*', 'eps']   FOLLOW(T') = ['$', ')', '+']
  FIRST(F) = ['(', 'id']    FOLLOW(F) = ['$', ')', '*', '+']
  M[E,(] = E -> T E'
  M[E,id] = E -> T E'
  M[E',$] = E' -> eps
  M[E',)] = E' -> eps
  M[E',+] = E' -> + T E'
  M[F,(] = F -> ( E )
  M[F,id] = F -> id
  M[T,(] = T -> F T'
  M[T,id] = T -> F T'
  M[T',$] = T' -> eps
  M[T',)] = T' -> eps
  M[T',*] = T' -> * F T'
  M[T',+] = T' -> eps
  conflicts: none  => the grammar is LL(1)
```

How to read it: an entry `M[A,a] = A -> $\alpha$` means "when A is on top of the stack and the next token is a, expand with α". The ε-productions $E' \to \varepsilon$ and $T' \to \varepsilon$ appear exactly under the tokens in $\text{FOLLOW}(E')$ and $\text{FOLLOW}(T')$: that is why FOLLOW sets matter. No cell has two entries, so the grammar is LL(1).

9.2.3 The predictive parser in action

Table-driven LL(1) parse of id + id * id

```
from lrtool import *
g = parse_grammar("""E -> T E'
E' -> + T E' | eps
T -> F T'
T' -> * F T' | eps
F -> ( E ) | id""")
ll1_parse(g, "E", ["id", "+", "id", "*", "id"])
```

Captured output

```
STACK (top at right)        INPUT              ACTION
$ E                         id + id * id $     E -> T E'
$ E' T                      id + id * id $     T -> F T'
$ E' T' F                   id + id * id $     F -> id
$ E' T' id                  id + id * id $     match id
$ E' T'                     + id * id $        T' -> eps
$ E'                        + id * id $        E' -> + T E'
$ E' T +                    + id * id $        match +
$ E' T                      id * id $          T -> F T'
$ E' T' F                   id * id $          F -> id
$ E' T' id                  id * id $          match id
$ E' T'                     * id $             T' -> * F T'
$ E' T' F *                 * id $             match *
$ E' T' F                   id $               F -> id
$ E' T' id                  id $               match id
$ E' T'                     $                  T' -> eps
$ E'                        $                  E' -> eps
$                           $                  accept
```

LL(1) checks examiners love (all verified by the script)

- $E \to E + T \mid T$: not LL(1) — **left recursion**. $S \to iEtS \mid iEtSeS \mid a$ (the dangling else): not LL(1) — common prefix, and ambiguous.
- $S \to AB,\ A \to a \mid \varepsilon,\ B \to b \mid \varepsilon$ *is* LL(1): FIRST(S)={a,b,ε}, FOLLOW(A)={b,$}, FOLLOW(B)={$}. And $S \to (S)S \mid \varepsilon$ (balanced parentheses) is LL(1).
- Palindromes $S \to aSa \mid bSb \mid a \mid b \mid \varepsilon$ cannot be parsed deterministically by *any* LL or LR parser: the middle of the string cannot be recognised with bounded lookahead (it is not a deterministic CFL).

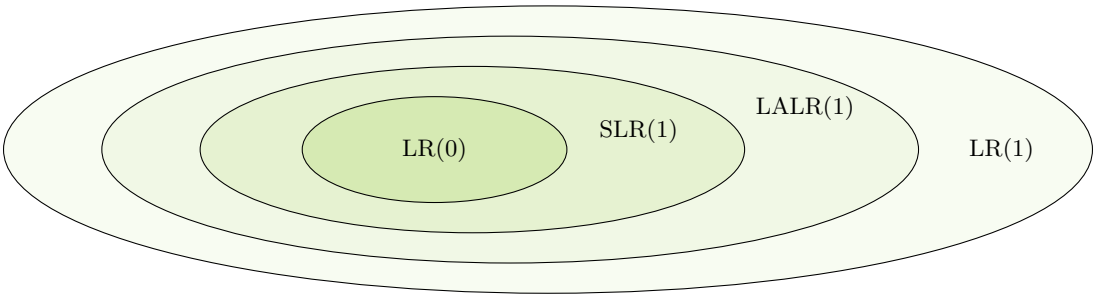
Error recovery: *panic mode* skips tokens until one in a *synchronising set* (usually FOLLOW of the current non-terminal plus statement-start keywords) is found; *phrase-level* recovery inserts or deletes tokens locally; *error productions* add known common mistakes to the grammar; *global correction* (minimum edits) is too expensive in practice.

9.3 Bottom-up parsing: LR family ★★★

Must know

A **shift-reduce** parser keeps a stack of grammar symbols. **Shift**: push the next token. **Reduce**: when the top of the stack matches the right side of a production (a **handle**), pop it and push the left side. **Accept** when the stack is the start symbol and input is exhausted. It traces a *rightmost derivation in reverse*. An LR parser decides by consulting an automaton whose states are sets of **LR items** $A \to \alpha \cdot \beta$ ("I have seen α, expect β").

Parser	Items / lookahead for a reduce	Power and size
LR(0)	items only; reduce in a state regardless of the next token	weakest; conflicts as soon as a state has a reduce plus anything else
SLR(1)	reduce $A \to \alpha$ on tokens in FOLLOW(A)	simple, small tables (same states as LR(0))
LALR(1)	merge LR(1) states that have the *same core*, keeping precise lookaheads	same number of states as SLR; **Yacc/Bison default**
LR(1) (canonical)	items carry one lookahead token $[A \to \alpha \cdot \beta,\ a]$	most powerful; many more states



LR(0) $\subsetneq$ SLR(1) $\subsetneq$ LALR(1) $\subsetneq$ LR(1) **as grammar classes**; every LL(k) grammar is LR(k); LR(1) languages = deterministic CFLs

9.3.1 The SLR(1) construction, verified

Construction recipe: (1) augment with $S' \to S$; (2) *closure*(I): while an item has the dot before a non-terminal B, add $B \to \cdot\gamma$ for all its productions; (3) *goto*(I, X): move the dot over X in every item that allows it, then close; (4) build all states from I_0; (5) **shift** on terminal a if goto exists; **reduce** $A \to \alpha$ on each $a \in$ FOLLOW(A) when $A \to \alpha\cdot$ is in the state; **accept** on \$ for $S' \to S\cdot$; **goto** entries for non-terminals. If any cell receives two actions, the grammar is not SLR(1).

SLR(1) table for E -> E+T | T ; T -> T*F | F ; F -> (E) | id

```
from lrtool import *
g = parse_grammar("""E -> E + T | T
T -> T * F | F
F -> ( E ) | id""")
show_grammar(g)
print_slr(g, "E")
```

Captured output

```
  (1) E -> E + T
  (2) E -> T
  (3) T -> T * F
  (4) T -> F
  (5) F -> ( E )
  (6) F -> id
  state | +    *    (    )    id   $    | E   T   F
      0 |           s3        s4        | 1   2   5
      1 | s6                       acc  |
      2 | r2   s7        r2        r2   |
      3 |           s3        s4        | 8   2   5
      4 | r6   r6        r6        r6   |
      5 | r4   r4        r4        r4   |
      6 |           s3        s4        |     9   5
      7 |           s3        s4        |         10
      8 | s6             s11           |
      9 | r1   s7        r1        r1   |
     10 | r3   r3        r3        r3   |
     11 | r5   r5        r5        r5   |
  conflicts: 0
```

Reading the table: `s4` = "shift and go to state 4"; `r6` = "reduce by production 6 ($F \to id$)"; the right block is GOTO. There are **12 states** and no conflicts. State 4 contains the single complete item $F \to id\cdot$ so it reduces on any token in FOLLOW(F)= $\{+, *,), \$\}$. State 2 has both $E \to T\cdot$ (reduce on FOLLOW(E)= $\{+,), \$\}$) and $T \to T \cdot *F$ (shift on `*`); the two token sets are disjoint, so there is no conflict: this is what the lookahead buys over LR(0).

LR parse of id * id + id : each reduce is one step of a rightmost derivation (in reverse)

```
from lrtool import *
g = parse_grammar("""E -> E + T | T
T -> T * F | F
F -> ( E ) | id""")
lr_parse(g, "E", ["id", "*", "id", "+", "id"])
```

Captured output

```
STATES              SYMBOLS         INPUT           ACTION
0                                   id * id + id $  shift 4
0 4                 id              * id + id $     reduce 6: F -> id
0 5                 F               * id + id $     reduce 4: T -> F
0 2                 T               * id + id $     shift 7
0 2 7               T *             id + id $       shift 4
0 2 7 4             T * id          + id $          reduce 6: F -> id
0 2 7 10            T * F           + id $          reduce 3: T -> T * F
0 2                 T               + id $          reduce 2: E -> T
0 1                 E               + id $          shift 6
0 1 6               E +             id $            shift 4
0 1 6 4             E + id          $               reduce 6: F -> id
0 1 6 5             E + F           $               reduce 4: T -> F
0 1 6 9             E + T           $               reduce 1: E -> E + T
0 1                 E               $               accept
```

The parse performed **8 reductions** (6, 4, 6, 3, 2, 6, 4, 1), which is the number of productions in the rightmost derivation; **5 shifts** (one per token). Note that `id*id` was reduced to T *before* the `+` was shifted: that is precedence at work.

9.3.2 Conflicts and the grammar classes, verified

Conflicts

Shift/reduce: a state wants to shift token a and also reduce on a (typical cause: ambiguity, e.g. dangling else, or too weak a lookahead). **Reduce/reduce:** two different complete items could be reduced on the same token (typical cause: two non-terminals deriving the same string). Parser generators: default = prefer *shift*; for reduce/reduce choose the *earlier* production. **Ambiguous grammars are never LR(k)** for any k.

Which class does each grammar belong to? (counts of states and conflicts)

```
from lrtool import *
G = {
 "expr":  ("E", "E -> E + T | T\nT -> T * F | F\nF -> ( E ) | id"),
 "LL1":   ("E", "E -> T E'\nE' -> + T E' | eps\nT -> F T'\nT' -> * F T' | eps\nF -> ( E ) | id"),
 "G2":    ("S", "S -> L = R | R\nL -> * R | id\nR -> L"),
 "G3":    ("S", "S -> a A d | b B d | a B e | b A e\nA -> c\nB -> c"),
 "dangle":("S", "S -> i E t S | i E t S e S | a\nE -> b"),
 "AB":    ("S", "S -> A B\nA -> a | eps\nB -> b | eps"),
 "paren": ("S", "S -> ( S ) S | eps"),
 "palin": ("S", "S -> a S a | b S b | a | b | eps"),
 "ambig": ("E", "E -> E + E | E * E | id"),
}
for name, (start, text) in G.items(): classify(name, text, start)
```

Captured output

```
expr    LL(1)=no  | LR(0): 12 states | SLR conflicts  0 | LR(1): 22 st,  0 confl | LALR: 12 st,  0 confl
LL1     LL(1)=yes | LR(0): 16 states | SLR conflicts  0 | LR(1): 30 st,  0 confl | LALR: 16 st,  0 confl
G2      LL(1)=no  | LR(0): 10 states | SLR conflicts  1 | LR(1): 14 st,  0 confl | LALR: 10 st,  0 confl
G3      LL(1)=no  | LR(0): 13 states | SLR conflicts  2 | LR(1): 14 st,  0 confl | LALR: 13 st,  2 confl
dangle  LL(1)=no  | LR(0): 10 states | SLR conflicts  1 | LR(1): 17 st,  1 confl | LALR: 10 st,  1 confl
AB      LL(1)=yes | LR(0):  6 states | SLR conflicts  0 | LR(1):  6 st,  0 confl | LALR:  6 st,  0 confl
paren   LL(1)=yes | LR(0):  6 states | SLR conflicts  0 | LR(1): 10 st,  0 confl | LALR:  6 st,  0 confl
palin   LL(1)=no  | LR(0):  8 states | SLR conflicts  8 | LR(1): 20 st,  8 confl | LALR:  8 st,  4 confl
ambig   LL(1)=no  | LR(0):  7 states | SLR conflicts  4 | LR(1):  7 st,  4 confl | LALR:  7 st,  4 confl
```

Grammar	Lesson
`expr`	Left-recursive, so not LL(1), but SLR(1): **12 LR(0) states, 22 canonical LR(1) states, 12 LALR states**. LALR has exactly as many states as LR(0); canonical LR(1) splits states by lookahead.
`G2`	The pointer-assignment grammar $S \to L = R \mid R,\ L \to *R \mid id,\ R \to L$ is **not SLR(1)** (one shift/reduce conflict: FOLLOW(R) contains `=`, though a reduce $R \to L$ before `=` is never valid) but **is LALR(1)** and LR(1). A famous exam example.
`G3`	$S \to aAd \mid bBd \mid aBe \mid bAe,\ A \to c,\ B \to c$ is **LR(1) but not LALR(1)**: after `ac` and `bc` the LR(1) states differ only in lookaheads, merging them creates a reduce/reduce conflict. (Not SLR(1) either.)

Grammar	Lesson
`dangle`, `ambig`	Ambiguous: conflicts in *every* LR variant.
`palin`	Inherently non-deterministic context-free language: many conflicts everywhere.

9.4 Parser generators and what real compilers do ★★

- **Yacc/Bison:** LALR(1) generator. A rule file with semantic actions (`$$ = $1 + $3;`); precedence/associativity declarations (`%left '+' '-'`, `%left '*' '/'`, `%right '^'`, `%prec` for unary minus) resolve the shift/reduce conflicts of an ambiguous expression grammar *without* the layered rewrite. Bison also offers GLR parsing for ambiguous grammars.
- **ANTLR** generates LL(*)/ALL(*) recursive-descent parsers; **PEG/packrat** parsers use ordered choice.
- **Hand-written recursive descent** is what production compilers use: Clang and GCC's C and C++ front ends are hand-written. Reasons: precise error messages and recovery, easy context-sensitive tricks (C++ needs the symbol table to know whether `a * b;` declares a pointer or multiplies), speed, and flexibility. **Pratt (precedence-climbing) parsing** handles expressions compactly (next chapter).
- C and C++ are **not** context-free as written (typedef names, templates): parsing consults the symbol table (the "lexer hack").

9.5 Formal-language facts that appear in MCQs ★★

Chomsky class	Automaton	Examples / key facts
Type 3 regular	DFA / NFA	lexical tokens; closed under union, concatenation, star, complement, intersection; pumping lemma shows $a^n b^n$ is not regular
Type 2 context-free	PDA (non-deterministic)	balanced parentheses, $a^n b^n$, palindromes; closed under union, concatenation, star; **not** closed under intersection or complement; **DPDA** $\subsetneq$ **NPDA**
Type 1 context-sensitive	linear bounded automaton	$a^n b^n c^n$ (not context-free)
Type 0 recursively enumerable	Turing machine	halting problem's yes-instances

9.6 Practice ★★★

Q9.1 Compute FIRST and FOLLOW for $S \to AB,\ A \to a \mid \varepsilon,\ B \to b \mid \varepsilon$.

```
from lrtool import *
g = parse_grammar("S -> A B\nA -> a | eps\nB -> b | eps")
first, follow, _ = first_follow(g, "S")
for A in g: print("FIRST(%s) = %-12s FOLLOW(%s) = %s" % (A, sorted(first[A]), A, sorted(follow[A])))
```

Answer: captured output

```
FIRST(S) = ['a', 'b', 'eps'] FOLLOW(S) = ['$']
FIRST(A) = ['a', 'eps'] FOLLOW(A) = ['$', 'b']
FIRST(B) = ['b', 'eps'] FOLLOW(B) = ['$']
```

Why: FIRST(S) includes `a`; since A can vanish it also includes FIRST(B)={b}; since both can vanish, $\varepsilon \in$ FIRST(S). FOLLOW(A) gets FIRST(B)$\setminus\{\varepsilon\} = \{b\}$ and, because B vanishes, FOLLOW(S)={$}.

Q9.2 How many reduce moves does an LR parser make for `id+id*id` with the expression grammar?
Answer: One reduction per production in the rightmost derivation: $F \to id$ three times, $T \to F$ twice, $T \to T * F$ once, $E \to T$ once, $E \to E + T$ once, total **8**. (Shifts: one per token, so 5.)

Q9.3 True or false: (a) every LL(1) grammar is LR(1); (b) every LR(1) grammar is LL(1); (c) LALR(1) tables are smaller than canonical LR(1) tables; (d) an ambiguous grammar can be LR(1).
Answer: (a) **True**. (b) False: the left-recursive expression grammar is LR(1) but not LL(1). (c) **True** (merging same-core states; 12 vs. 22 above). (d) False: LR(1) grammars are unambiguous.

Q9.4 Which of these cannot be recognised by a regular expression/DFA? (i) strings over $\{a, b\}$ with equal numbers of a and b; (ii) strings ending in `ab`; (iii) binary numbers divisible by 5.
Answer: Only (i): counting needs unbounded memory. (ii) and (iii) are regular (3 and 5 states respectively).

Q9.5 In an SLR(1) table, when can a state have both a shift on a and a reduce on a without being a conflict?
Answer: Never: two actions in one cell is by definition a conflict. State 2 above has a shift on `*` and reduces on `+, ), $`: different cells, so no conflict.

Q9.6 Give the left-recursion-free equivalent of $A \to Ab \mid Ac \mid d$.
Answer: $A \to dA'$, $A' \to bA' \mid cA' \mid \varepsilon$ (generates $d(b|c)^*$).

10. Semantic Analysis, Intermediate Code, SSA, Runtime

In this chapter

Priority ★★★ for IR/SSA and runtime, ★★ for SDD/types. Time: 80 min. Intermediate representations (three-address code, SSA) are the language of optimisers and of LLVM/PTX. Runtime topics (activation records, parameter passing, GC) are classic MCQ material. The chapter ends with a **working mini compiler** (Appendix **??**) that is reused in the verification lab (Chapter 18).

10.1 Syntax-directed definitions and translation ★★

Must know

An **SDD** attaches *attributes* to grammar symbols and *semantic rules* to productions. **Synthesised** attributes are computed from children (bottom-up); **inherited** attributes from parent and siblings (top-down). **S-attributed**: synthesised only (evaluable during an LR parse by reduce-time actions). **L-attributed**: inherited attributes may depend only on the parent and *left* siblings (evaluable in one left-to-right depth-first pass, so suitable for LL parsers and recursive descent). A **translation scheme (SDT)** embeds the actions in the productions at the exact point they run.

Production	Semantic rule	Kind
$E \to E_1 + T$	$E.val = E_1.val + T.val$	synthesised
$T \to T_1 * F$	$T.val = T_1.val \times F.val$	synthesised
$F \to digit$	$F.val = digit.lexval$	synthesised (S-attributed overall)
$D \to T\ L$	$L.inh = T.type$	**inherited** (declaration `int a, b, c;`)
$L \to L_1,\ id$	$L_1.inh =$ $L.inh$; addtype($id.entry, L.inh$)	inherited, L-attributed

Typical uses: evaluating constant expressions, building the AST, *type checking*, generating intermediate code, filling the symbol table with declared types.

10.2 Types, scopes and symbol tables ★★

- **Static vs. dynamic typing:** types checked at compile time (C, C++, Rust) vs. at run time (Python). **Strong vs. weak:** whether implicit unsafe conversions are allowed (C is weakly typed: `int` to pointer casts).
- **Type equivalence:** *name* equivalence (two types are the same only if they have the same declared name; C structs) vs. *structural* equivalence (same shape). **Coercion**: implicit conversion inserted by the compiler (`int` to `float` in `a + 1.5`); **cast**: explicit.
- **Overloading** (several functions, one name, resolved by argument types) vs. **polymorphism**: parametric (templates/generics), subtype (virtual functions), ad hoc (overloading). **Type inference** (Hindley–Milner, `auto`) deduces types by unification: $\lambda x.\, x + 1$ gets type $int \to int$.
- **Symbol table:** maps names to attributes (type, scope, offset, storage class). Implementation: a hash table per scope in a *stack of scopes*; `enter_scope` pushes, `exit_scope` pops, lookup searches innermost to outermost ($O(1)$ per scope on average).
- **Static (lexical) scoping:** a name refers to the binding in the textually enclosing scope (nearly all languages). **Dynamic scoping:** it refers to the most recent binding on the *call stack* (early Lisp, shell variables).

Static vs dynamic scoping: x=10; f(){print x}; g(){x=20; f()}; g()

```
def run(dynamic):
    stack = [{"x": 10}]                          # the global frame
    out = []
    def lookup(name):
        frames = reversed(stack) if dynamic else [stack[0]]   # dynamic: search the call stack; static: f is
     defined at global level
        for fr in frames:
            if name in fr: return fr[name]
    def f(): out.append(lookup("x"))
    def g():
        stack.append({"x": 20}); f(); stack.pop()
    g()
    return out[0]
print("static scoping prints", run(False), "| dynamic scoping prints", run(True))
```

Answer: captured output

```
static scoping prints 10 | dynamic scoping prints 20
```

10.3 Intermediate representations ★★★

Form	Description
AST	tree of the program's structure; the front end's product; basis for semantic analysis and many source-level tools
DAG	AST with *shared* common subexpressions; detects redundancy inside a basic block
Postfix / stack code	operands then operator (`a b c * +`); bytecodes (JVM, CPython), WebAssembly
Three-address code (TAC)	at most one operator on the right: `x = y op z`; explicit temporaries; the standard optimiser IR
SSA	TAC in which every variable is assigned *exactly once*; merges use φ functions; LLVM IR, GCC GIMPLE-SSA, NVVM
Machine-level IR	instructions with virtual registers (LLVM MIR, GCC RTL); PTX is a *virtual-register ISA*

Quadruples, triples, indirect triples for `a = b * -c + b * -c`

Quadruples (op, arg1, arg2, result)					**Triples** (op, arg1, arg2)				**Indirect**	
0	minus	c		t1	0	minus	c		35	(0)
1	*	b	t1	t2	1	*	b	(0)	36	(1)
2	minus	c		t3	2	minus	c		37	(2)
3	*	b	t3	t4	3	*	b	(2)	38	(3)
4	+	t2	t4	t5	4	+	(1)	(3)	39	(4)
5	=	t5		a	5	=	a	(4)	40	(5)

Quadruples name their results (easy to move/delete instructions; optimisers prefer them); **triples** refer to results by position (compact: 3 fields, but moving an instruction invalidates references); **indirect triples** add a list of pointers to the triples so reordering only permutes the list. A **DAG** would build `minus c` and `b * minus c` once and give the `+` node the same child twice.

Address calculations and control-flow translation

- Row-major array `A[n1][n2]` with element width w: **addr(A[i][j]) = base + (i × n2 + j) × w**. Example: `int A[10][20]`, `&A[3][4]` = base $+(3 \cdot 20 + 4) \cdot 4$ = base $+256$. Column-major (Fortran): base $+ (j \cdot n_1 + i) \cdot w$.
- **Boolean expressions** are translated either to *numerical* values (0/1) or, in conditions, to **jumping code**: `a < b && c < d` becomes two conditional jumps with no `&&` instruction at run time (short-circuit evaluation). **Backpatching** fills in jump targets in one pass: emit jumps with unknown targets onto lists (`makelist`), combine lists (`merge`), and patch them (`backpatch`) once the target label is known.

- `switch` becomes a jump table (dense cases) or a binary decision tree/hash (sparse cases). Procedure calls: `param x` instructions then `call f, n`.

10.4 Static single assignment (SSA) ★★★

Must know

In SSA each variable has exactly *one* definition. Where two control-flow paths merge, a φ function selects the value by the incoming edge: $x_3 = \varphi(x_1, x_2)$. SSA makes def–use chains explicit, which makes constant propagation, dead-code elimination, value numbering and register allocation simpler and faster. **Construction (Cytron et al.):** (1) compute the dominator tree (node d *dominates* n if every path from entry to n passes through d); (2) compute **dominance frontiers**: DF(d) = nodes where d's dominance *ends* (successors of nodes d dominates, which d does not strictly dominate); (3) insert a φ for variable v at the *iterated* DF of every block that defines v; (4) **rename** by a dominator-tree walk with a stack of current versions.

The script below applies this to the TAC that the mini compiler produced for a loop (the exact program is in Section 10.6).

Dominators, dominance frontiers, phi placement and SSA renaming

```
import ssa
ssa.report("""n = 10; s = 0; i = 1;
while (i <= n) {
  if (i % 2 == 0 && i > 2) { s = s + i * 2; } else { s = s + i; }
  i = i + 1;
}
print(s);""")
```

Captured output

```
Basic blocks and successors:
  B0: n = 10; s = 0; i = 1                        -> [1]
  B1: L1:; if i > n goto L3                       -> [7, 2]
  B2: t1 = i % 2; if t1 != 0 goto L5              -> [5, 3]
  B3: if i <= 2 goto L5                           -> [5, 4]
  B4: t2 = i * 2; s = s + t2; goto L6             -> [6]
  B5: L5:; s = s + i                              -> [6]
  B6: L6:; i = i + 1; goto L1                     -> [1]
  B7: L3:; print s                                -> []
immediate dominators: {1: 0, 2: 1, 3: 2, 4: 3, 5: 2, 6: 2, 7: 1}
dominance frontiers : {1: [1], 2: [1], 3: [5, 6], 4: [6], 5: [6], 6: [1]}
phi functions needed: {'B1': ['s', 'i'], 'B6': ['s']}

SSA form:
  B0:
      n1 = 10
      s1 = 0
      i1 = 1
  B1:
      s2 = phi(s1 [B0], s5 [B6])
      i2 = phi(i1 [B0], i3 [B6])
      if i2 > n1 goto B7
  B2:
      t1 = i2 % 2
      if t1 != 0 goto B5
  B3:
      if i2 <= 2 goto B5
  B4:
      t2 = i2 * 2
      s3 = s2 + t2
      goto B6
  B5:
      s4 = s2 + i2
  B6:
      s5 = phi(s3 [B4], s4 [B5])
      i3 = i2 + 1
      goto B1
  B7:
      print s2
```

Reading the output

- **Loop header** B1 is a join of the entry edge (B0) and the back edge (B6): both `s` and `i` get a φ there (s_2, i_2). Every loop-carried variable needs one.
- B6 is a join of B4 and B5 (the two arms of the `if`); `s` was defined in both, so $s_5 = \varphi(s_3, s_4)$. `i` is not modified in the arms, so it needs no φ at B6.
- $\mathrm{DF}(B3) = \{B5, B6\}$: B3 dominates B4 but not B5 (also reachable from B2), so its dominance ends there. $\mathrm{DF}(B6) = \{B1\}$ is the back edge.
- Temporaries (`t1`, `t2`) are already single-assignment and need no renaming.

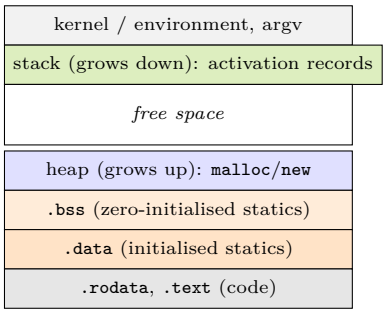
SSA gotchas

φ functions are not real instructions: they are eliminated before code generation by inserting copies on the incoming edges ("out-of-SSA"), and naive copy insertion causes the *lost-copy* and *swap* problems (parallel copies need a temporary). In LLVM, front ends emit `alloca`/`load`/`store` for locals and the `mem2reg`/`sroa` pass promotes them to SSA registers using exactly this algorithm (Chapter 12). *Minimal* SSA inserts a φ wherever DF says; *pruned* SSA also drops φs for variables that are dead at the join.

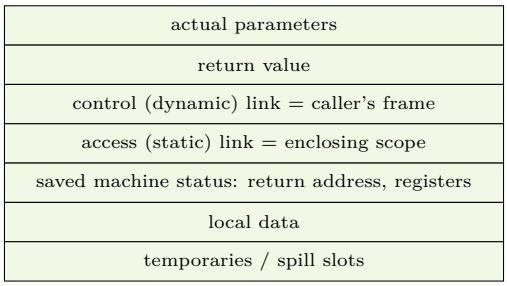
10.5 Runtime environments ★★★

10.5.1 Storage organisation and activation records

Process memory (high → low)



One activation record (frame)



Must know

- A **calling sequence** splits work between *caller* (evaluate and pass arguments, save caller-saved registers, `call`) and *callee* (prologue: push frame pointer, allocate locals; epilogue: restore, return). Frames form a stack because calls nest; a frame lives exactly as long as its call.
- **Nested procedures** (Pascal, Python closures, GCC nested functions) need the **access link** (follow k links to reach a variable k scopes out) or a **display** (array indexed by nesting depth: $O(1)$ access, but updated on every call). C has no nested functions, so it needs neither.
- **Tail-call optimisation:** a call in tail position (`return f(x);`) can reuse the caller's frame, turning recursion into a loop; guaranteed in Scheme, optional in C/C++ compilers, absent in CPython.
- **Stack overflow** = unbounded recursion; **dangling reference** = pointer into a frame that has returned.

10.5.2 Parameter-passing mechanisms: the classic `swap(i, a[i])` question

Mechanism	Semantics
call by value	callee gets a *copy* of the value; changes are invisible to the caller (C, Java primitives)
call by reference	callee gets the *address* (alias); the address of the actual is computed **once, at the call** (C++ `T&`, Fortran)
copy-restore (value-result)	copy in at call, copy out at return to the addresses fixed at the call
call by name	the actual parameter is a *thunk* re-evaluated at **every use** in the caller's environment (Algol 60, macros; Jensen's device)

swap(x, y) called as swap(i, a[i]) with i=1, a[1]=10, a[2]=20

```
def fresh(): return {"i": 1}, {1: 10, 2: 20, 10: 99}      # a[10] exists only so call-by-name has somewhere to
    write

def by_value():
    s, a = fresh(); x, y = s["i"], a[s["i"]]; t = x; x = y; y = t     # callee works on copies
    return s["i"], a
def by_reference():
    s, a = fresh(); idx = s["i"]                                       # address of a[i] fixed at the call
    t = s["i"]; s["i"] = a[idx]; a[idx] = t
    return s["i"], a
def by_value_result():
    s, a = fresh(); idx = s["i"]; x, y = s["i"], a[idx]                # copy in
    t = x; x = y; y = t
    s["i"] = x; a[idx] = y                                             # copy out to the addresses fixed at the
    call
    return s["i"], a
def by_name():
    s, a = fresh()
    x_get = lambda: s["i"];     x_set = lambda v: s.__setitem__("i", v)
    y_get = lambda: a[s["i"]];  y_set = lambda v: a.__setitem__(s["i"], v)   # a[i] re-evaluated at every use
    t = x_get(); x_set(y_get()); y_set(t)
    return s["i"], a

for name, f in (("value", by_value), ("reference", by_reference), ("value-result", by_value_result), ("name",
    by_name)):
    i, a = f(); print("%-13s i=%-2d a[1]=%-2d a[2]=%-2d a[10]=%d" % (name, i, a[1], a[2], a[10]))
```

Answer: captured output

```
value         i=1  a[1]=10 a[2]=20 a[10]=99
reference     i=10 a[1]=1  a[2]=20 a[10]=99
value-result  i=10 a[1]=1  a[2]=20 a[10]=99
name          i=10 a[1]=10 a[2]=20 a[10]=1
```

Why call-by-name differs: after `x = y` sets `i` to 10, the later `y = t` re-evaluates `a[i]` as `a[10]`, so it writes the wrong element, while by reference the address `a[1]` was frozen at the call.

10.5.3 Heap management and garbage collection

Technique	Key facts
Free-list allocation	first fit (fast), best fit (least waste, slow, tiny fragments), worst fit; **coalescing** adjacent free blocks fights *external* fragmentation; rounding sizes causes *internal* fragmentation; segregated lists (size classes) are the practical choice
Reference counting	count pointers to each object, free at zero; immediate, incremental; **cannot reclaim cycles**; overhead on every pointer store (CPython, Swift ARC, `shared_ptr`)
Mark-and-sweep	mark everything reachable from roots (stack, registers, globals), sweep the rest; handles cycles; cost $\propto$ heap size; leaves fragmentation; "stop the world"
Copying (semi-space, Cheney)	copy live objects to a fresh space; cost $\propto$ live data; compacts, allocation is a pointer bump; wastes half the memory
Generational	most objects die young: collect the nursery often, the old generation rarely; needs a write barrier
Conservative (Boehm)	treats any word that looks like a pointer as one; works for C/C++ but can retain garbage

10.6 Lab: a complete mini compiler in 250 lines ★★

Interviewers who ask about "compiler projects" love candidates who built the whole pipeline once. This one (source in Appendix **??**) is also the *system under test* in Chapter 18. Language: integer variables, arithmetic, comparisons, `&& || !`, `if/else`, `while`, `print`; division truncates toward zero and division by zero traps. **Pipeline:** regex lexer → recursive descent + precedence climbing parser → AST → TAC with jumping code → optimiser (constant folding/propagation, algebraic simplification, local CSE, dead-code elimination, jump clean-up, branch inversion, temp coalescing) → TAC virtual machine; a separate **reference interpreter** over the AST serves as the semantic oracle.

Front end: tokens and AST (note how precedence shapes the tree)

```
import tiny
print(tiny.lex("x = 1 + 2 * 3;")[:-1])
print(tiny.parse("x = 1 + 2 * 3;"))
print(tiny.parse("y = a - b - c;"))          # left-associative: (a - b) - c
```

Captured output

```
[('id', 'x'), ('op', '='), ('num', 1), ('op', '+'), ('num', 2), ('op', '*'), ('num', 3), ('op', ';')]
('block', [('assign', 'x', ('bin', '+', ('num', 1), ('bin', '*', ('num', 2), ('num', 3))))])
('block', [('assign', 'y', ('bin', '-', ('bin', '-', ('var', 'a'), ('var', 'b')), ('var', 'c')))])
```

TAC with jumping code for the loop, before and after optimisation

```
import tiny
SRC = """n = 10; s = 0; i = 1;
while (i <= n) {
  if (i % 2 == 0 && i > 2) { s = s + i * 2; } else { s = s + i; }
  i = i + 1;
}
print(s);"""
raw = tiny.compile_source(SRC, opt=False)
opt = tiny.compile_source(SRC, opt=True)
print("=== raw TAC (%d instructions)" % len([i for i in raw if i[0] != "label"]));       print(tiny.show(raw))
print("=== optimised TAC (%d instructions)" % len([i for i in opt if i[0] != "label"])); print(tiny.show(opt))
ref = tiny.interpret(tiny.parse(SRC))
r1, r2 = tiny.run_tac(raw), tiny.run_tac(opt)
print("reference interpreter:", ref)
print("raw TAC on the VM    :", r1[:2], "executed", r1[2], "instructions")
print("optimised TAC on VM  :", r2[:2], "executed", r2[2], "instructions")
```

Captured output

```
=== raw TAC (20 instructions)
    n = 10
    s = 0
    i = 1
L1:
    if i <= n goto L2
    goto L3
L2:
    t1 = i % 2
    if t1 == 0 goto L7
    goto L5
L7:
    if i > 2 goto L4
    goto L5
L4:
    t2 = i * 2
    t3 = s + t2
    s = t3
    goto L6
L5:
    t4 = s + i
    s = t4
L6:
    t5 = i + 1
    i = t5
    goto L1
L3:
    print s
=== optimised TAC (14 instructions)
    n = 10
    s = 0
    i = 1
L1:
    if i > n goto L3
    t1 = i % 2
    if t1 != 0 goto L5
    if i <= 2 goto L5
    t2 = i * 2
    s = s + t2
    goto L6
L5:
```

```
    s = s + i
L6:
    i = i + 1
    goto L1
L3:
    print s
reference interpreter: ([83], None)
raw TAC on the VM    : ([83], None) executed 105 instructions
optimised TAC on VM  : ([83], None) executed 78 instructions
```

Observe: (1) `i % 2 == 0 && i > 2` became two conditional jumps: there is no `&&` at run time. (2) After branch inversion the pattern `if c goto A; goto B; A:` became `if !c goto B`. (3) Temporaries were coalesced (`t3 = s + t2; s = t3` became `s = s + t2`). (4) Same output (83) from all three executors, with fewer executed instructions.

Local CSE and constant folding

```
import tiny
cse = "a = b * -c + b * -c; print(a);"
print("--- CSE: raw");  print(tiny.show(tiny.compile_source(cse, opt=False)))
print("--- CSE: optimised (the second b * -c is reused: the DAG idea)"); print(tiny.show(tiny.compile_source(cse))
    )
print(tiny.interpret(tiny.parse(cse), env0={"b": 3, "c": 4}), tiny.run_tac(tiny.compile_source(cse), env0={"b": 3,
     "c": 4})[:2])
cf = "x = 2 * 3 + 4; y = x * 1; z = y + 0; print(z / 2); if (x > 5 || z < 0) { print(1); } else { print(0); }"
print("--- constant folding + dead branch: raw has %d instructions, optimised:" % len([i for i in tiny.
    compile_source(cf, opt=False) if i[0] != "label"]))
print(tiny.show(tiny.compile_source(cf)))
```

Captured output

```
--- CSE: raw
    t1 = -c
    t2 = b * t1
    t3 = -c
    t4 = b * t3
    t5 = t2 + t4
    a = t5
    print a
--- CSE: optimised (the second b * -c is reused: the DAG idea)
    t1 = -c
    t2 = b * t1
    t5 = t2 + t2
    print t5
([-24], None) ([-24], None)
--- constant folding + dead branch: raw has 16 instructions, optimised:
    print 5
    print 1
```

10.7 Practice ★★★

Q10.1 For the grammar $E \to E + T \mid T$, $T \to T * F \mid F$, $F \to \mathit{digit}$ with $F.val = \mathit{digit.lexval}$ etc., what is `E.val` for `3*5+4`?
Answer: 19: $T.val = 3 \cdot 5 = 15$, $E.val = 15 + 4$. (S-attributed: computed bottom-up.)

Q10.2 Which is true: (a) synthesised attributes may depend on siblings; (b) L-attributed SDDs can be evaluated during top-down parsing; (c) every S-attributed SDD is L-attributed?
Answer: (b) and (c). (a) is false: synthesised attributes depend on children (and the node itself) only.

Q10.3 Translate `x = a[i] + b` to three-address code if `a` is an `int` array.
Answer: `t1 = i * 4; t2 = a[t1]; t3 = t2 + b; x = t3`. (Scaling by the element width; in real IRs this is an address computation then a load.)

Q10.4 A procedure is called with arguments by name vs. reference: when do results differ?
Answer: When the actual parameter's *expression* depends on a variable that the procedure modifies (`a[i]` with `i` changed): by name re-evaluates it at each use; by reference fixed the address at the call (see the swap output).

Q10.5 Why does SSA place φ functions at dominance frontiers?
Answer: A definition of v in block B reaches every block B dominates unambiguously. Just outside that region (the dominance frontier) another definition may also reach, so the value becomes path-dependent: exactly where a φ is needed. Iterating covers the new definitions created by the φs themselves.

Q10.6 Which GC can reclaim a cycle of two objects that point to each other: reference counting or mark-and-sweep?
Answer: Mark-and-sweep (the cycle is unreachable from roots). Reference counting leaves both counts at 1 (the `shared_ptr` cycle leak seen in Chapter 5).

11. Optimisation, Code Generation and Linking

In this chapter

Priority ★★★. Time: 100 min. This is where compiler interviews concentrate (S2, S4, S6, S8, S10): loop unrolling, inlining, data-flow and dependence analysis, pointer analysis, matrix-multiplication optimisations, graph-colouring register allocation. Every analysis result below is produced by code in the book (Appendices **??**–**??**); the experiments (assembly, vectoriser remarks, timings, linker errors) are real tool output.

11.1 What an optimiser is allowed to do ★★★

Must know

- **The as-if rule:** the compiler may transform the program in any way as long as the *observable behaviour* (I/O, volatile accesses, and the results of well-defined programs) is unchanged. **Legality** = the analysis proves the transformation safe; **profitability** = a cost model predicts it helps. Many bugs are legality bugs (wrong analysis); many missed speed-ups are profitability judgements.
- **UB is the optimiser's licence:** because signed overflow, out-of-bounds access and aliasing violations "cannot happen", loops can be widened, checks removed, loads reordered (Chapter 4, Section 4.4).
- **Levels (GCC/Clang):** `-O0` none (debuggable); `-O1` cheap clean-ups; `-O2` the default for releases (no large code growth); `-O3` adds aggressive loop transforms, vectorisation, larger inlining; `-Os`/`-Oz` optimise for size; `-Ofast` = `-O3` plus `-ffast-math` (breaks IEEE semantics: assumes no NaN/Inf, allows reassociation); `-Og` GCC's debug-friendly level.
- **Phase ordering problem:** optimisations enable and disable each other (inlining enables constant propagation, which enables dead-code elimination, which shrinks the function so it is inlined elsewhere); no single order is best, so pipelines run *cleanup* passes repeatedly. Whether a transformation is machine-independent (on IR) or machine-dependent (peephole, scheduling, register allocation) decides where it sits.

11.2 Local optimisations (within a basic block) ★★★

Optimisation	**Example**
Constant folding	`x = 2 * 3 + 4` → `x = 10`; careful: fold with the *target's* semantics (integer division, overflow, floating-point rounding)
Constant / copy propagation	after `y = x; z = y + 1` use `x` directly; after `c = 5` replace uses of `c` by 5
Algebraic simplification	`x+0`, `x*1`, `x*0`, `x-x`, `x&0`; (not for floating point without `-ffast-math`: `x*0` may be NaN)
Strength reduction	replace an expensive operation by a cheaper equivalent: `x*8` → shift/`lea`, `x/2^k` → shift with fix-up, `x%16` → `and`
Common subexpression elimination (CSE) / **value numbering**	compute `b * -c` once and reuse; value numbering assigns equal numbers to equal computations using a hash table
Dead-code elimination	delete instructions whose results are never used and have no side effects; *dead stores*; unreachable blocks
Peephole optimisation	slide a small window over generated code: remove `mov r,r`, jump-to-next, `push/pop` pairs, replace sequences by one idiom

Strength reduction in real compiler output (GCC `-O2`, x86-64). Note that signed `x/2` is *not* a plain shift: negative odd values must round toward zero.

```
int mul9(int x)             { return x * 9; }
int mul8(int x)             { return x * 8; }
unsigned udiv8(unsigned x)  { return x / 8; }
int sdiv2(int x)            { return x / 2; }
unsigned umod16(unsigned x) { return x % 16; }
```

```c
int times1000(int x)        { return x * 1000; }
```

```
$ gcc -O2 -S -masm=intel sr.c
```

```
mul9:
    lea    eax, [rdi+rdi*8]
    ret
mul8:
    lea    eax, 0[0+rdi*8]
    ret
udiv8:
    mov    eax, edi
    shr    eax, 3
    ret
sdiv2:
    mov    eax, edi
    shr    eax, 31
    add    eax, edi
    sar    eax
    ret
umod16:
    mov    eax, edi
    and    eax, 15
    ret
times1000:
    imul    eax, edi, 1000
    ret
```

Reading it: `x*9` is `x + 8x`, one `lea`; `x*8` is a scaled address computation; unsigned `x/8` is `shr 3`; `x%16` is `and 15`; signed `x/2` adds the sign bit (`shr 31`) before the arithmetic shift (`sar`) to round toward zero; `x*1000` stays a real `imul` (no cheaper form). This is also where *compiler verification* lives: every one of these rewrites must be exactly right for *all* inputs, including `INT_MIN`.

11.3 Data-flow analysis ★★★

The framework

A data-flow problem assigns each basic block B a set $IN[B]$ and $OUT[B]$ satisfying equations built from per-block **gen**/**kill** sets. The solution is the *fixed point* of iterating them; it terminates because the transfer functions are *monotone* on a lattice of finite height. Processing blocks in **reverse post-order** (forward problems) or post-order (backward) converges in few passes (roughly the loop-nesting depth plus two).

Analysis	Direction	Meet	Equations / used for
Reaching definitions	forward	$\cup$ (may)	$OUT = gen \cup (IN - kill)$, $IN = \bigcup_P OUT[P]$; constant/copy propagation, loop invariants, uninitialised-use warnings
Live variables	backward	$\cup$ (may)	$IN = use \cup (OUT - def)$, $OUT = \bigcup_S IN[S]$; **register allocation**, dead-code elimination
Available expressions	forward	$\cap$ (must)	$OUT = gen \cup (IN - kill)$, $IN = \bigcap_P OUT[P]$, init all non-entry to U; **global CSE**
Very busy expressions	backward	$\cap$ (must)	code hoisting, partial-redundancy elimination (PRE)

May analyses ask "is it *possible*?" (union at joins; initialise to $\emptyset$); **must** analyses ask "is it *certain* on every path?" (intersection; initialise to the universe).

Reaching definitions and live variables on the loop CFG (iterated to a fixed point)

```
import dataflow
dataflow.report("""n = 10; s = 0; i = 1;
while (i <= n) {
  if (i % 2 == 0 && i > 2) { s = s + i * 2; } else { s = s + i; }
  i = i + 1;
}
```

```
print(s);""", which=("rd", "live"))
```

Captured output

```
Basic blocks:
  B0: n = 10; s = 0; i = 1                          -> [1]
  B1: L1:; if i > n goto L3                         -> [7, 2]
  B2: t1 = i % 2; if t1 != 0 goto L5                -> [5, 3]
  B3: if i <= 2 goto L5                             -> [5, 4]
  B4: t2 = i * 2; s = s + t2; goto L6               -> [6]
  B5: L5:; s = s + i                                -> [6]
  B6: L6:; i = i + 1; goto L1                       -> [1]
  B7: L3:; print s                                  -> []

Reaching definitions (forward, union) -- stable after 3 passes
  definitions: d1:n@B0  d2:s@B0  d3:i@B0  d4:t1@B2  d5:t2@B4  d6:s@B4  d7:s@B5  d8:i@B6
  B0  gen={d1,d2,d3}   kill={d6,d7,d8}       IN={}                     OUT={d1,d2,d3}
  B1  gen={}           kill={}               IN={d1,d2,d3,d4,d5,d6,d7,d8} OUT={d1,d2,d3,d4,d5,d6,d7,d8}
  B2  gen={d4}         kill={}               IN={d1,d2,d3,d4,d5,d6,d7,d8} OUT={d1,d2,d3,d4,d5,d6,d7,d8}
  B3  gen={}           kill={}               IN={d1,d2,d3,d4,d5,d6,d7,d8} OUT={d1,d2,d3,d4,d5,d6,d7,d8}
  B4  gen={d5,d6}      kill={d2,d7}          IN={d1,d2,d3,d4,d5,d6,d7,d8} OUT={d1,d3,d4,d5,d6,d8}
  B5  gen={d7}         kill={d2,d6}          IN={d1,d2,d3,d4,d5,d6,d7,d8} OUT={d1,d3,d4,d5,d7,d8}
  B6  gen={d8}         kill={d3}             IN={d1,d3,d4,d5,d6,d7,d8} OUT={d1,d4,d5,d6,d7,d8}
  B7  gen={}           kill={}               IN={d1,d2,d3,d4,d5,d6,d7,d8} OUT={d1,d2,d3,d4,d5,d6,d7,d8}

Live variables (backward, union) -- stable after 3 passes
  B0  use={}         def={i,n,s}    IN={}             OUT={i,n,s}
  B1  use={i,n}      def={}         IN={i,n,s}        OUT={i,n,s}
  B2  use={i}        def={t1}       IN={i,n,s}        OUT={i,n,s}
  B3  use={i}        def={}         IN={i,n,s}        OUT={i,n,s}
  B4  use={i,s}      def={s,t2}     IN={i,n,s}        OUT={i,n,s}
  B5  use={i,s}      def={s}        IN={i,n,s}        OUT={i,n,s}
  B6  use={i}        def={i}        IN={i,n,s}        OUT={i,n,s}
  B7  use={s}        def={}         IN={s}            OUT={}
```

How to read it

- d_2 (`s = 0`) reaches the loop header B1 (first iteration) but is killed on *both* arms of the `if` (B4 defines d_6, B5 defines d_7), so it is absent from $IN[B6]$. Every in-loop definition reaches B1 through the back edge.
- Live variables flow *backwards*: B7 uses `s`, so `s` is live out of B1; `n` is live throughout the loop because B1 re-reads it; `t1` and `t2` are never live across a block boundary: they could live in a register that is reused. Both analyses stabilised in 3 passes.

Available expressions and global CSE. In the program below `a + b` is computed in B0, again on one branch, and again at the join. Is the third computation redundant?

```
import dataflow
dataflow.report("""x = a + b;
if (c) { y = a + b; } else { z = a * b; }
w = a + b;
print(x + y + z + w);""", which=("avail",))
```

Captured output

```
Basic blocks:
  B0: x = a + b; if c == 0 goto L2                  -> [2, 1]
  B1: y = a + b; goto L3                            -> [3]
  B2: L2:; z = a * b                                -> [3]
  B3: L3:; t4 = a + b; t5 = x + y; t6 = t5 + z; t7 = t6 + t4; print t7 -> []

Available expressions (forward, intersection) -- stable after 2 passes
  B0  gen={a + b}              IN={}                    OUT={a + b}
  B1  gen={a + b}              IN={a + b}               OUT={a + b}
  B2  gen={a * b}              IN={a + b}               OUT={a * b, a + b}
  B3  gen={a + b, t5 + z, t6 + t4, x + y} IN={a + b}              OUT={a + b, t5 + z, t6 + t4, x + y}
```

Why: $IN[B3] = OUT[B1] \cap OUT[B2] = \{a+b\} \cap \{a*b,\ a+b\} = \{a+b\}$. The expression is available on *every* path into the join, so the third `a + b` can be replaced by a copy of the earlier value (global CSE). `a * b` is not available (only one path computes it). Local value numbering alone could not see this because the computations live in different blocks.

Facts to quote

Bit-vector frameworks (the four above) are *distributive*, so the iterative *maximal fixed point* equals the *meet-over-all-paths* ideal. **Constant propagation** is monotone but *not* distributive (the join of x=2, y=3 and x=3, y=2 loses x+y=5), so it needs a richer lattice (constant / $\top$ / $\bot$). Iteration order matters for speed, never for the result. Inter-procedural data-flow needs call-graph and context-sensitivity decisions.

11.4 Loop optimisations and dependence analysis ★★★

Natural loops

A **back edge** is an edge $n \to h$ where h dominates n. The **natural loop** of that edge is h plus all nodes that can reach n without passing through h; h is the **header**. Loops are the hot spots, so most optimisations target them. A **pre-header** block before the header is where hoisted code is placed.

Transformation	What it does, when it is legal, why it helps
Loop-invariant code motion (LICM)	hoist computations whose operands do not change in the loop to the pre-header. Legal if the instruction cannot trap or execute speculatively unsafely (e.g. a division needs a guard), and its value is not used before definition
Induction-variable strength reduction / elimination	replace `a[i*4]` address arithmetic by a pointer incremented by 4 per iteration; eliminate a second induction variable that tracks the first
Loop unrolling	replicate the body k times (plus a remainder loop). *Helps:* fewer branch and counter instructions, more instruction-level parallelism, enables scheduling, CSE across iterations and vectorisation. *Costs:* larger code (I-cache pressure), more register pressure. Compiler decides by trip count and body size
Loop peeling / unswitching	peel the first iterations (to remove an iteration-dependent special case); move a loop-invariant `if` outside the loop (duplicating the loop)
Loop fusion / fission	merge adjacent loops over the same range (fewer passes over memory) / split a loop (enables vectorising part of it, reduces pressure)
Loop interchange	swap nested loop order to make the innermost loop access memory with stride 1 (legal only if no dependence direction vector is reversed)
Tiling (blocking)	split iteration space into blocks that fit in cache/registers/shared memory, reusing data before eviction
Vectorisation	turn n scalar iterations into SIMD operations; needs no loop-carried dependence (or a recognised reduction) and known/checked aliasing
Software pipelining	overlap iterations in the schedule (VLIW/DSP, GPUs' asynchronous copies)

Natural loop and loop-invariant instruction detection (uses reaching definitions)

```
import dataflow
dataflow.report("""a = k + 1; b = 7; s = 0; i = 0;
while (i < n) { c = a * b; s = s + c + i; i = i + 1; }
print(s);""", which=("loops",))
```

Captured output

```
Basic blocks:
  B0: a = k + 1; b = 7; s = 0; i = 0                -> [1]
  B1: L1:; if i >= n goto L3                        -> [3, 2]
  B2: t2 = a * b; t3 = s + t2; s = t3 + i; i = i + 1; goto L1 -> [1]
  B3: L3:; print s                                  -> []

Natural loop with header B1, body [1, 2]
  loop-invariant instructions: ['t2 = a * b  (B2)']
```

Why: `t2 = a * b` is invariant: `a` and `b` have all their reaching definitions *outside* the loop (B0). `s + t2` and `i + 1` are not: `s` and `i` are redefined inside the loop. LICM would move the multiplication to the pre-header, saving one instruction per iteration. `k` and `n` are program inputs (no definition at all), hence invariant.

11.4.1 Dependences decide legality

Must know

For two statements touching the same memory: **flow (true, RAW)**: write then read; **anti (WAR)**: read then write; **output (WAW)**: write then write. Only *flow* dependences carry real data; anti/output can be removed by renaming. A dependence is **loop-carried** if source and sink are in different iterations; its **distance** is the iteration difference (the direction vector gives the sign). A loop can be *parallelised or vectorised* when it has no loop-carried dependence (reductions excepted). **Loop interchange** is legal iff no dependence's direction vector becomes lexicographically negative. **GCD test:** for `a[c1*i+k1] = ...  a[c2*j+k2]`, a dependence is possible only if $\gcd(c_1, c_2) \mid (k_2 - k_1)$. It ignores loop bounds, so it may report a dependence that does not exist, but never misses one (a *conservative* test).

GCD test versus brute-force enumeration of the iteration space

```
from math import gcd
def gcd_test(c1, k1, c2, k2): return (k2 - k1) % gcd(c1, c2) == 0              # write a[c1*i+k1], read a[c2*j+k2]
def brute(c1, k1, c2, k2, n=60):
    pairs = {(i, j) for i in range(n) for j in range(n) if c1 * i + k1 == c2 * j + k2}
    return bool(pairs), sorted({j - i for i, j in pairs}, key=abs)[:3]
cases = [("a[i]   = a[i-1] + 1", 1, 0, 1, -1), ("a[2*i] = a[2*i+1]", 2, 0, 2, 1),
         ("a[i]   = a[i] + 1", 1, 0, 1, 0), ("a[2*i] = a[4*j+2]", 2, 0, 4, 2), ("a[3*i] = a[6*j+1]", 3, 0, 6, 1)]
print("%-22s %-10s %-24s" % ("loop", "GCD test", "brute force (distances j-i)"))
for text, c1, k1, c2, k2 in cases:
    exists, dist = brute(c1, k1, c2, k2)
    print("%-22s %-10s %s %s" % (text, "may depend" if gcd_test(c1, k1, c2, k2) else "independent", "dependent" if
      exists else "independent", dist))
```

Captured output

```
loop                   GCD test   brute force (distances j-i)
a[i]   = a[i-1] + 1    may depend dependent [1]
a[2*i] = a[2*i+1]      independent independent []
a[i]   = a[i] + 1      may depend dependent [0]
a[2*i] = a[4*j+2]      may depend dependent [-1, -2, -3]
a[3*i] = a[6*j+1]      independent independent []
```

Reading it: `a[i] = a[i-1] + 1` has distance $+1$: a loop-carried flow dependence (each iteration needs the previous result), so it cannot be vectorised or run in parallel. `a[2*i] = a[2*i+1]` writes even and reads odd elements: provably independent. `a[i] = a[i] + 1` has distance 0 (loop-independent). The fourth loop's distance is *not constant* ($-1, -2, -3, \ldots$: reads at iteration j precede the overwrite at iteration $2j+1$, an anti-dependence), so legality of vectorisation/interchange needs a more precise test (Banerjee, Omega); the fifth is independent by the GCD test. The GCD test never missed a real dependence here.

11.4.2 What a real vectoriser says (Clang remarks, real output)

```
void add_arrays(int *a, const int *b, const int *c, int n) { for (int i = 0; i < n; i++) a[i] = b[i] + c[i]; }
void recurrence(int *a, int n)                               { for (int i = 1; i < n; i++) a[i] = a[i-1] + 1; }
int  sum(const int *a, int n)                                { int s = 0; for (int i = 0; i < n; i++) s += a[i];
     return s; }
```

```
$ clang -O2 -c vec.c -Rpass=loop-vectorize -Rpass-missed=loop-vectorize -Rpass=loop-unroll
```

```
line 1: remark: unrolled loop by a factor of 4 with run-time trip count [-Rpass=loop-unroll]
line 2: remark: loop not vectorized [-Rpass-missed=loop-vectorize]
line 2: remark: unrolled loop by a factor of 4 with run-time trip count [-Rpass=loop-unroll]
```

Reading it: line 1 and 3 vectorised (4 ints per SIMD register, interleaved by 2; the reduction on line 3 is recognised); line 2 is *not* vectorised because of the loop-carried dependence; the scalar remainder loops were unrolled by 4. These remarks are the first tool to use when asked "why is my loop slow?" or "why did the optimiser not fire?". Note that `add_arrays` may alias (`a` could overlap `b`), so the vectoriser emits a *run-time alias check*; `restrict` removes the need.

11.4.3 Matrix multiplication "and all applicable optimisations" (S6): a measured answer

The full answer to the Shanghai interview question is a *list with reasons*: (1) **loop interchange** to make the innermost accesses stride-1; (2) **tiling** for cache reuse (and registers/shared memory on GPUs); (3) **register**

blocking and **unrolling** to keep accumulators in registers and expose ILP; (4) **vectorisation**; (5) **multithreading** across output tiles; (6) **transposing** or packing `B` for contiguous access; (7) **prefetching**/software pipelining; (8) algorithmic: Strassen's $O(n^{2.81})$ for very large n; (9) use a tuned library (BLAS, cuBLAS) or Tensor Cores. Experiment (768×768 doubles, GCC `-O2`, this sandbox; absolute times vary by machine, ratios are the lesson):

Textbook order vs. loop interchange vs. tiling

```c
#include <stdio.h>
#include <time.h>
#define N 768
static double A[N][N], B[N][N], C[N][N];
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); return t.tv_sec + t.tv_nsec * 1e
    -9; }
static void clear(void) { for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) C[i][j] = 0; }
static double checksum(void) { double s = 0; for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) s += C[i][j]
    * ((i + j) % 7 + 1); return s; }
static void mm_ijk(void) {                    /* B is walked down a column: stride N*8 bytes, a cache miss per
    access */
    for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) { double s = 0; for (int k = 0; k < N; k++) s += A[i][
    k] * B[k][j]; C[i][j] = s; }
}
static void mm_ikj(void) {                    /* interchange: B and C walked along rows, vectorisable inner loop */
    for (int i = 0; i < N; i++) for (int k = 0; k < N; k++) { double a = A[i][k]; for (int j = 0; j < N; j++) C[i
    ][j] += a * B[k][j]; }
}
#define BS 64
static void mm_tiled(void) {                  /* blocking: each 64x64 tile of B and C is reused while it is hot in
    cache */
    for (int ii = 0; ii < N; ii += BS) for (int kk = 0; kk < N; kk += BS) for (int jj = 0; jj < N; jj += BS)
        for (int i = ii; i < ii + BS; i++) for (int k = kk; k < kk + BS; k++) { double a = A[i][k]; for (int j =
    jj; j < jj + BS; j++) C[i][j] += a * B[k][j]; }
}
int main(void) {
    for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) { A[i][j] = (i * 3 + j) % 10 * 0.5; B[i][j] = (i + j *
     2) % 9 * 0.25; }
    void (*fn[])(void) = { mm_ijk, mm_ikj, mm_tiled };
    const char *name[] = { "ijk (textbook)", "ikj (interchange)", "tiled 64x64" };
    double base = 0;
    for (int v = 0; v < 3; v++) {
        clear(); double t0 = now(); fn[v](); double t = now() - t0;
        if (v == 0) base = t;
        printf("%-18s %6.2f s   speed-up %4.1fx   checksum %.1f\n", name[v], t, base / t, checksum());
    }
    return 0;
}
```

Captured output (timings are measurements)

```
ijk (textbook)       0.49 s   speed-up  1.0x   checksum 4076851668.4
ikj (interchange)    0.14 s   speed-up  3.5x   checksum 4076851668.4
tiled 64x64          0.12 s   speed-up  4.2x   checksum 4076851668.4
```

Reading it: identical checksums (same arithmetic, same summation order per element: interchange and tiling are *legal* here because they preserve the dependences). Interchange alone gives a multi-fold gain by turning cache misses into streaming accesses; tiling adds reuse. Floating-point note: transformations that change the *order of additions* (parallel or vectorised reductions) change rounding, so compilers only do them with `-ffast-math` or explicit permission; verification tests must compare with a tolerance (Chapter 17).

11.4.4 Alias and pointer analysis (asked in S4)

Optimisations need to know whether two pointers can refer to the same memory. **Results:** must-alias, may-alias, no-alias. **Sources of no-alias facts:** distinct local objects, `restrict`, C/C++ type-based alias analysis (an `int*` cannot alias a `float*`), `noalias` function attributes, GPU address spaces (shared vs. global never alias). **Points-to algorithms:** *Andersen* (inclusion-based: each assignment $p = q$ adds $pts(q) \subseteq pts(p)$; precise, roughly $O(n^3)$) vs. *Steensgaard* (unification-based: near-linear, less precise). **Flow-sensitive** (order matters) vs. **insensitive**; **context-sensitive** (distinguish call sites) vs. insensitive. Imprecise alias information is the main obstacle to vectorising C code.

11.5 Interprocedural optimisation ★★

- **Inlining:** replace a call by the callee's body. *Benefits:* no call overhead, exposes constant propagation, CSE and dead-code elimination across the boundary (often the biggest win). *Costs:* code growth, I-cache pressure, compile time. Heuristics: callee size, call-site hotness, `inline`/`always_inline`/`noinline`. Recursion limits inlining.
- **Tail-call elimination**, **devirtualisation** (turn a virtual call into a direct one when the dynamic type is known), **IPSCCP** (propagate constants across functions), **dead-argument elimination**, **function specialisation/cloning**.
- **LTO/ThinLTO:** defer optimisation to link time so it sees the whole program; **PGO:** use measured branch/call frequencies to drive layout and inlining; **escape analysis:** an object that never escapes the function can live on the stack/in registers (scalar replacement of aggregates).
- **JIT compilers** add tiered compilation (interpret, then compile hot code), inline caches, and *deoptimisation* (bail out when an assumption breaks). The PTX→SASS step in a GPU driver is such a JIT.

11.6 Code generation ★★★

The three back-end problems

Instruction selection: map IR operations to target instructions, using addressing modes and fused operations (`lea`, FMA); done by tree/DAG pattern matching (LLVM SelectionDAG, GlobalISel). **Register allocation:** assign the unlimited virtual registers to the k physical ones, spilling to memory when necessary. **Instruction scheduling:** order instructions to hide latencies and fill issue slots without changing dependences. They interact (scheduling extends live ranges; allocation constrains scheduling): another phase-ordering problem.

11.6.1 Register allocation by graph colouring (S6 again)

Method (Chaitin–Briggs): (1) *liveness* gives each variable's live range; (2) build the **interference graph**: an edge joins two variables live at the same time; (3) **simplify**: repeatedly remove a node of degree $< k$ (it can always be coloured later) and push it; (4) if none exists, choose a **spill candidate** (optimistically push the highest degree node); (5) **select**: pop nodes and give each a colour (register) unused by its coloured neighbours; a node with none is *spilled*: stored to the stack and reloaded around uses, then the whole process repeats. **Coalescing** merges the two ends of a copy (`mov a,b`) when they do not interfere. **Linear scan** is a much faster, slightly worse alternative for JITs.

Interference graph and colouring for print((a+b)*c + (a-c)*b) with 4 registers, then 3

```
import backend
SRC = "print((a + b) * c + (a - c) * b);"
backend.allocate(SRC, 4)
print()
backend.allocate(SRC, 3)
```

Captured output

```
registers available: k = 4
TAC:
    t1 = a + b
    t2 = t1 * c
    t3 = a - c
    t4 = t3 * b
    t5 = t2 + t4
    print t5
live after each instruction: [['a', 'b', 'c', 't1'], ['a', 'b', 'c', 't2'], ['b', 't2', 't3'], ['t2', 't4'], ['t5
    '], []]
interference edges: ['a-b', 'a-c', 'a-t1', 'a-t2', 'b-c', 'b-t1', 'b-t2', 'b-t3', 'c-t1', 'c-t2', 't2-t3', 't2-t4
    ']
colouring: {'a': 'r3', 'b': 'r2', 'c': 'r0', 't1': 'r1', 't2': 'r1', 't3': 'r0', 't4': 'r0', 't5': 'r0'} | spilled
    : none
allocated code:
    r1 = r3 + r2
    r1 = r1 * r0
    r0 = r3 - r0
    r0 = r0 * r2
    r0 = r1 + r0
    print r0
```

```
registers available: k = 3
TAC:
    t1 = a + b
    t2 = t1 * c
    t3 = a - c
    t4 = t3 * b
    t5 = t2 + t4
    print t5
live after each instruction: [['a', 'b', 'c', 't1'], ['a', 'b', 'c', 't2'], ['b', 't2', 't3'], ['t2', 't4'], ['t5
    '], []]
interference edges: ['a-b', 'a-c', 'a-t1', 'a-t2', 'b-c', 'b-t1', 'b-t2', 'b-t3', 'c-t1', 'c-t2', 't2-t3', 't2-t4
    ']
colouring: {'b': 'r2', 'c': 'r1', 't1': 'r0', 't2': 'r0', 't3': 'r1', 't4': 'r1', 't5': 'r0'} | spilled: ['a']
```

Reading it: `a, b, c` are inputs live together and `t1`/`t2` interfere with all three (they are computed while `a, b, c` are still needed), so **four** values are simultaneously live: the graph needs 4 colours. With $k = 4$ the allocator reuses `r1` for `t1` then `t2` and `r0` for `c`, `t3`, `t4`, `t5` (their live ranges do not overlap). With $k = 3$ no colouring exists, so a variable (`a`) is **spilled**. *GPU link:* each GPU thread has a register budget; more registers per thread means fewer resident warps (lower occupancy) and, past the cap, spills to local memory (Chapter 15).

11.6.2 Instruction scheduling

List scheduling hides load latency (in-order pipeline: ld=3, add=1, mul=2 cycles)

```
import backend
lat  = {"ld a": 3, "ld b": 3, "add": 1, "ld c": 3, "mul": 2, "st d": 1}
deps = {"ld a": [], "ld b": [], "add": ["ld a", "ld b"], "ld c": [], "mul": ["add", "ld c"], "st d": ["mul"]}
naive = ["ld a", "ld b", "add", "ld c", "mul", "st d"]
print("source order:");  t1 = backend.cycles(naive, lat, deps, show=True);  print("  -> %d cycles" % t1)
better = backend.list_schedule(lat, deps)
print("list-scheduled order:", better); t2 = backend.cycles(better, lat, deps, show=True); print("  -> %d cycles"
    % t2)
```

Captured output

```
source order:
    cycle  0: issue ld a                  result ready at 3
    cycle  1: issue ld b                  result ready at 4
    cycle  4: issue add                   result ready at 5
    cycle  5: issue ld c                  result ready at 8
    cycle  8: issue mul                   result ready at 10
    cycle 10: issue st d                  result ready at 11
  -> 11 cycles
list-scheduled order: ['ld a', 'ld b', 'ld c', 'add', 'mul', 'st d']
    cycle  0: issue ld a                  result ready at 3
    cycle  1: issue ld b                  result ready at 4
    cycle  2: issue ld c                  result ready at 5
    cycle  4: issue add                   result ready at 5
    cycle  5: issue mul                   result ready at 7
    cycle  7: issue st d                  result ready at 8
  -> 8 cycles
```

Why: in source order the `add` must wait for `ld b` and the `mul` for `ld c` (each load takes 3 cycles) so the pipeline stalls; hoisting `ld c` to run in the shadow of the earlier loads removes the stall (11 → 8 cycles). **List scheduling** greedily picks, among ready instructions, the one with the longest remaining latency path (*critical path*). Note: NVIDIA GPUs do not reorder instructions in hardware, so `ptxas` must schedule statically; other warps hide the rest.

11.7 Linking and loading ★★

Must know

- An **object file** (ELF on Linux) has sections: `.text` (code), `.data`, `.bss`, `.rodata`, a **symbol table** (`.symtab`: defined and undefined names) and **relocation** entries (`.rela.text`: "patch this address once the final location of symbol S is known").
- The **linker** (1) resolves each undefined symbol to exactly one definition, (2) assigns final addresses to sections, (3) applies relocations. **Strong** symbols (functions, initialised globals) cannot be defined twice; **weak** ones can be overridden. Order matters for static libraries (`.a`): the linker only pulls members that resolve *currently* undefined symbols, so put libraries *after* the objects that need them.

- **Static linking** copies library code into the executable. **Dynamic linking** records dependencies (`DT_NEEDED`); the loader (`ld.so`) maps shared objects and resolves symbols at load or lazily at first call through the **PLT/GOT**. Shared libraries use position-independent code (`-fPIC`). ASLR randomises load addresses.
- The **loader** maps segments with the right permissions (text r-x, data rw-, bss zero pages), sets up the stack with `argv`/`envp`, and jumps to the entry point (`_start`, which calls `main`).

```
$ gcc m.c -o m        # helper() is declared but never defined
```

```
undefined reference to `helper'
```

```
$ gcc -c d1.c d2.c && gcc d1.o d2.o -o dd   # two strong definitions of x
```

```
multiple definition of `x'; d1.o:(.data+0x0): first defined here
```

```
$ gcc -c d1.c && readelf -S -W d1.o | sed -n 's/^ *\[ *[0-9]*\] \([^ ]*\) .*/\1/p'
```

```
 .text .rela.text .data .bss .comment .note.GNU-stack .note.gnu.property .eh_frame .rela.eh_frame .symtab .strtab
    .shstrtab
```

11.8 Practice ★★★

Q11.1 Which of these is a *backward* data-flow problem: reaching definitions, live variables, available expressions?
Answer: Live variables (information flows from uses back to definitions). The other two are forward.

Q11.2 A block has $use = \{a\}$, $def = \{b\}$ and its successors have $IN = \{b, c\}$. What are OUT and IN of the block (live variables)?
Answer: $OUT = \{b, c\}$ (union over successors); $IN = use \cup (OUT - def) = \{a\} \cup (\{b, c\} - \{b\}) = \{a, c\}$.

Q11.3 Why is the meet for available expressions intersection, and what is its initial value for non-entry blocks?
Answer: An expression is available only if it is available along *every* incoming path (must analysis), so $\cap$. Start from the universal set (so intersection can only shrink) and $IN[entry] = \emptyset$; starting from $\emptyset$ would wrongly make everything unavailable forever.

Q11.4 Give two benefits and two costs of loop unrolling, and say how a compiler decides.
Answer: Benefits: fewer branches/counter updates; more instruction-level parallelism and scheduling freedom (and enables vectorisation and cross-iteration CSE). Costs: larger code (I-cache misses), higher register pressure (spills). Decision: a cost model using the trip count (known or estimated), body size, target issue width and register budget, and a remainder loop for trip counts not divisible by the factor.

Q11.5 Is it always legal to hoist `x = a / b` out of a loop where `a, b` are invariant?
Answer: No. If the loop might execute zero times or the division is guarded, executing it unconditionally in the pre-header could trap (division by zero) when the original program would not. Hoist only if the instruction is safe to speculate or dominates all loop exits.

Q11.6 What is the difference between *liveness* and *reaching definitions*?
Answer: Reaching definitions: which assignments to v *may reach* a point (forward, about the past; used to propagate constants). Liveness: whether the value of v *may be read* before being overwritten (backward, about the future; used for register allocation and dead-code elimination).

Q11.7 Colour the graph with edges a–b, b–c, c–d, d–a (a 4-cycle). What is the minimum number of registers?
Answer: 2 (bipartite): a, c share one colour, b, d the other. A triangle would need 3; a 5-cycle needs 3.

Q11.8 Which are machine-dependent: peephole optimisation, common-subexpression elimination, register allocation, loop-invariant code motion?
Answer: Peephole optimisation (on target code idioms) and register allocation. CSE and LICM work on the machine-independent IR.

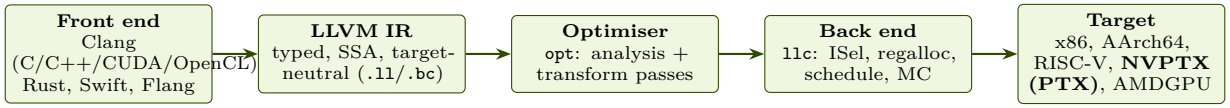
Part IV
LLVM, the NVIDIA Toolchain and GPUs

12. LLVM Crash Course: IR, Passes, Back Ends

In this chapter

Priority ★★★ (vocabulary) / ★★ (depth). Time: 70 min. NVIDIA's CUDA compiler is built on LLVM technology and NVIDIA lists LLVM/MLIR experience as a "stand-out" skill (J3, J6); interviewers ask IR questions (S8, S10: `getelementptr`, `inbounds`). Every IR and assembly listing in this chapter was generated by the real tools in the build sandbox (Clang/LLVM 18). Output details (value numbering, attribute spelling) change between LLVM versions; the *ideas* do not.

12.1 Architecture and the tools ★★★



Command	Purpose
`clang -S -emit-llvm f.c -o f.ll`	C to textual IR (`-c -emit-llvm` for bitcode `.bc`)
`opt -passes=mem2reg,instcombine -S f.ll`	run chosen passes; `-passes='default<O2>'` is the whole `-O2` pipeline
`llc -O2 -march=nvptx64 -mcpu=sm_80 f.ll`	IR to target assembly (PTX here); `-mtriple=`, `-mattr=`
`lli`, `llvm-as`, `llvm-dis`, `llvm-link`, `llvm-extract`	interpret/JIT; text↔bitcode; link modules; pull out functions
`llvm-reduce`, `bugpoint`, `llvm-stress`	shrink a failing IR file; legacy reducer; random IR generator (Chapter 17)
`FileCheck`, `lit`, `not`, `count`	the test infrastructure (next chapter)
`llvm-mca`, `llvm-objdump`, `llvm-nm`	throughput analysis, disassembly, symbols

Why three-phase? Each language compiles once to IR; every optimisation is written once on IR; every new CPU/GPU needs only a back end (the $M + N$ argument of Chapter 8). The same IR below was lowered by four back ends:

f.ll: x*9 + 1

```
define i32 @f(i32 %x) {
  %m = mul i32 %x, 9
  %a = add i32 %m, 1
  ret i32 %a
}
```

```
$ llc -O2 -march=<target> f.ll   # x86-64, aarch64, riscv64, nvptx64
```

```
--- x86-64
    lea    eax, [rdi + 8*rdi + 1]
    ret
--- aarch64
    add    w8, w0, w0, lsl #3
    add    w0, w8, #1
    ret
--- riscv64
    slli    a1, a0, 3
    add    a0, a1, a0
    addiw    a0, a0, 1
    ret
```

```
--- nvptx64
    ld.param.u32    %r1, [f_param_0];
    mad.lo.s32    %r2, %r1, 9, 1;
    st.param.b32    [func_retval0+0], %r2;
    ret;
```

Observe: x86 does it with one `lea` (address arithmetic computes $x + 8x + 1$); AArch64 uses a shifted-operand add; RISC-V has no such fused form, so three instructions; NVPTX emits a PTX multiply-add `mad.lo.s32` with the constants as immediates. One IR, four instruction selectors: this is why a verification team runs the *same test* against many targets (including the Arm and RISC-V embedded ones).

12.2 IR in one page ★★★

Reading LLVM IR

- A **module** has globals and **functions**; a function has **basic blocks** (labels), each ending in exactly one *terminator* (`br`, `ret`, `switch`, `unreachable`). Every instruction producing a value defines a **virtual register** `%name` exactly once (**SSA**). Unnamed values are numbered `%0, %1, \ldots` in order.
- **Types:** `i1 i8 i32 i64` (any width), `float double half bfloat`, `ptr` (a single *opaque* pointer type since LLVM 15: the pointee type lives on the *access*, not the pointer), vectors `<4 x i32>`, arrays `[8 x i32]`, structs `{i32, double}`.
- **Instructions:** arithmetic (`add sub mul sdiv udiv srem shl lshr ashr and or xor, fadd fmul \ldots`); comparison `icmp slt/ult/eq\ldots` (signed vs. unsigned is in the *predicate*, not the type) and `fcmp`; memory `alloca load store getelementptr`; control `br phi select call`; casts `trunc zext sext bitcast ptrtoint fptosi sitofp`; atomics `atomicrmw cmpxchg fence`.
- **Flags and attributes** carry facts for the optimiser: `nsw`/`nuw` (no signed/unsigned wrap), `inbounds`, `fast`/`nnan`/`contract` (floating-point), `noalias`, `readonly`, `nounwind`, `align N`; **metadata** (`!tbaa`, `!llvm.loop`, debug info) is advisory and may be dropped.

12.2.1 From C to optimised IR: the full story on one function

```
int sum_to(int n) { int s = 0; for (int i = 0; i < n; i++) s += i; return s; }
```

Step 1: Clang at `-O0`. The front end is simple-minded: every local variable becomes a stack slot (`alloca`) with explicit `load`/`store`; `nsw` records that signed overflow is undefined in C.

```
$ clang -O0 -Xclang -disable-O0-optnone -S -emit-llvm sum.c -o sum0.ll
```

```
define dso_local i32 @sum_to(i32 noundef %0) {
  %2 = alloca i32, align 4
  %3 = alloca i32, align 4
  %4 = alloca i32, align 4
  store i32 %0, ptr %2, align 4
  store i32 0, ptr %3, align 4
  store i32 0, ptr %4, align 4
  br label %5
5:                                                ; preds = %13, %1
  %6 = load i32, ptr %4, align 4
  %7 = load i32, ptr %2, align 4
  %8 = icmp slt i32 %6, %7
  br i1 %8, label %9, label %16
9:                                                ; preds = %5
  %10 = load i32, ptr %4, align 4
  %11 = load i32, ptr %3, align 4
  %12 = add nsw i32 %11, %10
  store i32 %12, ptr %3, align 4
  br label %13
13:                                               ; preds = %9
  %14 = load i32, ptr %4, align 4
  %15 = add nsw i32 %14, 1
  store i32 %15, ptr %4, align 4
  br label %5, !llvm.loop !6
16:                                               ; preds = %5
  %17 = load i32, ptr %3, align 4
  ret i32 %17
}
```

Step 2: `mem2reg` promotes the stack slots to SSA values. This is exactly the dominance-frontier algorithm of Chapter 10: the loop header gets one φ per loop-carried variable (`%.01` is `s`, `%.0` is `i`).

```
$ opt -passes=mem2reg -S sum0.ll
```

```
define dso_local i32 @sum_to(i32 noundef %0) {
  br label %2
2:                                                ; preds = %6, %1
  %.01 = phi i32 [ 0, %1 ], [ %5, %6 ]
  %.0 = phi i32 [ 0, %1 ], [ %7, %6 ]
  %3 = icmp slt i32 %.0, %0
  br i1 %3, label %4, label %8
4:                                                ; preds = %2
  %5 = add nsw i32 %.01, %.0
  br label %6
6:                                                ; preds = %4
  %7 = add nsw i32 %.0, 1
  br label %2, !llvm.loop !6
8:                                                ; preds = %2
  ret i32 %.01
}
```

Step 3: the full `-O2` pipeline. Scalar-evolution analysis recognises that `s` is the sum of an arithmetic progression and replaces the *entire loop* by the closed form $n(n-1)/2$ (computed in 33-bit arithmetic so that the intermediate product cannot overflow), then `loop-deletion` removes the loop. A guard keeps the result 0 for $n \leq 0$.

```
$ clang -O2 -S -emit-llvm sum.c -o -
```

```
define dso_local i32 @sum_to(i32 noundef %0) local_unnamed_addr {
  %2 = icmp sgt i32 %0, 0
  br i1 %2, label %3, label %13
3:                                                ; preds = %1
  %4 = add nsw i32 %0, -1
  %5 = zext nneg i32 %4 to i33
  %6 = add nsw i32 %0, -2
  %7 = zext i32 %6 to i33
  %8 = mul i33 %5, %7
  %9 = lshr i33 %8, 1
  %10 = trunc i33 %9 to i32
  %11 = add i32 %10, %0
  %12 = add i32 %11, -1
  br label %13
13:                                               ; preds = %3, %1
  %14 = phi i32 [ 0, %1 ], [ %12, %3 ]
  ret i32 %14
}
```

What this one example teaches

(1) Front ends emit naive IR and rely on the optimiser. (2) `mem2reg`/`sroa` is the gateway pass: nothing else works well on memory. (3) The optimiser reasons *algebraically* (induction variables, closed forms), not just locally. (4) It can only do this because signed overflow is undefined (`nsw`): with unsigned or wrapping arithmetic the transformation would need extra care.

12.3 Memory addressing, flags, poison ★★★

12.3.1 `getelementptr` (GEP) and `inbounds` (asked in S10)

Must know

`getelementptr` only **computes an address**: it never reads or writes memory. Form: `getelementptr <type>, ptr <base>, <idx0>, <idx1>, \ldots` where the first index steps over whole *objects of the source type* (`base + idx0 * sizeof(type)`) and later indices select into aggregates (struct field numbers must be constants). `inbounds` promises that the base and the result both point within (or one past) the same allocated object; if violated the result is **poison**. That promise lets the optimiser assume non-overlap and reorder accesses.

```
struct S { int a; double b; char c; };
```

```c
int    get_idx(int *p, long i)           { return p[i]; }
double get_b(struct S *s)                { return s->b; }
int    get_2d(int m[4][8], int i, int j) { return m[i][j]; }
int    add_s(int x)                      { return x + 1; }
unsigned add_u(unsigned x)               { return x + 1; }
int    mul8(int x)                       { return x * 8; }
```

```
$ clang -O1 -S -emit-llvm gep.c -o -   (metadata and attributes trimmed)
```

```
%struct.S = type { i32, double, i8 }
define i32 @get_idx(ptr nocapture readonly %0, i64 %1) {
  %3 = getelementptr inbounds i32, ptr %0, i64 %1
  %4 = load i32, ptr %3, align 4
  ret i32 %4
}
define double @get_b(ptr nocapture readonly %0) {
  %2 = getelementptr inbounds %struct.S, ptr %0, i64 0, i32 1
  %3 = load double, ptr %2, align 8
  ret double %3
}
define i32 @get_2d(ptr nocapture readonly %0, i32 %1, i32 %2) {
  %4 = sext i32 %1 to i64
  %5 = sext i32 %2 to i64
  %6 = getelementptr inbounds [8 x i32], ptr %0, i64 %4, i64 %5
  %7 = load i32, ptr %6, align 4
  ret i32 %7
}
define i32 @add_s(i32 %0) {
  %2 = add nsw i32 %0, 1
  ret i32 %2
}
define i32 @add_u(i32 %0) {
  %2 = add i32 %0, 1
  ret i32 %2
}
define i32 @mul8(i32 %0) {
  %2 = shl nsw i32 %0, 3
  ret i32 %2
}
```

Reading it: `p[i]` → `getelementptr inbounds i32, ptr %0, i64 %1` (scaled by 4); `s->b` → field index 1 of `%struct.S` (the byte offset, 8, comes from the data layout, not from the IR); the 2-D access indexes `[8 x i32]` rows then columns (the row stride is part of the *type*, which is why the array parameter must carry its inner dimension). **Signed** `x+1` becomes `add nsw`, **unsigned** `x+1` plain `add` (wraps by definition): the signedness of C arithmetic is encoded as flags, and `x*8` became `shl nsw 3`.

12.3.2 Poison, undef, freeze, and why miscompilations happen

Deferred undefined behaviour

`poison`: the result of an instruction whose assumption was violated (`nsw` overflow, out-of-bounds `inbounds` GEP, out-of-range shift, `nnan` producing NaN). Any instruction using poison yields poison; using it in a branch condition or as an address, or storing it to externally visible memory, is *real* undefined behaviour. **`undef`**: an arbitrary but *unspecified* value that may differ at each use (being phased out). **`freeze`**: converts poison/undef to some *fixed* arbitrary value, stopping propagation.

Most real LLVM miscompilations are *refinement errors*: a pass assumes a flag it may not (e.g. keeps `nsw` when it widened an operation, or moves a `select`'s operand out of its guard so poison leaks). The tool for catching these is **translation validation**: *Alive2* checks, with an SMT solver, that the optimised function *refines* the original for all inputs (it found 47 new LLVM bugs when run over the existing test-suite, PLDI 2021). Chapter 17 returns to this.

12.4 The pass manager and the passes you should recognise ★★★

Must know

LLVM has **analysis passes** (compute information: `domtree`, `loops`, `scalar-evolution`, alias analysis, `memoryssa`, block frequency) and **transform passes**. The *new pass manager* (`opt -passes=`) runs a pipeline on modules/functions/loops, caches analyses, and **invalidates** an analysis unless the transform declares it preserved: forgetting to preserve or invalidate correctly is a classic bug class. Passes run in a deliberate order (cleanup after inlining, loop passes before the vectoriser).

Pass	What it does
`sroa`, `mem2reg`	break up/promote `alloca`s to SSA values (the gateway)
`instcombine`	peephole algebra and canonicalisation on IR (`mul x,8` → `shl x,3`, `x+0` → `x`); the largest pass; frequent bug site
`simplifycfg`	merge/remove blocks, fold constant branches, turn small `if`s into `select`
`early-cse`, `gvn`	redundancy elimination: value numbering, redundant-load elimination (uses alias analysis, MemorySSA)
`licm`	hoist loop-invariant code, sink stores
`loop-rotate`, `indvars`, `loop-deletion`, `loop-idiom`	canonical loop shape; induction-variable simplification using scalar evolution; delete dead loops; recognise `memset`/`memcpy` loops
`loop-unroll`, `loop-vectorize`, `slp-vectorizer`	unroll; vectorise loops; vectorise straight-line code
`inline`, `ipsccp`, `deadargelim`, `globalopt`	interprocedural: inlining, constant propagation across calls, dead-argument removal, global-variable optimisation
`jump-threading`, `reassociate`, `adce`, `tailcallelim`	thread branches through known conditions, canonicalise operand order, aggressive DCE, tail recursion

Debugging a miscompile with the pass manager. `-opt-bisect-limit=N` runs only the first N pass-invocations. Binary-search N to find the single pass invocation that turns a good program into a bad one: the standard first step of triage (the `git bisect` of optimisation passes).

```
$ clang -O2 -Xclang -disable-llvm-passes -S -emit-llvm sum.c -o sum_raw.ll && opt -passes='default<O2>' -opt-
    bisect-limit=6 -S sum_raw.ll -o /dev/null
```

```
BISECT: running pass (1) Annotation2MetadataPass on [module]
BISECT: running pass (2) ForceFunctionAttrsPass on [module]
BISECT: running pass (3) InferFunctionAttrsPass on [module]
BISECT: running pass (4) LowerExpectIntrinsicPass on sum_to
BISECT: running pass (5) SimplifyCFGPass on sum_to
BISECT: running pass (6) SROAPass on sum_to
BISECT: NOT running pass (7) EarlyCSEPass on sum_to
BISECT: NOT running pass (8) OpenMPOptPass on [module]
BISECT: NOT running pass (9) IPSCCPPass on [module]
```

Other instruments: `-debug-pass-manager` (which passes run), `-print-after-all` / `-print-changed` (IR after each pass), `-Rpass=\ldots`, `-Rpass-missed=\ldots`, `-Rpass-analysis=\ldots` (optimisation remarks, e.g. the inliner's cost/threshold decisions below), `-mllvm -debug-only=\ldots` (pass internals, debug builds).

```
static int square(int x) { return x * x; }
int big(int x) { int s = 0; for (int i = 0; i < 100; i++) s += (x * i) ^ (s >> 3) + i * i + 12345 * x; return s; }
int use(int a) { return square(a) + big(a); }
```

```
$ clang -O2 -c inl.c -Rpass=inline
```

```
line 3: remark: 'square' inlined into 'use' with (cost=-15030, threshold=337) at callsite use:0:25; [-Rpass=inline
    ]
line 3: remark: 'big' inlined into 'use' with (cost=20, threshold=225) at callsite use:0:37; [-Rpass=inline]
```

Reading it: the inliner computes a *cost* for each call site and inlines when it is below a *threshold*: `square` has hugely negative cost (inlining makes it vanish into a multiply), `big` was inlined because its single call site made the copy free. This is the "benefit versus cost" pattern interviewers want.

12.5 Vectorisation in IR ★★

```
void add_arrays(int *restrict a, const int *restrict b, const int *restrict c, int n) { for (int i = 0; i < n; i
    ++) a[i] = b[i] + c[i]; }
```

```
$ clang -O2 -S -emit-llvm vec2.c -o - | grep '<4 x i32>'
```

```
%15 = load <4 x i32>, ptr %13, align 4
%16 = load <4 x i32>, ptr %14, align 4
%19 = load <4 x i32>, ptr %17, align 4
%20 = load <4 x i32>, ptr %18, align 4
%21 = add nsw <4 x i32> %19, %15
%22 = add nsw <4 x i32> %20, %16
store <4 x i32> %21, ptr %23, align 4
store <4 x i32> %22, ptr %24, align 4
```

Reading it: the loop body now moves `<4 x i32>` vectors (SSE: 128 bits = 4 ints), interleaved by 2 (two vectors per iteration to hide latency), with scalar remainder loops for the leftover iterations. `restrict` removed the need for a run-time overlap check. On a GPU the analogue of this SIMD loop is *the thread grid itself*: each thread computes one `a[i]`.

12.6 The back end: from IR to machine code ★★

1. **Instruction selection** turns IR into target *MachineInstrs* with virtual registers, using patterns written in TableGen (`.td`) files. Two implementations: **SelectionDAG** (per-basic-block DAG: build, legalise types/-operations, combine, select, schedule) and the newer **GlobalISel** (IRTranslator, Legalizer, RegBankSelect, InstructionSelect).
2. **Register allocation** (greedy allocator with live-range splitting by default; fast allocator at `-O0`) maps virtual to physical registers and inserts spills. **Scheduling**, prologue/epilogue insertion, branch relaxation, peepholes follow, then the **MC layer** emits assembly or an object file.

```
$ llc -O2 -march=x86-64 -stop-after=finalize-isel f.ll -o - | sed -n '/bb.0/,/RET/p'
```

```
  bb.0 (%ir-block.0):
    liveins: $edi

    %0:gr32 = COPY $edi
    %2:gr64 = IMPLICIT_DEF
    %1:gr64_nosp = INSERT_SUBREG %2, %0, %subreg.sub_32bit
    %3:gr32 = LEA64_32r %1, 8, %1, 1, $noreg
    $eax = COPY %3
    RET 0, $eax
```

This is **MachineIR (MIR)** right after instruction selection: virtual registers (`%0:gr32`) with register *classes*, and the `LEA64_32r` pattern already chosen. `-stop-after=<pass>` and `-run-pass=<pass>` let you run a back-end pass in isolation on an MIR file: the same trick used for back-end regression tests.

12.7 MLIR, Triton and the AI-compiler ecosystem ★★

- **MLIR** (Multi-Level IR, part of the LLVM project) generalises LLVM IR into *dialects* of operations (each with types, attributes and *regions*), so a compiler can keep high-level structure (tensors, loops, GPU kernels) and *progressively lower*: e.g. `linalg` → `scf`/`affine` loops → `gpu` → `nvvm` → `llvm` dialect → LLVM IR → PTX. NVIDIA job postings for "LLVM and MLIR compiler engineer" (J6) refer to this stack.
- **Triton** (OpenAI) compiles block-level Python kernels through MLIR and LLVM to PTX; **torch.compile/TorchInductor** generates Triton or C++; **XLA/OpenXLA** (JAX/TensorFlow), **TVM**, **TensorRT** are other ML compilers (Chapter 19).

12.8 GCC in one paragraph ★

GCC's pipeline: front ends → **GENERIC** → **GIMPLE** (three-address, SSA-capable) → **RTL** (low-level, register-transfer) → assembly. Dump intermediate forms with `-fdump-tree-all` and `-fdump-rtl-all`; plugin pass infrastructure exists; its regression suite uses **DejaGnu** (`dg-*` directives in test comments), the analogue of lit+FileCheck. Both compilers matter to a verifier: differential testing GCC against Clang is a classic bug-finding method.

12.9 Practice ★★★

Q12.1 Why does Clang emit `alloca`/`load`/`store` for every local, and which pass fixes it?
Answer: It keeps the front end trivial: it need not compute SSA or dominance. `mem2reg`/`sroa` promote allocas to SSA registers, inserting φ nodes at dominance frontiers.

Q12.2 Does `getelementptr` access memory? What does `inbounds` mean?
Answer: No: it is pure address arithmetic. `inbounds` asserts the computed address stays within the same allocated object (or one past the end); violating it yields poison, enabling alias reasoning.

Q12.3 What is the difference between `add`, `add nsw` and `add nuw`?
Answer: Plain `add` wraps modulo 2^n. `nsw`: signed overflow produces *poison*; `nuw`: unsigned overflow produces poison. C's signed overflow is UB, so Clang emits `nsw`; unsigned C arithmetic wraps, so it emits plain `add`.

Q12.4 How do you find which optimisation pass broke a program?
Answer: Bisect: `-opt-bisect-limit=N` (or `-mllvm -opt-bisect-limit=N` from Clang) disables all pass invocations after the Nth. Binary-search the smallest N that miscompiles; the N-th pass invocation (printed as `BISECT: running pass (N) \ldots`) is the culprit. Then reduce the IR (`llvm-reduce`) and report.

Q12.5 Why can a vectoriser not vectorise `a[i] = a[i-1] + 1`, and what would let it vectorise `a[i] = b[i] + c[i]` without a run-time check?
Answer: The first has a loop-carried flow dependence of distance 1. The second needs proof that `a` does not overlap `b` and `c`: `restrict` (or `noalias`), or the compiler emits a run-time overlap check and a scalar fallback loop.

Q12.6 Name the parts of LLVM's back end and the TableGen files' role.
Answer: Instruction selection (SelectionDAG/GlobalISel), scheduling, register allocation, prologue/epilogue, MC emission. TableGen `.td` files declaratively describe registers, instructions, and instruction-selection patterns, from which C++ tables are generated.

13. Testing Compilers the LLVM Way: lit and FileCheck

In this chapter

Priority ★★. Time: 35 min. The vast majority of regression tests in LLVM (tens of thousands) are tiny files with a `RUN:` line and `CHECK:` lines. A compiler-verification engineer must read and write such tests fluently; NVIDIA's own test frameworks differ, but the ideas (small inputs, exact expectations, automatic discovery, XFAIL tracking) are universal. Everything below is run for real with `lit` 23 and LLVM 18's `FileCheck`.

13.1 The test pyramid for a compiler ★★

Level	Example in LLVM	What it checks
Unit tests	GoogleTest C++ (`unittests/`)	APIs and data structures (APInt, pass-manager plumbing)
Regression tests	`.ll`/`.c`/`.mir` files run by **lit**, verified by **FileCheck**	the *output* of one pass or tool for one tiny input: transformation happened (or did not), diagnostics, code selected
Execution tests	compile, run, compare output (`lli`, or build+run)	the *behaviour* of generated code
Integration / benchmark suites	`llvm-test-suite`, SPEC CPU, application builds	large real programs: correctness and performance over time
Fuzzing / differential	`llvm-isel-fuzzer`, Csmith, YARPGen	bugs nobody thought to write a test for (Chapter 17)

Rule: every fixed compiler bug leaves behind a *minimal regression test*: the reduced reproducer plus checks that fail before the fix and pass after it.

13.2 lit: the test driver ★★

Must know

`lit` discovers test files (by suffix, configured in `lit.cfg.py`), runs the shell commands in each file's `RUN:` lines, and classifies the result: **PASS** (all commands exit 0), **FAIL**, **XFAIL** (expected failure, declared with `XFAIL:`), **XPASS** (an XFAIL test that now passes: reported as an error so stale expectations get removed), **UNSUPPORTED** (`UNSUPPORTED:` or an unmet `REQUIRES:`), **UNRESOLVED**, **TIMEOUT**.

Feature	Meaning
`RUN: cmd | FileCheck %s`	a command line; the test fails if any command in the chain fails (pipelines fail if *any* stage fails)
`%s`, `%S`, `%t`, `%p`	the test file path; its directory; a unique temp file; the same as `%S`. Typical: `clang -S -emit-llvm %s -o %t.ll`
`REQUIRES: nvptx-registered-target`	run only if the feature exists (targets built, OS, assertions-enabled build ...)
`UNSUPPORTED: system-windows`, `XFAIL: *`	skip on a configuration; mark as a known failure
`not cmd`, `not --crash cmd`, `count N cmd`	invert the exit status (expect failure); expect a crash; require exactly N lines of output

Feature	Meaning
`lit -v`, `-a`, `-j N`, `--filter=RE`, `--timeout=S`, `--shard=I/N`, `--xunit-xml-output=f.xml`	show failures, show all commands, parallelism, select tests, time limit, split across CI machines, JUnit-style report for dashboards

13.3 FileCheck: match the output, not the accident ★★

Directive	Semantics
`CHECK: text`	find *text* somewhere after the previous match (substring match; patterns are matched in order)
`CHECK-NEXT:` / `CHECK-SAME:`	must be on the *next* line / on the *same* line as the previous match
`CHECK-NOT:`	text must *not* appear between the surrounding matches (use to prove an instruction disappeared)
`CHECK-LABEL:`	anchors a function or section; the other checks are matched *within* it, so one failing function does not confuse the next
`CHECK-DAG:`	a group of lines that may appear in *any* order (e.g. independent loads)
`CHECK-COUNT-N:`, `CHECK-EMPTY:`	exactly N consecutive matches; an empty line
`{{regex}}`	inline regular expression: `add i32 {{%[0-9]+}}, 1`
`[[NAME:regex]]`, `[[NAME]]`	**capture** a value then require it again: `[[R:%.*]] = shl i32 %x, 3 \ldots ret i32 [[R]]`. Numeric variants `[[#N:]]`, `[[#N+1]]`
`--check-prefix=P`, `--implicit-check-not=T`	several expectation sets in one file (e.g. per optimisation level); forbid `T` everywhere
`--dump-input=fail`, `--strict-whitespace`, `--match-full-lines`, `-DVAR=value`	show the annotated input on failure; whitespace-exact; whole-line matches; pass substitutions from `RUN:`

13.3.1 A real mini test suite

Four tests for the `instcombine` pass: the first two pass, one is an expected failure (a known bug) and one is *wrong on purpose* so you can see how FileCheck explains a failure.

tests/mul8.ll

```
; RUN: opt -passes=instcombine -S %s | FileCheck %s

define i32 @mul8(i32 %x) {
  %r = mul i32 %x, 8
  ret i32 %r
}
; CHECK-LABEL: define i32 @mul8(
; CHECK-NEXT:    [[R:%.*]] = shl i32 %x, 3
; CHECK-NEXT:    ret i32 [[R]]
```

tests/identity.ll

```
; RUN: opt -passes=instcombine -S %s | FileCheck %s
; Algebraic identities must disappear entirely.

define i32 @ident(i32 %x) {
  %a = add i32 %x, 0
  %b = mul i32 %a, 1
  %c = xor i32 %b, 0
  ret i32 %c
```

```
}
; CHECK-LABEL: @ident(
; CHECK-NOT:      add
; CHECK-NOT:      mul
; CHECK-NOT:      xor
; CHECK:          ret i32 %x
```

tests/xfail_example.ll

```
; RUN: opt -passes=instcombine -S %s | FileCheck %s
; XFAIL: *
; Known missed optimisation, tracked elsewhere: XFAIL keeps the suite green until it is fixed.

define i32 @f(i32 %x) {
  %r = mul i32 %x, 3
  ret i32 %r
}
; CHECK: shl i32 %x, 2
```

tests/wrong_expectation.ll (wrong on purpose)

```
; RUN: opt -passes=instcombine -S %s | FileCheck %s

define i32 @f(i32 %x) {
  %r = mul i32 %x, 8
  ret i32 %r
}
; CHECK-LABEL: @f(
; CHECK: mul i32 %x, 8
```

lit.cfg.py

```
import os, lit.formats
config.name = "mini-llvm-tests"
config.test_format = lit.formats.ShTest(execute_external=False)      # lit's built-in shell: pipes and redirects
    work everywhere
config.suffixes = [".ll"]
config.test_source_root = os.path.dirname(__file__)
config.environment["PATH"] = os.pathsep.join(["/usr/lib/llvm-18/bin", config.environment["PATH"]])   # find opt
    and FileCheck
```

```
$ lit -v -j1 tests
```

```
-- Testing: 4 tests, 1 workers --
FAIL: mini-llvm-tests :: tests/wrong_expectation.ll (1 of 4)
******************** TEST 'mini-llvm-tests :: tests/wrong_expectation.ll' FAILED ********************
Exit Code: 1

Command Output (stdout):
--
# RUN: at line 1
opt -passes=instcombine -S tests/wrong_expectation.ll | FileCheck tests/wrong_expectation.ll
# executed command: opt -passes=instcombine -S tests/wrong_expectation.ll
# executed command: FileCheck tests/wrong_expectation.ll
# .---command stderr------------
# | tests/wrong_expectation.ll:8:10: error: CHECK: expected string not found in input
# | ; CHECK: mul i32 %x, 8
# |          ^
# | <stdin>:4:15: note: scanning from here
# | define i32 @f(i32 %x) {
# |               ^
# | <stdin>:5:7: note: possible intended match here
# |  %r = shl i32 %x, 3
# |       ^
# |
# | Input file: <stdin>
# | Check file: tests/wrong_expectation.ll
# |
# | -dump-input=help explains the following input dump.
# |
# | Input was:
# | <<<<<<
# |            1: ; ModuleID = 'tests/wrong_expectation.ll'
# |            2: source_filename = "tests/wrong_expectation.ll"
```

```
# |            3:
# |            4: define i32 @f(i32 %x) {
# | check:8'0                  X~~~~~~~~ error: no match found
# |            5:  %r = shl i32 %x, 3
# | check:8'0     ~~~~~~~~~~~~~~~~~~~
# | check:8'1           ?             possible intended match
# |            6:  ret i32 %r
# | check:8'0     ~~~~~~~~~~~~
# |            7: }
# | check:8'0     ~~
# | >>>>>>
# `----------------------------
# error: command failed with exit status: 1

--

********************
PASS: mini-llvm-tests :: tests/mul8.ll (2 of 4)
XFAIL: mini-llvm-tests :: tests/xfail_example.ll (3 of 4)
PASS: mini-llvm-tests :: tests/identity.ll (4 of 4)
********************
Failed Tests (1):
  mini-llvm-tests :: tests/wrong_expectation.ll



Total Discovered Tests: 4
  Passed           : 2 (50.00%)
  Expectedly Failed: 1 (25.00%)
  Failed           : 1 (25.00%)
```

Reading lit's report

- **PASS** for the first two; **XFAIL** for the known-bug test (counted as success; if the bug were fixed it would show **XPASS** and fail the run, forcing the `XFAIL` line to be deleted).
- FileCheck's failure message is the model of a good diagnostic: the *exact check line*, where it scanned from, and a *possible intended match* (`%r = shl i32 %x, 3`): the optimiser did *better* than the test expected (strength reduction), so here the test, not the compiler, is wrong.
- `CHECK-NOT` in `identity.ll` proves the three no-op instructions were removed; `[[R:%.*]]` captured the register name so the test does not depend on how values happen to be numbered.

13.4 Writing tests that survive refactoring ★★

Must know

- **Minimal and focused:** one behaviour per test; reduce with `llvm-reduce`/`creduce` before committing. A 3-line test is read; a 300-line test is ignored.
- **Check intent, not accident:** capture registers with `[[VAR:%.*]]`, use `CHECK-DAG` where order is unspecified, `CHECK-LABEL` per function, and avoid pinning unrelated instructions or exact scheduling unless that is what is under test.
- **Generated checks:** LLVM's `utils/update_test_checks.py` (IR) and `update_llc_test_checks.py` (assembly) write `CHECK` lines from current output. Fast, but they *bless whatever the compiler does today*: always review the diff for semantic sense.
- **Target selection:** pin `-mtriple=nvptx64-nvidia-cuda -mcpu=sm_80`; guard with `REQUIRES: nvptx-registered-target`; keep target-independent tests target-independent.
- **Negative tests:** `not opt \ldots` for invalid input that must be rejected; Clang's `-verify` with `// expected-error {{message}}` comments for diagnostics; `not --crash` documents a currently-crashing input.
- **Execution tests** compile and *run*: compare output or exit status; keep them deterministic (Chapter 17). Separate *performance* tests from correctness tests; performance needs statistics, not CHECK lines.

For NVIDIA-style work the analogue is a tiny *PTX* or *CUDA* file, a `RUN:` that assembles/compiles it for a target, and checks on either the generated *code* (a given PTX instruction becomes the expected SASS) or the executed *result* on a GPU or simulator. Chapter 18 implements a Python mini-FileCheck and test runner you can describe in an interview.

13.5 Practice ★★

Q13.1 Which directive proves that an optimisation removed an instruction, and where must it be placed?
Answer: `CHECK-NOT: <text>`. It applies between the matches that surround it (so put it between two positive checks, or after `CHECK-LABEL`), or use `--implicit-check-not`. It must be bracketed by positive checks or it will search to the end of the input.

Q13.2 Why is hard-coding `%3 = add i32 \ldots` in a check line fragile, and what replaces it?
Answer: Value numbering changes when unrelated instructions change. Capture with `[[V:%.*]] = add i32 \ldots` and reuse `[[V]]`.

Q13.3 What does **XPASS** mean and why does lit treat it as a failure?
Answer: A test marked `XFAIL` passed: the bug it tracked was fixed (or the test changed). Failing the run forces maintainers to delete the stale `XFAIL`, keeping the expectation file honest.

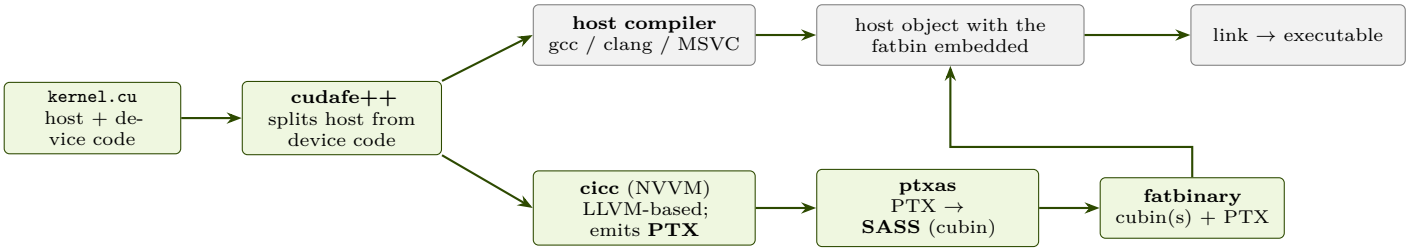
Q13.4 You are asked to add a test for a miscompilation. List what the test directory should contain.
Answer: A minimal reproducer (reduced), a `RUN:` line that invokes the exact tool and flags, checks that distinguish correct from incorrect output (code pattern or runtime value), a comment linking the bug report, and `REQUIRES:` for the needed target/feature. It must fail on the unfixed compiler and pass on the fixed one.

14. NVIDIA's Stack: nvcc, NVVM, PTX and SASS

In this chapter

Priority ★★★ for the pipeline and PTX vocabulary, ★★ for details. Time: 90 min. The role description names PTX, CUDA and GPU assembly explicitly. The sandbox has no CUDA SDK or GPU, so this chapter is grounded in what the *open-source LLVM NVPTX back end* generates for real CUDA kernels (Clang's CUDA front end, no SDK needed for device-only compilation). It is the same instruction set (PTX) that NVIDIA's own `cicc`/NVVM emits; instruction *choices* and register names differ from NVIDIA's closed compiler, *syntax and semantics* do not. Sources: NVIDIA nvcc documentation and the PTX ISA manual (PDF, versions up to 9.1 exist); read PTX ISA chapters 5–9 if you have an hour spare.

14.1 The CUDA compilation pipeline ★★★



Must know

- `nvcc` is a *driver*: it separates device from host code, sends the device part through NVIDIA's compilers and assembler, embeds the result (a **fat binary**) in the host object, and forwards all ordinary C++ compilation to a host compiler. `nvcc -dryrun` prints the exact sub-commands; `--keep` keeps the intermediates (`.ptx`, `.cubin`, `.fatbin`).
- **NVVM / `cicc`** is the LLVM-based compiler that turns the device program (in *NVVM IR*, an LLVM IR dialect) into **PTX**. `ptxas` is the optimising assembler: PTX → architecture-specific machine code (**SASS**, in a **cubin** ELF file). Register allocation, instruction scheduling and most low-level optimisation happen in `ptxas`.
- **Alternative front ends/APIs: NVRTC** compiles CUDA C++ strings to PTX/cubin at run time; **libNVVM** compiles NVVM IR; **nvJitLink** links at run time; **Clang** can compile CUDA directly (`clang++ -x cuda --cuda-gpu-arch=sm_80`) through the open-source NVPTX back end; Triton, XLA and other compilers also emit PTX.
- **The driver can JIT:** if a binary lacks SASS for the installed GPU it compiles the embedded *PTX* at load time (result cached).

nvcc option	Meaning
`-arch=sm_80`	shorthand for `-gencode arch=compute_80,code=[sm_80,compute_80]`: SASS for sm_80 *and* PTX for compute_80
`-gencode arch=compute_70,code=sm_70`	virtual arch (PTX feature level) used to *generate*, real arch (SASS) to *emit*; repeat for a multi-architecture fat binary
`-ptx`, `-cubin`, `-fatbin`, `--keep`	stop at PTX / cubin / fatbin; keep intermediates
`-Xptxas -v`, `--maxrregcount=N`, `-lineinfo`, `-G`	report registers/shared memory/spills per kernel; cap registers per thread; line info for profilers; device debug build (disables most optimisation)
`--use_fast_math`	shorthand for `-ftz=true -prec-div=false -prec-sqrt=false --fmad=true` plus approximate intrinsics
`--fmad=false`, `-ftz=true`	forbid fused multiply-add contraction; flush denormals to zero
`-rdc=true`	relocatable device code: separate compilation of device code, then device linking

Virtual vs. real architecture (a favourite question)

`compute_XY` = virtual architecture = the PTX feature level. `sm_XY` = real architecture = SASS for that hardware generation. **PTX is forward compatible:** PTX for `compute_70` can be JIT-compiled for any GPU of compute capability ≥7.0. **SASS is not:** a cubin built for `sm_XY` runs on devices of the same major version and minor ≥ Y (sm_80 SASS runs on sm_86, not the reverse, and never across major versions). So a robust library ships SASS for the architectures it was tested on *plus* PTX for the newest virtual architecture. A recent generation is the Hopper family (`sm_90`/`sm_90a`); Blackwell adds further `sm_100`/`sm_120`-family targets with architecture-specific suffixes: do not memorise numbers, understand the rule.

14.2 NVVM IR: LLVM IR with GPU conventions ★★

Compile a kernel with Clang and read the IR (the stub header replaces `cuda_runtime.h`).

vecadd.cu

```
#include "cuda_stub.h"
__global__ void vecadd(const float* a, const float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
```

```
$ clang++ -x cuda --cuda-device-only --cuda-gpu-arch=sm_80 -O2 -S -emit-llvm vecadd.cu -o -
```

```
target datalayout = "e-i64:64-i128:128-v16:16-v32:32-n16:32:64"
target triple = "nvptx64-nvidia-cuda"
define dso_local void @_Z6vecaddPKfS0_Pfi(ptr %0, ptr %1, ptr %2, i32 %3) {
  %5 = tail call i32 @llvm.nvvm.read.ptx.sreg.ctaid.x()
  %6 = tail call i32 @llvm.nvvm.read.ptx.sreg.ntid.x()
  %7 = mul i32 %5, %6
  %8 = tail call i32 @llvm.nvvm.read.ptx.sreg.tid.x()
  %9 = add i32 %7, %8
  %10 = icmp slt i32 %9, %3
  br i1 %10, label %11, label %19
11:                                               ; preds = %4
  %12 = sext i32 %9 to i64
  %13 = getelementptr inbounds float, ptr %2, i64 %12
  %14 = getelementptr inbounds float, ptr %1, i64 %12
  %15 = getelementptr inbounds float, ptr %0, i64 %12
  %16 = load float, ptr %15, align 4
  %17 = load float, ptr %14, align 4
  %18 = fadd contract float %16, %17
  store float %18, ptr %13, align 4
  br label %19
19:                                               ; preds = %11, %4
  ret void
}
declare i32 @llvm.nvvm.read.ptx.sreg.ctaid.x() #1
declare i32 @llvm.nvvm.read.ptx.sreg.ntid.x() #1
declare i32 @llvm.nvvm.read.ptx.sreg.tid.x() #1
!nvvm.annotations = !{!0}
!0 = !{ptr @_Z6vecaddPKfS0_Pfi, !"kernel", i32 1}
```

CUDA construct	NVVM IR	PTX
`threadIdx.x`	`call i32 @llvm.nvvm.read.ptx.sreg.tid.x()`	`mov.u32 %r, %tid.x;`
`blockIdx.x`, `blockDim.x`, `gridDim.x`	`...sreg.ctaid.x`, `...ntid.x`, `...nctaid.x`	`%ctaid.x`, `%ntid.x`, `%nctaid.x`
`__global__` kernel	`!nvvm.annotations = !{ptr @f, !"kernel", i32 1}` (newer LLVM: `ptx_kernel` calling convention)	`.visible .entry`
`__shared__ float s[256]`	global in `addrspace(3)`	`.shared .align 4 .b8 s[1024];`
`__syncthreads()`	`call void @llvm.nvvm.barrier0()`	`bar.sync 0;`
`atomicAdd(&x, v)`	`atomicrmw add`	`atom.global.add.u32`

CUDA construct	NVVM IR	PTX
`__shfl_down_sync(m,v,d)`	`@llvm.nvvm.shfl.sync.down.f32`	`shfl.sync.down.b32`

Conventions: triple `nvptx64-nvidia-cuda`; **address spaces**: 0 generic, 1 global, 3 shared, 4 constant, 5 local; thread and block coordinates are *intrinsics* reading special registers; kernels are marked by metadata. A pointer in the generic space is converted with `addrspacecast` (PTX `cvta`).

14.3 PTX: the virtual ISA ★★★

PTX in ten facts

1. PTX is a **low-level, assembly-like, text** instruction set for a *virtual* GPU, documented publicly and versioned (`.version 8.x`); it is the stable target for compilers and the unit that NVIDIA's driver can JIT.
2. It has **unlimited typed virtual registers**: declared in banks like `.reg .b32 %r<10>;` (meaning `%r0..%r9`); predicates are `.pred`. `ptxas` maps them to the real register file.
3. **Types:** untyped bit-size `.b8/.b16/.b32/.b64`; unsigned `.u*`; signed `.s*`; floating `.f16 .bf16 .f32 .f64` (plus packed and low-precision tensor types). Instructions state their type: `add.s32`, `add.f32`. There is no 8-bit arithmetic: bytes are loaded/stored as `.u8/.s8` and computed in 16/32 bits.
4. **Instruction format:** `[@{!}pred] opcode[.mod]*.type dst, src1, src2;`. Modifiers select rounding (`.rn .rz .rm .rp`), saturation (`.sat`), flush-to-zero (`.ftz`), approximation (`.approx`), width (`.lo .hi .wide`), comparison operator (`setp.lt`), cache and memory-order qualifiers.
5. **Predication:** any instruction can be guarded by a predicate register: `@%p1 bra L;`, `@!%p2 st.global.f32 [%rd], %f;`. Short conditionals become predicated or `selp` (select) code with *no branch*.
6. **State spaces** name where data lives (next table); `ld/st` must say which. `cvta` converts between a space-specific and a generic address.
7. **Special registers:** `%tid.{x,y,z}`, `%ntid`, `%ctaid`, `%nctaid`, `%laneid`, `%warpid`, `%smid`, `%clock`, `%globaltimer`, lane masks.
8. **Control flow:** labels and `bra`; `bra.uni` asserts that all active threads of a warp take the branch together (no divergence); `call`, `ret`, `exit`.
9. **Memory consistency:** `ld/st/atom/fence` take *ordering* (`.relaxed .acquire .release .sc`) and *scope* (`.cta .cluster .gpu .sys`) qualifiers; `bar.sync` is a CTA-wide barrier with memory ordering among participants.
10. **Entry points:** `.entry` = kernel (callable from the host, parameters in `.param` space); `.func` = device function. `.visible`/`.extern`/`.weak` control linkage.

State space	Scope / lifetime	CUDA analogue and notes
`.reg`	per thread	scalars/locals in registers; fastest
`.local`	per thread (off-chip, cached)	register spills, large per-thread arrays; *slow*, despite the name
`.shared`	per CTA (thread block); on-chip	`__shared__`; banked; needs `bar.sync` to communicate
`.global`	all threads, host-visible	device memory (`cudaMalloc`); cached in L2 (and L1)
`.const`	read-only, all threads	`__constant__`; broadcast-friendly cache
`.param`	kernel/function arguments	read with `ld.param`; kernel parameters live in a constant bank

14.3.1 Reading the real PTX of `vecadd`, line by line

```
$ clang++ -x cuda --cuda-device-only --cuda-gpu-arch=sm_80 -O2 -S vecadd.cu -o -   (comments trimmed)
```

```
.version 7.0
.target sm_80
.address_size 64
.visible .entry _Z6vecaddPKfS0_Pfi(
    .param .u64 p0,
```

```
    .param .u64 p1,
    .param .u64 p2,
    .param .u32 p3
)
{
    .reg .pred     %p<2>;
    .reg .b32     %r<6>;
    .reg .f32     %f<4>;
    .reg .b64     %rd<11>;
    ld.param.u32     %r1, [p3];
    mov.u32     %r2, %ctaid.x;
    mov.u32     %r3, %ntid.x;
    mov.u32     %r4, %tid.x;
    mad.lo.s32     %r5, %r2, %r3, %r4;
    setp.ge.s32     %p1, %r5, %r1;
    @%p1 bra     $L__BB0_2;
    ld.param.u64     %rd4, [p0];
    ld.param.u64     %rd5, [p2];
    cvta.to.global.u64     %rd6, %rd5;
    ld.param.u64     %rd7, [p1];
    cvta.to.global.u64     %rd8, %rd7;
    cvta.to.global.u64     %rd9, %rd4;
    mul.wide.s32     %rd10, %r5, 4;
    add.s64     %rd1, %rd6, %rd10;
    add.s64     %rd2, %rd8, %rd10;
    add.s64     %rd3, %rd9, %rd10;
    ld.global.f32     %f1, [%rd3];
    ld.global.f32     %f2, [%rd2];
    add.f32     %f3, %f1, %f2;
    st.global.f32     [%rd1], %f3;
$L__BB0_2:
    ret;
}
```

PTX	Meaning
`.version 7.0 / .target sm_80 / .address_size 64`	PTX ISA version, target architecture, 64-bit addresses
`.visible .entry _Z6vecadd\ldots( .param .u64 p0\ldots )`	kernel entry point (the C++ mangled name); four parameters: three pointers (64-bit) and an `int`
`.reg .pred %p<2>; .reg .b32 %r<6>; \ldots`	register banks: 2 predicates, 6 32-bit, 4 floats, 11 64-bit registers
`ld.param.u32 %r1, [p3]`	load the argument `n`
`mov.u32 %r2, %ctaid.x; mov %r3, %ntid.x; mov %r4, %tid.x`	read `blockIdx.x`, `blockDim.x`, `threadIdx.x`
`mad.lo.s32 %r5, %r2, %r3, %r4`	$i = \text{ctaid} \times \text{ntid} + \text{tid}$: a fused multiply-add keeping the *low* 32 bits
`setp.ge.s32 %p1, %r5, %r1; @%p1 bra $L__BB0_2;`	predicate $p_1 = (i \geq n)$; if true branch to the `ret` (the `if (i < n)` guard)
`cvta.to.global.u64 %rd6, %rd5`	convert a generic pointer parameter to a *global*-space address
`mul.wide.s32 %rd10, %r5, 4; add.s64 %rd1, %rd6, %rd10`	byte offset $= i \times 4$ widened to 64 bits (`.wide`), added to the base: `&c[i]`
`ld.global.f32 %f1, [%rd3]; ld.global.f32 %f2, [%rd2]`	the two loads `a[i]`, `b[i]` (global memory)
`add.f32 %f3, %f1, %f2; st.global.f32 [%rd1], %f3`	the addition and the store to `c[i]`
`ret;`	thread exit

Things a reviewer should notice: (1) no loop: the grid *is* the loop; (2) the bounds guard is a predicated branch, and threads with $i \geq n$ simply return; (3) `fadd` compiled to plain `add.f32` (clang's default contraction allows fusing; for NVCC `--fmad` controls the same); (4) every pointer is converted with `cvta`; `ptxas` will later turn the 64-bit offset arithmetic into `IMAD.WIDE` instructions.

14.4 More kernels, more PTX ★★★

Five small kernels, compiled for `sm_80` with PTX 7.8 enabled (needed for warp shuffles). Only the instructions that matter are shown.

kernels.cu

```
#include "cuda_stub.h"
#define BLOCK 256

__global__ void reduce_sum(const float* in, float* out, int n) {      // block-level tree reduction in shared memory
    __shared__ float smem[BLOCK];
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    smem[tid] = (i < n) ? in[i] : 0.0f;
    __syncthreads();
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) smem[tid] += smem[tid + s];
        __syncthreads();
    }
    if (tid == 0) out[blockIdx.x] = smem[0];
}
__global__ void hist(const unsigned char* data, int* bins, int n) {   // histogram with atomics
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) atomicAdd(&bins[data[i]], 1);
}
__global__ void warp_sum(const float* in, float* out) {               // warp-level reduction with shuffles
    float v = in[threadIdx.x];
    for (int o = 16; o > 0; o >>= 1) v += __shfl_down_sync(0xffffffffu, v, o);
    if (threadIdx.x == 0) out[0] = v;
}
__global__ void fma_k(float* o, const float* a, const float* b, const float* c) { *o = *a * *b + *c; }
__global__ void branchy(int* a) {                                     // short if/else: if-converted
    int i = threadIdx.x;
    if (i % 2 == 0) a[i] = a[i] * 2; else a[i] = a[i] + 1;
}
```

```
$ clang++ -x cuda --cuda-device-only --cuda-gpu-arch=sm_80 -O2 -S kernels.cu -o kernels.ptx
```

```
// ===== reduce_sum: shared memory, barriers, predicated loop
    .shared .align 4 .b8 smem[1024];
    setp.ge.s32     %p1, %r7, %r6;
    @%p1 bra     $L__BB0_2;
    st.shared.f32     [%rd3], %f8;
    bar.sync     0;
    setp.lt.u32     %p2, %r8, 2;
    @%p2 bra     $L__BB0_3;
    bra.uni     $L__BB0_6;
    setp.eq.s32     %p5, %r1, 0;
    @%p5 bra     $L__BB0_4;
    bra.uni     $L__BB0_5;
    ld.shared.f32     %f7, [smem];
    bar.sync     0;
    setp.lt.u32     %p4, %r8, 4;
    @%p4 bra     $L__BB0_3;
    setp.ge.s32     %p3, %r1, %r5;
    @%p3 bra     $L__BB0_8;
    ld.shared.f32     %f4, [%rd5];
    ld.shared.f32     %f5, [%rd3];
    add.f32     %f6, %f4, %f5;
    st.shared.f32     [%rd3], %f6;
    bra.uni     $L__BB0_8;
// ===== hist: global atomic
    ld.global.u8     %r6, [%rd2];
    atom.global.add.u32     %r7, [%rd8], 1;
// ===== warp_sum: shuffle reduction (fully unrolled)
    shfl.sync.down.b32    %f3, %f2, 16, 31, -1;
    add.f32     %f4, %f2, %f3;
    shfl.sync.down.b32    %f5, %f4, 8, 31, -1;
    add.f32     %f6, %f4, %f5;
    shfl.sync.down.b32    %f7, %f6, 4, 31, -1;
    add.f32     %f8, %f6, %f7;
// ===== branchy: if/else became a select (no branch, no divergence)
    and.b32      %r2, %r1, 1;
    setp.eq.b32     %p1, %r2, 1;
    add.s32     %r4, %r3, 1;
    shl.b32     %r5, %r3, 1;
    selp.b32     %r6, %r4, %r5, %p1;
```

What each kernel teaches

- **reduce_sum.** `__shared__` becomes a `.shared` array; `__syncthreads()` is `bar.sync 0`; the shrinking-stride loop is a predicated branch structure; threads with `tid >= s` skip the add but *still* reach the next

barrier (a barrier must be executed by *all* threads of the block that are not exited, otherwise the kernel deadlocks or is undefined: classic CUDA bug #1).

- **hist.** `atomicAdd` on global memory becomes `atom.global.add.u32`, which returns the old value (here unused). Atomics serialise on contention; privatise histograms in shared memory when bins are few.
- **warp_sum.** The loop was fully unrolled (5 steps for 32 lanes) into `shfl.sync.down.b32`: threads of a warp exchange registers with *no shared memory and no barrier*; the first operand after the destination is the source value, then the lane offset, the clamp, and the **member mask** (`-1` = all 32 lanes). Since Volta (independent thread scheduling) the mask is mandatory.
- **branchy.** For a tiny `if/else` the compiler computed *both* results and picked one with `selp`: no branch, therefore no divergence. A long or side-effecting body would stay a real branch; then lanes of one warp taking different sides would execute *serially* with the other lanes masked off.

14.4.1 Floating-point contraction: the same source, two PTX results

```
$ clang++ ... -ffp-contract=fast vs -ffp-contract=off   (kernel fma_k: *o = *a * *b + *c)
```

```
// -ffp-contract=fast
    fma.rn.f32     %f4, %f1, %f2, %f3;
// -ffp-contract=off
    mul.rn.f32     %f3, %f1, %f2;
    add.rn.f32     %f5, %f3, %f4;
```

Why this matters for verification

`fma.rn.f32` rounds *once*; `mul.rn` then `add.rn` round *twice*: results can differ in the last bit. NVCC contracts by default (`--fmad=true`); host compilers may or may not. A test that compares GPU and CPU floating-point results bit-for-bit will "fail" for reasons that are *not* bugs. Verification tests either pin the semantics (explicit `.rn` instructions that PTX never contracts, `--fmad=false`) or compare within a ULP tolerance (Chapter 17). Other classic differences: `-ftz` (denormals flushed), approximate `div/sqrt/sin` variants, and reduction order (floating-point addition is not associative, so parallel reductions are non-deterministic unless the order is fixed).

14.5 A generated coverage matrix: how the back end lowers each IR operation ★★

This is a miniature of what a compiler-verification script does: *generate* a test per (operation, type) pair, *run* the compiler under test, *parse* the result, and *report*. If a cell said `ERROR`, that would be a back-end crash worth filing.

IR operation x type -> PTX instruction chosen by the NVPTX back end (llc, sm_80)

```
import subprocess

STEM = {"add": "add", "sub": "sub", "mul": "mul", "sdiv": "div", "udiv": "div", "srem": "rem", "and": "and", "xor"
    : "xor",
       "shl": "shl", "lshr": "shr", "ashr": "shr", "fadd": "add", "fmul": "mul", "fdiv": "div"}

def lower(op, ty):
    ir = f"define {ty} @t({ty} %a, {ty} %b) {{\n  %r = {op} {ty} %a, %b\n  ret {ty} %r\n}}\n"
    p = subprocess.run(["llc", "-O2", "-march=nvptx64", "-mcpu=sm_80", "-o", "-"], input=ir, capture_output=True,
     text=True)
    if p.returncode: return "ERROR"
    ins = [l.strip().split()[0].rstrip(";") for l in p.stdout.splitlines() if l.strip() and l.strip()[0].isalpha()
     ]
    main = next((i for i in ins if i.startswith(STEM[op] + ".")), "?")
    mark = ""
    if any(i.startswith("cvt.") for i in ins): mark += "*"                          # widening/narrowing helper
     instructions
    if "bra.uni" in ins: mark += "+"                                               # fast-path branch
    return main + mark

types = ["i8", "i16", "i32", "i64", "float", "double"]
print("%-6s" % "op" + "".join("%-14s" % t for t in types))
for op in ["add", "sub", "mul", "sdiv", "udiv", "srem", "and", "xor", "shl", "lshr", "ashr", "fadd", "fmul", "fdiv
    "]:
    row = []
    for t in types:
```

```
        isfp = t in ("float", "double")
        row.append(lower(op, t) if (op.startswith("f") == isfp) else "-")
    print("%-6s" % op + "".join("%-14s" % c for c in row))
print("\n* = helper cvt (widening/narrowing) instructions added;  + = extra branch (fast path)")
```

Captured output

```
op    i8            i16           i32           i64           float         double
add   add.s16*      add.s16*      add.s32       add.s64       -             -
sub   sub.s16*      sub.s16*      sub.s32       sub.s64       -             -
mul   mul.lo.s16*   mul.lo.s16*   mul.lo.s32    mul.lo.s64    -             -
sdiv  div.s16*      div.s16*      div.s32       div.s64*+     -             -
udiv  div.u16*      div.u16*      div.u32       div.u64*+     -             -
srem  rem.s16*      rem.s16*      rem.s32       rem.s64*+     -             -
and   and.b16*      and.b16*      and.b32       and.b64       -             -
xor   xor.b16*      xor.b16*      xor.b32       xor.b64       -             -
shl   shl.b16*      shl.b16*      shl.b32       shl.b64       -             -
lshr  shr.u16*      shr.u16*      shr.u32       shr.u64       -             -
ashr  shr.s16*      shr.s16*      shr.s32       shr.s64       -             -
fadd  -             -             -             -             add.rn.f32    add.rn.f64
fmul  -             -             -             -             mul.rn.f32    mul.rn.f64
fdiv  -             -             -             -             div.rn.f32    div.rn.f64

* = helper cvt (widening/narrowing) instructions added;  + = extra branch (fast path)
```

What the table shows: (1) `i8`/`i16` arithmetic is done with **16-bit PTX instructions** (`add.s16`) and widened/masked with helpers: PTX has no 8-bit ALU, so *type legalisation* is a source of bugs; (2) `i32` maps one-to-one (`mul.lo.s32`: the low half of the 64-bit product); (3) **`i64` division** carries a *branch*: the back end first tests whether both operands fit in 32 bits and uses the much cheaper 32-bit divide when they do ("bypass slow division"): a target-specific optimisation, and exactly the sort of path-dependent code that needs tests with values on both sides of the threshold; (4) floating-point operations carry the explicit rounding modifier `.rn`; (5) shifts distinguish arithmetic (`shr.s`) from logical (`shr.u`).

A verification engineer reads a table like this and asks . . .

Which *input values* reach every path (boundaries: 0, ±1, `INT_MIN`, `INT_MAX`, values around 2^{32} for the 64-bit divide fast path, shift amounts ≥ width)? Is the *runtime result* identical to a reference for each cell, not only that the compiler did not crash? That is the difference between a smoke test and verification.

14.6 SASS: the machine code ★★

SASS is the native instruction set of the GPU generation (e.g. Volta/Ampere/Hopper), produced by `ptxas` and stored in the **cubin**. NVIDIA documents PTX in detail but SASS only at the level of instruction lists (*CUDA Binary Utilities* manual), and it changes between architectures. Inspect it with `cuobjdump -sass a.out` or `nvdisasm kernel.cubin`; see PTX with `cuobjdump -ptx`.

Typical SASS mnemonic	Role (varies by architecture)
`S2R`, `MOV`, `CS2R`	read a special register (thread/block ids, clock), move, read a 64-bit special register
`IMAD`, `IADD3`, `LEA`, `LOP3`, `SHF`	integer multiply-add (also used for moves and address math), 3-input add, address/shift, 3-input logic, funnel shift
`ISETP`, `FSETP`, `SEL`, `@P0 \ldots`	compare to predicate, select, predicated execution
`FADD`, `FMUL`, `FFMA`, `MUFU`	float add/mul/fused multiply-add; special-function unit (reciprocal, rsqrt, sin, exp2)
`LDG`, `STG`, `LDS`, `STS`, `LDL`, `LDC`	global, shared, local, constant memory load/store
`BAR.SYNC`, `BRA`, `EXIT`, `WARPSYNC`, `BSYNC`	barriers, branches, thread exit, warp-level (re)convergence
`SHFL`, `VOTE`, `ATOM/ATOMG`, `RED`	warp shuffle/vote, atomics
`HMMA`, `IMMA`, `DMMA`	tensor-core matrix-multiply-accumulate (FP16/BF16, INT8, FP64)

What `ptxas` adds on top of PTX: register allocation (against the per-thread register limit), **static instruction scheduling** (GPUs issue in order; compiler-inserted control information tells the hardware about dependences and stalls), instruction selection (PTX `mad.lo.s32` may become `IMAD`; `mul.wide` `IMAD.WIDE`), constant-bank parameter access (`c[0x0][\ldots]`), memory-op fusion, divergence/reconvergence management, and many peepholes. Typical statistics to watch via `-Xptxas -v`: *registers per thread*, *spill stores/loads*, *shared memory per block*, *stack frame*.

14.7 How a team verifies a GPU compiler (the job, concretely) ★★★

1. **ISA conformance:** for every PTX instruction × type × modifier combination, generate a PTX program, run it on hardware/simulator, and compare with a *golden model* written in Python/C++ from the ISA text. Boundary values for every operand. (The matrix above is the cheap, compile-only cousin.)
2. **Optimisation-level and architecture differential testing:** compile the same program with `ptxas -O0..-O3`, different `--maxrregcount`, and for different `sm_XX`; run all; results must match (modulo documented FP differences). Compare JIT (driver) with offline `ptxas`; compare old PTX versions on new GPUs (forward compatibility).
3. **Random program generation (CLsmith/Csmith-style)** for CUDA/PTX: deterministic kernels built from well-defined operations, with inter-thread communication only through disciplined patterns (barriers, atomics) so the expected result is independent of the schedule. Chapter 17.
4. **Control-flow and convergence stress:** nested/irregular divergence, `bra.uni`, barriers inside branches (legal only with uniform conditions), `__syncwarp` masks, early exits, loops with divergent trip counts, *independent thread scheduling* hazards.
5. **Memory-model litmus tests:** message-passing and store-buffering patterns with each ordering/scope qualifier (`.relaxed/.acquire/.release/.sc` at `.cta/.gpu/.sys`), run millions of times to expose illegal outcomes.
6. **Resource limits:** maximum registers (spills), huge shared memory, many parameters, very large kernels (compile time), deep call stacks, large constant banks, odd launch configurations (block sizes not multiples of 32).
7. **Numerics:** IEEE rounding modes, denormals (`.ftz`), NaN/Inf/−0, saturation, conversions between all types; math-function accuracy in ULPs against a high-precision reference; FMA contraction on/off.
8. **Sanitised runs:** `compute-sanitizer` tools (`memcheck`, `racecheck`, `synccheck`, `initcheck`) over the test corpus catch bugs in *tests* (which then do not poison the results) and sometimes in the compiler.
9. **Regression infrastructure:** nightly builds of the compiler, sharded execution over a GPU farm (and boards for embedded), results database, automatic triage (de-duplicate by crash signature), bisection to the offending change, reduction to a small PTX/CUDA reproducer, KPIs (pass rate, crash rate, compile-time and register-usage trends).

14.8 Embedded targets, cross-compilation and functional safety ★★

- NVIDIA's embedded platforms combine Arm CPU cores with GPU/accelerator IP (Jetson, DRIVE-class SoCs), and GPUs contain small control processors that moved from the proprietary Falcon ISA to **RISC-V** (NVIDIA states 10–40 RISC-V cores per chip; reported by Tom's Hardware). "Embedded compilers" in the job description plausibly include toolchains for such cores; the verification method is the same: build tests for the *host* architecture, **cross-compile** for the target, and execute on boards, simulators or emulators (e.g. QEMU).
- **Cross-compilation** is a single flag with Clang. The same C source for three targets (real output):

```
int mul9(int x)           { return x * 9; }
unsigned udiv8(unsigned x) { return x / 8; }
```

```
$ clang --target=<triple> -O2 -S xc.c -o -
```

```
--- aarch64-linux-gnu
mul9:                                   // @mul9
    add    w0, w0, w0, lsl #3
    ret
udiv8:                                  // @udiv8
    lsr    w0, w0, #3
    ret
--- riscv64-unknown-elf
mul9:                                   # @mul9
    slli    a1, a0, 3
    addw    a0, a0, a1
    ret
```

```
udiv8:                                  # @udiv8
    srliw    a0, a0, 3
    ret
--- riscv32-unknown-elf
mul9:                                   # @mul9
    slli    a1, a0, 3
    add    a0, a0, a1
    ret
udiv8:                                  # @udiv8
    srli    a0, a0, 3
    ret
```

Notice the RV32 vs. RV64 difference (`add` vs. `addw`, `srli` vs. `srliw`): word-size-specific semantics are a classic place for miscompilations (sign extension of 32-bit results in 64-bit registers).

- **Functional safety:** automotive software follows **ISO 26262** (hazard levels ASIL A–D); compilers used to build safety code are subject to **tool confidence** analysis, often via *tool qualification* evidence such as large validation suites and documented known-bug lists. Coding-standard compliance (MISRA C) is checked by static analysers. One senior NVIDIA compiler-verification posting asks for ISO 26262 qualification experience (J3 / search listings); for an intern it is background to recognise, not to master.
- Embedded constraints that shape tests: code size and stack depth limits, no floating-point hardware on some cores (soft-float libraries), alignment/endianness, interrupt safety, deterministic timing.

14.9 Practice ★★★

Q14.1 What is the difference between PTX and SASS, and why does NVIDIA keep both?
Answer: PTX is a documented, stable, virtual ISA (unlimited typed registers, text form) that serves as the compiler target and as a forward-compatible distribution format (the driver JIT-compiles it). SASS is the native machine code of one GPU generation, produced by `ptxas`. The split lets hardware change freely while software keeps working, and puts hardware-specific scheduling and register allocation in one place.

Q14.2 A binary was built with `-arch=sm_70`. It runs on an sm_80 GPU but is slow on first launch. Why?
Answer: `-arch=sm_70` embeds sm_70 SASS and `compute_70` PTX. SASS for sm_70 cannot run on sm_80 (different major version), so the driver JIT-compiles the embedded PTX for sm_80 at first use (then caches it). Build with `-gencode` for sm_80 as well to avoid the JIT.

Q14.3 Read: `@%p1 bra $L; ld.shared.f32 %f1, [%rd2]; bar.sync 0;`. What does each instruction do and what can go wrong with the barrier?
Answer: If predicate `p1` is true the thread branches to label `L`; otherwise it loads a 32-bit float from shared memory, then waits at a CTA-wide barrier. If the barrier sits in code that only *some* threads of the block execute (here, the branch may skip it), the other threads wait forever: deadlock or undefined behaviour. `bar.sync` must be reached by all non-exited threads of the CTA.

Q14.4 Why does `fadd` lower to `add.rn.f32` and what does `.rn` do?
Answer: `.rn` = round to nearest even, the IEEE default. PTX makes rounding explicit. An `add.rn` followed by `mul.rn` is not fused into an FMA by `ptxas`, whereas an instruction without an explicit rounding modifier may be contracted.

Q14.5 Why is `__shfl_down_sync(0xffffffff, \ldots)` preferred over reading a neighbour lane through shared memory?
Answer: It exchanges registers directly inside a warp: no shared-memory traffic, no bank conflicts, no `bar.sync`. The mask names the participating lanes, which Volta-and-later hardware needs because threads in a warp can be at different program counters.

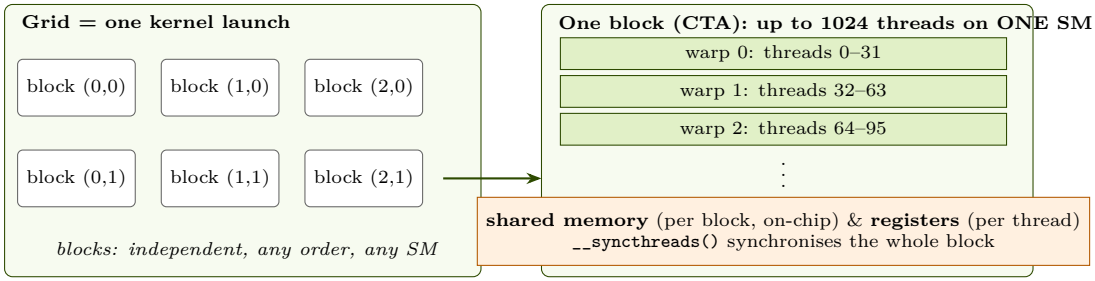
Q14.6 Which parts of a PTX test would you randomise, and which must be deterministic?
Answer: Randomise *input data* and program structure under a fixed, printed seed. Keep deterministic: the program's meaning (no data races, no uninitialised reads, defined behaviour for every operation), the launch configuration per test, and the comparison oracle (tolerances stated). A failing run must be reproducible from the seed.

15. CUDA and GPU Architecture

In this chapter

Priority ★★ (vocabulary ★★★). Time: 75 min. You will *write* CUDA and PTX test programs in this job, so you need the programmer-visible model (threads, warps, blocks, memory spaces) cold, and a feeling for what makes GPU code fast or wrong. The sandbox has no GPU and no CUDA SDK, so this chapter uses two honest substitutes: (1) four tiny *numeric models* in Python that answer the classic interview questions with numbers (occupancy, coalescing, bank conflicts, warp activity); (2) a 60-line **CUDA-on-CPU emulator**: the *same kernel source* is executed by `g++` on the CPU (each CUDA thread is a `std::thread`), checked against a CPU reference, race-checked with ThreadSanitizer, and compiled to real PTX by Clang. Timing numbers on a real GPU are therefore *not* measured here; hardware figures quoted from NVIDIA's public documents are marked as such.

15.1 The mental model ★★★



SIMT in six sentences

1. A CPU spends transistors on *latency* (big caches, out-of-order cores, branch prediction); a GPU spends them on *throughput*: thousands of simple lanes, and it hides memory latency by switching among many resident threads at zero cost (all their registers stay on chip).
2. You write **scalar code for one thread**; a launch `kernel<<<grid, block>>>(args)` creates `grid` × `block` threads. Each finds its work from `threadIdx`, `blockIdx`, `blockDim`, `gridDim`.
3. The hardware groups 32 consecutive threads into a **warp**, the unit that is scheduled and issues one instruction at a time (**SIMT**: single instruction, multiple threads).
4. A **block** (CTA, cooperative thread array) is resident on *one* SM until it finishes; its threads share `__shared__` memory and can call `__syncthreads()`.
5. Blocks of one grid are **independent**: they may run in any order, concurrently or one after another. The only cross-block communication inside a kernel is atomics and global memory; kernel boundaries are the global barrier. That rule is what lets one binary scale from a 2-SM laptop GPU to a 132-SM datacentre part.
6. Linear thread index: `i = blockIdx.x * blockDim.x + threadIdx.x`; blocks needed: `(n + block - 1) / block` (ceiling division); a **grid-stride loop** (`for (i = tid; i < n; i += gridDim.x * blockDim.x)`) makes a kernel correct for *any* launch size.

Software	Hardware	Limits (public NVIDIA figures; verify in the CUDA C++ Programming Guide)
thread	one lane of a warp; owns registers; has a program counter since Volta	up to 255 registers per thread
warp	32 threads, the scheduling/issue unit; a scheduler issues one warp instruction per cycle	warp size 32 on all NVIDIA GPUs to date
block / CTA	runs on one SM; own shared memory; barrier	≤ 1024 threads; dims $\leq (1024, 1024, 64)$; 48 KB static shared memory (up to ≈163 KB opt-in on A100)
cluster (Hopper+)	blocks co-scheduled on one GPC; *distributed* shared memory among them	portable cluster size ≤ 8

Software	Hardware	Limits (public NVIDIA figures; verify in the CUDA C++ Programming Guide)
grid	all blocks of a launch, spread over SMs	`gridDim.x` $\leq 2^{31}-1$; `y`, `z` ≤ 65535
SM	4 sub-partitions, each: warp scheduler, register file, FP32/INT32/FP64 units, tensor core, load/store, special-function unit; unified L1/shared memory	A100: 108 SMs, 2048 threads, 64K registers, up to 164 KB shared per SM; H100 (SXM): 132 SMs, up to 228 KB shared

Sources: NVIDIA CUDA C++ Programming Guide (chapters "Programming Model", "Hardware Implementation", appendix "Compute Capabilities"), the A100 and H100 architecture white papers, and the textbook *Programming Massively Parallel Processors* (Kirk & Hwu). NVIDIA's developer blog has a good "CUDA refresher" series.

15.2 Execution: warps, divergence, barriers ★★★

- **Latency hiding.** A load takes hundreds of cycles; while warp A waits, the scheduler issues instructions from warp B, C… This is why GPUs want *many* resident warps (**occupancy**, Section 15.4) and why a GPU kernel is fast only if there is enough parallelism and the memory accesses are friendly.
- **Divergence.** If lanes of one warp take different sides of a branch, the hardware executes *both* paths one after the other, with the non-participating lanes masked off, and reconverges afterwards. Cost ≈ sum of the paths. Branches that are uniform per warp (e.g. depending on `threadIdx.x / 32` or on `blockIdx`) cost nothing extra. Short `if`/`else` bodies are *if-converted* into predicated instructions or `selp` (we saw this in PTX, Chapter 14).
- **Independent thread scheduling** (Volta, 2017, compute capability 7.0, and later). Every thread has its own program counter and call stack, so divergent threads of a warp may interleave and spin-locks between lanes no longer deadlock the warp. The price: *reconvergence is not guaranteed at the end of an `if`*. Old "warp-synchronous" code that relied on lock-step (e.g. the last 32 steps of a reduction without barriers on `volatile` shared memory) is now **wrong**; use `__syncwarp(mask)` or the warp primitives `__shfl_*_sync`, `__ballot_sync`, `__activemask`, `__any_sync`, `__all_sync`, `__match_*_sync`, which all take an explicit **member mask**.
- **Barrier rules.** `__syncthreads()` must be reached by every non-exited thread of the block; if it sits inside a branch the condition must be *block-uniform*, else the behaviour is undefined (hang or silent wrong data). The barrier also orders memory: all shared- and global-memory writes before it are visible to the block after it.
- **Visibility across blocks.** There is no ordering between blocks within a kernel except through atomics and fences (`__threadfence()`, `cuda::atomic_ref` with scopes `cta`/`device`/`system` — the PTX `.scope` qualifiers). `volatile` is not a synchronisation primitive.

15.2.1 Worked number: why "sequential addressing" wins in a reduction

A block of 256 threads sums 256 values in shared memory in 8 steps. Version A (the textbook "interleaved addressing") lets thread `tid` with `tid % (2*s) == 0` add `sdata[tid + s]` for `s` $= 1, 2, 4, \ldots, 128$. Version B ("sequential addressing") lets `tid < s` add `sdata[tid + s]` for `s` $= 128, 64, \ldots, 1$. Count the warps that still have *any* active lane (each such warp occupies an issue slot) and the bank-conflict degree of the `sdata` accesses:

gpumodel.py: four tiny models

```
"""gpumodel.py -- four tiny models that answer the classic GPU interview questions with numbers."""
from math import ceil

# ---- 1. occupancy (simplified model of compute capability 8.0, e.g. A100) --------------------
SM = dict(max_threads=2048, max_blocks=32, regs=65536, smem=164 * 1024, max_warps=64)

def occupancy(threads, regs_per_thread, smem_per_block=0, sm=SM):
    warps = ceil(threads / 32)
    regs_block = warps * 32 * ceil(regs_per_thread / 8) * 8      # registers are allocated per warp in units of 256
    lim = {
        "threads": sm["max_threads"] // (warps * 32),
        "blocks":  sm["max_blocks"],
        "regs":    sm["regs"] // regs_block if regs_block else 99,
        "smem":    sm["smem"] // smem_per_block if smem_per_block else 99,
    }
    blocks = min(lim.values())
    limiter = [k for k, v in lim.items() if v == blocks][0]
    return blocks, blocks * warps / sm["max_warps"], limiter

# ---- 2. global-memory coalescing: 32-byte sectors touched by one warp ------------------------------
def sectors(addr_of_lane, elem=4):
    s = set()
```

```
    for lane in range(32):
        a = addr_of_lane(lane)
        s.update(range(a // 32, (a + elem - 1) // 32 + 1))
    return len(s)

# ---- 3. shared-memory bank conflicts: 32 banks x 4 bytes; same word = broadcast ----------
def conflict_degree(word_of_lane, active=range(32)):
    banks = {}
    for lane in active:
        w = word_of_lane(lane)
        banks.setdefault(w % 32, set()).add(w)
    return max(len(v) for v in banks.values())

# ---- 4. how many warps still issue instructions in a step (block of `threads` threads) ----
def warps_active(pred, threads=256):
    return len({t // 32 for t in range(threads) if pred(t)})
```

```
from gpumodel import conflict_degree, warps_active
print("step | A1: tid%(2s)==0      | A2: index=2*s*tid       | B: tid<s, index=tid")
print("     | warps  conflicts     | warps  worst conflict   | warps  worst conflict")
tot = [0, 0, 0]
for k in range(8):
    s = 1 << k                                  # stride used by the interleaved versions at this step
    n = 128 >> k                                # active threads in A2 and B (they halve every step)
    a1 = warps_active(lambda t: t % (2 * s) == 0)
    c1 = conflict_degree(lambda l: l, active=[l for l in range(32) if l % (2 * s) == 0])
    a2 = warps_active(lambda t: t < n)
    c2 = conflict_degree(lambda l: 2 * s * l, active=range(min(32, n)))
    b = warps_active(lambda t: t < n)
    cb = conflict_degree(lambda l: l, active=range(min(32, n)))
    tot[0] += a1; tot[1] += a2; tot[2] += b
    print(f"  {k}  |  {a1}    {c1}-way          |  {a2}     {c2}-way             |  {b}     {cb}-way")
print(f"total warp-steps: A1={tot[0]}  A2={tot[1]}  B={tot[2]}")
```

Captured output

```
step | A1: tid%(2s)==0      | A2: index=2*s*tid       | B: tid<s, index=tid
     | warps  conflicts     | warps  worst conflict   | warps  worst conflict
  0  |  8    1-way          |  4     2-way            |  4     1-way
  1  |  8    1-way          |  2     4-way            |  2     1-way
  2  |  8    1-way          |  1     8-way            |  1     1-way
  3  |  8    1-way          |  1     8-way            |  1     1-way
  4  |  8    1-way          |  1     8-way            |  1     1-way
  5  |  4    1-way          |  1     4-way            |  1     1-way
  6  |  2    1-way          |  1     2-way            |  1     1-way
  7  |  1    1-way          |  1     1-way            |  1     1-way
total warp-steps: A1=47  A2=12  B=12
```

Reading the result. A1 keeps *every* warp busy (with a few active lanes) for five of the eight steps: 47 warp-steps, caused by **divergence**. A2 removes the divergence by compacting the active threads (12 warp-steps) but its strided shared-memory index creates up to 8-way **bank conflicts**. B has A2's 12 warp-steps and no conflicts: that is why every production reduction is built this way (NVIDIA's classic deck *Optimizing Parallel Reduction in CUDA*, Mark Harris, walks through seven such versions and reports a speed-up of about 30× from first to last on its test GPU). Modern code goes one step further and finishes with warp shuffles (Section 15.5), skipping shared memory and barriers for the last 32 elements.

15.3 Memory hierarchy and access patterns ★★★

Space	Scope	Where / speed (order of magnitude)	Rule of thumb
registers	thread	on-chip register file; 1 cycle	255 max per thread; too many ⇒ *spills* to local memory and fewer resident warps
local memory	thread	off-chip (L1/L2 cached); slow like global	spills and dynamically indexed per-thread arrays; "local" means *private*, not fast
shared `__shared__`	block	on-chip SRAM, unified with L1; ∼20–30 cycles	32 banks of 4 bytes; explicit tiling; needs `__syncthreads()` between writers and readers
L1 / L2	SM / device	L2 shared by all SMs (40 MB on A100); atomics are resolved in L2	caches, transparent; L2 residency controls exist on newer parts
global	grid + host	HBM or GDDR; hundreds of cycles; 1.5–3 TB/s class	**coalesce**: a warp should touch few 32-byte sectors
constant `__constant__`	grid, read-only	64 KB + cache	fast when all lanes of a warp read the *same* address (broadcast); serialises otherwise

Space	Scope	Where / speed (order of magnitude)	Rule of thumb
texture / `__ldg`	grid, read-only	read-only data cache	2-D/spatial locality, free clamping and interpolation
host: pageable, pinned, managed	CPU + GPU	PCIe or NVLink; ∼10–50 GB/s link	transfers dominate small kernels; pinned memory enables async copies; `cudaMallocManaged` migrates pages on demand

15.3.1 Coalescing: how many sectors does a warp touch?

Global memory is served in 32-byte **sectors** (128-byte cache lines, 4 sectors each). A warp-wide 4-byte load needs at least $32 \times 4 = 128$ bytes $= 4$ sectors; every extra sector is wasted bandwidth. Model: count the sectors touched, for a handful of typical patterns.

```
from gpumodel import sectors, conflict_degree
patterns = [                                                # (access, byte address touched by lane l)
    ("a[i]                   unit stride",           lambda l: 4 * l),
    ("a[i+1]                 misaligned by one",     lambda l: 4 * (l + 1)),
    ("a[2*i]                 stride 2",              lambda l: 8 * l),
    ("a[4*i]                 stride 4",              lambda l: 16 * l),
    ("a[32*i]                a column of a matrix",  lambda l: 128 * l),
    ("p[i].x   struct{x,y,z} array of structures",   lambda l: 12 * l),
    ("p[i].x   struct of 4 floats",                  lambda l: 16 * l),
    ("x[i]     separate arrays   structure of arrays", lambda l: 4 * l),
]
print("-- one warp, one 4-byte load per lane; useful = share of the fetched bytes that are used")
for name, addr in patterns:
    n = sectors(addr)
    print(f"{name:46s} sectors={n:2d}  useful={128 / (32 * n) * 100:5.1f}%")
banks = [                                                   # (access, shared-memory word read by lane l)
    ("s[tid]",                                      lambda l: l),
    ("s[2*tid]",                                    lambda l: 2 * l),
    ("s[4*tid]",                                    lambda l: 4 * l),
    ("s[32*tid]",                                   lambda l: 32 * l),
    ("s[33*tid]",                                   lambda l: 33 * l),
    ("s[0]  all lanes read one word (broadcast)",   lambda l: 0),
    ("tile[tid][0]   float tile[32][32]",           lambda l: l * 32),
    ("tile[tid][0]   float tile[32][33] (padded)",  lambda l: l * 33),
]
print()
print("-- shared memory: 32 banks x 4 bytes; conflict degree = serialised passes for one warp")
for name, word in banks:
    print(f"{name:46s} {conflict_degree(word)}-way")
```

Captured output

```
-- one warp, one 4-byte load per lane; useful = share of the fetched bytes that are used
a[i]                   unit stride            sectors= 4  useful=100.0%
a[i+1]                 misaligned by one      sectors= 5  useful= 80.0%
a[2*i]                 stride 2               sectors= 8  useful= 50.0%
a[4*i]                 stride 4               sectors=16  useful= 25.0%
a[32*i]                a column of a matrix   sectors=32  useful= 12.5%
p[i].x   struct{x,y,z} array of structures    sectors=12  useful= 33.3%
p[i].x   struct of 4 floats                   sectors=16  useful= 25.0%
x[i]     separate arrays   structure of arrays sectors= 4  useful=100.0%

-- shared memory: 32 banks x 4 bytes; conflict degree = serialised passes for one warp
s[tid]                                        1-way
s[2*tid]                                      2-way
s[4*tid]                                      4-way
s[32*tid]                                     32-way
s[33*tid]                                     1-way
s[0]  all lanes read one word (broadcast)     1-way
tile[tid][0]   float tile[32][32]             32-way
tile[tid][0]   float tile[32][33] (padded)    1-way
```

Two rules that explain most GPU memory performance

1. **Make neighbouring threads touch neighbouring addresses** in global memory (the lane index should be the *fastest-varying* index: `a[row * N + col]` with `col = threadIdx.x + ...`). Prefer **structure of arrays** to array of structures; a column walk costs 32 sectors where a row walk costs 4.
2. **Make neighbouring threads touch different shared-memory banks.** Same bank, different words = serialised; same word = broadcast (free). The classical fix for a transposed or column access is **padding the row by one word**: `float tile[32][33]` (used in the transpose kernel below), because the extra

word shifts each row by one bank.

15.4 Occupancy and the roofline ★★

Occupancy = resident warps per SM / maximum warps per SM. A block can become resident only if *all* its needs fit: warp slots (threads), block slots, registers (allocated per warp), and shared memory. The scarcest resource decides. The model below is simplified (it ignores allocation granularities beyond the register unit); NVIDIA's Occupancy Calculator spreadsheet and the API `cudaOccupancyMaxActiveBlocksPerMultiprocessor` are the exact tools.

```
from gpumodel import occupancy
configs = [                          # (label, threads/block, registers/thread, shared bytes/block)
    ("256 thr, 32 regs/thr",                    256,  32, 0),
    ("256 thr, 64 regs/thr",                    256,  64, 0),
    ("256 thr, 128 regs/thr",                   256, 128, 0),
    ("256 thr, 32 regs, 48 KB shared/block",    256,  32, 48 * 1024),
    ("1024 thr, 64 regs/thr",                  1024,  64, 0),
    ("32 thr, 32 regs/thr (tiny blocks)",        32,  32, 0),
    ("128 thr, 255 regs/thr",                   128, 255, 0),
]
print("A100-like SM: 2048 threads, 32 blocks, 65536 registers, 164 KB shared memory, 64 warps")
for name, threads, regs, smem in configs:
    blocks, occ, limiter = occupancy(threads, regs, smem)
    print(f"{name:40s} blocks/SM={blocks:2d}  occupancy={occ * 100:5.1f}%  limited by: {limiter}")
```

Captured output

```
A100-like SM: 2048 threads, 32 blocks, 65536 registers, 164 KB shared memory, 64 warps
256 thr, 32 regs/thr                     blocks/SM= 8  occupancy=100.0%  limited by: threads
256 thr, 64 regs/thr                     blocks/SM= 4  occupancy= 50.0%  limited by: regs
256 thr, 128 regs/thr                    blocks/SM= 2  occupancy= 25.0%  limited by: regs
256 thr, 32 regs, 48 KB shared/block     blocks/SM= 3  occupancy= 37.5%  limited by: smem
1024 thr, 64 regs/thr                    blocks/SM= 1  occupancy= 50.0%  limited by: regs
32 thr, 32 regs/thr (tiny blocks)        blocks/SM=32  occupancy= 50.0%  limited by: blocks
128 thr, 255 regs/thr                    blocks/SM= 2  occupancy= 12.5%  limited by: regs
```

"Maximise occupancy" is not the goal

Occupancy is a means to hide latency, not a score. A kernel with 25% occupancy whose threads each keep several independent loads/FMAs in flight (instruction-level parallelism, more registers per thread) often beats a 100%-occupancy kernel (V. Volkov, *Better performance at lower occupancy*, GTC 2010). Conversely, very low occupancy (a few warps) cannot hide a 400-cycle load. Tools: `-Xptxas -v` (registers, spills, shared bytes), `__launch_bounds__(maxThreads, minBlocks)` and `--maxrregcount` to steer register allocation, Nsight Compute for achieved occupancy.

15.4.1 Compute-bound or memory-bound? A 10-line roofline

```
bw, peak = 1555e9, 19.5e12        # A100 40GB datasheet: HBM2e bandwidth (B/s), FP32 peak (FLOP/s)
print(f"ridge point = peak / bandwidth = {peak / bw:.1f} FLOP per byte of DRAM traffic\n")
kernels = {
    "vecadd  c=a+b   (1 FLOP per 12 B)": 1 / 12,
    "naive matmul    (2N FLOP per 8N B)": 2 / 8,
    "tiled 16x16     (traffic / 16)":     2 * 16 / 8,
    "tiled 32x32     (traffic / 32)":     2 * 32 / 8,
}
for name, ai in kernels.items():
    rate = min(peak, ai * bw)
    print(f"{name:36s} {ai:5.2f} FLOP/B -> at most {rate / 1e9:7.0f} GFLOP/s = {rate / peak * 100:5.1f}% of peak")
```

Captured output

```
ridge point = peak / bandwidth = 12.5 FLOP per byte of DRAM traffic

vecadd  c=a+b   (1 FLOP per 12 B)     0.08 FLOP/B -> at most     130 GFLOP/s =   0.7% of peak
naive matmul    (2N FLOP per 8N B)    0.25 FLOP/B -> at most     389 GFLOP/s =   2.0% of peak
tiled 16x16     (traffic / 16)        4.00 FLOP/B -> at most    6220 GFLOP/s =  31.9% of peak
tiled 32x32     (traffic / 32)        8.00 FLOP/B -> at most   12440 GFLOP/s =  63.8% of peak
```

Vector addition is hopelessly memory-bound (under 1% of peak FLOPs): its speed is set by DRAM bandwidth alone, so the only optimisations that matter are coalescing and not moving data (fusion). Tiling cuts the global traffic of a matrix multiply by the tile edge T (each loaded element is reused T times from shared memory), raising intensity to $T/4$ FLOP/B; getting beyond the ridge point needs register tiling and, on recent GPUs, tensor cores — exactly what libraries such as cuBLAS/CUTLASS do.

15.5 Run CUDA without a GPU: a 60-line emulator ★★

Idea. Keep the kernels in a `.cu` file that begins with `#include "cuda_dual.h"`. Under **Clang** (`__CUDA__` defined) the header pulls in the device-only stub from Chapter 14, so the file compiles to real PTX. Under **g++** the header turns CUDA into ordinary C++: every CUDA thread of a block becomes a `std::thread`, `__syncthreads()` becomes a `std::barrier`, `__shared__` becomes a `static` array (blocks run one at a time, so a static array *is* that block's shared memory), atomics use `std::atomic_ref`, and warp shuffles use a per-warp barrier plus exchange slots. Threads that finish call `arrive_and_drop()`, mimicking the hardware rule that exited threads do not block barriers.

cuda_dual.h (CPU emulation + device-only stub)

```
// cuda_dual.h -- one kernel source, two compilers.
//   clang++ -x cuda --cuda-device-only ...   -> PTX  (device-only stub from the previous chapter)
//   g++ -std=c++20 -pthread                  -> runs on the CPU: every CUDA thread is a std::thread
#pragma once
#ifdef __CUDA__
#include "cuda_stub.h"
__device__ static inline int __shfl_down_sync(unsigned m, int v, int d) { return __nvvm_shfl_sync_down_i32(m, v, d, 0x1f); }
#else
#include <algorithm>
#include <atomic>
#include <barrier>
#include <cstdint>
#include <cstdio>
#include <cstring>
#include <memory>
#include <thread>
#include <vector>

#define __global__
#define __device__
#define __host__
#define __shared__ static          // blocks run one at a time, so a static array *is* that block's shared memory
#define __launch_bounds__(...)

struct dim3 { unsigned x, y, z; dim3(unsigned x_ = 1, unsigned y_ = 1, unsigned z_ = 1) : x(x_), y(y_), z(z_) {} };
inline thread_local dim3 threadIdx;
inline dim3 blockIdx, blockDim, gridDim;

namespace emu {
inline std::barrier<>* block_bar;                                  // __syncthreads
inline std::vector<std::unique_ptr<std::barrier<>>> warp_bar;  // __syncwarp and shuffles
inline uint32_t warp_val[32][32];                                  // [warp][lane] exchange slots for shuffles
inline thread_local unsigned lin_tid;
}
inline void __syncthreads()                { emu::block_bar->arrive_and_wait(); }
inline void __syncwarp(unsigned = ~0u)     { emu::warp_bar[emu::lin_tid / 32]->arrive_and_wait(); }
template <class T> inline T __shfl_down_sync(unsigned, T v, int delta, int width = 32) {
    static_assert(sizeof(T) == 4, "4-byte values only");
    unsigned w = emu::lin_tid / 32, lane = emu::lin_tid % 32;
    std::memcpy(&emu::warp_val[w][lane], &v, 4);
    emu::warp_bar[w]->arrive_and_wait();                           // everybody has published its value
    T r = v;                                                       // out-of-range source lane: keep own value
    if ((lane % width) + delta < (unsigned)width) std::memcpy(&r, &emu::warp_val[w][lane + delta], 4);
    emu::warp_bar[w]->arrive_and_wait();                           // everybody has read before slots are reused
    return r;
}
inline int   atomicAdd(int* p, int v)     { return std::atomic_ref<int>(*p).fetch_add(v); }
inline float atomicAdd(float* p, float v) { return std::atomic_ref<float>(*p).fetch_add(v); }

template <class K, class... A>
void launch(dim3 grid, dim3 block, K kernel, A... args) {
    gridDim = grid; blockDim = block;
    const unsigned n = block.x * block.y * block.z, nw = (n + 31) / 32;
    for (unsigned bz = 0; bz < grid.z; ++bz) for (unsigned by = 0; by < grid.y; ++by) for (unsigned bx = 0; bx < grid.x; ++bx) {
        blockIdx = dim3(bx, by, bz);
        std::barrier<> bar(n);
        emu::block_bar = &bar;
        emu::warp_bar.clear();
        for (unsigned w = 0; w < nw; ++w) emu::warp_bar.push_back(std::make_unique<std::barrier<>>(std::min(32u, n - 32 * w)));
        std::vector<std::thread> ts;
        for (unsigned t = 0; t < n; ++t)
            ts.emplace_back([=, &bar] {
                emu::lin_tid = t;
                threadIdx = dim3(t % block.x, (t / block.x) % block.y, t / (block.x * block.y));
                kernel(args...);
                bar.arrive_and_drop();                             // an exited thread no longer blocks barriers
                emu::warp_bar[t / 32]->arrive_and_drop();
            });
        for (auto& th : ts) th.join();
    }
}
#define LAUNCH(kernel, grid, block, ...) launch(dim3(grid), dim3(block), kernel, __VA_ARGS__)
#endif
```

What the emulator is good for: checking *logic* (indexing, tiling, grid-stride loops, barriers, atomics) against a CPU reference, running unit tests on a machine without a GPU, and race detection with ThreadSanitizer (Section 15.6). **What it cannot do:** no lock-step warps (so no divergence or timing information), no GPU memory model (it obeys the C++ one), no performance, one OS thread per CUDA thread (fine for blocks up to 1024). For the real thing, a free Colab GPU runtime has `nvcc` and an NVIDIA GPU.

15.5.1 Six classic kernels, one source

`vecadd_gs` (grid-stride loop), `reduce_shfl` (warp-shuffle + shared-memory block reduction), `matmul_tiled` (shared-memory tiling), `transpose32` (padded tile), `hist_priv` (privatised histogram with shared-memory atomics), `scan_hs` (Hillis–Steele inclusive scan with ping-pong buffers).

k15.cu: the kernels

```
#include "cuda_dual.h"
#define TILE 16

__global__ void vecadd_gs(const int* a, const int* b, int* c, int n) {          // grid-stride loop
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n; i += gridDim.x * blockDim.x)
        c[i] = a[i] + b[i];
}

__global__ void reduce_shfl(const int* in, int* out, int n) {                    // block sum: shuffle + shared
    __shared__ int warp_sums[32];
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int v = (i < n) ? in[i] : 0;
    for (int o = 16; o > 0; o >>= 1) v += __shfl_down_sync(0xffffffffu, v, o);   // sum inside each warp
    int lane = threadIdx.x % 32, warp = threadIdx.x / 32;
    if (lane == 0) warp_sums[warp] = v;
    __syncthreads();
    if (warp == 0) {                                                             // first warp adds the warp sums
        v = (lane < (int)(blockDim.x / 32)) ? warp_sums[lane] : 0;
        for (int o = 16; o > 0; o >>= 1) v += __shfl_down_sync(0xffffffffu, v, o);
        if (lane == 0) atomicAdd(out, v);
    }
}

__global__ void matmul_tiled(const float* A, const float* B, float* C, int N) {  // C = A * B, N multiple of TILE
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];
    int row = blockIdx.y * TILE + threadIdx.y, col = blockIdx.x * TILE + threadIdx.x;
    float acc = 0.0f;
    for (int t = 0; t < N / TILE; ++t) {
        As[threadIdx.y][threadIdx.x] = A[row * N + t * TILE + threadIdx.x];
        Bs[threadIdx.y][threadIdx.x] = B[(t * TILE + threadIdx.y) * N + col];
        __syncthreads();                                                         // tile fully loaded
        for (int k = 0; k < TILE; ++k) acc += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        __syncthreads();                                                         // everyone done before overwrite
    }
    C[row * N + col] = acc;
}

__global__ void transpose32(const float* in, float* out, int N) {                // 32x32 tile, padded to dodge bank conflicts
    __shared__ float tile[32][33];
    int x = blockIdx.x * 32 + threadIdx.x, y = blockIdx.y * 32 + threadIdx.y;
    for (int j = 0; j < 32; j += 8) tile[threadIdx.y + j][threadIdx.x] = in[(y + j) * N + x];
    __syncthreads();
    x = blockIdx.y * 32 + threadIdx.x; y = blockIdx.x * 32 + threadIdx.y;        // swapped block coordinates
    for (int j = 0; j < 32; j += 8) out[(y + j) * N + x] = tile[threadIdx.x][threadIdx.y + j];
}

#define BINS 16
__global__ void hist_priv(const unsigned char* data, int* bins, int n) {         // privatised histogram
    __shared__ int local[BINS];
    if (threadIdx.x < BINS) local[threadIdx.x] = 0;
    __syncthreads();
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n; i += gridDim.x * blockDim.x)
        atomicAdd(&local[data[i] % BINS], 1);                                    // contention stays on-chip
    __syncthreads();
    if (threadIdx.x < BINS) atomicAdd(&bins[threadIdx.x], local[threadIdx.x]);   // one global atomic per bin per block
}

__global__ void scan_hs(const int* in, int* out, int n) {                        // Hillis-Steele inclusive scan, one block
    __shared__ int buf[2][256];
    int t = threadIdx.x, cur = 0;
    buf[0][t] = (t < n) ? in[t] : 0;
    __syncthreads();
    for (int d = 1; d < n; d <<= 1) {
        buf[1 - cur][t] = (t >= d) ? buf[cur][t] + buf[cur][t - d] : buf[cur][t];
        cur = 1 - cur;                                                           // ping-pong: read one buffer, write the other
        __syncthreads();
    }
    if (t < n) out[t] = buf[cur][t];
}
```

Host harness for the CPU build. `LAUNCH(kernel, grid, block, args...)` stands for `kernel<<<grid, block>>>(args...)`; sizes are deliberately awkward (1000 elements, 100-element scan) to test the guards.

host15.cpp: CPU harness

```
#include "k15.cu"
#include <cstdio>
#include <numeric>
#include <vector>
using std::vector;

static void report(const char* name, bool ok) { std::printf("%-14s %s\n", name, ok ? "OK" : "MISMATCH"); }

int main() {
    const int n = 1000;                                  // deliberately not a multiple of the block size
    vector<int> a(n), b(n), c(n, -1), r(n);
    for (int i = 0; i < n; ++i) { a[i] = i; b[i] = 3 * i + 1; r[i] = a[i] + b[i]; }
    LAUNCH(vecadd_gs, 4, 64, a.data(), b.data(), c.data(), n);     // 256 threads cover 1000 elements
    report("vecadd_gs", c == r);

    vector<int> in(n);
    for (int i = 0; i < n; ++i) in[i] = (i * 7) % 13;
    int sum = 0, ref = std::accumulate(in.begin(), in.end(), 0);
    LAUNCH(reduce_shfl, (n + 255) / 256, 256, in.data(), &sum, n);
    std::printf("%-14s emulated=%d reference=%d %s\n", "reduce_shfl", sum, ref, sum == ref ? "OK" : "MISMATCH");

    const int N = 32;
    vector<float> A(N * N), B(N * N), C(N * N), R(N * N, 0.0f);
    for (int i = 0; i < N * N; ++i) { A[i] = float(i % 5); B[i] = float(i % 7 - 3); }
    for (int i = 0; i < N; ++i)
        for (int j = 0; j < N; ++j)
            for (int k = 0; k < N; ++k) R[i * N + j] += A[i * N + k] * B[k * N + j];
    LAUNCH(matmul_tiled, dim3(N / TILE, N / TILE), dim3(TILE, TILE), A.data(), B.data(), C.data(), N);
    report("matmul_tiled", C == R);

    vector<float> T(N * N, 0.0f), RT(N * N);
    for (int i = 0; i < N; ++i)
        for (int j = 0; j < N; ++j) RT[j * N + i] = A[i * N + j];
    LAUNCH(transpose32, dim3(N / 32, N / 32), dim3(32, 8), A.data(), T.data(), N);
    report("transpose32", T == RT);

    vector<unsigned char> d(5000);
    for (size_t i = 0; i < d.size(); ++i) d[i] = (unsigned char)((i * i + 3 * i) & 0xff);
    vector<int> h(BINS, 0), hr(BINS, 0);
    for (unsigned char x : d) hr[x % BINS]++;
    LAUNCH(hist_priv, 3, 128, d.data(), h.data(), (int)d.size());
    report("hist_priv", h == hr);

    vector<int> s(100), sr(100), so(100, -1);
    for (int i = 0; i < 100; ++i) s[i] = i % 4 + 1;
    std::partial_sum(s.begin(), s.end(), sr.begin());
    LAUNCH(scan_hs, 1, 128, s.data(), so.data(), 100);
    report("scan_hs", so == sr);
    return 0;
}
```

```
$ g++ -std=c++20 -pthread host15.cpp -o host15 && ./host15
```

```
vecadd_gs      OK
reduce_shfl    emulated=6000 reference=6000 OK
matmul_tiled   OK
transpose32    OK
hist_priv      OK
scan_hs        OK
```

15.5.2 The same kernels as PTX: what a compiler test would look at

```
$ clang++ -x cuda --cuda-device-only --cuda-gpu-arch=sm_80 -O2 -S k15.cu -o k15.ptx   # then count instructions per kernel
```

```
kernel        bar.sync shfl.sync atom.shared atom.global ld.shared st.shared ld.global st.global     fma
vecadd_gs            0         0           0           0         0         0         2         1       0
reduce_shfl          1        10           0           1         1         1         1         0       0
matmul_tiled         2         0           0           0        32         2         2         1      16
transpose32          1         0           0           0         4         4         4         4       0
hist_priv            2         0           1           1         1         1         1         0       0
scan_hs              2         0           0           0         3         2         1         1       0
```

Static instruction counts per kernel, read as a verification engineer would: `matmul_tiled` has 2 barriers (load phase, compute phase) and the 16-iteration inner loop was fully unrolled into 16 `fma.rn.f32` (the `acc += a*b` contraction); `reduce_shfl` contains 10 `shfl.sync` (2 unrolled warp reductions × 5 steps), one barrier and one global atomic; `transpose32` has one barrier, four loads and four stores (the `j` loop unrolled four times, no branches); `hist_priv` shows one `atom.shared` and one `atom.global`: the source used the same generic `atomicAdd(int*,`

`int)` for both, and the compiler's *address-space inference* proved which memory each pointer targets. A bug in that inference is exactly the kind of defect a GPU-compiler test suite is written to catch. The IR also shows the attribute that protects barriers:

```
$ clang++ ... -S -emit-llvm k15.cu | grep -E 'barrier0|attributes #2'
```

```
declare void @llvm.nvvm.barrier0() #2
attributes #2 = { convergent nocallback nounwind }
```

`convergent` means "this operation communicates with other threads that must execute it together": an optimiser may not make it *control-dependent on more conditions* than before (no jump threading or sinking of a barrier or shuffle into a divergent branch) — a rule every LLVM-based GPU compiler must implement and test.

15.6 Race detection with ThreadSanitizer ★★

On a GPU the tool is `compute-sanitizer --tool racecheck` (and `synccheck` for barrier misuse, `memcheck` for out-of-bounds, `initcheck` for uninitialised memory). In our emulator the C++ ThreadSanitizer does the same job for shared-memory bugs, because the barrier is the only thing ordering the threads of a block. Two classic bugs, each with a one-line description:

kbug.cu: two kernels with a missing barrier

```
#include "cuda_dual.h"
__global__ void scan_inplace_bug(const int* in, int* out, int n) {     // BUG: in-place Hillis-Steele, single buffer
    __shared__ int x[256];
    int t = threadIdx.x;
    x[t] = (t < n) ? in[t] : 0;
    __syncthreads();
    for (int d = 1; d < n; d <<= 1) {
        int v = (t >= d) ? x[t - d] : 0;      // read a neighbour's slot ...
        x[t] += v;                            // ... while that neighbour may already be overwriting it
        __syncthreads();
    }
    if (t < n) out[t] = x[t];
}
__global__ void tile_sync_bug(const float* A, float* out) {            // BUG: second barrier missing
    __shared__ float tile[32];
    float acc = 0;
    for (int t = 0; t < 4; ++t) {
        tile[threadIdx.x] = A[t * 32 + threadIdx.x];
        __syncthreads();
        acc += tile[(threadIdx.x + 1) % 32];  // read the neighbour's element
        /* missing __syncthreads() here */
    }
    out[threadIdx.x] = acc;
}
```

```
$ g++ -std=c++20 -g -fsanitize=thread hostbug.cpp && ./a.out   # report the kernel names only
```

```
ThreadSanitizer: data race in kernel scan_inplace_bug
ThreadSanitizer: data race in kernel tile_sync_bug
--- number of ThreadSanitizer reports for the six correct kernels (host15.cpp):
0
```

Reading it. The in-place scan reads `x[t-d]` while thread `t-d` may already be writing it in the same step (a read-after-write hazard inside one iteration, fixed by the ping-pong buffer in `scan_hs` or by a barrier between the read and the write); the tile kernel overwrites `tile[]` for the next `t` while a slow neighbour is still reading the old value. The correct kernels produce **zero** reports. On the GPU a hazard like this may give correct answers 999 times out of 1000 — which is why *tools* rather than test runs find them.

15.7 The CUDA bug catalogue ★★

Bug	Symptom	Catch / fix
missing barrier between shared-memory writers and readers (or before the buffer is overwritten)	intermittent wrong results; right 999 times in 1000	`compute-sanitizer --tool racecheck`; TSan on the emulator; barrier after each producer phase *and* before reuse

Bug	Symptom	Catch / fix
`__syncthreads()` inside a branch that is not block-uniform, or after a thread-dependent early `return`	hang or garbage; undefined behaviour	`--tool synccheck`; guard the *work*, not the barrier
no `if (i < n)` guard, or grid size rounded *down*	out-of-bounds writes; tail elements never computed	`--tool memcheck`; `(n + block - 1) / block`
32-bit index overflow (`row * N + col`, `int` loop counter)	wrong addresses for big arrays	use `size_t` / `long long`; see `mul.wide` in Chapter 14
warp-synchronous code without `__syncwarp`, or a wrong shuffle/ballot mask	rare wrong results on Volta and later	always pass the real participation mask; full mask `0xffffffff` only if the whole warp executes the call
non-atomic read-modify-write on shared data (`count++`)	lost updates	`atomicAdd`, or a reduction
float `atomicAdd` or a different summation order	results differ in the last bits between runs/devices	not a bug: floating-point addition is not associative; compare with a tolerance, or use a fixed-order tree
uninitialised shared or local memory	results change from launch to launch	`--tool initcheck`; (our emulator's static arrays start at zero and *hide* this bug)
host pointer used on the device or vice versa; forgotten copy	`invalid argument`, zeros, crashes	name pointers `h_x`/`d_x`; check every API call
errors seen late: a launch is *asynchronous*	failure surfaces at the next API call that synchronises	`CHECK(cudaGetLastError())` after the launch; `cudaDeviceSynchronize()` while debugging
too many resources (1024 threads × many registers; too much shared memory)	`too many resources requested for launch`	smaller block, `__launch_bounds__`, `--maxrregcount`

15.8 Host-side API cheat sheet ★★

The standard pattern (not compiled here: there is no CUDA SDK in the sandbox; the device half is exactly what we compiled above):

saxpy.cu: allocate, copy, launch, check, copy back

```
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>
#define CHECK(call) do { cudaError_t e = (call); if (e != cudaSuccess) {                        \
    std::fprintf(stderr, "%s:%d: %s\n", __FILE__, __LINE__, cudaGetErrorString(e)); std::exit(1); } } while (0)

__global__ void saxpy(int n, float a, const float* x, float* y) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) y[i] = a * x[i] + y[i];
}

int main() {
    const int n = 1 << 20;
    const size_t bytes = n * sizeof(float);
    float *hx = (float*)std::malloc(bytes), *hy = (float*)std::malloc(bytes), *dx, *dy;
    for (int i = 0; i < n; ++i) { hx[i] = 1.0f; hy[i] = 2.0f; }

    CHECK(cudaMalloc(&dx, bytes));  CHECK(cudaMalloc(&dy, bytes));
    CHECK(cudaMemcpy(dx, hx, bytes, cudaMemcpyHostToDevice));
    CHECK(cudaMemcpy(dy, hy, bytes, cudaMemcpyHostToDevice));

    int block = 256, grid = (n + block - 1) / block;     // ceiling division
    saxpy<<<grid, block>>>(n, 3.0f, dx, dy);              // asynchronous
    CHECK(cudaGetLastError());                             // launch-configuration errors
    CHECK(cudaDeviceSynchronize());                        // errors raised while the kernel ran

    CHECK(cudaMemcpy(hy, dy, bytes, cudaMemcpyDeviceToHost));
    std::printf("y[0] = %f (expect 5)\n", hy[0]);
    cudaFree(dx); cudaFree(dy); std::free(hx); std::free(hy);
}
```

API	Meaning
`cudaMalloc`, `cudaFree`, `cudaMemcpy(dst, src, bytes, kind)`	device allocation; blocking copy (`cudaMemcpyHostToDevice`, `cudaMemcpyDeviceToHost`, ...)
`cudaMallocHost`, `cudaMemcpyAsync(..., stream)`	pinned host memory and asynchronous copies that can overlap with kernels
`cudaMallocManaged`, `cudaMemPrefetchAsync`	unified (managed) memory: one pointer valid on host and device, pages migrate on demand
`cudaStream_t`, `cudaStreamCreate`, `<<<grid, block, shmem, stream>>>`	independent queues of work; same-stream work is ordered, different streams may overlap
`cudaEvent_t`, `cudaEventRecord`, `cudaEventElapsedTime`	GPU-side timestamps: the right way to time kernels (host timers miss asynchrony)
`cudaDeviceSynchronize`, `cudaStreamSynchronize`	host waits for all (or one stream's) work
`cudaGetLastError`, `cudaGetErrorString`, `cudaGetDeviceProperties`	error handling; query SM count, shared memory, compute capability
occupancy API	`cudaOccupancyMaxActiveBlocksPerMultiprocessor` gives the exact number of resident blocks for a given kernel and block size

Build and inspect: `nvcc -O2 -arch=sm_80 -lineinfo saxpy.cu -o saxpy`; `cuobjdump -sass -ptx saxpy` shows what is inside; `compute-sanitizer ./saxpy`; `ncu ./saxpy` (Nsight Compute: per-kernel hardware counters), `nsys profile ./saxpy` (Nsight Systems: timeline).

15.9 Recent hardware features: the one-line versions ★

Feature	Since	What it is, and why compiler people care
Tensor Cores (`wmma`, `mma.sync`, `wgmma`)	Volta (2017); 4th gen Hopper	small matrix-multiply-accumulate per warp (or warp group), mixed precision: FP16/BF16/TF32/INT8/FP8 inputs, wider accumulators; new PTX instruction families with fragment register layouts that the register allocator and scheduler must respect
Asynchronous copy: `cp.async`; **TMA** (tensor memory accelerator)	Ampere; Hopper	copy global→shared without staging through registers; TMA moves whole multi-dimensional tiles; synchronised with `mbarrier`s: new ordering rules to verify
Thread block clusters, distributed shared memory	Hopper	blocks of a cluster can read each other's shared memory: a new PTX memory scope (`.cluster`)
Cooperative Groups	CUDA 9	named groups (tile, warp, block, grid) with `sync()`, `reduce()`...; compile down to `bar`/`shfl`; grid-wide sync needs a cooperative launch
CUDA Graphs	CUDA 10	record a DAG of kernels/copies once and replay it with low CPU overhead
Dynamic parallelism	CUDA 5	kernels launching kernels
MIG, MPS	Ampere; Kepler	hardware partitioning / software sharing of one GPU among tenants
Lower-precision formats	Hopper onwards (FP8; newer parts add narrower formats)	more types in PTX and in NVVM IR; more FP semantics (rounding, saturation, NaN handling) for tests to cover

Ecosystem names worth recognising: cuBLAS, cuDNN, CUTLASS (templated GEMM building blocks), CUB and Thrust (parallel primitives), NCCL (multi-GPU collectives), TensorRT, NVRTC, **Triton** (a Python DSL whose compiler lowers to LLVM IR and PTX), Nsight Systems/Compute, `compute-sanitizer`.

Why the compiler-verification team cares about all of this

Each item above is a place where a GPU compiler can be wrong, and a test generator or a hand-written kernel can show it: **barriers and `convergent` operations** moved across divergent control flow; **address-space inference** (generic → shared/global/local); **if-conversion and divergence analysis** (`selp`, `bra.uni`); **memory ordering** around `bar.sync`, fences, atomics and scopes; **FP contraction, flush-to-zero and rounding** flags (`--fmad`, `-ftz`, `-prec-div`); **integer semantics** (`nsw` assumptions on index arithmetic, 32-bit vs 64-bit address math); **register allocation and spills** under `__launch_bounds__`; **vectorised loads** (`ld.v4`) that need alignment proofs; **warp-level primitives** with partial masks. Chapter 17 turns this list into a test plan.

15.10 Practice questions ★★

Q15.1 Define thread, warp, block and grid. Why should the block size be a multiple of 32, and what is a sensible default?
Answer: A *thread* runs the kernel body on its own registers; a *warp* is 32 consecutive threads issued together; a *block* is a group of up to 1024 threads that lives on one SM, shares shared memory and can synchronise; a *grid* is all blocks of a launch. Warps are the scheduling unit, so a block of 100 threads still occupies 4 warps and wastes 28 lanes. 128 or 256 is a good default: enough warps to hide latency, a divisor of the 2048-thread SM limit, and small enough that registers and shared memory still allow several resident blocks.

Q15.2 What is warp divergence, when is it free, and how do you reduce it?
Answer: Lanes of one warp taking different paths: the hardware runs the paths serially with masks, so cost ≈ sum of paths. Free when the condition is uniform across the warp (e.g. `threadIdx.x / 32`, `blockIdx`-dependent) and when the compiler predicates a short body. Reduce it by regrouping work so that neighbouring threads take the same path (sort/bin inputs, compact active threads as in reduction version B), and by replacing branches with arithmetic or `select`.

Q15.3 Difference between `__syncthreads()`, `__syncwarp()`, `__threadfence()` and `cudaDeviceSynchronize()`?
Answer: `__syncthreads()`: barrier for all threads of the block, with memory ordering for shared and global writes among them. `__syncwarp(mask)`: barrier and memory ordering among the named lanes of one warp. `__threadfence()`: only orders the calling thread's memory writes as seen by other threads of the device — *no* waiting. `cudaDeviceSynchronize()`: host-side call that blocks until all previously issued device work is finished.

Q15.4 Why does the tiled matrix multiply have two `__syncthreads()` per tile?
Answer: The first makes sure the whole tile has been loaded before any thread starts using it; the second makes sure all threads have finished using the tile before the next iteration overwrites it. Dropping the second one is the race that ThreadSanitizer flagged in `tile_sync_bug`.

Q15.5 A kernel uses 64 registers per thread, 256 threads per block and no shared memory. What is the occupancy on an A100-like SM, and what would you try?
Answer: Registers per block = $256 \times 64 = 16384$, so $65536/16384 = 4$ blocks = 1024 threads = 50% (table above). Try `--maxrregcount=32` or `__launch_bounds__(256, 8)` only if profiling shows latency stalls; the forced register cut may cause spills that cost more than the extra occupancy gains.

Q15.6 Why can the sum of a float array differ between CPU and GPU, and between two GPU runs?
Answer: Floating-point addition is not associative; the GPU adds in a tree (and with `atomicAdd` in an order that depends on scheduling), the CPU adds left to right. Results are all valid approximations. Compare with a relative tolerance, or use a fixed reduction tree (and integers or Kahan summation) for bit-reproducible results.

Q15.7 What is a bank conflict? Show a fix.
Answer: Two or more lanes of a warp access *different* 32-bit words in the same one of the 32 shared-memory banks in one instruction; the accesses are serialised. Same word = broadcast, no conflict. Column access to a `float tile[32][32]` is 32-way; padding the declaration to `tile[32][33]` makes it conflict-free (checked in the model above and used in `transpose32`).

Q15.8 Coalesced vs. uncoalesced: how many 32-byte sectors does `a[32*i]` touch per warp, and what is the fix?
Answer: 32 sectors for 128 useful bytes (12.5% efficiency) versus 4 for `a[i]`. Fix: change the data layout (structure of arrays), swap loop and thread indexing so the lane index is the fastest-varying one, or stage a tile through shared memory so the global loads are coalesced and the strided access happens on-chip.

Q15.9 PTX versus SASS versus cubin, and what does `-arch=sm_80` do?
Answer: PTX is NVIDIA's virtual ISA (text, forward compatible, JIT-able); SASS is the native machine code of one GPU generation; a cubin is the ELF container with SASS. `-arch=sm_80` is shorthand for `-gencode arch=compute_80,code=[sm_80,compute_80]`: emit SASS for sm_80 and also embed compute_80 PTX so newer GPUs can JIT it.

Q15.10 You are asked to test that a GPU compiler handles `__syncthreads()` correctly. What do you write?
Answer: Kernels where the barrier sits in straight-line code, in uniform loops, in nested uniform branches and after a thread-dependent *value* (but not a thread-dependent *control path*); each has a CPU reference (the emulator above can serve). Compile at several optimisation levels and `sm_` targets, run, compare outputs (differential testing), check the PTX with FileCheck for the expected `bar.sync` count and position, run `racecheck`/`synccheck`, and add negative tests for transformations that must *not* happen (barrier sunk into a divergent branch). Chapters 17 and 18 build such a harness.

16. Parallel Programming: Threads, Atomics, OpenMP, Patterns

In this chapter

Priority ★★ (the job description lists "parallel programming" as a stand-out skill). Time: 60 min. Everything here was compiled and run in the sandbox (4 virtual CPUs, GCC 13, ThreadSanitizer, OpenMP, Python 3.13). **Timings are real but machine-dependent**: read them for the *ratio*, not the absolute numbers; counts of race outcomes vary from run to run by design. Chapter 15 applied the same ideas (races, barriers, atomics, reductions, scans) to GPUs.

16.1 Vocabulary and speed-up laws ★★

- **Concurrency** = several tasks in progress, structured so they *may* interleave (one core is enough); **parallelism** = several tasks executing *simultaneously* (needs several execution units).
- **Flynn's taxonomy:** SISD (one instruction stream, one data stream: classic CPU core), **SIMD** (one instruction on many data: AVX/NEON vector units), MISD (rare), **MIMD** (independent cores: multicore CPUs, clusters). A GPU is **SIMT**: SIMD hardware presented as many scalar threads with per-thread state.
- **Kinds of parallelism:** *data* (same operation on many elements: GPU kernels, `#pragma omp parallel for`), *task* (different jobs: compile 100 files at once), *pipeline* (stages in series, each busy on a different item).
- **Speed-up** $S(p) = T_1/T_p$, **efficiency** $E = S/p$. **Amdahl:** if a fraction f of the work is parallelisable, $S(p) \le 1/\big((1-f) + f/p\big)$, so the serial part caps the speed-up at $1/(1-f)$. **Gustafson:** if the problem grows with the machine, scaled speed-up is $(1-f) + fp$.
- **Work and span:** $W = T_1$ (total operations), $S_\infty = T_\infty$ (critical-path length, "depth"); **Brent's bound** $T_p \le W/p + S_\infty$; available parallelism $= W/S_\infty$. A good parallel algorithm has work close to the best sequential algorithm and a small span.

```
from math import log2
from itertools import accumulate

def amdahl(f, p):    return 1 / ((1 - f) + f / p)            # fixed problem size
def gustafson(f, p): return (1 - f) + f * p                  # problem grows with p

print("Amdahl speed-up S(p) = 1/((1-f)+f/p)")
print("   f      p=4     p=16     p=64   p=1024   limit 1/(1-f)")
for f in (0.50, 0.90, 0.95, 0.99):
    row = "  ".join(f"{amdahl(f, p):7.2f}" for p in (4, 16, 64, 1024))
    print(f" {f:4.2f}  {row}   {1 / (1 - f):8.1f}")
print(f"\nGustafson with f=0.95, p=64: {gustafson(0.95, 64):.1f}   (Amdahl, same f and p: {amdahl(0.95, 64):.1f})")

# ---- prefix sum (scan): work = number of additions, span = number of parallel steps ----------------
def hillis_steele(a):                                        # inclusive; every step adds a shifted copy
    a, work, steps, d = list(a), 0, 0, 1
    while d < len(a):
        a = [a[i] + a[i - d] if i >= d else a[i] for i in range(len(a))]
        work += len(a) - d; steps += 1; d *= 2
    return a, work, steps

def blelloch(a):                                             # exclusive; up-sweep then down-sweep; n = 2**k
    a, n, work, steps, d = list(a), len(a), 0, 0, 1
    while d < n:                                             # up-sweep: build partial sums (a reduction tree)
        for i in range(2 * d - 1, n, 2 * d): a[i] += a[i - d]; work += 1
        steps += 1; d *= 2
    a[n - 1] = 0
    d = n // 2
    while d >= 1:                                            # down-sweep: distribute the prefixes
        for i in range(2 * d - 1, n, 2 * d): a[i - d], a[i] = a[i], a[i] + a[i - d]; work += 1
        steps += 1; d //= 2
    return a, work, steps

n = 1024
data = [(i * 7) % 10 for i in range(n)]
ref = list(accumulate(data))
hs, w1, s1 = hillis_steele(data)
bl, w2, s2 = blelloch(data)
assert hs == ref and [x + y for x, y in zip(bl, data)] == ref                # both verified against itertools
print(f"\nscan of n={n}: sequential work={n - 1} span={n - 1} | Hillis-Steele work={w1} span={s1} | Blelloch work={w2} span={s2}")
N, p = 2 ** 20, 1024                                                         # closed forms, checked above for n=1024
W_hs, S_hs, W_bl, S_bl = N * int(log2(N)) - N + 1, int(log2(N)), 2 * (N - 1), 2 * int(log2(N))
print(f"Brent bound T_p <= W/p + S for n=2**20, p={p}: Hillis-Steele {W_hs / p + S_hs:.0f}, Blelloch {W_bl / p + S_bl:.0f}")
```

Captured output

```
Amdahl speed-up S(p) = 1/((1-f)+f/p)
   f      p=4     p=16     p=64   p=1024   limit 1/(1-f)
 0.50     1.60     1.88     1.97     2.00        2.0
 0.90     3.08     6.40     8.77     9.91       10.0
 0.95     3.48     9.14    15.42    19.64       20.0
 0.99     3.88    13.91    39.26    91.18      100.0

Gustafson with f=0.95, p=64: 60.8   (Amdahl, same f and p: 15.4)

scan of n=1024: sequential work=1023 span=1023 | Hillis-Steele work=9217 span=10 | Blelloch work=2046 span=20
Brent bound T_p <= W/p + S for n=2**20, p=1024: Hillis-Steele 19476, Blelloch 2088
```

Read-out. With 95% parallel code, 1024 processors give 19.6×, never more than 20× (Amdahl). A prefix sum shows why *work-efficiency* matters: Hillis–Steele has the shorter span (10 steps) but does $n \log_2 n$ additions (what `scan_hs` in Chapter 15 does inside one block); Blelloch has twice the span but only $2(n-1)$ additions, which wins as soon as there are fewer processors than elements — the Brent bound above predicts ∼9× fewer time steps for it.

16.2 Data races, mutexes and atomics ★★★

Data race (C/C++ definition)

Two threads access the *same memory location*, at least one access is a *write*, at least one is not atomic, and nothing orders them (no *happens-before* edge from a mutex, atomic acquire/release, thread start/join…). A data race is **undefined behaviour**: not "a slightly wrong number" but a licence for the compiler to assume it never happens (it may keep the variable in a register across the whole loop, hoist loads, tear writes). A *race condition* is the broader logic bug where the result depends on timing; it can exist even without a data race.

race.cpp

```cpp
#include <cstdio>
#include <thread>
int counter = 0;                                    // plain int: concurrent increments are a data race
void work() { for (int i = 0; i < 1000000; ++i) counter++; }
int main() {
    std::thread a(work), b(work);
    a.join(); b.join();
    std::printf("expected 2000000, got %d\n", counter);
}
```

```
$ g++ -O0 -pthread race.cpp && ./a.out      # then again under ThreadSanitizer: g++ -g -fsanitize=thread race.cpp
```

```
expected 2000000, got 1117832
SUMMARY: ThreadSanitizer: data race race.cpp:4 in work()
WARNING: ThreadSanitizer: data race
```

A large share of the increments is lost (the number changes on every run), because `counter++` is three steps (load, add, store) and two threads interleave them. ThreadSanitizer (`-fsanitize=thread`) names the racing line without needing the bad interleaving to show up in the result — it is the first tool to reach for. Three correct ways to sum an array of 20 million ones with 4 threads:

```cpp
#include <atomic>
#include <chrono>
#include <cstdio>
#include <mutex>
#include <thread>
#include <vector>
using Clock = std::chrono::steady_clock;
template <class Body, class Result>
void run(const char* name, const std::vector<int>& data, Body body, Result result) {
    auto t0 = Clock::now();
    std::vector<std::thread> ts;
    for (int t = 0; t < 4; ++t) ts.emplace_back([&, t] { body(t, data.size() / 4); });
    for (auto& th : ts) th.join();
    double ms = std::chrono::duration<double, std::milli>(Clock::now() - t0).count();
    std::printf("%-42s sum=%ld  %8.1f ms\n", name, (long)result(), ms);
}
int main() {
    std::vector<int> data(20000000, 1);
    long locked = 0, partial = 0;
    std::mutex m;
    std::atomic<long> atom{0};
    run("lock a mutex for every element", data, [&](int t, size_t n) {
```

```
        for (size_t i = t * n; i < (t + 1) * n; ++i) { std::lock_guard<std::mutex> g(m); locked += data[i]; } },
        [&] { return locked; });
    run("atomic fetch_add for every element", data, [&](int t, size_t n) {
        for (size_t i = t * n; i < (t + 1) * n; ++i) atom.fetch_add(data[i], std::memory_order_relaxed); },
        [&] { return atom.load(); });
    run("private partial sum, one lock at the end", data, [&](int t, size_t n) {
        long local = 0;
        for (size_t i = t * n; i < (t + 1) * n; ++i) local += data[i];
        std::lock_guard<std::mutex> g(m); partial += local; },
        [&] { return partial; });
}
```

Captured output (4 threads, 20 million elements)

```
lock a mutex for every element            sum=20000000    1378.2 ms
atomic fetch_add for every element        sum=20000000     755.8 ms
private partial sum, one lock at the end  sum=20000000       5.2 ms
```

All three are exact; the ranking is the lesson: **contention is the cost**. A mutex per element serialises everything and adds lock overhead; an atomic is cheaper but still bounces one cache line between cores; private partial sums touch shared state once per thread — the same structure as a GPU reduction (per-thread, then per-warp, then per-block, then global).

Mutex, atomic, or neither?

- **Prefer no sharing:** give each thread its own data and combine once (map–reduce). **Then prefer a mutex** (`std::lock_guard`, `std::scoped_lock`, RAII: unlock on every path, including exceptions). **Atomics** only for simple counters/flags or when you can prove the memory ordering.
- A **mutex** gives mutual exclusion *and* a happens-before edge (unlock → next lock). A **semaphore** counts permits and need not be owned by the thread that releases it. A **spinlock** busy-waits: good only for very short critical sections on a dedicated core. A **reader–writer lock** (`std::shared_mutex`) lets many readers in.
- `volatile` is **not** a synchronisation tool in C/C++ (no atomicity, no ordering between threads); use `std::atomic`. `std::shared_ptr`'s *control block* is thread-safe, the pointee is not.
- **Thread basics:** every `std::thread` must be `join()`ed or `detach()`ed before destruction (else `std::terminate`); prefer `std::jthread` (C++20, joins automatically, supports stop tokens); exceptions do not cross threads (use `std::future`/`std::promise`, `std::async`). C++20 also adds `std::latch`, `std::barrier`, `std::counting_semaphore`.

16.3 Deadlock, livelock, starvation ★★

Deadlock needs all four Coffman conditions: *mutual exclusion*, *hold and wait*, *no preemption*, *circular wait*. Break one: acquire locks in a **global order**, take all at once with `std::scoped_lock(a, b)` (a deadlock-avoidance algorithm), use `try_lock` with back-off, or avoid holding locks while calling unknown code. **Livelock:** threads keep reacting to each other and make no progress (two people stepping aside in a corridor). **Starvation:** a thread never gets the resource (unfair locks, priority inversion). ThreadSanitizer finds *potential* deadlocks from the lock-order graph even in a run that did not hang:

deadlock.cpp: opposite lock orders, run one after the other so nothing hangs

```
#include <mutex>
#include <thread>
std::mutex A, B;
int main() {
    std::thread t1([] { std::lock_guard<std::mutex> a(A); std::lock_guard<std::mutex> b(B); });   // order A, B
    t1.join();
    std::thread t2([] { std::lock_guard<std::mutex> b(B); std::lock_guard<std::mutex> a(A); });   // order B, A
    t2.join();
}
```

```
$ g++ -g -fsanitize=thread deadlock.cpp && ./a.out 2>&1 | grep ...
```

```
  Cycle in lock order graph: Mn (0xADDR) => Mn (0xADDR) => Mn
WARNING: ThreadSanitizer: lock-order-inversion (potential deadlock)
```

16.4 Condition variables and producer–consumer ★★

A **condition variable** lets a thread sleep until some *predicate over shared state* becomes true. Rules: (1) always wait *with a predicate* (`cv.wait(lock, pred)`), because of spurious wake-ups and because another consumer may have taken the item first; (2) change the shared state *while holding the mutex*, then notify; (3) `notify_one` wakes one waiter, `notify_all` all (needed for shutdown or when waiters wait for different predicates).

pc.cpp: bounded queue, 3 producers x 1000 items, 2 consumers

```
#include <condition_variable>
#include <cstdio>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>
class BoundedQueue {
    std::queue<int> q_; const size_t cap_;
    std::mutex m_; std::condition_variable not_full_, not_empty_;
    bool closed_ = false;
public:
    explicit BoundedQueue(size_t cap) : cap_(cap) {}
    void push(int v) {
        std::unique_lock<std::mutex> lk(m_);
        not_full_.wait(lk, [&] { return q_.size() < cap_; });          // predicate form: immune to spurious wake-ups
        q_.push(v);
        not_empty_.notify_one();
    }
    bool pop(int& v) {                                                  // false when closed and drained
        std::unique_lock<std::mutex> lk(m_);
        not_empty_.wait(lk, [&] { return !q_.empty() || closed_; });
        if (q_.empty()) return false;
        v = q_.front(); q_.pop();
        not_full_.notify_one();
        return true;
    }
    void close() { { std::lock_guard<std::mutex> lk(m_); closed_ = true; } not_empty_.notify_all(); }
};
int main() {
    BoundedQueue q(4);
    long produced = 0, consumed[2] = {0, 0};
    std::vector<std::thread> prod, cons;
    for (int p = 0; p < 3; ++p) prod.emplace_back([&q, p] { for (int i = 1; i <= 1000; ++i) q.push(p * 1000 + i); });
    for (int p = 0; p < 3; ++p) for (int i = 1; i <= 1000; ++i) produced += p * 1000 + i;
    for (int c = 0; c < 2; ++c) cons.emplace_back([&q, &consumed, c] { int v; while (q.pop(v)) consumed[c] += v; });
    for (auto& t : prod) t.join();
    q.close();
    for (auto& t : cons) t.join();
    std::printf("produced sum = %ld, consumed sum = %ld -> %s\n", produced, consumed[0] + consumed[1],
                produced == consumed[0] + consumed[1] ? "OK" : "LOST OR DUPLICATED ITEMS");
}
```

```
$ g++ -g -fsanitize=thread pc.cpp && ./a.out
```

```
produced sum = 4501500, consumed sum = 4501500 -> OK
ThreadSanitizer reports: 0
```

16.5 The memory model: why `seq_cst` is the default ★★

Compilers and CPUs reorder memory operations that look independent in one thread. The C++ memory model says: **a data-race-free program behaves as if the threads' operations were interleaved in some order (sequential consistency)**. Atomics let you build the happens-before edges yourself, at three strengths:

`std::memory_order`	Guarantee
`relaxed`	atomicity only; no ordering with other variables (fine for statistics counters)
`release` (store) / `acquire` (load)	if the acquire load reads the value written by the release store, then *everything the writer did before the store is visible to the reader after the load*: the "publish a pointer/flag" pattern, and exactly what a mutex unlock/lock does
`seq_cst` (default)	acquire/release plus one global total order of all `seq_cst` operations; the cost is a fence (or locked instruction) on stores on x86 and more on ARM/POWER

Store-buffering litmus test. Thread 1 does `x=1; r1=y`, thread 2 does `y=1; r2=x`, initially `x=y=0`. Under sequential consistency at least one thread sees the other's store, so `r1=r2=0` is impossible. x86 is only "total store

order": each core has a store buffer, so a later load can complete before an earlier store becomes visible. Measure it (spin barrier to start both threads together; the count *varies a lot* from run to run, what matters is zero versus non-zero):

```
#include <atomic>
#include <cstdio>
#include <thread>
struct SpinBarrier {                                   // makes both threads leave at (almost) the same instant
    std::atomic<int> arrived{0}, gen{0};
    int n;
    explicit SpinBarrier(int n_) : n(n_) {}
    void wait() {
        int g = gen.load();
        if (arrived.fetch_add(1) + 1 == n) { arrived.store(0); gen.fetch_add(1); }
        else while (gen.load() == g) { }
    }
};
template <std::memory_order MO> int both_zero(int iters) {
    std::atomic<int> x{0}, y{0};
    int r1 = 0, r2 = 0, both = 0;
    SpinBarrier start(2), done(2);
    std::thread t2([&] {
        for (int i = 0; i < iters; ++i) { start.wait(); y.store(1, MO); r2 = x.load(MO); done.wait(); }
    });
    for (int i = 0; i < iters; ++i) {
        x.store(0); y.store(0);
        start.wait();
        x.store(1, MO); r1 = y.load(MO);
        done.wait();
        if (r1 == 0 && r2 == 0) ++both;
    }
    t2.join();
    return both;
}
int main() {
    const int N = 300000;
    std::printf("relaxed : r1 = r2 = 0 in %d of %d runs\n", both_zero<std::memory_order_relaxed>(N), N);
    std::printf("seq_cst : r1 = r2 = 0 in %d of %d runs\n", both_zero<std::memory_order_seq_cst>(N), N);
}
```

Captured output

```
relaxed : r1 = r2 = 0 in 79940 of 300000 runs
seq_cst : r1 = r2 = 0 in 0 of 300000 runs
```

Message passing is the other classic: publish data, then raise a flag. With a release store and an acquire load the reader must see the data; ThreadSanitizer *proves* the ordering by reporting nothing, and it flags the race as soon as the orders are weakened to `relaxed`, even though an x86 machine would probably still print 42 (x86 never reorders store–store or load–load; ARM and POWER do, and so does the compiler):

mp.cpp

```
#include <atomic>
#include <cstdio>
#include <thread>
#ifndef PUBLISH
#define PUBLISH std::memory_order_release
#define CONSUME std::memory_order_acquire
#endif
int data = 0;                                          // ordinary (non-atomic) payload
std::atomic<int> flag{0};
int main() {
    std::thread producer([] { data = 42; flag.store(1, PUBLISH); });
    std::thread consumer([] { while (flag.load(CONSUME) == 0) { } std::printf("data=%d\n", data); });
    producer.join(); consumer.join();
}
```

```
$ g++ -g -fsanitize=thread mp.cpp && ./a.out        # release/acquire, then -DPUBLISH=relaxed -DCONSUME=relaxed
```

```
--- release / acquire
data=42
--- relaxed / relaxed
WARNING: ThreadSanitizer: data race
data=42
```

Memory-model traps

(1) A plain `bool done;` polled in a loop is a data race: the compiler may read it once and spin forever; use `std::atomic<bool>`. **(2)** Double-checked locking needs acquire/release on the pointer (or just use a function-local `static`, which C++11 initialises thread-safely, or `std::call_once`). **(3)** "It works on my x86 laptop" proves nothing about ARM, POWER or a GPU. **(4)** GPUs have their own (scoped) memory model: `.cta/.gpu/.sys` scopes in PTX; `__syncthreads()` orders within a block only.

16.6 False sharing and cache effects ★

Cores keep data in **64-byte cache lines**; the coherence protocol (MESI) lets only one core hold a line in a writable state. If four threads update four *different* counters that happen to sit in one line, the line ping-pongs between cores: **false sharing**. Fix: pad/align per-thread data to the line size (`alignas(64)`, or `std::hardware_destructive_interference_size`).

```
#include <chrono>
#include <cstdio>
#include <thread>
#include <vector>
struct Packed { volatile long v = 0; };                 // four of these share one 64-byte cache line
struct Padded { alignas(64) volatile long v = 0; };     // one cache line each
template <class S> double run() {
    S slot[4];
    auto t0 = std::chrono::steady_clock::now();
    std::vector<std::thread> ts;
    for (int t = 0; t < 4; ++t) ts.emplace_back([&slot, t] { for (long i = 0; i < 50000000; ++i) slot[t].v = slot[t].v + 1; });
    for (auto& th : ts) th.join();
    return std::chrono::duration<double, std::milli>(std::chrono::steady_clock::now() - t0).count();
}
int main() {
    double a = run<Packed>(), b = run<Padded>();
    std::printf("sizeof: packed=%zu padded=%zu bytes\n", sizeof(Packed), sizeof(Padded));
    std::printf("adjacent counters (false sharing): %7.1f ms\npadded to 64 bytes               : %7.1f ms\nslow-down: %.1fx\n", a,
      b, a / b);
}
```

Captured output

```
sizeof: packed=8 padded=64 bytes
adjacent counters (false sharing):   166.6 ms
padded to 64 bytes               :    71.5 ms
slow-down: 2.3x
```

Each thread only ever touches its own counter, yet the unpadded version is markedly slower (the ratio depends on the machine and on the other load; 2–10× is typical). The GPU analogue is the memory-side rules of Section 15.3: coalescing and bank conflicts.

16.7 OpenMP ★★

OpenMP adds `#pragma omp` directives to C/C++/Fortran; the compiler turns an annotated loop into a fork–join team of threads (`-fopenmp`; the number of threads comes from `OMP_NUM_THREADS` or `omp_set_num_threads`).

Directive / clause	Meaning
`#pragma omp parallel`	start a team; the following block runs on every thread
`#pragma omp for` / `parallel for`	split loop iterations among the threads (implicit barrier at the end unless `nowait`)
`schedule(static | dynamic,k | guided)`	iteration assignment: fixed chunks (cheap, for uniform work), on demand (for uneven work), shrinking chunks
`reduction(+:sum)`	private copy per thread, combined at the end (also `*`, `min`, `max`, `&&`…)
`private`, `firstprivate`, `shared`, `default(none)`	data-sharing; loop variable is private; `default(none)` forces you to decide for every variable
`critical`, `atomic`, `barrier`, `single`, `master`	mutual exclusion on a block / on one update, team barrier, one thread only
`task`, `taskwait`	irregular parallelism (recursion, linked structures)
`simd`, `target`	vectorise a loop; offload to an accelerator (GPU)

```cpp
#include <omp.h>
#include <cstdio>
#include <initializer_list>
int main() {
    const long N = 400000000;                       // midpoint rule for the integral of 4/(1+x^2) on [0,1], which is pi
    const double h = 1.0 / N;
    double t1 = 0;
    for (int threads : {1, 2, 4}) {
        omp_set_num_threads(threads);
        double t0 = omp_get_wtime(), sum = 0.0;
        #pragma omp parallel for reduction(+ : sum) schedule(static)
        for (long i = 0; i < N; ++i) { double x = (i + 0.5) * h; sum += 4.0 / (1.0 + x * x); }
        double t = omp_get_wtime() - t0;
        if (threads == 1) t1 = t;
        std::printf("threads=%d  pi=%.10f  time=%.3f s  speed-up=%.2f\n", threads, sum * h, t, t1 / t);
    }
}
```

Captured output

```
threads=1  pi=3.1415926536  time=0.606 s  speed-up=1.00
threads=2  pi=3.1415926536  time=0.306 s  speed-up=1.98
threads=4  pi=3.1415926536  time=0.214 s  speed-up=2.83
```

The same value of π with every thread count (the reduction combines partial sums; floating-point order differs in the last bits only), and a speed-up below p because the sandbox's virtual CPUs are shared and the serial parts and memory system cap it (Amdahl again). The classic mistake is to forget the `reduction` clause. The three variants, compiled without optimisation:

```cpp
#include <omp.h>
#include <cstdio>
int main() {
    const int N = 1000000;
    long wrong = 0, by_atomic = 0, by_reduction = 0;
    #pragma omp parallel for
    for (int i = 0; i < N; ++i) wrong += 1;                     // data race on `wrong`: lost updates
    #pragma omp parallel for
    for (int i = 0; i < N; ++i) {
        #pragma omp atomic
        by_atomic += 1;                                         // correct, but every update is a locked instruction
    }
    #pragma omp parallel for reduction(+ : by_reduction)
    for (int i = 0; i < N; ++i) by_reduction += 1;              // correct and scalable
    std::printf("no clause: %s | atomic: %ld | reduction: %ld | expected %d\n",
                wrong == N ? "correct by luck" : "WRONG", by_atomic, by_reduction, N);
}
```

Captured output

```
no clause: WRONG | atomic: 1000000 | reduction: 1000000 | expected 1000000
```

16.8 Parallel patterns and their GPU forms ★★

Pattern	Idea	CPU / GPU form
map	independent element-wise function	`parallel for`, `std::transform(std::execution::par, \ldots)`; one thread per element (`vecadd`)
reduce	combine all elements with an associative operator	per-thread partials → tree; GPU: warp shuffles, shared memory, one atomic per block (`reduce_shfl`)
scan (prefix sum)	$y_i = x_0 \oplus \cdots \oplus x_i$; the building block of compaction, sorting, radix sort, allocation	Hillis–Steele (work $n \log n$), Blelloch (work $2n$); GPU libraries: CUB, Thrust
stencil	each output reads a neighbourhood	tiling with halo cells in shared memory
gather / scatter	indirect reads / writes	coalescing is the problem; scatter needs atomics or conflict-free indices
histogram	many-to-few update with atomics	privatise per thread/block (`hist_priv`), merge at the end
pipeline, producer–consumer	stages connected by queues	thread pool + bounded queue; GPU: streams

Pattern	Idea	CPU / GPU form
divide and conquer	recursion on independent halves	OpenMP tasks, `std::async`; GPU: usually flattened (parallel merge/bitonic sort)

Determinism warning: floating-point addition is not associative, so a parallel reduction may differ from the serial sum in the last bits *and* from run to run if the order depends on scheduling (atomics). Test with tolerances or integer data; for bit-reproducible results fix the reduction tree.

16.9 Python concurrency (you will use it in the automation lab) ★★★

- **The GIL** (global interpreter lock) lets only one thread execute Python bytecode at a time. **Threads** therefore speed up *I/O-bound* work (waiting for files, sockets, child processes) but not pure-Python CPU work. **Processes** (`multiprocessing`, `concurrent.futures.ProcessPoolExecutor`) have separate interpreters and scale CPU-bound work, at the cost of start-up time and pickling of arguments/results. `asyncio` multiplexes many I/O waits on one thread. Python 3.13 offers an *experimental* free-threaded build (PEP 703); the normal build still has the GIL.
- **For a compiler test farm** the right pattern is a *thread pool that launches subprocesses*: each worker thread blocks in `subprocess.run(["clang", ...])` while the real CPU work happens in the child, so the GIL is irrelevant (Chapter 18).

```python
import os, sys, time
from concurrent.futures import ProcessPoolExecutor, ThreadPoolExecutor

def burn(n):                                # CPU-bound pure-Python work
    s = 0
    for i in range(n):
        s += i * i % 7
    return s

if __name__ == "__main__":
    jobs = [3_000_000] * 4
    t = time.perf_counter(); r0 = [burn(n) for n in jobs];              t_seq = time.perf_counter() - t
    t = time.perf_counter()
    with ThreadPoolExecutor(4) as ex: r1 = list(ex.map(burn, jobs))
    t_thr = time.perf_counter() - t
    t = time.perf_counter()
    with ProcessPoolExecutor(4) as ex: r2 = list(ex.map(burn, jobs))
    t_proc = time.perf_counter() - t
    assert r0 == r1 == r2
    gil = sys._is_gil_enabled() if hasattr(sys, "_is_gil_enabled") else True
    print(f"sequential {t_seq:.2f}s | 4 threads {t_thr:.2f}s | 4 processes {t_proc:.2f}s   (cpus={os.cpu_count()}, GIL enabled: {
     gil})")
```

Captured output (pure-Python CPU-bound work, 4 jobs)

```
sequential 0.90s | 4 threads 0.95s | 4 processes 0.35s   (cpus=4, GIL enabled: True)
```

Threads give no speed-up (slightly slower: lock hand-over), processes do. Use `if __name__ == "__main__":` around process pools (on Windows and macOS child processes re-import the module).

16.10 Practice questions ★★

Q16.1 Two threads each execute `x++` one million times on a shared `int x = 0`. What are the possible final values, and what exactly is wrong?

Answer: Formally the program has undefined behaviour (data race). In practice anything from 1 000 001 to 2 000 000 can appear (each increment is load/add/store; lost updates), and an optimising compiler may even turn the loop into a single add. Fix: `std::atomic<int>`, a mutex, or per-thread counters summed after `join`.

Q16.2 Mutex versus semaphore versus spinlock?

Answer: A mutex has an owner and provides mutual exclusion (blocking waiters, usually via the OS); a (counting) semaphore is a permit counter that any thread may release (signalling, resource pools); a spinlock busy-waits and is only sensible for very short critical sections on multiple cores (a preempted lock holder makes spinners burn their whole time slice).

Q16.3 List the Coffman conditions and show how lock ordering prevents deadlock.
Answer: Mutual exclusion, hold-and-wait, no preemption, circular wait. If all threads acquire locks in one global order (e.g. by address), a cycle in the wait-for graph cannot form, so circular wait is impossible. `std::scoped_lock(a, b)` does this internally.

Q16.4 Why must `condition_variable::wait` be used with a predicate in a loop?
Answer: Spurious wake-ups are allowed, and when several consumers are woken only one finds the item; the waiter must re-check the condition under the mutex. The predicate overload `wait(lock, pred)` is exactly `while (!pred()) wait(lock);`.

Q16.5 What is false sharing and how do you find and fix it?
Answer: Independent variables in one cache line written by different cores make the line bounce between caches. Symptoms: no scaling with threads although there is no lock; profilers show cache-line contention (e.g. `perf c2c`). Fix: pad or align per-thread data to 64 bytes, or accumulate in thread-local variables.

Q16.6 What does `memory_order_acquire/release` guarantee, and why is `seq_cst` the default?
Answer: If an acquire load reads the value written by a release store, all writes that happened before the store are visible after the load (happens-before). `seq_cst` additionally gives a single total order of all `seq_cst` operations, the model most people reason in; weaker orders are an optimisation to be used only with a proof (and with a tool such as ThreadSanitizer).

Q16.7 An OpenMP loop sums an array into a shared `double` and gives different answers on each run. Two distinct reasons?
Answer: (1) A data race if there is no `reduction/atomic/critical` (lost updates). (2) Even with a correct `reduction`, floating-point addition is not associative and the combination order may vary, so results differ in the last bits; compare with a tolerance.

Q16.8 Amdahl's law: 10% of a program is serial. What is the best possible speed-up, and the speed-up on 8 cores?
Answer: Limit $1/0.1 = 10\times$. On 8 cores $1/(0.1 + 0.9/8) = 1/0.2125 \approx 4.7\times$.

Q16.9 Why is Python threading useless for CPU-bound code, and what do you use instead?
Answer: The GIL serialises bytecode execution. Use `multiprocessing`/`ProcessPoolExecutor`, native extensions that release the GIL (NumPy), or — for driving compilers — threads that wait on `subprocess` children.

Q16.10 How do GPU concepts map to these CPU ideas?
Answer: Block barrier ↔ `std::barrier`; shared memory ↔ per-core scratchpad; `atomicAdd` ↔ `fetch_add`; per-thread partial sums then a tree ↔ OpenMP `reduction`; bank conflicts ↔ false sharing (both are "unrelated data collide in a hardware unit"); CUDA's scoped memory model ↔ C++ memory orders with narrower scopes.

Part V

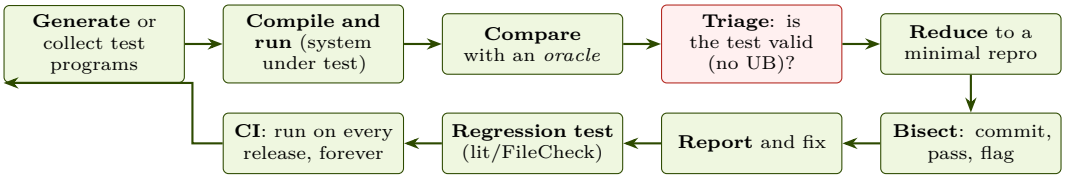
Compiler Verification and the Automation Lab

17. Compiler Verification: Oracles, Differential Testing, Reduction

In this chapter

Priority ★★★. Time: 90 min. This chapter *is* the job description: "implement programs or GPU code to verify NVIDIA GPU / embedded compilers", "design and implement tools, libraries, and automation", "scalable verification infrastructure … across compiler releases". In an interview for this team "how would you test a compiler?" is far more likely than any single algorithm question. Demonstrations were run in the sandbox (GCC 13, Clang/LLVM 18, `git`, `valgrind`); the facts about published tools (bug counts, venues) come from the papers' own abstracts as retrieved while preparing this manual. Chapter 18 then builds the whole workflow in Python so that you can *say* "I built one" and mean it.

17.1 What "verifying a compiler" means ★★★



A compiler is correct when, for every program with defined behaviour, the generated code shows one of the behaviours the language allows (the compiler may *remove* non-determinism, never *add* behaviours: this is called **refinement**). Verification is therefore much more than "does it compile my test files":

Property	Question	Typical failure
Correctness (semantic preservation)	does the generated code do what the source specifies?	*wrong-code* bug (miscompilation): silent, the most dangerous class
Robustness	does it accept every valid input without crashing or hanging?	*ICE* (internal compiler error), assertion, segfault, infinite loop in a pass
Conformance	does it implement the standard (C++, CUDA, PTX ISA) and the documented extensions?	rejects valid code, accepts invalid code, wrong diagnostics
Determinism, reproducibility	same source, same flags ⇒ same binary?	pass order depends on hash-map iteration; timestamps in output
Compatibility	across releases, architectures (`sm_XY`), host compilers, libraries, ABIs	*regression*: passes in release N, fails in $N{+}1$; PTX version or ABI mismatch
Performance	code quality and compile time/memory	3× slower kernel after a pass change; register spills; 10× compile-time blow-up
Tooling	debug info, optimisation remarks, sanitizer interplay	debugger shows the wrong line; remarks claim a loop was vectorised when it was not
Safety evidence (embedded, automotive)	can you show how and how well the compiler was tested?	missing coverage of a transformation; no traceability from requirement to test

17.2 The oracle problem ★★★

A test is an *input* plus an *oracle* that decides pass or fail. For ordinary software you write the expected output by hand; for a compiler tested on millions of generated programs nobody knows the expected output. All of compiler-testing research is about getting oracles:

Oracle	How it works	Catches	Misses
Golden output (lit/FileCheck, self-checking programs)	expected text or result written down	regressions, known patterns	anything nobody thought of
Differential	compare two or more implementations: `-O0` vs `-O3`, GCC vs Clang, release N vs $N+1$, x86 vs AArch64 vs GPU, compiled vs interpreted	wrong code, crashes, inconsistent targets	bugs common to all implementations
Metamorphic (EMI)	transform a program so that its behaviour must not change; compare the two runs of the *same* compiler	optimiser bugs, no reference needed	bugs that affect both variants identically
Reference model	interpreter, emulator or high-precision library (e.g. `mpmath` for math functions, CPU for GPU kernels)	semantic bugs in the system under test	bugs in the reference itself ("common-mode" if code is shared)
Property / invariant	verifier passes (`opt -verify`, `llc -verify-machineinstrs`), idempotence, determinism, `ptxas` accepts the PTX, sanitizers stay silent	internal inconsistencies, ICEs	wrong-but-consistent code
Proof	translation validation (Alive2), verified compiler (CompCert)	*all* behaviours of the checked region	scale: bounded loops, memory models

Say it in one breath

"A compiler test needs an input and an oracle. I would combine cheap golden tests for regressions (lit/FileCheck), *differential testing* across optimisation levels, compilers, releases and targets on randomly generated, UB-free programs, *metamorphic* variants where no reference exists, automatic *reduction* of every failure, and *bisection* to the guilty change; all of it in CI, sharded, with timeouts and flake handling."

17.3 Differential testing in practice ★★★

Why it works: independently written implementations rarely fail in the same way on the same input, so any disagreement is a *candidate* bug. **Prerequisite:** the input must have fully defined behaviour (no undefined, unspecified or implementation-defined dependence, no data races, no uninitialised reads), otherwise the compilers are *allowed* to disagree and you drown in false alarms. Here is a miniature harness (five builds: two compilers × two optimisation levels, plus GCC with `-funsigned-char` to imitate an Arm-Linux target) run on one clean program and six classic offenders:

ub_cases.c: seven tiny programs, split into files by split_cases.sh

```
//// clean
#include <stdio.h>
int main(void) { unsigned h = 17u; for (int i = 0; i < 10; ++i) h = h * 31u + (unsigned)i; printf("%u\n", h); return 0; }
//// ovf
#include <limits.h>
#include <stdio.h>
int bigger(int x) { int y = x + 1; return y > x; }                  /* signed overflow: undefined behaviour */
int main(void) { printf("%d\n", bigger(INT_MAX)); return 0; }
//// uninit
#include <stdio.h>
int f(int c) { int x; if (c) x = 5; return x; }                     /* x uninitialised when c == 0 */
int main(void) { printf("%d\n", f(0)); return 0; }
//// alias
#include <stdio.h>
int punned(int *i, float *f) { *i = 1; *f = 2.0f; return *i; }      /* strict aliasing: int* and float* may not alias */
int main(void) { int x = 0; printf("%d\n", punned(&x, (float *)&x)); return 0; }
//// order
#include <stdio.h>
static int n = 0;
static int next(void) { return ++n; }
int main(void) { printf("%d %d\n", next(), next()); return 0; }      /* argument evaluation order is unspecified */
//// shift
#include <stdio.h>
static unsigned shl(unsigned v, unsigned s) { return v << s; }       /* shift count >= 32: undefined */
```

```
int main(void) { printf("%u\n", shl(1u, 32u)); return 0; }
//// char
#include <stdio.h>
int main(void) { char c = (char)200; printf("%d\n", c < 0); return 0; }   /* is plain char signed? implementation-defined */
```

difftest.sh

```
#!/bin/bash
# difftest.sh file.c...   build every file 5 ways, run each, and flag any disagreement between the outputs
builds=("gcc -O0" "gcc -O2" "clang -O0" "clang -O2" "gcc -O2 -funsigned-char")
printf "%-8s" program; for b in "${builds[@]}"; do printf " %-14s" "$(echo "$b" | sed 's/-funsigned-char/-uchar/')"; done; echo "
     verdict"
for f in "$@"; do
  outs=()
  for b in "${builds[@]}"; do
    if $b -w "$f" -o /tmp/dt.bin 2>/dev/null; then outs+=("$(/tmp/dt.bin | tr '\n' ' ' | sed 's/ $//')"); else outs+=("COMPILE-ERR
     "); fi
  done
  verdict=agree; for o in "${outs[@]}"; do [ "$o" != "${outs[0]}" ] && verdict=DISAGREE; done
  printf "%-8s" "$(basename "$f" .c | sed 's/^v_//')"; for o in "${outs[@]}"; do printf " %-14s" "$o"; done; echo " $verdict"
done
```

split_cases.sh

```
#!/bin/bash
# split_cases.sh FILE.c   write every section that starts with "//// name" to its own file v_name.c
awk '/^\/\/\/\/ /{f = "v_" $2 ".c"; next} {print > f}' "$1"
```

```
$ ./split_cases.sh ub_cases.c && ./difftest.sh v_clean.c v_ovf.c v_uninit.c v_alias.c v_order.c v_shift.c v_char.c
```

```
program  gcc -O0        gcc -O2        clang -O0      clang -O2      gcc -O2 -uchar verdict
clean    3976595158     3976595158     3976595158     3976595158     3976595158     agree
ovf      0              1              0              1              1              DISAGREE
uninit   32752          5              0              5              5              DISAGREE
alias    1073741824     1              1073741824     1              1              DISAGREE
order    2 1            2 1            1 2            1 2            2 1            DISAGREE
shift    1              0              1              1015479064     0              DISAGREE
char     1              1              1              1              0              DISAGREE
```

Six of the seven programs make the builds **disagree**, yet *none* of the six disagreements is a compiler bug: each program is wrong, or at best non-portable. Only a disagreement on `clean` would be a compiler bug. This is why generated tests must be UB-free, and why a mismatch is the *start* of an investigation, not its conclusion. Which tool tells you whose fault it is?

diagnose.sh

```
#!/bin/bash
# diagnose.sh   which tool explains each disagreement? (first diagnostic line only)
first() { grep -m1 "$1" | sed -E 's/^[^ ]+ //'; }
echo "ovf    UBSan      : $(gcc -g -O0 -fsanitize=undefined v_ovf.c -o /tmp/d1 && /tmp/d1 2>&1 | first 'runtime error')"
echo "ovf    UBSan -O1  : $(gcc -g -O1 -fsanitize=undefined v_ovf.c -o /tmp/d1b && /tmp/d1b 2>&1 | first 'runtime error' | grep .
     || echo '(silent)')"
echo "shift  UBSan      : $(gcc -g -O0 -fsanitize=undefined v_shift.c -o /tmp/d2 && /tmp/d2 2>&1 | first 'runtime error')"
echo "uninit valgrind   : $(gcc -g -O0 v_uninit.c -o /tmp/d3 && valgrind -q /tmp/d3 2>&1 | first 'uninitialised')"
echo "alias  -Wstrict-aliasing=2 : $(gcc -O2 -Wstrict-aliasing=2 -c v_alias.c -o /dev/null 2>&1 | first 'warning')"
echo "order  (no tool)  : unspecified, not undefined: both orders are conforming; fix the test"
echo "char   (no tool)  : implementation-defined: compare against the documented ABI of each target"
```

```
$ ./diagnose.sh
```

```
ovf    UBSan      : runtime error: signed integer overflow: 2147483647 + 1 cannot be represented in type 'int'
ovf    UBSan -O1  : (silent)
shift  UBSan      : runtime error: shift exponent 32 is too large for 32-bit type 'unsigned int'
uninit valgrind   : Conditional jump or move depends on uninitialised value(s)
alias  -Wstrict-aliasing=2 : warning: dereferencing type-punned pointer will break strict-aliasing rules [-Wstrict-aliasing]
order  (no tool)  : unspecified, not undefined: both orders are conforming; fix the test
char   (no tool)  : implementation-defined: compare against the documented ABI of each target
```

"`-O0` and `-O2` disagree": the checklist before you file a compiler bug

1. **Is it deterministic?** Run the same binary several times (data race, uninitialised memory, address-dependent behaviour, timing).
2. **Ask the tools:** `-Wall -Wextra`, UBSan (`-fsanitize=undefined`), ASan, MSan or `valgrind` (uninitialised), TSan (races), `-Wstrict-aliasing`. Run them at more than one optimisation level and with more than one tool: sanitizer instrumentation interacts with the optimiser (compare the two UBSan lines above), so one silent run proves nothing.

3. **Unspecified or implementation-defined?** Evaluation order, `char` signedness, `sizeof(long)`, struct layout, bit-fields, shifts of negative numbers, floating-point contraction and excess precision (`-ffp-contract=off`, `-fno-fast-math`), `printf` of pointers.
4. **Strict aliasing / lifetime:** does `-fno-strict-aliasing` change the result? Dangling pointers, out-of-bounds reads?
5. **Localise:** compare IR or assembly; bisect over passes (`-mllvm -opt-bisect-limit=N`), flags, commits (Section 17.7).
6. **Only now** *reduce* the program, re-check that the reduced program is still UB-free, and file the report (Section 17.12).

Where this wins in the real world

Differential testing with random programs found over **325 previously unknown bugs** in GCC, LLVM and commercial C compilers in three years (**Csmith**: Yang, Chen, Eide, Regehr, PLDI 2011; every compiler tested crashed and miscompiled valid input at least once; in the verified passes of CompCert no wrong-code bug was found); **YARPGen** (Livinskii, Babokin, Regehr, OOPSLA 2020) reported more than 220 defects in GCC, LLVM and Intel's compiler; **CLsmith** (Lidbury, Lascu, Chong, Donaldson, PLDI 2015) found bugs in every OpenCL device/compiler configuration it tested (21 in the final version, from CPUs, GPUs and other accelerators).

17.4 Generating test programs ★★★

- **Defined behaviour by construction** (the hardest requirement): initialise everything; guard or wrap arithmetic (`safe_div`, unsigned arithmetic for wrap-around, masked shift counts); bounded loops; no out-of-range indices; no pointer tricks; for GPUs, *race freedom* (each thread writes only its own output slot, barriers only in block-uniform control flow, no float atomics whose order matters).
- **Deterministic and self-reporting:** no clocks, no addresses, fixed seeds; end the program by printing a *checksum* of all state, so that any difference is visible with one comparison.
- **Diversity beyond uniform randomness:** YARPGen's *generation policies* bias the generator towards constructs that make optimisers fire (re-using sub-expressions, loop nests with known trip counts, copy chains); Chapter 18 shows how much a policy matters.
- **Cheap and reducible:** millions of programs per night, each small enough to reduce automatically.

Tool	Target	Idea
Csmith	C	UB-free random C with a global checksum; static analysis keeps programs valid
YARPGen	C, C++ (loops, SIMD)	UB avoided *without* run-time checks; generation policies increase optimiser coverage
CLsmith	OpenCL kernels (GPU)	Csmith-style modes for many-core code plus an EMI mode; found bugs in every OpenCL device/compiler configuration tested
GraphicsFuzz / **spirv-fuzz**	GLSL/SPIR-V shaders	metamorphic transformations of real shaders (Donaldson et al., OOPSLA 2017); several driver compilers affected
llvm-stress, **libFuzzer**-based fuzzers (`clang-fuzzer`, `llvm-isel-fuzzer`)	LLVM IR, source, instruction selection	random or coverage-guided inputs aimed at *crashes* and verifier failures rather than wrong code
Learning-based (DeepSmith, NNSmith)	OpenCL, deep-learning compilers	language models or graph generators trained on real code/models; weaker validity guarantees, new input shapes

Crash-testing LLVM's back ends with random IR takes ten lines of shell. We feed `llvm-stress` modules to `llc` for four targets (including NVPTX, the open-source PTX back end) with the machine verifier enabled, and classify every outcome as *ok*, *diagnosed* (a clean error message: fine) or *crash* (signal, assertion, timeout: a bug):

stress.sh

```
#!/bin/bash
# stress.sh N   crash-test several LLVM back ends with random IR from llvm-stress; classify every outcome
N=${1:-100}
printf "%-22s %6s %10s %8s\n" "llc target" "ok" "diagnosed" "crash"
for target in "-march=x86-64" "-march=aarch64" "-march=riscv64" "-march=nvptx64 -mcpu=sm_80"; do
  ok=0; err=0; crash=0
  for seed in $(seq 1 $N); do
    llvm-stress -seed=$seed -size=300 -o /tmp/s.ll 2>/dev/null
    timeout 20 llc -O2 -verify-machineinstrs $target /tmp/s.ll -o /dev/null >/tmp/s.out 2>&1; rc=$?
    if [ $rc -eq 0 ]; then ok=$((ok+1))
    elif [ $rc -ge 124 ]; then crash=$((crash+1)); echo "seed $seed: $(grep -m1 -E 'Stack dump|Assertion|LLVM ERROR' /tmp/s.out)"
      >> crashes.txt
    else err=$((err+1)); fi
  done
  printf "%-22s %6d %10d %8d\n" "$(echo $target | sed 's/-march=//')" $ok $err $crash
done
```

```
$ ./stress.sh 40
```

```
llc target                ok  diagnosed    crash
x86-64                    40          0        0
aarch64                   40          0        0
riscv64                   40          0        0
nvptx64 -mcpu=sm_80       40          0        0
```

Zero crashes is the normal result for a mature back end and a generator that has run for years: the infrastructure (classification, seed logging, timeouts) is the product, and a campaign that finds nothing is evidence, not failure. New bugs come from *new kinds of input* (new generators, new instructions, new architectures such as a new `sm_` target) — which is exactly the situation a team that ships a new GPU generation every two years is in.

Designing a generator for GPU kernels (a favourite whiteboard question)

Each thread owns one output element (`out[gid] = \ldots`), so there are no races; the program is a straight-line or bounded-loop expression DAG over *integer* arithmetic with safe-math wrappers (division guarded, shifts masked, signed overflow avoided by computing in `unsigned`); floating point only as exactly-representable values or with a tolerance; barriers appear only at points that every thread reaches, with shared-memory phases (write, `__syncthreads()`, read neighbour); divergence is created by conditions on `threadIdx` and on data; the host compares a checksum against a CPU run of the *same* source (the dual-compilation trick of Chapter 15), then across `-O0`/`-O3`, across `sm_` targets and across compiler versions.

17.5 Metamorphic testing and EMI ★★

When there is no second implementation, make the program its own oracle: apply a transformation whose effect on behaviour is known ("must be identical") and compare the two runs of the *same* compiler. **Equivalence Modulo Inputs** (EMI; Le, Afshari, Su, PLDI 2014) runs a program on an input, finds code that was *not executed*, and deletes or mutates it: the variants must behave identically *on that input*; a difference means the optimiser used a fact derived from dead code. The authors' tool *Orion* reported 147 confirmed GCC and LLVM bugs in eleven months, mostly miscompilations.

Relation ("must not change the output")	What it stresses
delete or insert code that is dead for the chosen input (EMI)	constant propagation, jump threading, alias facts, loop optimisations
replace `e` by `e+0`, `1*e`, `-(-e)`, `e-0`	algebraic simplification, instruction selection
wrap statements in `if (true-but-opaque)`; add an always-false branch	control-flow clean-up, branch folding, CFG shape
rename variables, reorder independent statements or functions	value numbering, scheduling, register allocation
compile with different but equivalent flags (e.g. vectoriser on/off)	the optimiser pipeline itself

Relation ("must not change the output")	What it stresses
GPU: change the block size for a block-size-independent kernel; permute blocks; pad inputs; split one kernel into two	launch-configuration assumptions, shared-memory sizing, barrier placement

Limit: a bug that damages the base program and every variant *in the same way* stays invisible (a "common-mode" failure); Chapter 18 measures exactly this: two seeded bugs that differential testing catches immediately are never seen by the metamorphic checker.

17.6 Test-case reduction ★★★

A 20 000-line failing program is useless to the person who must fix the compiler. **Delta debugging** (Zeller and Hildebrandt, 2002) shrinks an input automatically: split it into n chunks, try removing each chunk (and its complement), keep any smaller input that still fails, otherwise double n (smaller chunks). `ddmin` needs about n^2 test runs in the worst case and a few $\log n$ in the best; **hierarchical** delta debugging (Misherghi and Su, 2006) works on the syntax tree so that most candidates are valid; **C-Reduce** (Regehr et al., PLDI 2012; `cvise` is a Python port) adds C-aware passes (inline a function, remove a parameter, simplify an expression); `llvm-reduce` does the same for LLVM IR (functions, basic blocks, instructions, metadata, arguments…). All of them need only an **interestingness test**: a script that exits 0 exactly when the candidate still shows *the* bug.

interesting.sh: the test that drives the reducer

```
#!/bin/bash
# exit 0  <=>  "interesting": the PTX generated for this IR still contains a vector store st.v4.u32
llc -O2 -march=nvptx64 -mcpu=sm_80 "$1" -o - 2>/dev/null | grep -q 'st\.v4\.u32'
```

```
$ chmod +x interesting.sh; llvm-stress -seed=1 -size=300 -o big.ll; llvm-reduce --test=./interesting.sh big.ll -o reduced.ll; wc -
    l big.ll reduced.ll; cat reduced.ll
```

```
original is interesting
357 big.ll
9 reduced.ll
366 total
--- reduced.ll:

define void @autogen_SD1() {
BB:
  br label %CF239

CF239:                                            ; preds = %CF239, %BB
  store <4 x i32> zeroinitializer, ptr null, align 16
  br label %CF239
}
--- its PTX:
    st.v4.u32     [%rd1], {%r1, %r1, %r1, %r1};
```

357 lines became 9 (a vector store of zeros inside a loop), and the property is checked on the *PTX*, not on the IR: that is how you would reduce "the compiler emits this instruction (or crashes, or miscompiles)". For a real miscompile the test is: compile at `-O0` and `-O3`, run both, outputs differ.

Reduction traps

(1) The reducer introduces undefined behaviour: deleting an initialisation makes the miscompile "reproduce" because the program now reads garbage. Put the UB checks (`-Wall -Werror=uninitialized`, sanitizers, a warning-free build with a second compiler) *inside* the interestingness test. **(2) A different bug:** the reduced program crashes with another message; compare the crash signature or the exact output difference, not just "fails". **(3) Non-determinism:** a flaky interestingness test makes the reducer wander; make it deterministic or run it several times. **(4) Slow tests:** give every command a timeout; most candidates are rejected, so the test must be cheap.

17.7 Isolating the culprit: bisection and triage ★★

A reduced test case says *what* fails; bisection says *which change* (or pass, or flag) is responsible. Bisection is binary search on an ordered set, so 10 000 commits need only $\lceil \log_2 10000 \rceil = 14$ test runs, provided the test is a deterministic script. `git bisect run` automates it: exit code **0 = good**, **1–127 (except 125) = bad**, **125 = cannot test this commit (skip)**, **>127 = abort**.

bisect_demo.sh: a 20-commit history, the bug arrives with commit 13

```
#!/bin/bash
set -e
rm -rf /tmp/bisect_repo && mkdir /tmp/bisect_repo && cd /tmp/bisect_repo
git init -q . && git config user.email t@example.com && git config user.name tester
cat > cc.sh <<'EOS'
#!/bin/bash
# toy "compiler": a / b with C semantics (truncation toward zero)
a=$1; b=$2
echo $(( a / b ))
EOS
cat > test.sh <<'EOS'
#!/bin/bash
[ "$(bash cc.sh -7 2)" = "-3" ]          # exit 0 = good build, 1 = bad build
EOS
for i in $(seq 1 20); do
  echo "# change $i" >> cc.sh
  if [ $i -eq 13 ]; then sed -i 's|echo $(( a / b ))|echo $(( (a - (a % b + b) % b) / b ))|' cc.sh; fi   # floor division sneaks in
  git add -A && git commit -q -m "commit $i$( [ $i -eq 13 ] && echo ': refactor division lowering')"
done
git bisect start HEAD "$(git rev-list --max-parents=0 HEAD)" >/dev/null
git bisect run bash test.sh > bisect.out 2>&1 || true
echo "history: 20 commits; test runs needed: $(grep -c running bisect.out) (log2 20 = 4.3)"
echo "first bad commit: $(git log -1 --format=%s refs/bisect/bad)"
```

```
$ ./bisect_demo.sh
```

```
history: 20 commits; test runs needed: 5 (log2 20 = 4.3)
first bad commit: commit 13: refactor division lowering
```

Dimension	Tool	Note
commit / release	`git bisect run`; bisect between two nightly builds or between release branches	make the test a script with the exit-code protocol above; skip commits that do not build (125)
optimisation pass	LLVM: `-mllvm -opt-bisect-limit=N` (each transformation gets a number; binary-search N); `-print-after-all`, `-verify-each`; GCC: `-fno-<pass>`, `-fdbg-cnt=name:limit`, `-fdump-tree-all`	finds the pass that makes output wrong; then dump IR before/after it
function / translation unit	compile half the functions (or files) optimised, half not; `llvm-reduce` on the module	in a large build, bisect over object files
flags	remove options one by one (delta debugging on the command line)	`-O3 -march=\ldots{} -ffast-math\ldots` often hides a single culprit flag
target / architecture	same source for `sm_70`, `sm_80`, `sm_90`; x86 vs AArch64	separates generic from target-specific bugs

Triage turns a pile of failures into a short, ranked list of *distinct* problems: (1) **classify** (crash, wrong code, hang, performance, diagnostic); (2) **deduplicate** by a normalised signature (assertion text plus the top stack frames with addresses and line numbers masked; for wrong code, the failing test area); (3) **rank** (silent wrong code > crash on valid code > rejects valid code > performance regression > diagnostics; weigh by how many users or tests are affected and whether it is a regression); (4) **route** to the owning component using the guilty pass or commit. Chapter 18 implements steps 1 and 2.

17.8 Formal and semi-formal methods in one page ★★

- **Translation validation** checks *each compilation* instead of the compiler: prove that the output refines the input for this program. **Alive2** (Lopes, Lee, Hur, Liu, Regehr; PLDI 2021) does it for LLVM IR with an SMT solver; run over LLVM's own unit tests it found 47 new bugs (28 fixed at publication) and led to eight corrections of the LLVM Language Reference. It is *bounded* (loops are unrolled up to a limit), so it can miss bugs, but everything it reports is real.
- **A verified compiler** (CompCert; Leroy) comes with a machine-checked proof that compilation preserves semantics for the verified passes; Csmith found no wrong-code bugs in the verified passes, which is the strongest argument for proof; testing remains the way to cover what is not proved (parsers, assemblers, the hardware, the specification itself).
- **Refinement vocabulary** to use correctly: `undef` and `poison` are the IR's ways of saying "unspecified value" and "deferred undefined behaviour"; `nsw`/`nuw`/`inbounds` *promise* something that, if false, yields poison; an optimisation is valid if every behaviour of the result is a behaviour of the source. Most real LLVM miscompilations are refinement errors: a pass keeps a flag it may not keep, or speculates something that may trap (Chapter 12).
- **Limits:** proofs and checkers need a formal semantics of the source and target (for PTX and for the GPU memory model this is research-level work), cost time, and cannot cover the hardware.

17.9 Test adequacy: coverage and mutation ★★

"How good is the test suite?" has three practical answers. **Code coverage of the compiler** (`gcov`/`llvm-cov`, built with `-fprofile-instr-generate`) shows which passes and branches no test reaches. **Feature coverage** counts what the *domain* offers: every PTX opcode × type × modifier (the matrix idea of Chapter 14), every `sm_` target, every CUDA language feature. **Mutation testing** injects known faults into the compiler (change `<` to `<=`, delete a cache invalidation) and counts how many the test suite notices: the *mutation score*. Surviving mutants say precisely which test is missing. The lab in Chapter 18 does this with eight seeded bugs and shows that a naive random generator kills only 5 of 8.

17.10 A test plan for a GPU and embedded compiler ★★★

NVIDIA's internal infrastructure is not public; what follows is the standard layered structure used by industrial compiler teams (LLVM, GCC and vendor toolchains), adapted to PTX/CUDA and embedded targets. Use it as the skeleton for any "design a verification flow for X" question.

Layer	What is checked	Oracle / tools	Notes
L0 unit and regression tests	one pass or lowering step on a tiny IR/PTX input	lit + FileCheck; IR and machine verifiers (`opt -verify-each`, `llc -verify-machineinstrs`)	milliseconds; every commit; tens of thousands of tests
L1 instruction × type matrix	every PTX instruction × type × modifier × operand class × boundary value (0, ±1, MIN/MAX, NaN, ±0, denormals, rounding modes)	tests *generated from a machine-readable ISA table*; CPU soft-float or `mpmath` reference; `ptxas` acceptance; SASS pattern checks	finds wrong encodings and semantics; scales with the ISA
L2 language conformance	CUDA C++ and C++ features: templates, lambdas, `constexpr`, cooperative groups, atomics, `__managed__`	self-checking programs (return 0/1); expected-diagnostic tests for invalid code	one feature per test; compile-fail tests matter as much as run tests
L3 random + differential	generated kernels and C programs (Section above)	`-O0` vs `-O3`; `sm_XY` vs `sm_ZW`; GPU vs CPU; release N vs $N-1$; checksum; auto-reduction	nightly, millions of programs, seeds logged for replay
L4 numerics	rounding modes, flush-to-zero, FMA contraction, fast-math flags, math-library accuracy, conversions (FP16, BF16, TF32, FP8)	ULP error against a high-precision reference; *bit-exact* where IEEE-754 demands it (`+ - * / sqrt` with `.rn`)	results must match across architectures unless documented

Layer	What is checked	Oracle / tools	Notes
L5 memory model and concurrency	fences, scoped atomics, barrier semantics, divergence and reconvergence, race freedom	GPU *litmus tests* run millions of times (Alglave et al., ASPLOS 2015, studied real GPUs); `compute-sanitizer`; checks that `convergent` operations are not moved	non-deterministic by nature: statistics, not single runs
L6 real code	CUDA samples, library builds (BLAS/DNN-style), ML-compiler kernels (Triton), open-source applications, "build the world"	application self-checks, golden outputs with tolerances	finds integration and scale problems no unit test sees
L7 performance and size	kernel time, SASS instruction count, registers, spills, occupancy, code size, compile time and memory	tracked per commit; thresholds with noise bands (median of N runs); `-Xptxas -v` metrics	a 20% slowdown is a bug even if the output is right
L8 embedded / cross	cross-compilation (x86 host → AArch64 Linux or QNX target), ABI, alignment, `char` signedness, safety coding rules	board farm and simulators (QEMU-style); reproducible builds; MISRA/CERT static analysis; coverage evidence	qualification evidence (below)

Infrastructure that makes the layers usable: (1) *scheduling and sharding* with a stable hash of the test name (`hash()` in Python is randomised per process; use `zlib.crc32`); (2) *hermetic, pinned environments* (containers; compiler, driver and GPU model recorded with every result); (3) *timeouts and resource limits* on every command; (4) *retry and flake policy* (next section); (5) a *result database* and dashboards: pass rate per release and per architecture, time-to-green; (6) *automatic reduction, bisection and triage* so that a human sees one ranked list of distinct problems; (7) *repro bundles*: source, flags, compiler version, GPU, driver, seed and log in one archive; (8) *hardware farm management* (queues, GPU health checks, resets after hangs, priority tiers: pre-commit smoke tests, nightly full runs, weekly soaks).

Embedded and safety-related targets

Automotive platforms are developed under ISO 26262, which treats a compiler as a *software tool* (Part 8, "confidence in the use of software tools"): you classify how likely the tool is to introduce or fail to detect an error and how likely you are to notice, obtain a *tool confidence level*, and raise confidence by evidence: increased confidence from use, evaluation of the development process, or *validation* — a documented, systematic compiler test campaign. Practical consequences for a verification engineer: traceability from each requirement to its tests, coverage reports of compiler features, per-release known-bug lists, reproducible builds, and strict control of what changed between two releases. (Check the standard for exact wording; interviewers want the idea.)

17.11 Flaky tests and infrastructure hygiene ★★

A **flaky test** passes and fails on the same code and inputs. At scale it is the largest tax on a verification team: people stop trusting red results, and real regressions hide among false alarms.

Typical cause	Remedy
data races, timing assumptions, too-short timeouts	fix the test; synchronise instead of sleeping; calibrate timeouts from measured run times
shared state or order dependence (temp files, ports, environment)	private temp directories per test; unique ports; no reliance on test order
resource contention (CPU, disk, a GPU shared by parallel jobs)	limit parallelism per resource; pin tests to devices (`CUDA_VISIBLE_DEVICES`); queue GPU jobs

Typical cause	Remedy
randomness without a logged seed	fixed or logged seeds; every failure must be replayable
hardware and infrastructure (thermal throttling, ECC errors, a dead node, licence or network servers)	health checks; classify *infrastructure* failures separately from *product* failures; auto-drain bad nodes
non-deterministic compiler output (hash-map iteration order, timestamps)	this is a product bug of its own (non-reproducible builds): fix it in the compiler

Policy: (1) *never retry silently*: a test that fails then passes is reported as **FLAKY** and counted (Chapter 18's `litlite` does this); (2) track a flake rate per test and alert when it rises; (3) **quarantine** with an owner and an expiry date — a skipped test is lost coverage; (4) re-run a failing test k times to separate always-failing (real) from sometimes-failing (flaky); (5) timeouts are results, not noise: a hang is a bug until proven otherwise; (6) store seed, command line and environment with every failure.

17.12 Writing the bug report ★★

A report a compiler engineer will thank you for

Title	`[component] symptom`, e.g. "`[NVPTX] wrong result: signed division by constant folds with floor semantics`".
Environment	compiler version or commit, target (`sm_80`), host OS, driver, *exact* command line.
Reproducer	the *reduced*, UB-free source (sanitizer-clean, warning-free), and the commands to build and run it.
Expected / actual	with the justification (spec section, result from another compiler or from the interpreter) and the verbatim output, crash message or diff.
Analysis	bisection result (commit or pass), IR or PTX before and after the guilty pass, what you already ruled out (UB, flags).
Impact	always or intermittent; regression since which release; which code is affected; workaround (flag or source change).

One bug per report; text as text (no screenshots); attach files; do not guess at the root cause in the title.

17.13 Practice questions ★★★

Q17.1 How would you test a compiler?
Answer: Layers (Section 17.10): *(1)* fast regression tests with lit/FileCheck for each pass; *(2)* conformance tests for the language and, for GPUs, a generated instruction×type×modifier matrix against a CPU reference; *(3)* random UB-free programs with *differential* testing across optimisation levels, targets, compilers and releases, and *metamorphic* variants where no reference exists; *(4)* automatic reduction of every failure and bisection to the commit or pass; *(5)* numerics, memory-model and performance suites; *(6)* real applications; *(7)* all of it in sharded CI with timeouts, flake handling, triage by signature, and coverage/mutation measurements of the suite itself.

Q17.2 What is a test oracle? Name four that are usable for a compiler.
Answer: The mechanism that decides whether an observed behaviour is correct. Golden expected output (FileCheck, self-checking programs); a second implementation (differential); a behaviour-preserving transformation of the input (metamorphic/EMI); a reference model (interpreter, CPU for GPU code, high-precision maths library); invariants (verifiers, sanitizers, idempotence, determinism); proofs (translation validation).

Q17.3 Differential versus metamorphic testing: strengths, weaknesses, one example each.
Answer: Differential needs two implementations (e.g. `-O0` vs `-O3`) and finds any bug on which they disagree, but not bugs shared by both; it needs defined-behaviour inputs. Metamorphic needs only one implementation and a transformation with a known effect (insert code that is dead for the input, replace `e` by `e+0`), so it works where there is no reference, but it only sees bugs that treat the original and the variant differently. In the lab (Chapter 18) two seeded bugs were found by differential but never by metamorphic testing for exactly that reason.

Q17.4 A program prints different results at `-O0` and `-O2`. Is it a compiler bug?
Answer: Usually not. Check determinism; run UBSan/ASan/MSan/valgrind/TSan and `-Wall -Wextra`; look for unspecified or implementation-defined behaviour (evaluation order, `char` signedness, FP contraction), strict-aliasing and lifetime violations. Only if the program is clean, localise (pass bisection, IR diff), reduce, re-check the reduced program, and report. The table of six programs earlier in this chapter shows six disagreements and zero compiler bugs.

Q17.5 Why must generated programs avoid undefined behaviour, and how do Csmith or YARPGen manage it?
Answer: With UB the compiler may legally do anything, so a mismatch proves nothing and testing drowns in false positives. Generators avoid UB by construction: initialised variables, wrapped or guarded arithmetic (safe-math helpers, unsigned wrap-around, masked shifts), bounded loops, valid indices, static analysis of pointers (Csmith), and tracking value ranges during generation so that no operation can overflow (YARPGen, without run-time checks).

Q17.6 Explain delta debugging and the interestingness test; name two pitfalls.
Answer: Repeatedly remove chunks of the input (halves, quarters, ...) and keep any smaller input for which a test script still reports the failure; stop when no single removal works (1-minimal). Worst case $\sim n^2$ test runs. Pitfalls: the reduced program drifts into undefined behaviour or into a different bug (so check the *same* symptom and run sanitizers inside the test), and flaky or slow tests (give timeouts, make them deterministic). Hierarchical variants and C-Reduce/`llvm-reduce` use the syntax so that most candidates are valid.

Q17.7 Between a good and a bad build lie 10 000 commits. How many builds do you need, and what if some commits do not build?
Answer: $\lceil \log_2 10\,000 \rceil = 14$ test runs by binary search. Commits that do not build return exit code 125 ("skip") from the `git bisect run` script; git then picks a neighbour. A non-monotonic history (the bug appears, disappears and reappears) can mislead bisection: make sure the test checks *one* bug.

Q17.8 What is a flaky test and how should the infrastructure treat it?
Answer: One that both passes and fails on identical code. Log retries and report FLAKY instead of hiding it; keep a per-test flake rate; quarantine with owner and expiry rather than silently skipping; classify infrastructure versus product failures; store seeds and environment so a failure is replayable; treat timeouts as results.

Q17.9 A new instruction `foo.bf16x2` was added to the compiler. How do you verify it?
Answer: Build the test matrix from the ISA description: every type and modifier combination (rounding, saturation, flush-to-zero), operand classes (register, immediate), and *boundary values* (0, ±1, max/min, infinities, NaNs with payloads, denormals). Compute expected results with a bit-exact CPU reference (soft float or a high-precision library). Check that `ptxas` accepts the PTX and that the SASS is plausible (FileCheck), run on every supported `sm_` target, compare across optimisation levels, include the instruction in random-kernel generation, and add regression tests for each bug found.

Q17.10 Translation validation versus a verified compiler?
Answer: A verified compiler (CompCert) carries a machine-checked proof for the compiler itself: all future compilations are correct. Translation validation checks each *individual* compilation (source vs. output, usually with an SMT solver, e.g. Alive2 for LLVM IR): no proof of the compiler is needed, and it can be applied to an existing, unverified compiler, but it checks only what it is run on and is typically bounded.

Q17.11 Design a random generator for CUDA kernels whose results are deterministic.
Answer: One output slot per thread (no races); expression DAGs over integers with safe-math wrappers; bounded loops; initialised memory; barriers only in block-uniform control flow, shared-memory phases as write–barrier–read; divergence from `threadIdx`- and data-dependent conditions; no floating-point atomics (or tolerance-based comparison); print a checksum per kernel. Oracle: the same source compiled for the CPU (as in Chapter 15), plus differential comparison across optimisation levels, `sm_` targets and releases; reduce failures automatically.

Q17.12 The compiler crashes and prints "Running pass `X` on function `Y`". First steps?
Answer: Save the exact command line and compiler version; re-run to confirm determinism; reduce the input (`llvm-reduce` with the crash message as interestingness test); get the IR just before the pass (`-mllvm -print-before=X`, or `-mllvm -opt-bisect-limit`, `llc -stop-before=X`); run with assertions or verifiers enabled (`-verify-each`, `-verify-machineinstrs`); check for a duplicate in the tracker by crash signature; file with the reduced input.

Q17.13 How do you know your test suite is any good?
Answer: Coverage of the compiler (which passes and branches no test reaches), feature/matrix coverage (instructions × types × targets), and *mutation testing*: inject known bugs and measure the fraction the suite catches; every surviving mutant names a missing test. Also: bug-escape analysis (for each bug found later by users, why did no test catch it?).

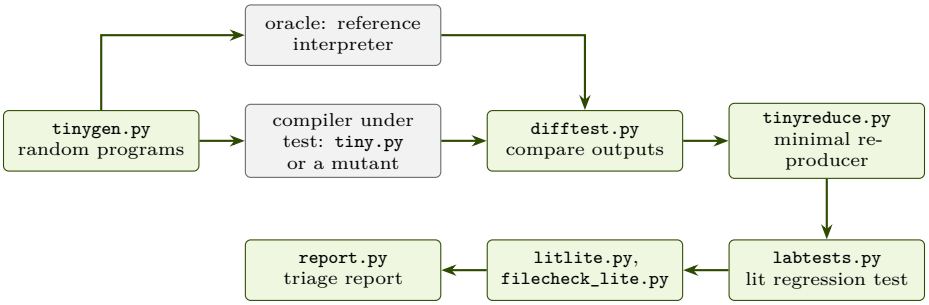
Q17.14 The same CUDA kernel gives slightly different floating-point results on two GPU generations. Bug?
Answer: Not necessarily: fused multiply-add contraction (`--fmad`), different math-library implementations (documented ULP error bounds), flush-to-zero defaults, and different reduction orders all change the last bits. Compare with a tolerance derived from the documented bounds; for bit-exact IEEE operations (`+ - * /`, `sqrt` with round-to-nearest) results *must* match, so a difference there is a bug. Re-run with `--fmad=false -ftz=false -prec-div=true -prec-sqrt=true` to separate contraction effects from real defects.

18. The Automation Lab: A Mini Verification Framework in Python

In this chapter

Priority ★★★ for the ideas, ★★ for the code. Time: 120 min to read and run everything; 45 min if you only read the results. The job asks you to "design and implement tools, libraries, and automation to support compiler development and verification" and lists Python as a core skill. This chapter builds, from scratch and in about 500 lines, the workflow of Chapter 17: a random program generator, a differential tester, metamorphic variants, an automatic test-case reducer, seeded bugs to *measure* the whole thing (mutation testing), a mini `FileCheck` and a mini `lit` with timeouts, retries, flake detection and sharding, and a triage report. The system under test is the **Tiny compiler** of Chapter 10 (source → three-address code → optimiser → virtual machine; listing in Appendix **??**). Every listing is the *actual file* the build wrote and ran; every number is captured output of that run, with fixed seeds so you can reproduce it.

18.1 What we build ★★★



Lab file	What it does	Industry counterpart
`tinygen.py`	random, always-terminating, UB-free programs; generation *policies*; metamorphic variants	Csmith, YARPGen, CLsmith, GraphicsFuzz
`difftest.py`	compile and run, compare with the reference interpreter; classify pass / miscompile / ICE	cross-compiler, cross-`-O`, cross-target harnesses
`mutants.py`, `study.py`	eight seeded compiler bugs; which testing method catches which; reproducers	mutation testing; fault-injection studies
`tinyreduce.py`	hierarchical delta debugging on the syntax tree	C-Reduce, `cvise`, `llvm-reduce`
`labtests.py`, `tinyc.py`	turn a reproducer into a lit test with golden output	the "add a regression test" step of every bug fix
`filecheck_lite.py`	`CHECK`, `CHECK-NEXT`, `CHECK-NOT`, `{{regex}}`	LLVM `FileCheck` (cross-validated against it below)
`litlite.py`	discover tests, run `RUN:` lines in parallel; timeout, retry, XFAIL, shard, JSON	LLVM `lit`
`report.py`	summary table and failure grouping by normalised signature	nightly dashboards, triage bots

The language. Tiny has integer variables (unbounded), arithmetic with C-style truncating division, comparisons, `&&`, `||`, `!`, assignment, `if`/`else`, `while` and `print`; division by zero is a defined *trap* (so there is no undefined

behaviour to avoid, only traps to preserve). The optimiser does constant folding and propagation, copy propagation, algebraic simplification, common-subexpression elimination, dead-code elimination, branch and jump clean-up, and temporary coalescing; it was fuzzed against the interpreter before this chapter existed, so *the pristine compiler is expected to pass everything.*

18.2 Step 1: a random program generator ★★★

Design rules (the same as for Csmith, in miniature): **every program terminates** (each loop has its own counter variable per nesting level, which the body never assigns); **no variable is read before it is set**; **divisors are usually** ≥ 1 (`rhs*rhs+1`), but one in ten is a raw expression that may trap; **variables `c`–`f` are scratch** (only `a` and `b` are printed at the end), so dead code arises naturally; everything is driven by one seed. Three **policies** change *what* is generated, not how valid it is: `plain`; `reuse` (re-use small earlier expressions, sometimes with swapped operands, to feed common-subexpression elimination); `full` (`reuse` plus small *idioms* that optimisers like to rewrite: copy-then-clobber, a dead division, `x-y` next to `y-x`). A separate random stream applies **meaning-preserving edits** when `meta=True` (used in Step 4): value-preserving wrappers (`e+0`, `1*e`, `-(-e)`, `e-0`) and never-taken branches.

tinygen.py: the generator

```
"""tinygen.py -- random generator of well-defined Tiny programs.
Every loop terminates and no variable is read before it is set."""
import random

VARS = ["a", "b", "c", "d", "e", "f"]      # a and b are printed at the end; c..f are scratch (can become dead)
CONSTS = [0, 1, 2, 3, 5, 7, 10, 100]
OPS = ["+", "-", "*", "/", "%", "<", "<=", ">", ">=", "==", "!=", "&&", "||"]

def nodes(e):                               # number of AST nodes of an expression tuple
    return 1 + sum(nodes(x) for x in e[2:]) if e[0] in ("bin", "un") else 1

def show(e, m=None):                        # expression tuple -> source; an RNG m adds identity rewrites
    k = e[0]
    if k == "num": s = str(e[1]) if e[1] >= 0 else "(-%d)" % -e[1]
    elif k == "var": s = e[1]
    elif k == "un": s = "(%s%s)" % (e[1], show(e[2], m))
    else: s = "(%s %s %s)" % (show(e[2], m), e[1], show(e[3], m))
    if m is not None and k in ("bin", "un") and m.random() < 0.15:      # metamorphic edit: value-preserving wrapper
        s = m.choice(["(%s + 0)", "(1 * %s)", "(%s - 0)", "(-(-%s))"]) % s
    return s

class Gen:
    def __init__(self, seed, policy="full", meta=False):    # policy: "plain" | "reuse" | "full" (reuse + idioms)
        self.r, self.policy, self.recent, self.meta = random.Random(seed), policy, [], meta
        self.m = random.Random(seed * 7919 + 1)             # separate stream: edits never disturb the base program

    def sh(self, e):
        return show(e, self.m if self.meta else None)

    def dead_branch(self):                                  # a branch that is never taken: v*v + 1 >= 1
        v, w = self.m.choice(VARS), self.m.choice(VARS)
        cond = show(("bin", "==", ("bin", "+", ("bin", "*", ("var", v), ("var", v)), ("num", 1)), ("num", 0)))
        return "if (%s) { %s = %s + 1; print(%s); } " % (cond, w, v, w)

    def expr(self, depth):
        r = self.r
        small = [x for x in self.recent if nodes(x) <= 7]   # (keeps programs from growing exponentially)
        if self.policy != "plain" and small and r.random() < 0.25:      # policy: re-use an earlier small expression
            e = r.choice(small)
            if e[1] in ("+", "-", "*") and r.random() < 0.5:            # ... sometimes with swapped operands
                e = ("bin", e[1], e[3], e[2])
            return e
        if depth == 0 or r.random() < 0.25:
            if r.random() < 0.5: return ("num", r.choice(CONSTS) * r.choice([1, 1, -1]))
            return ("var", r.choice(VARS))
        x = r.random()
        if x < 0.06: return ("un", "-", self.expr(depth - 1))
        if x < 0.10: return ("un", "!", self.expr(depth - 1))
        op, lhs, rhs = r.choice(OPS), self.expr(depth - 1), self.expr(depth - 1)
        if op in ("/", "%") and r.random() < 0.9:           # usually keep the divisor >= 1: rhs*rhs + 1
            rhs = ("bin", "+", ("bin", "*", rhs, rhs), ("num", 1))
        e = ("bin", op, lhs, rhs)
        self.recent = (self.recent + [e])[-6:]
        return e

    def idiom(self):                                        # policy: small patterns that optimisers like to rewrite
        r, sh = self.r, self.sh
        v, w = r.sample(VARS, 2)
        k = r.randint(0, 2)
```

```
        if k == 0:                                          # copy, then clobber the source
            return "%s = %s; %s = %s; print(%s);" % (v, w, w, sh(self.expr(2)), v)
        if k == 1:                                          # a dead division that may trap
            return "z = %s / %s;" % (sh(self.expr(2)), sh(self.expr(2)))
        vv, ww = ("var", v), ("var", w)                     # x-y next to y-x
        return "%s = %s; %s = %s;" % (v, sh(("bin", "-", vv, ww)), w, sh(("bin", "-", ww, vv)))

    def stmt(self, depth, level):
        text = self.stmt0(depth, level)
        return self.dead_branch() + text if self.meta and self.m.random() < 0.15 else text

    def stmt0(self, depth, level):
        r, x = self.r, self.r.random()
        if self.policy == "full" and r.random() < 0.12: return self.idiom()
        if depth == 0 or x < 0.55:
            v = r.choice(VARS)
            if r.random() < 0.2: return "%s = %s;" % (v, r.choice(VARS))       # plain copy: feeds copy propagation
            return "%s = %s;" % (v, self.sh(self.expr(3)))
        if x < 0.65: return "print(%s);" % self.sh(self.expr(3))
        if x < 0.85:
            s = "if (%s) { %s }" % (self.sh(self.expr(2)), self.block(depth - 1, level))
            return s + (" else { %s }" % self.block(depth - 1, level) if r.random() < 0.5 else "")
        i, n = "i%d" % level, r.randint(1, 4)               # one loop counter per nesting level: loops terminate
        body = self.block(depth - 1, level + 1)
        return "%s = 0; while (%s < %d) { %s %s = %s + 1; }" % (i, i, n, body, i, i)

    def block(self, depth, level):
        return " ".join(self.stmt(depth, level) for _ in range(self.r.randint(1, 3)))

def program(seed, stmts=10, policy="full", meta=False):
    g = Gen(seed, policy, meta)
    init = " ".join("%s = %d;" % (v, g.r.choice(CONSTS)) for v in VARS)
    return init + "\n" + "\n".join(g.stmt(2, 0) for _ in range(stmts)) + "\nprint(a); print(b);\n"
```

```
import statistics, tiny
from tinygen import program
srcs = [program(s) for s in range(300)]
asts = [tiny.parse(x) for x in srcs]                        # every generated program must parse ...
res = [tiny.interpret(a) for a in asts]                     # ... and run to completion or to a defined trap
print("300 programs: parse errors 0 (a parse error would have raised); mean length %d characters (max %d)"
      % (statistics.mean(len(x) for x in srcs), max(len(x) for x in srcs)))
print("end in a trap: %d | hit the step limit: %d | mean number of printed values: %.1f"
      % (sum(t == "division by zero" for _, t in res), sum(t == "step limit" for _, t in res), statistics.mean(len(o) for o, _ in
     res)))
print("with a loop: %d | with if/else: %d | with a division or %%: %d"
      % (sum("while" in x for x in srcs), sum("else" in x for x in srcs), sum(("/" in x) or ("%" in x) for x in srcs)))
print("\n--- program(7), first lines:")
for line in program(7).split("\n")[:5]: print(line[:110] + (" ..." if len(line) > 110 else ""))
```

Captured output

```
300 programs: parse errors 0 (a parse error would have raised); mean length 985 characters (max 2450)
end in a trap: 110 | hit the step limit: 0 | mean number of printed values: 3.7
with a loop: 266 | with if/else: 226 | with a division or %: 297

--- program(7), first lines:
a = 7; b = 2; c = 10; d = 0; e = 1; f = 1;
e = b; b = 3; print(e);
e = a; a = ((2 / (((-1) * (-1)) + 1)) + f); print(e);
f = (f - a); a = (a - f);
f = (((2 / (((-1) * (-1)) + 1)) / (((10 % ((100 * 100) + 1)) * (10 % ((100 * 100) + 1))) + 1)) < ((c || a) < ( ...
```

18.3 Step 2: differential testing against a reference interpreter ★★★

The oracle is the *tree-walking interpreter of the pristine compiler*: a second, independent implementation of the language (it shares only the arithmetic helper with the virtual machine). The system under test compiles the program to three-address code, optimises it and runs it on the virtual machine. Outcomes: **pass**, **MISCOMPILE** (different output or trap behaviour), **ICE** (the compiler raised an exception), **skip** (step limit reached on either side: inconclusive, not a failure). Note the design choice that matters most later: the oracle always comes from the *pristine* module even when the system under test is a mutant; otherwise a bug in shared code would affect both sides and cancel out (a common-mode failure).

difftest.py: oracle, classification, campaign

```
"""difftest.py -- differential and metamorphic testing of a Tiny compiler."""
import tiny as golden                  # the pristine compiler: its tree-walking interpreter is the oracle
from tinygen import program

def check(src, compiler, opt=True):
```

```
    ref = golden.interpret(golden.parse(src))          # (output list, trap message or None)
    try:
        out, trap, _ = compiler.run_tac(compiler.compile_source(src, opt))
    except Exception as e:                             # an exception inside the compiler is an ICE
        return "ICE", "%s: %s" % (type(e).__name__, e)
    if "step limit" in (ref[1], trap): return "skip", None
    return ("pass", None) if (ref[0], ref[1]) == (out, trap) else ("MISCOMPILE", (ref, (out, trap)))

def campaign(compiler, n=300, first=0, opt=True):
    """Run n random programs; return {'pass': k, ...} and the list of (seed, kind, detail) failures."""
    counts, failures = {}, []
    for seed in range(first, first + n):
        kind, detail = check(program(seed), compiler, opt)
        counts[kind] = counts.get(kind, 0) + 1
        if kind in ("MISCOMPILE", "ICE"): failures.append((seed, kind, detail))
    return counts, failures

def run_sut(src, compiler, opt=True):
    out, trap, _ = compiler.run_tac(compiler.compile_source(src, opt))
    return out, trap

def meta_check(seed, compiler):
    """Oracle-free (metamorphic): a program and its meaning-preserving variant must
    behave identically under the SAME compiler."""
    try:
        a, b = run_sut(program(seed), compiler), run_sut(program(seed, meta=True), compiler)
    except Exception as e:
        return "ICE", "%s: %s" % (type(e).__name__, e)
    if "step limit" in (a[1], b[1]): return "skip", None
    return ("pass", None) if a == b else ("MISCOMPILE", (a, b))
```

```
import difftest, tiny
print("pristine compiler vs reference interpreter, 300 random programs")
for opt in (False, True):
    counts, failures = difftest.campaign(tiny, 300, opt=opt)
    print("  optimiser %-3s: %s" % ("on" if opt else "off", counts))
```

Captured output

```
pristine compiler vs reference interpreter, 300 random programs
  optimiser off: {'pass': 300}
  optimiser on : {'pass': 300}
```

All 300 programs agree with and without the optimiser: the baseline is clean, so any later disagreement is a seeded bug. (A clean baseline is itself a result: it took one earlier round of fuzzing, three bugs in the optimiser and a rewrite of its jump clean-up to get here.)

18.4 Step 3: seeded bugs, so that the method can be measured ★★★

How do you know a random tester is any good? Break the compiler on purpose and see whether the tester notices (**mutation testing**, or fault injection). Each entry below rewrites one spot of `tiny.py` and loads the result as a separate module; the table of what they model is under the listing.

mutants.py: eight seeded bugs

```
"""mutants.py -- seeded compiler bugs: each entry rewrites one spot of tiny.py.
Every mutant is loaded as its own module, so the pristine compiler stays available as the oracle."""
import os, re, types
SRC = open(os.path.join(os.path.dirname(os.path.abspath(__file__)), "tiny.py")).read()

MUTANTS = {                                    # name: (regex of the original text, replacement)
  # / is folded with floor instead of truncation toward zero
  "fold-floor-div": (r'BUGGY_FOLD = False', 'BUGGY_FOLD = True'),
  # x - 0  ==>  0
  "sub-zero": (r'elif op == "-" and b == 0: res = a',
               'elif op == "-" and b == 0: res = 0'),
  # value numbering treats a-b and b-a as the same expression
  "cse-commutes-all": (r'key = \(op, a, b\) if op not in "\+\*" else \(op,\) \+ tuple\(sorted\(\(a, b\), key=str\)\)',
                       'key = (op,) + tuple(sorted((a, b), key=str))'),
  # off-by-one when a branch is inverted: "<" becomes ">" instead of ">="
  "invert-lt": (r'INVERT = \{"<": ">=",', 'INVERT = {"<": ">",'),
  # copy facts are not invalidated when their source is redefined
  "stale-copy": (r' *for k in \[k for k, v in env.items\(\) if v == name\]: del env\[k\]\n', ''),
  # dead-code elimination also deletes divisions that could trap
  "dce-trap": (r'used and\n\s+not \(ins\[0\] == "bin".*?\)\)\)\]', 'used)]'),
  # index error in a helper: the compiler crashes (ICE) on valid input
  "uses-crash": (r'elif k == "un": xs = \(ins\[3\],\)', 'elif k == "un": xs = (ins[4],)'),
  # coalesces temporaries that are used more than once
  "coalesce-multi": (r'count.get\(ins\[1\]\) == 1', 'count.get(ins[1], 0) >= 1'),
```

```
}

def load(name):
    pattern, repl = MUTANTS[name]
    text, n = re.subn(pattern, lambda m: repl, SRC, flags=re.S)
    assert n == 1, "mutation site for %s must match exactly once (matched %d)" % (name, n)
    mod = types.ModuleType("tiny_" + name.replace("-", "_"))
    exec(compile(text, "<mutant %s>" % name, "exec"), mod.__dict__)
    return mod
```

Mutant	The bug	Real-world family
`fold-floor-div`	constant-folds `/` with floor instead of truncation toward zero	wrong constant folding (rounding, `INT_MIN / -1`, shifts)
`sub-zero`	simplifies `x - 0` to `0`	wrong algebraic identity
`cse-commutes-all`	value numbering treats `a-b` and `b-a` as the same expression	non-commutative operator handled as commutative
`invert-lt`	inverts `<` into `>` instead of `>=` when a branch is flipped	off-by-one at a boundary in branch folding
`stale-copy`	copy facts are not invalidated when the source is redefined	stale data-flow fact
`dce-trap`	dead-code elimination deletes a division that could trap	removing an instruction with an observable effect
`uses-crash`	index error in a helper function	ICE: crash on valid input
`coalesce-multi`	merges a temporary that is used more than once	unsound optimisation precondition

18.5 Step 4: metamorphic variants (no reference needed) ★★

`meta_check` in `difftest.py` compiles the program and a variant that differs only by value-preserving wrappers and never-taken branches; the *same* compiler must treat both identically. The variant is built by the generator's second random stream, so the base program is unchanged. First check that the transformations really preserve meaning, and look at one variant:

```
import difflib, tiny
from tinygen import program
bad = sum(tiny.interpret(tiny.parse(program(s))) != tiny.interpret(tiny.parse(program(s, meta=True))) for s in range(300))
print("300 seeds: interpreter disagrees between a program and its metamorphic variant in %d cases" % bad)
print("\n--- first differences, program(3) vs program(3, meta=True) (cut to 100 columns):")
a, b = program(3).split("\n"), program(3, meta=True).split("\n")
for l in list(difflib.unified_diff(a, b, lineterm="", n=0))[2:8]: print(l[:100])
```

Captured output

```
300 seeds: interpreter disagrees between a program and its metamorphic variant in 0 cases

--- first differences, program(3) vs program(3, meta=True) (cut to 100 columns):
@@ -2,2 +2,2 @@
-i0 = 0; while (i0 < 2) { i1 = 0; while (i1 < 4) { f = e; a = (a - b); b = (b - a); i1 = i1 + 1; } i
-c = (((a * 100) && (e % ((e * e) + 1))) && (-(e && b)));
+i0 = 0; while (i0 < 2) { i1 = 0; while (i1 < 4) { if ((((a * a) + 1) == 0)) { a = a + 1; print(a);
+c = ((((a * 100) && (e % ((e * e) + 1))) + 0) && (-(e && b)));
@@ -5,7 +5,7 @@
```

18.6 Step 5: automatic test-case reduction ★★★

A failing random program has hundreds of tokens; the person who fixes the bug wants five. `tinyreduce.py` implements the idea of C-Reduce in miniature: **greedy hierarchical reduction** on the syntax tree. `shrinks` yields simpler variants of a node, most aggressive first: delete chunks of statements (halves, quarters, single statements: the ddmin idea), splice a nested block into its parent, keep only the `then`- or `else`-part of an `if`, run a loop body once, replace an expression by one of its operands or by a constant, shrink literals. `reduce` accepts the first smaller candidate that is still *interesting* and starts again, until nothing helps (a local minimum, "1-minimal" for these transformations). `same_failure` is the interestingness test: it demands the *same kind* of failure (for crashes, the same exception text), because reducing towards a different bug is the classic trap.

tinyreduce.py: hierarchical delta debugging

```
"""tinyreduce.py -- hierarchical delta debugging for Tiny programs: a miniature C-Reduce."""
import tiny, difftest

def unparse(n):                                  # AST -> source text
    k = n[0]
    if k == "num":   return str(n[1]) if n[1] >= 0 else "(-%d)" % -n[1]
    if k == "var":   return n[1]
    if k == "un":    return "(%s%s)" % (n[1], unparse(n[2]))
    if k == "bin":   return "(%s %s %s)" % (unparse(n[2]), n[1], unparse(n[3]))
    if k == "assign": return "%s = %s;" % (n[1], unparse(n[2]))
    if k == "print": return "print(%s);" % unparse(n[1])
    if k == "block": return "{ " + " ".join(unparse(s) for s in n[1]) + " }"
    if k == "if":    return "if (%s) %s" % (unparse(n[1]), unparse(n[2])) + (" else " + unparse(n[3]) if n[3] else "")
    if k == "while": return "while (%s) %s" % (unparse(n[1]), unparse(n[2]))

def source(ast): return "\n".join(unparse(s) for s in ast[1])

def size(n):                                # smaller = simpler; a literal costs its bit length (100 -> 0 is progress)
    if isinstance(n, tuple): return 1 + sum(size(x) for x in n[1:])
    if isinstance(n, list):  return sum(size(x) for x in n)
    return n.bit_length() if isinstance(n, int) and n > 0 else 0

def shrinks(n):                                  # candidate simplifications of a node, most aggressive first
    k = n[0]
    if k == "block":
        ss, step = n[1], max(len(n[1]) // 2, 1)
        while ss:                                # 1. delete chunks of statements: halves, quarters, ..., singles
            for i in range(0, len(ss), step):
                yield ("block", ss[:i] + ss[i + step:])
            if step == 1: break
            step //= 2
        for i, s in enumerate(ss):               # 2. splice a nested block into its parent
            if s[0] == "block": yield ("block", ss[:i] + s[1] + ss[i + 1:])
        for i, s in enumerate(ss):               # 3. simplify one statement in place
            for s2 in shrinks(s): yield ("block", ss[:i] + [s2] + ss[i + 1:])
    elif k == "if":
        yield n[2]                               # keep only the then-part ...
        if n[3] is not None: yield n[3]; yield ("if", n[1], n[2], None)     # ... or the else-part, or drop the else
        for c in shrinks(n[1]): yield ("if", c, n[2], n[3])
        for t in shrinks(n[2]): yield ("if", n[1], t, n[3])
        if n[3] is not None:
            for e in shrinks(n[3]): yield ("if", n[1], n[2], e)
    elif k == "while":
        yield n[2]                               # run the body once, drop the loop
        for c in shrinks(n[1]): yield ("while", c, n[2])
        for b in shrinks(n[2]): yield ("while", n[1], b)
    elif k in ("assign", "print"):
        i = 2 if k == "assign" else 1
        for e in shrinks(n[i]): yield n[:i] + (e,) + n[i + 1:]
    elif k == "bin":
        yield n[2]; yield n[3]; yield ("num", 0); yield ("num", 1)         # replace by an operand or a constant
        for l in shrinks(n[2]): yield ("bin", n[1], l, n[3])
        for r in shrinks(n[3]): yield ("bin", n[1], n[2], r)
    elif k == "un":
        yield n[2]; yield ("num", 0)
        for e in shrinks(n[2]): yield ("un", n[1], e)
    elif k == "var":
        yield ("num", 0)
    elif k == "num" and n[1] != 0:
        yield ("num", 0)
        if n[1] > 1: yield ("num", n[1] // 2)

def reduce(ast, interesting):
    """Greedy reduction: take the first simpler candidate that is still interesting, restart,
    stop when no candidate helps."""
    tests, progress = 0, True
    while progress:
        progress = False
        for cand in shrinks(ast):
            if size(cand) >= size(ast): continue
            tests += 1
            if interesting(cand):
                ast, progress = cand, True
                break
    return ast, tests

def same_failure(compiler, kind, detail):
    """Interestingness test: the candidate must still fail the SAME way
    (a different crash would be a different bug)."""
    def interesting(ast):
        k, d = difftest.check(source(ast), compiler)
        return k == kind and (kind != "ICE" or d == detail)
```

```
    return interesting
```

```
import tiny, mutants, difftest
import tinyreduce as R
from tinygen import program
m = mutants.load("stale-copy")
seed = next(s for s in range(500) if difftest.check(program(s), m)[0] == "MISCOMPILE")
src = program(seed)
accepted, check = [], R.same_failure(m, "MISCOMPILE", None)
def interesting(ast):
    ok = check(ast)
    if ok: accepted.append(R.size(ast))
    return ok
small, tests = R.reduce(tiny.parse(src), interesting)
print("mutant stale-copy, first failing seed %d: %d characters, %d lines" % (seed, len(src), len(src.splitlines())))
print("AST size after each accepted reduction:", accepted)
print("%d candidate programs tested; result (%d characters):" % (tests, len(R.source(small))))
print(R.source(small))
print("reference interpreter prints", tiny.interpret(small)[0], "- the buggy compiler prints", difftest.run_sut(R.source(small), m
    )[0])
```

Captured output

```
mutant stale-copy, first failing seed 0: 761 characters, 12 lines
AST size after each accepted reduction: [204, 141, 131, 129, 125, 120, 119, 73, 71, 66, 59, 58, 28, 23, 10, 9, 8]
46 candidate programs tested; result (23 characters):
e = a;
a = 1;
print(e);
reference interpreter prints [0] - the buggy compiler prints [1]
```

Why this matters. From a 570–2 000-character random program the reducer gets to a one-liner in a few hundred test runs (about a second): that is the artefact you attach to a bug report and keep as a regression test. In real C-Reduce the same loop runs hundreds of passes over C syntax with a compile-and-run script as the test, and takes minutes.

18.7 Step 6: the mutation study ★★★

`study.py` runs every mutant against 300 random programs with three generator policies (differential testing) and against the metamorphic checker, then reduces the first failing program of each caught mutant and writes it out as a regression test (next section). It uses **processes** (`ProcessPoolExecutor`), because the work is pure-Python CPU-bound (Chapter 16); each worker re-creates its own mutant module from the mutant's *name*, since module objects built with `exec` cannot be pickled.

study.py: the experiment

```
"""study.py -- mutation study: which seeded compiler bugs does each testing method catch,
and how small is the reproducer?"""
from concurrent.futures import ProcessPoolExecutor
import difftest, mutants, tiny
import tinyreduce as R
from labtests import write_lit_test
from tinygen import program

N = 300                                  # random programs per (mutant, method)
POLICIES = ("plain", "reuse", "full")

def differential(job):                   # oracle = reference interpreter of the pristine compiler
    name, policy = job
    m = mutants.load(name)
    bad = [s for s in range(N) if difftest.check(program(s, policy=policy), m)[0] in ("MISCOMPILE", "ICE")]
    return len(bad), (bad[0] if bad else None)

def metamorphic(name):                   # no oracle: program vs. meaning-preserving variant
    m = mutants.load(name)
    bad = [s for s in range(N) if difftest.meta_check(s, m)[0] in ("MISCOMPILE", "ICE")]
    return len(bad), (bad[0] if bad else None)

def cell(r):                             # (count, first failing seed) -> table cell
    return "%3d (seed %s)" % r if r[0] else "  0   (-)    "

if __name__ == "__main__":
    names = list(mutants.MUTANTS)
    jobs = [(n, p) for n in names for p in POLICIES]
    with ProcessPoolExecutor(4) as pool:       # CPU-bound pure Python: processes, not threads
        diff = dict(zip(jobs, pool.map(differential, jobs)))
        meta = dict(zip(names, pool.map(metamorphic, names)))
    row = "%-17s %-15s %-15s %-15s %-15s"
    print("bad results among %d random programs, per mutant (first failing seed in brackets)" % N)
```

```
    print(row % ("mutant", "plain", "+reuse", "+idioms", "metamorphic"))
    for n in names:
        print(row % (n, cell(diff[n, "plain"]), cell(diff[n, "reuse"]), cell(diff[n, "full"]), cell(meta[n])))
    killed = ["%d/%d" % (sum(diff[n, p][0] > 0 for n in names), len(names)) for p in POLICIES]
    killed.append("%d/%d" % (sum(meta[n][0] > 0 for n in names), len(names)))
    print(row % ("mutants killed", *killed))

    print("\nreduced reproducers (differential testing, '+idioms' generator, first failing seed)")
    for n in names:
        src = program(diff[n, "full"][1])
        m = mutants.load(n)
        kind, detail = difftest.check(src, m)
        small, tests = R.reduce(tiny.parse(src), R.same_failure(m, kind, detail))
        write_lit_test("tests_reduced/reduced_%s.tiny" % n.replace("-", "_"), small)
        print("%-17s %-10s %5d -> %3d chars in %4d tests:  %s"
              % (n, kind, len(src), len(R.source(small)), tests, R.source(small).replace("\n", " ")))
```

```
$ python3 study.py
```

```
bad results among 300 random programs, per mutant (first failing seed in brackets)
mutant           plain          +reuse          +idioms         metamorphic
fold-floor-div     3 (seed 4)     0   (-)         3 (seed 100)    0   (-)
sub-zero           6 (seed 38)    5 (seed 172)    4 (seed 27)    72 (seed 0)
cse-commutes-all   0   (-)        3 (seed 193)    1 (seed 168)    1 (seed 168)
invert-lt         96 (seed 3)   104 (seed 3)    110 (seed 0)     0   (-)
stale-copy         0   (-)        2 (seed 165)   30 (seed 0)     2 (seed 7)
dce-trap           0   (-)        0   (-)        78 (seed 0)     1 (seed 283)
uses-crash       266 (seed 0)   231 (seed 0)    215 (seed 3)   287 (seed 0)
coalesce-multi    13 (seed 24)   29 (seed 19)    49 (seed 10)    4 (seed 95)
mutants killed   5/8            6/8             8/8             6/8

reduced reproducers (differential testing, '+idioms' generator, first failing seed)
fold-floor-div    MISCOMPILE  1983 ->  39 chars in  229 tests:  if (((-1) / 2)) {  } else { print(0); }
sub-zero          MISCOMPILE   571 ->  22 chars in   59 tests:  print(((0 <= 0) - 0));
cse-commutes-all  MISCOMPILE   761 ->  42 chars in  218 tests:  e = 1; a = e; f = (c - a); print((a - c));
invert-lt         MISCOMPILE   761 ->  38 chars in   58 tests:  while ((i1 < 0)) { print(e); i1 = 1; }
stale-copy        MISCOMPILE   761 ->  23 chars in   46 tests:  e = a; a = 1; print(e);
dce-trap          MISCOMPILE   761 ->  12 chars in   17 tests:  z = (a / 0);
uses-crash        ICE         1219 ->   9 chars in   35 tests:  e = (-f);
coalesce-multi    MISCOMPILE  1375 ->  65 chars in  185 tests:  while ((i0 < 2)) { print(e); e = (d + 1); e = e; i0 = (i0 + 1); }
```

What the experiment teaches (all of it is visible in the table above)

1. **A naive random generator misses bugs.** The "plain" column kills five of eight mutants; `cse-commutes-all`, `stale-copy` and `dce-trap` survive 300 programs because the generator rarely produces the *pattern* that triggers them (a subtraction and its mirror image in one basic block; a copy followed by a clobber of its source; a dead trapping division). A surviving mutant is a precise request: "teach the tester this pattern".
2. **Generation policies work** (YARPGen's central idea): re-using sub-expressions finds `cse-commutes-all` and `stale-copy`; adding idioms finds `dce-trap` and everything else, eight of eight. But policies change the odds of *other* bugs too (`fold-floor-div` is rarer under "reuse"): rare-event bugs need many programs or a directed idiom.
3. **Different oracles see different bugs.** Metamorphic testing, with no reference at all, catches six of eight — including `sub-zero` 72 times in 300 programs, far more often than differential testing does — but never `fold-floor-div` or `invert-lt`: those bugs miscompile the program *and* its variant in the same way, so the two runs agree. Differential testing needs the oracle but catches them at once.
4. **Reduction makes every bug readable:** programs of 570–2 000 characters shrink to one-liners of 9–65 characters; the reducer's choices can look odd (the `fold-floor-div` reproducer is an `if`, because `(-1)/2` folds to −1, which is true, instead of 0, which is false) but they are *minimal and valid*.
5. **Crashes are the cheapest class:** `uses-crash` is found by nearly every program; most real ICE campaigns are won by volume, wrong-code campaigns by cleverness.

18.8 Step 7: from reproducer to regression test ★★★

Every fixed bug leaves behind a small test. The reducer's output becomes a **lit test**: a `RUN:` line that compiles and runs the program, and `CHECK:` lines with the *golden output computed by the reference interpreter*. `labtests.py` writes it; note the last line it adds, `CHECK-NOT: {{.}}` ("and nothing else may follow"): without it a FileCheck test passes when the compiler prints the expected lines *plus garbage afterwards*, and one of our seeded bugs slips through (we found this the hard way while writing the lab). `tinyc.py` is the command-line driver that tests call.

labtests.py: reproducer -> lit test

```
"""labtests.py -- write a lit test (RUN line + golden CHECK lines from the reference interpreter)."""
import os
import tiny
import tinyreduce as R

def write_lit_test(path, ast):
    out, trap = tiny.interpret(ast)
    if not out and not trap:                        # make sure there is something to check
        ast = ("block", ast[1] + [("print", ("num", 0))])
        out, trap = tiny.interpret(ast)
    checks = ["{{^%d$}}" % v for v in out] + (["{{^trap: %s$}}" % trap] if trap else [])
    lines = ["# RUN: %tinyc --run-opt %s | %filecheck %s", "# CHECK: " + checks[0]]
    lines += ["# CHECK-NEXT: " + c for c in checks[1:]]
    lines += ["# CHECK-NOT: {{.}}"]                 # ... and nothing after the last line
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    with open(path, "w") as f:
        f.write("\n".join(lines) + "\n" + R.source(ast) + "\n")
```

tinyc.py: the driver the tests call

```
#!/usr/bin/env python3
"""tinyc.py -- command-line driver for the Tiny compiler: the program the lit tests call.
usage: tinyc.py (--emit-tac | --emit-opt | --run | --run-opt) FILE
environment: TINYC_MUTANT=<name> selects a seeded-bug build"""
import os, sys
import tiny

def main(mode, path):
    lines = open(path).read().split("\n")
    src = "\n".join(l for l in lines if not l.startswith("#"))     # '#' lines carry test directives
    compiler = tiny
    if os.environ.get("TINYC_MUTANT"):
        import mutants
        compiler = mutants.load(os.environ["TINYC_MUTANT"])
    if mode in ("--emit-tac", "--emit-opt"):
        print(compiler.show(compiler.compile_source(src, opt=(mode == "--emit-opt"))))
    elif mode in ("--run", "--run-opt"):
        out, trap, _ = compiler.run_tac(compiler.compile_source(src, opt=(mode == "--run-opt")))
        for v in out: print(v)
        if trap: print("trap: " + trap)
    else:
        sys.exit("tinyc: unknown mode " + mode)

if __name__ == "__main__":
    main(sys.argv[1], sys.argv[2])
```

18.8.1 A mini FileCheck, cross-validated against the real one

FileCheck's model: the `CHECK:` patterns must match, in order, somewhere in the input; `CHECK-NEXT:` must match on the very next line; `CHECK-NOT:` must *not* match between the neighbouring positive matches (or to the end of the input). Patterns are literal text, blanks match any run of blanks, `{{...}}` contains a regular expression.

filecheck_lite.py: CHECK, CHECK-NEXT, CHECK-NOT, {{regex}}

```
#!/usr/bin/env python3
"""filecheck_lite.py CHECKFILE < output
A miniature FileCheck: CHECK, CHECK-NEXT, CHECK-NOT and {{regex}} islands."""
import re, sys

def directives(path):
    found = []
    for lineno, line in enumerate(open(path), 1):
        m = re.search(r"\b(CHECK(?:-NEXT|-NOT)?):\s?(.*?)\s*$", line)
        if m: found.append((m.group(1), m.group(2), lineno))
    return found

def to_regex(pattern):              # literal text; a run of blanks matches any run of blanks; {{...}} is a regex
    parts = re.split(r"(\{\{.*?\}\})", pattern)
    return re.compile("".join(p[2:-2] if p.startswith("{{") else
                              re.sub(r"(\\ )+", r"[ \\t]+", re.escape(p)) for p in parts))

def filecheck(path, text):
    lines, errors = text.split("\n"), []
    pos, last, nots = 0, -2, []      # pos: first unconsumed line; last: line of the previous positive match
    def search(rx, start, stop):
        return next((i for i in range(start, stop) if rx.search(lines[i])), None)
    def flush(stop):                 # pending CHECK-NOTs must not match between the previous match and `stop`
        for rx, pat, ln in nots:
            hit = search(rx, pos, stop)
            if hit is not None:
```

```
                errors.append("%s:%d: error: CHECK-NOT: excluded string found in input\n    %s"
                              % (path, ln, lines[hit]))
        nots.clear()
    for kind, pat, ln in directives(path):
        rx = to_regex(pat)
        if kind == "CHECK-NOT":
            nots.append((rx, pat, ln)); continue
        if kind == "CHECK-NEXT":
            nxt = last + 1
            i = nxt if last >= 0 and nxt < len(lines) and rx.search(lines[nxt]) else None
        else:
            i = search(rx, pos, len(lines))
        if i is None:
            errors.append("%s:%d: error: %s: expected string not found in input\n    %s: %s"
                          % (path, ln, kind, kind, pat))
            break
        flush(i)
        pos, last = i + 1, i
    else:
        flush(len(lines))
    return errors

if __name__ == "__main__":
    errs = filecheck(sys.argv[1], sys.stdin.read())
    for e in errs: print(e, file=sys.stderr)
    sys.exit(1 if errs else 0)
```

Six hand-written tests in the usual LLVM style (a pattern that must appear, one that must not, an exact output, a trap) are kept in one text file and split into test files by a tiny awk script:

tests_handwritten.txt: six hand-written tests

```
==== fold.tiny
# RUN: %tinyc --emit-opt %s | %filecheck %s
# CHECK: print 10
# CHECK-NOT: *
x = 2 * 3 + 4;
print(x);
==== cse.tiny
# RUN: %tinyc --emit-opt %s | %filecheck %s
# CHECK: = a + b
# CHECK-NOT: = a + b
y = (a + b) * (a + b);
print(y);
==== deadcode.tiny
# RUN: %tinyc --emit-opt %s | %filecheck %s
# CHECK-NOT: 5 *
# CHECK: print a
z = 5 * a;
print(a);
==== loop.tiny
# RUN: %tinyc --run-opt %s | %filecheck %s
# CHECK: 55
s = 0; i = 1;
while (i <= 10) { s = s + i; i = i + 1; }
print(s);
==== div_trap.tiny
# RUN: %tinyc --run-opt %s | %filecheck %s
# CHECK: 1
# CHECK-NEXT: trap: division by zero
print(1);
print(10 / (5 - 5));
print(2);
==== neg_div.tiny
# RUN: %tinyc --run-opt %s | %filecheck %s
# CHECK: -3
print((-7) / 2);
```

split_tests.sh

```
#!/bin/bash
# split_tests.sh FILE DIR   write every section that starts with "==== name" to DIR/name
mkdir -p "$2"
awk -v dir="$2" '/^==== /{f = dir "/" $2; next} {print > f}' "$1"
```

Does it behave like the real thing? `xcheck.py` runs both on every hand-written test against the output of the pristine compiler, of each of the eight mutants, and of mechanically damaged outputs (first line dropped, order reversed, a line repeated, blanks changed), and compares the verdicts:

xcheck.py: lite vs real FileCheck

```
"""xcheck.py -- cross-validate filecheck_lite against LLVM's real FileCheck on many (check file, input) pairs."""
import glob, os, subprocess
import mutants
from filecheck_lite import filecheck

def tinyc_output(test, mutant=None):
    env = dict(os.environ, TINYC_MUTANT=mutant) if mutant else dict(os.environ)
    mode = "--emit-opt" if "--emit-opt" in open(test).read() else "--run-opt"
    return subprocess.run(["python3", "tinyc.py", mode, test], capture_output=True, text=True, env=env).stdout

def real(test, text):
    fc = os.environ.get("FILECHECK", "FileCheck")
    return subprocess.run([fc, test], input=text, capture_output=True, text=True).returncode == 0

agree = total = 0
different = []
for test in sorted(glob.glob("tests/*.tiny")):
    outs = [tinyc_output(test)] + [tinyc_output(test, m) for m in mutants.MUTANTS]
    for text in list(outs):                                  # add mechanically damaged variants of every output
        lines = text.rstrip("\n").split("\n")
        if len(lines) > 1:
            outs.append("\n".join(lines[1:]))                # drop the first line
            outs.append("\n".join(lines[::-1]))              # reverse the order
        outs.append("\n".join(lines + lines[:1]))            # repeat a line at the end
        outs.append("\n".join("  " + l.replace(" ", "   ") for l in lines))    # change the blanks only
    for text in outs:
        a, b = (filecheck(test, text.rstrip("\n") + "\n") == []), real(test, text)
        total += 1
        agree += (a == b)
        if a != b: different.append((test, text))
print("%d of %d verdicts identical (lite vs real FileCheck)" % (agree, total))
for t, x in different[:3]: print("DIFFERENT:", t, repr(x))
```

```
$ ./split_tests.sh tests_handwritten.txt tests && FILECHECK=/usr/lib/llvm-18/bin/FileCheck python3 xcheck.py
```

```
198 of 198 verdicts identical (lite vs real FileCheck)
```

Identical verdicts on every case. (That is evidence, not proof: the test set is small, and real FileCheck has many more features: `CHECK-LABEL`, `CHECK-SAME`, variables `[[VAR:...]]`, `--match-full-lines`....)

18.9 Step 8: a mini lit ★★★

`litlite.py` is the test-running infrastructure in 64 lines. Features worth noticing, because each one answers an interview question: **discovery and `RUN:` lines** with substitutions (`%s` test file, `%t` private scratch name, `%tinyc`, `%filecheck`); every command runs under `bash -o pipefail`, so a failure of *any* stage of a pipeline fails the test; a **thread pool** (`ThreadPoolExecutor`) is the right tool here because each worker merely waits for a subprocess (Chapter 16); a **timeout** per command, reported as `TIMEOUT` and never retried (a hang is a result); **retries** that turn fail-then-pass into `FLAKY` instead of hiding it, with the scratch files cleaned once *before* the first attempt, not between attempts; `XFAIL`/`XPASS` (an expected failure that passes is itself a failure: somebody fixed the bug and must remove the marker); **sharding** by `zlib.crc32` of the test name (stable across runs and machines; Python's `hash()` is salted per process and would give different shards every time); a JSON result file; and an exit status that CI can use.

litlite.py: discovery, parallel execution, timeouts, retries, XFAIL, shards

```
#!/usr/bin/env python3
"""litlite.py -- a miniature lit: run the RUN lines of many tests in parallel,
with timeouts, retries, sharding and a JSON result file.
usage: litlite.py [-j N] [--timeout S] [--retries R] [--shard I/N] [--filecheck PROGRAM] [--json FILE] TEST_DIR"""
import argparse, concurrent.futures as cf, glob, json, os, re, subprocess, sys, time, zlib

def parse_test(path):
    runs, xfail = [], False
    for line in open(path):
        m = re.search(r"\b(RUN|XFAIL):\s*(.*?)\s*$", line)
        if m and m.group(1) == "RUN": runs.append(m.group(2))
        elif m: xfail = True                                 # "XFAIL: *": this test is expected to fail
    return runs, xfail

def run_test(path, args):
    runs, xfail = parse_test(path)
    name = os.path.basename(path)
    tmp = os.path.join(os.path.dirname(path), "Output", name + ".tmp")        # %t: private scratch name
    os.makedirs(os.path.dirname(tmp), exist_ok=True)
    for old in glob.glob(glob.escape(tmp) + "*"):             # clean once, NOT between retries
```

```
        os.remove(old)
    subst = {"%tinyc": "python3 tinyc.py", "%filecheck": args.filecheck, "%s": path, "%t": tmp}
    t0, status, log, attempts = time.time(), "FAIL", "", 0
    while attempts <= args.retries:
        attempts += 1
        status, log = "PASS", ""
        for cmd in runs:
            cmd = re.sub(r"%(tinyc|filecheck|s|t)\b", lambda m: subst[m.group(0)], cmd)
            try:
                p = subprocess.run(["bash", "-o", "pipefail", "-c", cmd],
                                   capture_output=True, text=True, timeout=args.timeout)
            except subprocess.TimeoutExpired:
                status, log = "TIMEOUT", "$ %s\ncommand timed out after %ss" % (cmd, args.timeout)
                break
            if p.returncode != 0:
                status, log = "FAIL", "$ %s\n%s%s" % (cmd, p.stdout, p.stderr)
                break
        if status in ("PASS", "TIMEOUT"): break               # a timeout is a result, not noise: never retried
    if status == "PASS" and attempts > 1: status = "FLAKY"    # failed, then passed on retry: report, do not hide
    if xfail: status = {"FAIL": "XFAIL", "PASS": "XPASS"}.get(status, status)
    return {"test": name, "status": status, "seconds": round(time.time() - t0, 2),
            "attempts": attempts, "log": log.strip()}

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("dir"); ap.add_argument("-j", type=int, default=4)
    ap.add_argument("--timeout", type=float, default=30); ap.add_argument("--retries", type=int, default=0)
    ap.add_argument("--shard", default="0/1"); ap.add_argument("--json")
    ap.add_argument("--filecheck", default="python3 filecheck_lite.py")
    args = ap.parse_args()
    i, n = map(int, args.shard.split("/"))
    tests = sorted(p for p in glob.glob(os.path.join(args.dir, "*")) if os.path.isfile(p))
    tests = [t for t in tests if zlib.crc32(os.path.basename(t).encode()) % n == i]   # stable across runs and hosts
    t0 = time.time()
    with cf.ThreadPoolExecutor(args.j) as pool:               # threads suffice: each worker just waits for a child
        results = list(pool.map(lambda t: run_test(t, args), tests))
    for k, r in enumerate(results, 1):
        print("%s: tiny :: %s (%d of %d) [%.2fs]" % (r["status"], r["test"], k, len(results), r["seconds"]))
    for r in results:
        if r["status"] in ("FAIL", "TIMEOUT", "XPASS", "FLAKY") and r["log"] or r["status"] == "FAIL":
            print("\n" + "*" * 20 + " TEST 'tiny :: %s' %s " % (r["test"], r["status"]) + "*" * 20)
            print(r["log"] + "\n" + "*" * 20)
    print("\nTesting Time: %.2fs" % (time.time() - t0))
    labels = (("Passed", "PASS"), ("Expectedly Failed", "XFAIL"), ("Failed", "FAIL"), ("Timed Out", "TIMEOUT"),
              ("Unexpectedly Passed", "XPASS"), ("Flaky", "FLAKY"))
    for label, key in labels:
        count = sum(r["status"] == key for r in results)
        if count: print("  %-20s: %d" % (label, count))
    if args.json: json.dump(results, open(args.json, "w"), indent=1)
    sys.exit(1 if any(r["status"] in ("FAIL", "TIMEOUT", "XPASS") for r in results) else 0)

if __name__ == "__main__":
    main()
```

First, the harness features on six deliberately “interesting” tests (the `demo/` directory): a pass, a failure with a crash-like message, a hang, a flaky test, an expected failure and an unexpected pass.

demo_tests.txt: six tests that exercise every status

```
==== pass.sh
# RUN: echo hello | grep -q hello
==== fail.sh
# RUN: echo "internal error: unreachable executed at line 42" >&2; exit 3
==== hang.sh
# RUN: sleep 5
==== flaky.sh
# RUN: test -f %t.marker || { touch %t.marker; echo "timing-dependent failure" >&2; exit 1; }
==== xfail.sh
# XFAIL: *
# RUN: exit 1
==== xpass.sh
# XFAIL: *
# RUN: true
```

```
$ ./split_tests.sh demo_tests.txt demo && python3 litlite.py -j 4 --timeout 1 --retries 1 demo
```

```
FAIL: tiny :: fail.sh (1 of 6) [time]
FLAKY: tiny :: flaky.sh (2 of 6) [time]
TIMEOUT: tiny :: hang.sh (3 of 6) [time]
PASS: tiny :: pass.sh (4 of 6) [time]
XFAIL: tiny :: xfail.sh (5 of 6) [time]
XPASS: tiny :: xpass.sh (6 of 6) [time]
```

```
******************** TEST 'tiny :: fail.sh' FAIL ********************
$ echo "internal error: unreachable executed at line 42" >&2; exit 3
internal error: unreachable executed at line 42
********************

******************** TEST 'tiny :: hang.sh' TIMEOUT ********************
$ sleep 5
command timed out after 1.0s
********************

  Passed              : 1
  Expectedly Failed   : 1
  Failed              : 1
  Timed Out           : 1
  Unexpectedly Passed : 1
  Flaky               : 1
```

Now the real suite, under the pristine compiler, and the effect of sharding and of parallelism on a generated suite of 40 tests (`gensuite.py` writes programs from the generator as lit tests, with golden output from the interpreter):

gensuite.py: generated lit tests

```
"""gensuite.py N -- write N generated programs as lit tests (golden output from the reference interpreter)."""
import sys, tiny
from labtests import write_lit_test
from tinygen import program

n, made, seed = int(sys.argv[1]), 0, 1000
while made < n:
    ast = tiny.parse(program(seed))
    if tiny.interpret(ast)[1] != "step limit":
        write_lit_test("tests_gen/gen_%03d.tiny" % made, ast)
        made += 1
    seed += 1
print("wrote %d tests to tests_gen/" % made)
```

```
$ ./split_tests.sh tests_handwritten.txt tests; python3 gensuite.py 40; python3 litlite.py tests; litlite -j1 / -j4 timing; three
     shards
```

```
wrote 40 tests to tests_gen/
PASS: tiny :: cse.tiny (1 of 10) [time]
PASS: tiny :: deadcode.tiny (2 of 10) [time]
PASS: tiny :: div_trap.tiny (3 of 10) [time]
PASS: tiny :: fold.tiny (4 of 10) [time]
FAIL: tiny :: identity.ll (5 of 10) [time]
PASS: tiny :: loop.tiny (6 of 10) [time]
FAIL: tiny :: mul8.ll (7 of 10) [time]
PASS: tiny :: neg_div.tiny (8 of 10) [time]
FAIL: tiny :: wrong_expectation.ll (9 of 10) [time]
XFAIL: tiny :: xfail_example.ll (10 of 10) [time]

******************** TEST 'tiny :: identity.ll' FAIL ********************
$ opt -passes=instcombine -S tests/identity.ll | FileCheck tests/identity.ll
bash: line 1: FileCheck: command not found
********************

******************** TEST 'tiny :: mul8.ll' FAIL ********************
$ opt -passes=instcombine -S tests/mul8.ll | FileCheck tests/mul8.ll
bash: line 1: FileCheck: command not found
********************

******************** TEST 'tiny :: wrong_expectation.ll' FAIL ********************
$ opt -passes=instcombine -S tests/wrong_expectation.ll | FileCheck tests/wrong_expectation.ll
bash: line 1: FileCheck: command not found
********************

  Passed              : 6
  Expectedly Failed   : 1
  Failed              : 3
40 generated tests, -j 1: 1.46 s;  -j 4: 0.64 s
shard sizes: [14, 15, 11] | disjoint and complete: True
```

18.9.1 Does the regression suite catch the seeded bugs?

The payoff of Steps 5–7: run both suites — the six hand-written tests and the eight tests that came out of the reducer — against every seeded-bug compiler (`TINYC_MUTANT` selects the build).

suite_vs_mutants.py

```
"""suite_vs_mutants.py -- run both regression suites against every seeded-bug compiler."""
import json, os, subprocess
import mutants

def failing(directory, mutant=None):
    env = dict(os.environ, TINYC_MUTANT=mutant) if mutant else dict(os.environ)
    subprocess.run(["python3", "litlite.py", directory, "--json", "r.json"], env=env, capture_output=True)
    return [r["test"] for r in json.load(open("r.json")) if r["status"] not in ("PASS", "XFAIL")]

def short(tests):
    names = [t.replace(".tiny", "") for t in tests]
    more = ", ..." if len(names) > 2 else ""
    return "%d" % len(names) + (" (" + ", ".join(names[:2]) + more + ")" if names else "")

print("%-17s %-30s %s" % ("compiler under test", "hand-written tests failing", "fuzzer-derived tests failing"))
for name in [None] + list(mutants.MUTANTS):
    a, b = failing("tests", name), failing("tests_reduced", name)
    print("%-17s %-30s %s" % (name or "pristine", short(a), short(b)))
```

```
$ python3 suite_vs_mutants.py
```

```
compiler under test hand-written tests failing     fuzzer-derived tests failing
pristine          3 (identity.ll, mul8.ll, ...)  0
fold-floor-div    4 (identity.ll, mul8.ll, ...)  1 (reduced_fold_floor_div)
sub-zero          3 (identity.ll, mul8.ll, ...)  1 (reduced_sub_zero)
cse-commutes-all  3 (identity.ll, mul8.ll, ...)  1 (reduced_cse_commutes_all)
invert-lt         3 (identity.ll, mul8.ll, ...)  2 (reduced_coalesce_multi, reduced_invert_lt)
stale-copy        3 (identity.ll, mul8.ll, ...)  1 (reduced_stale_copy)
dce-trap          3 (identity.ll, mul8.ll, ...)  1 (reduced_dce_trap)
uses-crash        3 (identity.ll, mul8.ll, ...)  1 (reduced_uses_crash)
coalesce-multi    3 (identity.ll, mul8.ll, ...)  1 (reduced_coalesce_multi)
```

The pristine compiler passes both suites (no false alarms). The six hand-written tests, which check typical optimiser patterns, notice *one* of the eight bugs; the tests derived from fuzzing notice all eight (each on its own bug, `invert-lt` also on a neighbour's reproducer). That is the usual shape of a mature suite: humans write the tests they can imagine; fuzzers and reducers add the ones nobody imagined, in their smallest form.

18.10 Step 9: reporting and triage ★★

A nightly run produces thousands of lines. `report.py` parses the `litlite` log with regular expressions, tabulates the statuses and slowest tests, and **groups failures by a normalised signature**: for a crash, the exception text plus the innermost frame; otherwise the first diagnostic line with addresses, file:line locations and numbers masked. Thirty failing tests that share one signature are *one* bug, not thirty. To see it in action we simulate three nightly runs of the 40-test generated suite against three different seeded-bug compilers and digest the concatenated log:

report.py: parse, summarise, group by signature

```
#!/usr/bin/env python3
"""report.py LOGFILE -- digest a litlite log: totals, slowest tests, failures grouped by signature."""
import collections, re, sys

RESULT = re.compile(r"^(PASS|FAIL|XFAIL|XPASS|TIMEOUT|FLAKY): tiny :: (\S+) \((\d+) of (\d+)\) \[([\d.]+)s\]$", re.M)
BLOCK = re.compile(r"^\*{20} TEST 'tiny :: (\S+)' (\w+) \*{20}\n(.*?)\n\*{20}$", re.M | re.S)

def signature(body):
    """Same defect => same key. Crash: exception line + innermost frame.
    Otherwise: the first diagnostic line with addresses, file:line and numbers masked."""
    lines = [l for l in body.splitlines() if l.strip() and not l.startswith("$")]
    exc = re.search(r"^(\w+(?:Error|Exception):.*)$", body, re.M)
    if lines and lines[0].startswith("Traceback") and exc:
        frames = [l.strip() for l in lines if l.lstrip().startswith("File ")]
        return "%s [%s]" % (exc.group(1), re.sub(r'File "(.*?)", line \d+, in ', "in ", frames[-1]))
    first = lines[0] if lines else "(no output)"
    first = re.sub(r"0x[0-9a-f]+", "0xADDR", first)
    first = re.sub(r"[\w./-]+\.(tiny|sh|py|c|cpp):\d+", "FILE:LINE", first)
    return re.sub(r"\d+", "N", first).strip()

def digest(log):
    results = [(s, t, float(sec)) for s, t, _, _, sec in RESULT.findall(log)]
    groups = collections.defaultdict(list)
    for test, status, body in BLOCK.findall(log):
        groups[(status, signature(body))].append(test)
    return results, groups

def markdown(log):
    results, groups = digest(log)
    out = ["| status | tests |", "|---|---|"]
```

```
    for status, n in collections.Counter(s for s, _, _ in results).most_common():
        out.append("| %s | %d |" % (status, n))
    out += ["", "| slowest test | seconds |", "|---|---|"]
    out += ["| %s | %.2f |" % (t, sec) for _, t, sec in sorted(results, key=lambda r: -r[2])[:3]]
    out += ["", "| failure signature | count | tests |", "|---|---|---|"]
    for (status, sig), tests in sorted(groups.items(), key=lambda kv: -len(kv[1])):
        names = ", ".join(sorted(tests)[:3]) + (", ..." if len(tests) > 3 else "")
        out.append("| %s: %s | %d | %s |" % (status, sig, len(tests), names))
    return "\n".join(out)

if __name__ == "__main__":
    print(markdown(open(sys.argv[1]).read()))
```

```
$ for m in uses-crash invert-lt fold-floor-div: TINYC_MUTANT=$m python3 litlite.py tests_gen >> nightly.log; python3 report.py
     nightly.log
```

```
| status | tests |
|---|---|
| PASS | 72 |
| FAIL | 48 |

| slowest test | seconds |
|---|---|
| gen_002.tiny | 0.11 |
| gen_007.tiny | 0.11 |
| gen_012.tiny | 0.10 |

| failure signature | count | tests |
|---|---|---|
| FAIL: IndexError: tuple index out of range [in uses] | 30 | gen_000.tiny, gen_001.tiny, gen_002.tiny, ... |
| FAIL: FILE:LINE: error: CHECK-NEXT: expected string not found in input | 16 | gen_000.tiny, gen_002.tiny, gen_004.tiny, ... |
| FAIL: FILE:LINE: error: CHECK: expected string not found in input | 2 | gen_014.tiny, gen_034.tiny |
```

The first group is the `uses-crash` bug seen through 30 tests; the other two groups are wrong-output failures whose first diagnostic is the failed `CHECK` directive. A real triage system keeps the signature database across nights ("new", "known", "fixed in build N"), links each signature to a bug-tracker entry, and alerts only on *new* signatures.

18.11 From the lab to NVIDIA's problem ★★★

Lab piece	GPU / embedded version	What changes
generator	CUDA kernel generator: one output slot per thread, safe-math integer DAGs, block-uniform barriers, checksum	the validity rules (no races, no divergent barriers) are the hard part
oracle	the same source compiled for the CPU (`cuda_dual.h`, Chapter 15); or another `sm_` target; or the previous release	the "second implementation" is cheap to build and cheap to run
system under test	`nvcc` / Clang with `-O0` … `-O3`, several `sm_` targets, run on real GPUs or simulators	execution needs a device farm: queues, health checks, GPU resets after hangs
mutants	seeded bugs in an open-source compiler (LLVM NVPTX) to evaluate the test methodology	the same experiment, bigger numbers
reducer	`llvm-reduce` on IR, `cvise` on CUDA source; interestingness = "outputs differ across `-O` levels and sanitizer is clean"	each test takes seconds, not milliseconds: reduction needs parallelism and caching
regression tests	lit + FileCheck on PTX (`CHECK: bar.sync`), plus execution tests	the checked artefact is assembly, not program output
infrastructure	shard across GPUs (`CUDA_VISIBLE_DEVICES`), per-test timeouts, flake policy, signature-based triage of `ptxas`/driver errors	scale: millions of tests per night

How to present this project in an interview (60 seconds)

"I built a miniature compiler-verification framework in Python around a toy compiler: a generator of terminating, UB-free programs with generation policies; a differential tester against a reference interpreter; a metamorphic checker; a hierarchical delta-debugging reducer; and a mini lit/FileCheck with timeouts, retries, flake detection and sharding. To evaluate it I seeded eight compiler bugs and measured which method finds which: the naive generator killed 5 of 8, policies brought it to 8 of 8, and metamorphic testing found a different 6 — the two missed ones were common-mode bugs. Every failure was reduced automatically to a one-liner and turned into a regression test, and the mini FileCheck agreed with LLVM's on all 198 cross-checks."

18.12 Exercises ★★

1. Add a ninth mutant (for instance, constant folding of `%` with floor semantics) and find the smallest generator change that kills it; plot kills against the number of programs (a learning curve).
2. Make `filecheck_lite.py` support `CHECK-LABEL` and `CHECK-SAME`; extend `xcheck.py` to cover them.
3. Add a token-level pass to `tinyreduce.py` (try deleting each token of the printed program and re-parsing) and compare the result sizes.
4. Teach `litlite.py` to run the previously failing tests first (test prioritisation) and to write a JUnit-style XML file.
5. Implement real EMI: instrument the interpreter to record which statements were executed, then delete or mutate only the statements that were *not* executed, and check against the compiler.
6. Port the idea to CUDA: write a generator that emits `__global__` kernels in the style of `k15.cu` (Chapter 15), compile them for the CPU with `cuda_dual.h`, and compare against Clang's PTX path on a machine that has a GPU.

18.13 Practice questions ★★★

Q18.1 Tell me about a tool you built to test something. What was the hardest design decision?
Answer: Use the lab: the oracle. A reference interpreter that shares code with the system under test cancels common-mode bugs, so the oracle always comes from the pristine module; second, validity: the generator must produce only programs with defined behaviour, otherwise every mismatch needs manual triage; third, evaluating the tool itself with seeded bugs rather than assuming that "300 passes" means anything.

Q18.2 Your random tester ran a million programs and found nothing. What do you conclude?
Answer: Very little. Measure the tester: mutation score on seeded bugs, coverage of the compiler, distribution of generated constructs. In the lab the naive generator passed 300 programs on three mutants that a policy-driven generator killed. A clean campaign is evidence only about the region the generator reaches.

Q18.3 Why use threads in the test runner but processes in the mutation study?
Answer: The runner's workers mostly wait for child processes, so the GIL does not matter and threads are cheap; the study is pure-Python computation, which threads would serialise, so it uses processes (and passes names, not module objects, to workers because objects created with `exec` cannot be pickled).

Q18.4 How do you shard tests across machines reproducibly?
Answer: Sort the tests, then assign each test to shard `crc32(name) % N` (or by index % N): a function of the name only, so every machine computes the same partition. Python's built-in `hash()` of a string is randomised per process and must not be used.

Q18.5 A test fails once in fifty runs. What does your infrastructure do?
Answer: It must not hide it: the retry is logged and the test reported as FLAKY with its flake rate; the failing run's seed, command line, environment and logs are kept so it can be replayed; a quarantine with owner and expiry date follows if it cannot be fixed immediately; timeouts are never retried; infrastructure failures are classified separately from product failures.

Q18.6 Why does a FileCheck test need a trailing `CHECK-NOT`?
Answer: `CHECK` lines only constrain what appears between and including the matches; extra output after the last match is invisible. For tests of the form "this is the complete output" add `CHECK-NOT: {{.}}` at the end (or use `--match-full-lines` together with an explicit empty check), otherwise a bug that prints extra lines passes.

Part VI

Machine Learning, CS Fundamentals and Aptitude

19. Machine Learning Exposure: Basics, Numerics, ML Compilers

In this chapter

Priority ★★ (★★★ for the basics). Time: 90 min. The placement notice lists "ML exposure" as an assessed skill and the job description lists compilers *and* machine learning as a stand-out combination, so expect (a) conceptual multiple-choice questions (overfitting, bias–variance, precision and recall, gradient descent, regularisation, CNN/RNN/Transformer basics), possibly a short NumPy task, and (b) in the interview, how ML meets compilers: compilers *for* ML (graph optimisation, operator fusion, Triton, TensorRT), ML *in* compilers (learned heuristics) and ML *for testing* compilers. Everything below was executed with NumPy 2.5 and scikit-learn 1.9; where a from-scratch implementation exists it is **cross-checked against scikit-learn** or against a finite-difference gradient, the same verification habit as in Chapters 17 and 18.

19.1 The ML map in one page ★★★

Paradigm	Data and goal	Examples
Supervised	inputs with labels; learn $x \mapsto y$	regression (price), classification (spam, digits)
Unsupervised	inputs only; find structure	clustering (k-means), dimensionality reduction (PCA), density estimation
Self-/semi-supervised	labels derived from the data itself, or few labels	language-model pre-training (predict the next token), contrastive learning
Reinforcement	agent acts, receives rewards; maximise return	games, robotics, and compiler heuristics (MLGO, Section 19.8)

Model	Idea	Strength	Weakness / cost
linear regression	minimise squared error; closed form $\hat{w} = (X^\top X)^{-1} X^\top y$ or gradient descent	fast, interpretable	assumes linearity; outliers; collinearity
logistic regression	linear score $\to$ sigmoid; cross-entropy loss; convex	probabilities, strong baseline	linear boundary
k-NN	vote of the k nearest training points	no training, non-linear	prediction $O(nd)$; scale features; curse of dimensionality
naive Bayes	Bayes rule with independent features	tiny data, very fast	independence assumption
decision tree	recursive splits by impurity (Gini, entropy)	interpretable, mixed types	overfits, unstable
random forest	many trees on bootstrap samples and random feature subsets (*bagging*)	lower variance, robust	less interpretable, memory
gradient boosting (XGBoost, LightGBM)	trees fitted one after another to the previous errors	best on tabular data	tuning; can overfit
SVM	maximum-margin hyperplane, kernels	small n, high d	training $O(n^2 - n^3)$; scaling needed
k-means	Lloyd: assign to nearest centre, recompute centres	simple, fast	k given; spherical clusters; initialisation
PCA	project on directions of maximal variance (SVD of centred data)	compression, visualisation	linear only

Model	Idea	Strength	Weakness / cost
neural networks	layers of affine maps and non-linearities trained by backpropagation (Section 19.6)	universal approximators, representation learning	data, compute, tuning

Workflow and pitfalls. Split the data into *train* (fit parameters), *validation* (choose hyper-parameters and models) and *test* (report once); use *k-fold cross-validation* when data is scarce (stratified for imbalanced classes; time-ordered splits for time series). **Data leakage** is the classic mistake: fitting a scaler or selecting features on the whole data set before splitting, using a feature that encodes the label, shuffling a time series. *Parameters* are learned (weights); *hyper-parameters* are chosen (learning rate, depth, λ, k). Always compare with a trivial baseline (majority class, mean prediction).

19.2 Learning is optimisation ★★★

A model has parameters w; a **loss** measures the error on the training data: mean squared error $\frac{1}{n}\sum(\hat{y}_i - y_i)^2$ for regression, **cross-entropy** $-\frac{1}{n}\sum\left[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\right]$ for classification. **Gradient descent** repeats $w \leftarrow w - \eta \nabla L(w)$ with learning rate η; *stochastic* and *mini-batch* variants estimate the gradient from a few samples (one *epoch* = one pass over the data; *iterations per epoch* = n/batch size); *momentum* and *Adam* adapt the step using past gradients. For a quadratic loss the iteration is stable exactly when $\eta < 2/\lambda_{\max}$, where $\lambda_{\max}$ is the largest eigenvalue of the Hessian: too large a step diverges, too small a step is slow. Feature **standardisation** (mean 0, variance 1) improves the conditioning of the problem, which is why it matters for gradient descent and for distance-based methods (k-NN, k-means, SVM).

```
import numpy as np
from sklearn.linear_model import LinearRegression

rng = np.random.default_rng(0)
n, d = 200, 3
X = rng.normal(size=(n, d))                                  # standardised features: mean 0, variance ~1
w_true, b_true = np.array([2.0, -1.0, 0.5]), 4.0
y = X @ w_true + b_true + rng.normal(scale=0.1, size=n)      # y = Xw + b + noise
A = np.hstack([X, np.ones((n, 1))])                          # absorb the bias as a column of ones

def gradient_descent(lr, steps=500):
    w = np.zeros(A.shape[1])
    for step in range(steps):
        w -= lr * 2 / n * A.T @ (A @ w - y)                  # gradient of the mean squared error
        if not np.all(np.isfinite(w)) or np.abs(w).max() > 1e6: return None, step      # diverged
    return w, steps

closed, *_ = np.linalg.lstsq(A, y, rcond=None)               # exact least-squares solution
sk = LinearRegression().fit(X, y)
print("closed form  :", np.round(closed, 4))
print("scikit-learn :", np.round(np.r_[sk.coef_, sk.intercept_], 4))
for lr in (0.001, 0.01, 0.1, 0.5, 0.99, 1.05):
    w, steps = gradient_descent(lr)
    msg = ("DIVERGED after %d steps" % steps if w is None
           else "max |w - closed form| = %.2e" % np.abs(w - closed).max())
    print("learning rate %-5s -> %s" % (lr, msg))
```

Captured output

```
closed form  : [ 1.991  -1.0096  0.5053  3.9964]
scikit-learn : [ 1.991  -1.0096  0.5053  3.9964]
learning rate 0.001 -> max |w - closed form| = 1.54e+00
learning rate 0.01  -> max |w - closed form| = 3.59e-04
learning rate 0.1   -> max |w - closed form| = 2.44e-15
learning rate 0.5   -> max |w - closed form| = 2.22e-15
learning rate 0.99  -> DIVERGED after 98 steps
learning rate 1.05  -> DIVERGED after 61 steps
```

Gradient descent reaches the closed-form (and scikit-learn) solution to 10^{-15} for $\eta = 0.1$ and 0.5, is still converging after 500 steps for $\eta = 0.01$, has barely started for $\eta = 0.001$, and **diverges for η just below 1**, because the Hessian $\frac{2}{n}A^\top A$ has $\lambda_{\max} \approx 2.1$ here (stability limit ≈ 0.95).

19.2.1 Gradient checking: how to *test* a derivative ★★★

Backpropagation bugs (a wrong sign, a missing $1/n$) do not crash: training just goes wrong. The universal test compares the analytic gradient with a **central finite difference** $\frac{L(w+\varepsilon e_j)-L(w-\varepsilon e_j)}{2\varepsilon}$ for every parameter and reports the *relative* error ($< 10^{-6}$ is healthy for double precision). This is differential testing with a numerical oracle, and it catches both injected bugs below:

```
import numpy as np
rng = np.random.default_rng(1)
X, y = rng.normal(size=(50, 4)), rng.choice([-1.0, 1.0], size=50)
w = rng.normal(size=4)

def loss(w):                                            # logistic loss, labels in {-1, +1}
    return np.mean(np.log1p(np.exp(-y * (X @ w))))
def grad(w):                                            # analytic gradient
    return -(X * (y / (1 + np.exp(y * (X @ w))))[:, None]).mean(axis=0)
def grad_missing_minus(w): return -grad(w)              # injected bug 1: wrong sign
def grad_no_mean(w): return grad(w) * len(y)            # injected bug 2: forgot the 1/n

def numeric(f, w, eps=1e-6):                            # central differences, one coordinate at a time
    g = np.zeros_like(w)
    for j in range(len(w)):
        e = np.zeros_like(w); e[j] = eps
        g[j] = (f(w + e) - f(w - e)) / (2 * eps)
    return g

def rel_error(a, b): return np.max(np.abs(a - b) / np.maximum(1e-12, np.maximum(np.abs(a), np.abs(b))))
num = numeric(loss, w)
for name, g in (("correct gradient", grad), ("wrong sign", grad_missing_minus), ("missing 1/n", grad_no_mean)):
    err = rel_error(g(w), num)
    print("%-17s relative error %.2e  -> %s" % (name, err, "OK" if err < 1e-6 else "BUG"))
```

Captured output

```
correct gradient  relative error 2.48e-09  -> OK
wrong sign        relative error 2.00e+00  -> BUG
missing 1/n       relative error 9.80e-01  -> BUG
```

19.3 Generalisation: overfitting, bias–variance, regularisation ★★★

Training error always falls as the model gets more flexible; *test* error falls and then rises. **High bias** (underfitting): the model is too simple, both errors are high and close. **High variance** (overfitting): the model memorises noise, training error is tiny and test error large. Expected test error $=$ bias^2 $+$ variance $+$ irreducible noise. Remedies for overfitting: more data, simpler model, **regularisation** (L2/ridge adds $\lambda\|w\|_2^2$ and shrinks weights; L1/lasso adds $\lambda\|w\|_1$ and drives some weights to exactly zero, i.e. feature selection), early stopping, dropout, data augmentation, ensembling, cross-validated choice of hyper-parameters.

```
import numpy as np
rng = np.random.default_rng(3)
f = lambda x: np.sin(2 * np.pi * x)                         # the unknown true function
xtr, xte = np.linspace(0, 1, 15), rng.uniform(size=500)     # test points lie inside the training range
ytr, yte = f(xtr) + rng.normal(scale=0.25, size=15), f(xte) + rng.normal(scale=0.25, size=500)

def design(x, degree):                                      # columns 1, x, x^2, ..., x^degree
    return np.vander(x, degree + 1, increasing=True)

def fit(x, y, degree, lam=0.0):                             # ridge regression as an augmented least-squares problem
    A = design(x, degree)
    R = np.sqrt(lam) * np.eye(degree + 1)
    R[0, 0] = 0                                             # do not penalise the intercept
    w, *_ = np.linalg.lstsq(np.vstack([A, R]), np.r_[y, np.zeros(degree + 1)], rcond=None)
    return w

mse = lambda w, x, y, deg: np.mean((design(x, deg) @ w - y) ** 2)

print("polynomial of degree d fitted to 15 noisy points (noise variance = 0.0625):")
print("degree  lambda    train MSE   test MSE")
for deg, lam in [(1, 0), (3, 0), (5, 0), (9, 0), (12, 0), (12, 1e-3), (12, 1e-1)]:
    w = fit(xtr, ytr, deg, lam)
    note = {(1, 0): "  <- underfit (high bias)", (12, 0): "  <- overfit (high variance)"}.get((deg, lam), "")
    print("%5d  %-8s  %9.4f  %9.4f%s" % (deg, lam, mse(w, xtr, ytr, deg), mse(w, xte, yte, deg), note))
```

Captured output

```
polynomial of degree d fitted to 15 noisy points (noise variance = 0.0625):
degree  lambda    train MSE   test MSE
    1  0            0.3084     0.3228  <- underfit (high bias)
    3  0            0.0224     0.1032
    5  0            0.0176     0.1034
    9  0            0.0062     0.1251
   12  0            0.0026     0.1342  <- overfit (high variance)
   12  0.001        0.0213     0.1115
   12  0.1          0.1564     0.2050
```

The test error is lowest for degree 3–5, rises for degrees 9 and 12 while the training error keeps falling, and a small ridge penalty ($\lambda = 10^{-3}$) repairs most of the damage of degree 12, whereas a large one ($\lambda = 0.1$) underfits again. (The noise floor is 0.0625; no model can do better on average.)

19.4 Metrics ★★★

For a binary classifier, the **confusion matrix** counts TP, FP, FN, TN. **Accuracy** $= \frac{TP+TN}{\text{all}}$; **precision** $= \frac{TP}{TP+FP}$ (of what I flagged, how much was right); **recall** (sensitivity, TPR) $= \frac{TP}{TP+FN}$ (of what exists, how much did I find); **F1** $= \frac{2PR}{P+R}$ (harmonic mean); **specificity** $= \frac{TN}{TN+FP}$. The **ROC curve** plots TPR against FPR as the threshold varies; **AUC** is the probability that a random positive scores higher than a random negative (threshold-free, ranking quality). With *heavy class imbalance* prefer precision–recall curves and F1 to accuracy and ROC. Regression: MAE (robust), MSE/RMSE (punishes large errors), $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$.

```python
import numpy as np
from sklearn import metrics

rng = np.random.default_rng(7)
y = (rng.random(1000) < 0.01).astype(int)                    # 1% positives
print("positives:", y.sum(), "of", len(y))
print("always predict 'negative': accuracy = %.3f, recall = %.3f   <- the accuracy paradox" % (1 - y.mean(), 0.0))

score = np.where(y == 1, rng.normal(1.5, 1, 1000), rng.normal(0, 1, 1000))     # a mediocre classifier's scores
pred = (score > 1.0).astype(int)

def confusion(y, p):                                         # TP, FP, FN, TN
    return (int(((y == 1) & (p == 1)).sum()), int(((y == 0) & (p == 1)).sum()),
            int(((y == 1) & (p == 0)).sum()), int(((y == 0) & (p == 0)).sum()))

def auc(y, s):                                               # rank form (Mann-Whitney U): P(score_pos > score_neg)
    order = np.argsort(s)
    ranks = np.empty(len(s))
    ranks[order] = np.arange(1, len(s) + 1)
    n1, n0 = y.sum(), (1 - y).sum()
    return (ranks[y == 1].sum() - n1 * (n1 + 1) / 2) / (n1 * n0)

tp, fp, fn, tn = confusion(y, pred)
precision, recall = tp / (tp + fp), tp / (tp + fn)
f1 = 2 * precision * recall / (precision + recall)
print("TP=%d FP=%d FN=%d TN=%d  precision=%.3f recall=%.3f F1=%.3f accuracy=%.3f"
      % (tp, fp, fn, tn, precision, recall, f1, (tp + tn) / len(y)))
print("ROC AUC (from scratch) = %.4f" % auc(y, score))
ok = (np.isclose(precision, metrics.precision_score(y, pred)) and np.isclose(recall, metrics.recall_score(y, pred))
      and np.isclose(f1, metrics.f1_score(y, pred)) and np.isclose(auc(y, score), metrics.roc_auc_score(y, score)))
print("all four agree with scikit-learn:", ok)
```

Captured output

```
positives: 8 of 1000
always predict 'negative': accuracy = 0.992, recall = 0.000   <- the accuracy paradox
TP=7 FP=159 FN=1 TN=833  precision=0.042 recall=0.875 F1=0.080 accuracy=0.840
ROC AUC (from scratch) = 0.9205
all four agree with scikit-learn: True
```

Predicting "negative" for everything is 99% accurate and useless (recall 0): the **accuracy paradox**. The mediocre classifier has a high AUC but a terrible precision at the chosen threshold because positives are rare: 159 false alarms for 7 true hits. All four hand-written metrics agree with scikit-learn.

19.5 Two classic unsupervised algorithms, from scratch ★★

k-means alternates *assignment* (each point to the nearest centre) and *update* (each centre becomes the mean of its points); it converges to a local minimum of the within-cluster sum of squares (*inertia*) that depends on the initialisation (hence k-means++ and several restarts), and the number of clusters k is chosen by the elbow

method or the silhouette score. **PCA** centres the data and takes the SVD $X_c = U\Sigma V^\top$: the rows of $V^\top$ are the principal directions, $\sigma_i^2/(n-1)$ the variances; keeping the first k directions is the best rank-k approximation in the least-squares sense.

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

rng = np.random.default_rng(5)
centers = np.array([[0, 0], [6, 1], [3, 6]])
X = np.vstack([c + rng.normal(scale=0.8, size=(100, 2)) for c in centers])     # three Gaussian blobs

def kmeans(X, init, iters=100):                                 # Lloyd's algorithm
    C = init.copy()
    for _ in range(iters):
        labels = np.argmin(((X[:, None, :] - C[None]) ** 2).sum(-1), axis=1)   # assignment step
        new = np.array([X[labels == k].mean(axis=0) for k in range(len(C))])    # update step
        if np.allclose(new, C): break
        C = new
    return C, labels, ((X - C[labels]) ** 2).sum()

init = X[[0, 100, 200]]
C, labels, inertia = kmeans(X, init)
ref = KMeans(n_clusters=3, init=init, n_init=1, algorithm="lloyd").fit(X)
print("k-means : inertia %.4f  (scikit-learn %.4f)  same labels: %s"
      % (inertia, ref.inertia_, np.array_equal(labels, ref.labels_)))

Z = rng.normal(size=(300, 5)) @ rng.normal(size=(5, 5))       # correlated 5-D data
Zc = Z - Z.mean(axis=0)                                       # PCA: centre, then SVD
U, S, Vt = np.linalg.svd(Zc, full_matrices=False)
ratio = S ** 2 / (S ** 2).sum()
sk = PCA(n_components=5).fit(Z)
print("PCA     : explained variance ratio", np.round(ratio, 4))
print("          agrees with scikit-learn:", np.allclose(ratio, sk.explained_variance_ratio_),
      "| components agree up to sign:", np.allclose(np.abs(Vt), np.abs(sk.components_)))
k = 2
approx = Zc @ Vt[:k].T @ Vt[:k]                               # project onto the first k components and back
print("          keeping %d of 5 components: %.1f%% of the variance, reconstruction MSE %.4f"
      % (k, 100 * ratio[:k].sum(), np.mean((Zc - approx) ** 2)))
```

Captured output

```
k-means : inertia 365.6482  (scikit-learn 365.6482)  same labels: True
PCA     : explained variance ratio [5.701e-01 3.026e-01 1.209e-01 6.000e-03 3.000e-04]
          agrees with scikit-learn: True | components agree up to sign: True
          keeping 2 of 5 components: 87.3% of the variance, reconstruction MSE 0.7862
```

19.6 Neural networks in 40 lines ★★★

A layer computes $h = \sigma(Wx+b)$; a network stacks layers; training minimises a loss by **backpropagation**, which is simply the chain rule applied layer by layer from the output backwards, reusing the intermediate activations of the forward pass. Softmax followed by cross-entropy has the beautifully simple gradient $\hat{p} - y$; $\tanh'(z) = 1 - \tanh^2 z$; $\sigma'(z) = \sigma(z)(1-\sigma(z)) \le 0.25$ (the reason deep sigmoid networks suffer **vanishing gradients**; ReLU has gradient 1 for positive inputs, is cheap and sparse, but can "die" for negative ones). The network below learns XOR, which no linear model can, and its gradients are verified against finite differences for all 17 parameters:

```
import numpy as np
rng = np.random.default_rng(2)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], float)
y = np.array([[0], [1], [1], [0]], float)                     # XOR: not linearly separable

H = 4                                                         # 2 -> 4 (tanh) -> 1 (sigmoid)
params = {"W1": rng.normal(size=(2, H)), "b1": np.zeros(H), "W2": rng.normal(size=(H, 1)), "b2": np.zeros(1)}
sigmoid = lambda z: 1 / (1 + np.exp(-z))

def forward(p, X):
    h = np.tanh(X @ p["W1"] + p["b1"])
    return h, sigmoid(h @ p["W2"] + p["b2"])

def loss(p):                                                  # binary cross-entropy
    _, out = forward(p, X)
    return -np.mean(y * np.log(out) + (1 - y) * np.log(1 - out))

def backward(p):                                              # backpropagation: the chain rule, layer by layer
    h, out = forward(p, X)
    d_out = (out - y) / len(X)                                # sigmoid + cross-entropy simplify to (out - y)
    d_h = (d_out @ p["W2"].T) * (1 - h ** 2)                  # tanh' = 1 - tanh^2
    return {"W2": h.T @ d_out, "b2": d_out.sum(0), "W1": X.T @ d_h, "b1": d_h.sum(0)}
```

```
def check(p, eps=1e-6):                                    # gradient check of EVERY parameter
    worst, g = 0.0, backward(p)
    for name, arr in p.items():
        for idx in np.ndindex(arr.shape):
            old = arr[idx]
            arr[idx] = old + eps; up = loss(p)
            arr[idx] = old - eps; down = loss(p)
            arr[idx] = old
            num = (up - down) / (2 * eps)
            worst = max(worst, abs(g[name][idx] - num) / max(1e-12, abs(g[name][idx]), abs(num)))
    return worst

print("parameters:", sum(a.size for a in params.values()), "= 2*4 + 4 + 4*1 + 1")
print("gradient check before training: worst relative error %.1e" % check(params))
for epoch in range(1, 2001):
    g = backward(params)
    for name in params: params[name] -= 0.5 * g[name]          # plain gradient descent, learning rate 0.5
    if epoch in (1, 100, 500, 2000): print("epoch %4d  loss %.4f" % (epoch, loss(params)))
probs = forward(params, X)[1].ravel()
print("predictions:", np.round(probs, 3), "-> classes", (probs > 0.5).astype(int))
```

Captured output

```
parameters: 17 = 2*4 + 4 + 4*1 + 1
gradient check before training: worst relative error 3.1e-08
epoch    1  loss 0.7098
epoch  100  loss 0.0862
epoch  500  loss 0.0083
epoch 2000  loss 0.0015
predictions: [0.001 0.998 0.998 0.002] -> classes [0 1 1 0]
```

Idea	Facts to remember
initialisation	never all zeros (symmetry); Xavier/Glorot for tanh, He for ReLU keep activation variance stable across layers
normalisation	BatchNorm normalises over the batch (needs a training/inference distinction); LayerNorm over features (Transformers)
regularisation	dropout randomly zeroes activations in training (scale at inference); weight decay = L2; data augmentation; early stopping
optimisers	SGD + momentum; Adam keeps running averages of the gradient and its square (adaptive per-parameter steps)
CNN	convolution = local, weight-shared filters; parameters $(k_h k_w C_{in} + 1)C_{out}$; output side $\lfloor (H + 2p - k)/s \rfloor + 1$; pooling downsamples; translation equivariance
RNN / LSTM	hidden state carries history; trained by back-propagation through time; vanishing/exploding gradients; LSTM gates ease them; sequential, hard to parallelise
Transformer	attention $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$: every token attends to every other; cost $O(n^2 d)$ in the sequence length n; fully parallel in training; about $12d^2$ parameters per block

The shape and parameter formulas are the kind of thing multiple-choice papers like, so check them by construction rather than by memory:

```
import numpy as np

def conv2d(x, w, stride, pad):                             # x: (C_in, H, W); w: (C_out, C_in, kh, kw)
    c_out, c_in, kh, kw = w.shape
    xp = np.pad(x, ((0, 0), (pad, pad), (pad, pad)))
    ho, wo = (xp.shape[1] - kh) // stride + 1, (xp.shape[2] - kw) // stride + 1
    y = np.zeros((c_out, ho, wo))
    for co in range(c_out):
        for i in range(ho):
            for j in range(wo):
                patch = xp[:, i * stride:i * stride + kh, j * stride:j * stride + kw]
                y[co, i, j] = (patch * w[co]).sum()
    return y

c_in, c_out, H, k, s, p = 3, 8, 32, 3, 2, 1
rng = np.random.default_rng(0)
x, w = rng.normal(size=(c_in, H, H)), rng.normal(size=(c_out, c_in, k, k))
y = conv2d(x, w, s, p)
```

```
ho = (H + 2 * p - k) // s + 1
macs = c_out * y.shape[1] * y.shape[2] * c_in * k * k
print("conv: output shape %s, formula floor((H+2p-k)/s)+1 = %d" % (y.shape, ho))
print("      parameters %d, formula (k*k*C_in + 1)*C_out = %d" % (w.size + c_out, (k * k * c_in + 1) * c_out))
print("      multiply-accumulates %d, formula H_out*W_out*C_out*k*k*C_in = %d" % (macs, ho * ho * c_out * k * k * c_in))

d = 64                                                  # one transformer block: attention + MLP + two LayerNorms
block = {"Wq": (d, d), "Wk": (d, d), "Wv": (d, d), "Wo": (d, d), "bq": (d,), "bk": (d,), "bv": (d,), "bo": (d,),
         "W1": (d, 4 * d), "b1": (4 * d,), "W2": (4 * d, d), "b2": (d,), "ln1": (2 * d,), "ln2": (2 * d,)}
exact = sum(int(np.prod(sh)) for sh in block.values())
print("transformer block, d=%d: parameters %d, rule of thumb 12*d^2 = %d (%.1f%% apart)"
      % (d, exact, 12 * d * d, 100 * (exact - 12 * d * d) / exact))
for n in (512, 2048, 8192):
    print("      sequence length %5d: attention score matrix has %d entries per head (grows as n^2)" % (n, n * n))
```

Captured output

```
conv: output shape (8, 16, 16), formula floor((H+2p-k)/s)+1 = 16
      parameters 224, formula (k*k*C_in + 1)*C_out = 224
      multiply-accumulates 55296, formula H_out*W_out*C_out*k*k*C_in = 55296
transformer block, d=64: parameters 49984, rule of thumb 12*d^2 = 49152 (1.7% apart)
      sequence length   512: attention score matrix has 262144 entries per head (grows as n^2)
      sequence length  2048: attention score matrix has 4194304 entries per head (grows as n^2)
      sequence length  8192: attention score matrix has 67108864 entries per head (grows as n^2)
```

19.6.1 Numerical drills (with answers computed)

```
import numpy as np
tp, fp, fn, tn = 40, 10, 20, 30
p, r = tp / (tp + fp), tp / (tp + fn)
print("1. confusion TP=40 FP=10 FN=20 TN=30: accuracy %.3f  precision %.3f  recall %.3f  F1 %.3f  specificity %.3f"
      % ((tp + tn) / 100, p, r, 2 * p * r / (p + r), tn / (tn + fp)))
print("2. dense 784 -> 128 -> 10: %d parameters" % (784 * 128 + 128 + 128 * 10 + 10))
print("3. conv 3x3, 16 -> 32 channels: %d parameters" % ((3 * 3 * 16 + 1) * 32))
print("4. conv on 224x224 input, k=7, stride 2, padding 3: output side %d" % ((224 + 2 * 3 - 7) // 2 + 1))
z = np.array([2.0, 1.0, 0.0])
sm = np.exp(z - z.max()) / np.exp(z - z.max()).sum()
print("5. softmax([2,1,0]) = %s;  cross-entropy for true class 0: %.3f" % (np.round(sm, 3), -np.log(sm[0])))
posterior = 0.5 * 0.2 / (0.5 * 0.2 + 0.05 * 0.8)
print("6. Bayes: P(spam)=0.2, P(w|spam)=0.5, P(w|ham)=0.05 -> P(spam|w) = %.3f" % posterior)
print("7. one gradient step: w=3, loss=(w-1)^2, lr=0.25 -> w = %.2f" % (3 - 0.25 * 2 * (3 - 1)))
print("8. 1000 samples, batch size 50, 3 epochs -> %d updates per epoch, %d in total" % (1000 // 50, 3 * 1000 // 50))
```

Captured output

```
1. confusion TP=40 FP=10 FN=20 TN=30: accuracy 0.700  precision 0.800  recall 0.667  F1 0.727  specificity 0.750
2. dense 784 -> 128 -> 10: 101770 parameters
3. conv 3x3, 16 -> 32 channels: 4640 parameters
4. conv on 224x224 input, k=7, stride 2, padding 3: output side 112
5. softmax([2,1,0]) = [0.665 0.245 0.09 ];  cross-entropy for true class 0: 0.408
6. Bayes: P(spam)=0.2, P(w|spam)=0.5, P(w|ham)=0.05 -> P(spam|w) = 0.714
7. one gradient step: w=3, loss=(w-1)^2, lr=0.25 -> w = 2.00
8. 1000 samples, batch size 50, 3 epochs -> 20 updates per epoch, 60 in total
```

19.7 Numerics every GPU and compiler person must know ★★★

Training and inference run in reduced precision for speed and memory, and compilers reorder, fuse and contract floating-point operations. The consequences are testable facts: formats have very different ranges (**bfloat16** keeps float32's exponent range with 7 mantissa bits; **float16** has more precision but overflows at 65 504; **FP8** comes in an E4M3 and an E5M2 flavour); a naive softmax overflows; the *order* of a summation changes the result by orders of magnitude; one outlier ruins a per-tensor quantisation scale. Mixed precision keeps FP16/BF16/FP8 inputs but *accumulates in FP32* (what tensor cores do).

```
import numpy as np
print("floating-point formats (binary16/32/64 from numpy; bfloat16 and FP8 from their specifications)")
print("%-10s %5s %5s %12s %12s %12s" % ("format", "exp", "mant", "epsilon", "max", "min normal"))
for name, dt in (("float16", np.float16), ("float32", np.float32), ("float64", np.float64)):
    i = np.finfo(dt)
    print("%-10s %5d %5d %12.3e %12.3e %12.3e" % (name, i.nexp, i.nmant, float(i.eps), float(i.max), float(i.tiny)))
print("%-10s %5d %5d %12.3e %12.3e %12.3e" % ("bfloat16", 8, 7, 2.0 ** -7, (2 - 2.0 ** -7) * 2.0 ** 127, 2.0 ** -126))
print("FP8 E4M3: max 448 (4 exponent, 3 mantissa bits); FP8 E5M2: max 57344 (5, 2)  [OCP/NVIDIA FP8 specification]")

x = np.array([1000.0, 1001.0, 1002.0], np.float32)
with np.errstate(all="ignore"):
    naive = np.exp(x) / np.exp(x).sum()
stable = np.exp(x - x.max()) / np.exp(x - x.max()).sum()
```

```python
print("\nsoftmax of [1000, 1001, 1002] in float32: naive =", naive, " stable =", np.round(stable, 4))
with np.errstate(all="ignore"):
    print("exp(12) in float16 =", np.exp(np.float16(12)), "(true value 162754.8; float16 max is 65504)")

a = np.full(10_000_000, 0.1, np.float32)                    # exact sum of ten million 0.1s: 1,000,000.0149...
seq = np.cumsum(a)[-1]                                      # left-to-right accumulation in float32
pairwise = a.sum()                                          # numpy's pairwise (tree-like) summation
exact = a.astype(np.float64).sum()
print("\nsum of 10^7 x 0.1 in float32: sequential = %.1f, pairwise = %.1f, float64 = %.1f" % (seq, pairwise, exact))
print("relative error: sequential %.1e, pairwise %.1e   <- the reduction order changes the answer"
      % (abs(seq - 1e6) / 1e6, abs(pairwise - 1e6) / 1e6))

rng = np.random.default_rng(0)
W = rng.normal(size=(64, 64))
W[0, 0] = 40.0                                              # one outlier ruins a single per-tensor scale

def quant_error(W, scale):                                  # int8 symmetric quantisation round trip
    q = np.clip(np.round(W / scale), -127, 127)
    return np.abs(q * scale - W).mean()

per_tensor = quant_error(W, np.abs(W).max() / 127)
per_row = np.mean([quant_error(W[i:i + 1], np.abs(W[i]).max() / 127) for i in range(64)])
print("\nint8 quantisation, mean abs error: per-tensor scale %.4f, per-row scale %.4f" % (per_tensor, per_row))
```

Captured output

```
floating-point formats (binary16/32/64 from numpy; bfloat16 and FP8 from their specifications)
format       exp  mant      epsilon          max    min normal
float16        5    10    9.766e-04    6.550e+04    6.104e-05
float32        8    23    1.192e-07    3.403e+38    1.175e-38
float64       11    52    2.220e-16   1.798e+308   2.225e-308
bfloat16       8     7    7.812e-03    3.390e+38    1.175e-38
FP8 E4M3: max 448 (4 exponent, 3 mantissa bits); FP8 E5M2: max 57344 (5, 2)  [OCP/NVIDIA FP8 specification]

softmax of [1000, 1001, 1002] in float32: naive = [nan nan nan]  stable = [0.09   0.2447 0.6652]
exp(12) in float16 = inf (true value 162754.8; float16 max is 65504)

sum of 10^7 x 0.1 in float32: sequential = 1087937.0, pairwise = 1000000.1, float64 = 1000000.0
relative error: sequential 8.8e-02, pairwise 1.2e-07   <- the reduction order changes the answer

int8 quantisation, mean abs error: per-tensor scale 0.0783, per-row scale 0.0060
```

This is the numerical background of Chapter 15's warning about reductions: a GPU tree reduction and a CPU loop *legitimately* differ, so a test must compare with a tolerance that scales with the problem. NumPy's `allclose(a, b, rtol, atol)` tests $|a-b| \le \text{atol} + \text{rtol}\,|b|$ element-wise, and is *not symmetric* (`b` is the reference): `allclose(1000, 1001.0005, rtol=1e-3, atol=0)` is true, the call with the arguments swapped is false. Choose `rtol` from the precision (about 10^{-6} for float32 element-wise operations, much looser after long reductions or reduced-precision inputs), and remember that `NaN != NaN`.

19.8 Compilers *for* machine learning ★★★

Deep-learning frameworks turn a model into a **graph of tensor operations**; ML compilers optimise the graph and generate kernels for the target (GPU, CPU, accelerator). The recurring ideas are the same as in Chapters 11 and 12 at a different level:

Technique	What it does and why it pays
operator fusion	merge element-wise ops (and their epilogues) into one kernel so intermediates stay in registers: memory traffic, not arithmetic, is the bottleneck for these ops
constant folding, CSE, DCE	the classical clean-ups on the graph (batch-norm folding into the preceding convolution is the standard example)
layout and dtype selection	NCHW vs NHWC, tensor-core friendly padding, FP16/BF16/FP8/INT8 with quantise/dequantise insertion
memory planning	reuse buffers of tensors with disjoint lifetimes
autotuning	try tile sizes, unrolling, vector widths; *learned cost models* (TVM/Ansor-style) predict which candidates are worth running
lowering	graph IR → loop/tile IR → LLVM IR → PTX → SASS, or calls into libraries (cuBLAS, cuDNN, CUTLASS)

The stack, by name. *ONNX*: interchange format. *XLA/StableHLO* and *MLIR*: graph and multi-level IRs (JAX and TensorFlow). *TVM*: graph IR plus tensor-level IR with autotuning. *TensorRT*: NVIDIA's inference optimiser (fusion, precision selection, kernel auto-selection). *PyTorch 2 / TorchInductor*: captures Python-level models with `torch.compile`, fuses element-wise operations and generates **Triton** kernels for GPUs (calling cuBLAS/CUTLASS-style libraries for matrix multiplication). **Triton** is a Python-embedded language for writing GPU kernels at the tile level; its compiler lowers the Python AST to Triton IR, to TritonGPU IR (adding layout encodings that say how tensors are spread over threads and warps), applies passes such as memory coalescing and software pipelining, then to LLVM IR and **PTX** (Chapters 12 and 14): the NVPTX back end you studied is the last compiler stage of most modern ML kernels.

19.8.1 A toy fusion pass, verified by random differential testing

The script below defines a tiny element-wise tensor IR, a reference executor (one "kernel" per operation, every intermediate through memory), a **compiler pass** (dead-code elimination, then fusion of the whole DAG into one tiled loop) and a test harness in the style of NNSmith: 300 random valid graphs, random sizes around the tile boundary, bit-exact comparison of fused and unfused results, and a count of how many random graphs are *useless* because their output is `inf`/`NaN`.

```
import numpy as np

# A tiny tensor-program IR: (op, output, inputs...) over named 1-D float32 tensors; every op is elementwise.
FUNCS = {"add": np.add, "sub": np.subtract, "mul": np.multiply, "relu": lambda a: np.maximum(a, 0),
         "tanh": np.tanh, "exp": np.exp}

def run_unfused(graph, env):
    """Reference semantics: one 'kernel' per op, every intermediate goes through memory."""
    env, traffic = dict(env), 0
    for op, out, *args in graph:
        env[out] = FUNCS[op](*[env[a] for a in args])
        traffic += (len(args) + 1) * env[out].size             # elements read + elements written
    return env, traffic

def fuse(graph, output):
    """The compiler pass: drop dead ops, then fuse the whole elementwise DAG into ONE loop over tiles."""
    needed, live = {output}, []
    for op, out, *args in reversed(graph):                     # dead-code elimination, last op to first
        if out in needed: live.append((op, out, *args)); needed.update(args)
    live.reverse()
    produced = {out for _, out, *_ in live}
    inputs = sorted({a for _, _, *args in live for a in args} - produced)
    return {"ops": live, "inputs": inputs, "output": output}

def run_fused(kernel, env, tile=1024):
    n = env[kernel["inputs"][0]].size
    out, traffic = np.empty(n, np.float32), 0
    for lo in range(0, n, tile):                               # intermediates live only in tile-sized temporaries
        local = {a: env[a][lo:lo + tile] for a in kernel["inputs"]}
        for op, name, *args in kernel["ops"]:
            local[name] = FUNCS[op](*[local[a] for a in args])
        out[lo:lo + tile] = local[kernel["output"]]
    traffic = (len(kernel["inputs"]) + 1) * n                  # read each input once, write the output once
    return out, traffic

def random_graph(rng, n_ops, safe):
    names, graph = ["x", "w", "b"], []
    for i in range(n_ops):
        op = rng.choice(list(FUNCS))
        args = [rng.choice(names) for _ in range(1 if op in ("relu", "tanh", "exp") else 2)]
        if op == "exp" and safe:                               # tanh first: exp's argument stays in [-1, 1], so no inf/NaN
            graph.append(("tanh", "s%d" % i, args[0]))
            args = ["s%d" % i]
        graph.append((op, "t%d" % i, *args)); names.append("t%d" % i)
    return graph, names[-1]

rng = np.random.default_rng(0)
for safe in (False, True):
    ok = bad = nonfinite = 0
    ratios = []
    for trial in range(300):
        graph, out = random_graph(rng, int(rng.integers(3, 12)), safe)
        n = int(rng.choice([1, 7, 1000, 1025, 5000]))          # sizes around the tile boundary
        env = {k: rng.normal(scale=3.0, size=n).astype(np.float32) for k in ("x", "w", "b")}
        with np.errstate(all="ignore"):
            ref_env, t_ref = run_unfused(graph, env)
            got, t_fused = run_fused(fuse(graph, out), env)
        same = np.array_equal(ref_env[out], got, equal_nan=True)  # elementwise ops, same order: must be bit-identical
        ok += same; bad += not same
        nonfinite += not np.all(np.isfinite(ref_env[out]))
```

```
        ratios.append(t_ref / t_fused)
    print("random graphs, exp %s: fused == unfused (bitwise) in %d of 300; outputs with inf/NaN: %d; "
          "memory traffic reduced %.1fx on average"
          % ("guarded by tanh" if safe else "unguarded", ok, nonfinite, np.mean(ratios)))

chain = [("mul", "t1", "x", "w"), ("add", "t2", "t1", "b"), ("relu", "t3", "t2"),
         ("exp", "t4", "t3"), ("add", "y", "t4", "x")]
env = {k: rng.normal(size=5000).astype(np.float32) for k in ("x", "w", "b")}
_, t1 = run_unfused(chain, env)
_, t2 = run_fused(fuse(chain, "y"), env)
print("y = exp(relu(x*w + b)) + x, 5000 elements: unfused moves %d elements, fused %d (%.2fx less)" % (t1, t2, t1 / t2))
```

Captured output

```
random graphs, exp unguarded: fused == unfused (bitwise) in 300 of 300; outputs with inf/NaN: 5; memory traffic reduced 7.0x
     on average
random graphs, exp guarded by tanh: fused == unfused (bitwise) in 300 of 300; outputs with inf/NaN: 0; memory traffic reduced
     8.0x on average
y = exp(relu(x*w + b)) + x, 5000 elements: unfused moves 65000 elements, fused 20000 (3.25x less)
```

Three lessons. **(1) Exactness:** fusion of element-wise operations changes *where* values live, not the arithmetic, so the results are bit-identical; if the pass also contracted `a*b+c` into a fused multiply-add, results would differ in the last bit and the test would need a tolerance (Chapter 14). **(2) Valid inputs matter:** when `exp` is unguarded, some random graphs overflow to `inf`/`NaN`, and `NaN == NaN` comparisons hide real differences; NNSmith searches for inputs (by gradient) that keep all intermediates finite, here we simply guard `exp` with `tanh`. **(3) Performance is a property to test:** the fused version moves 3–8× fewer elements; a performance test would assert a bound on that ratio.

19.9 Machine learning *in* and *for* compilers ★★

- **Learned heuristics inside a compiler. MLGO** (Trofin et al., Google; arXiv 2021) replaced LLVM's hand-written inlining-for-size heuristic with a neural policy trained by reinforcement learning (policy gradient, evolution strategies): up to 7% smaller code than `-Oz` on the paper's benchmarks; a second policy guides the register allocator's *eviction* decisions for performance; both are in the LLVM repository and used in production. Learned **cost models** predict kernel run times in autotuners. Verification consequence: a wrong prediction is a *performance* bug, never a correctness bug, *provided* the learned component only chooses among semantics-preserving options.
- **Learning to generate test programs. DeepSmith** (Cummins et al., ISSTA 2018) trains a language model on open-source OpenCL code, generates realistic programs and applies differential testing: in about 1 000 hours of testing it found bugs in every OpenCL compiler tested (67 bug reports). **NNSmith** (Liu et al., ASPLOS 2023) generates valid, diverse deep-learning models from operator specifications, searches for NaN-free inputs, and compares compilers/runtimes differentially: 72 new bugs in TVM, TensorRT, ONNX Runtime and PyTorch in its latest version.
- **Testing ML compilers and kernels.** Oracles: the framework's eager mode, a higher-precision run, another backend. Hazards: tolerances, non-determinism (atomics, reduction order), dynamic shapes, NaN/Inf propagation, dtype promotion rules, and kernels that are correct only for shapes that are multiples of the tile size (test the boundaries, as in Chapter 15).

Sources for this section

MLGO: a Machine Learning Guided Compiler Optimizations Framework (Trofin et al.); NNSmith: Generating Diverse and Valid Test Cases for Deep Learning Compilers; *Compiler Fuzzing through Deep Learning* (Cummins et al., ISSTA 2018); PyTorch's "Triton kernel compilation stages" blog post and the `torch.compile` documentation for the pipeline description. Figures are the papers' own headline numbers as retrieved while preparing this manual; check the papers before quoting them to an interviewer.

19.10 Practice questions ★★★

Q19.1 Explain the bias–variance trade-off and how you would diagnose and fix overfitting.
Answer: Total error = bias^2 + variance + noise. Underfitting: both training and validation error high (add capacity, features, train longer); overfitting: training error low, validation error high (more data, simpler model, regularisation, early stopping, dropout, augmentation, ensembling). Diagnose with learning curves and cross-validation.

Q19.2 A fraud detector is 99.5% accurate on a data set with 0.5% fraud. Is it good?
Answer: Not necessarily: predicting "no fraud" always gives 99.5%. Look at recall and precision of the fraud class, the PR curve and a cost-weighted metric; consider class weights or resampling.

Q19.3 Precision versus recall: when do you prefer which?
Answer: Prefer recall when a miss is costly (cancer screening, security alerts), precision when a false alarm is costly (spam filter deleting real mail, arrests). F1 balances them; the threshold moves you along the trade-off.

Q19.4 Why standardise features? Which algorithms need it?
Answer: Gradient descent converges faster with well-conditioned (similarly scaled) features; distance-based methods (k-NN, k-means, SVM with RBF kernel), regularised models and PCA are dominated by large-scale features otherwise. Trees do not need it. Fit the scaler on the training split only (leakage).

Q19.5 L1 versus L2 regularisation?
Answer: L2 (ridge) penalises squared weights, shrinks all weights smoothly, has a closed form for linear models; L1 (lasso) penalises absolute values, produces sparse solutions (exact zeros), so it performs feature selection. Elastic net mixes both.

Q19.6 Why ReLU instead of sigmoid in deep networks?
Answer: Sigmoid saturates (derivative ≤ 0.25, near 0 for large $|z|$), so gradients vanish through many layers and outputs are not zero-centred; ReLU has gradient 1 for positive inputs, is cheap, and gives sparse activations. Drawbacks: dead units (use Leaky ReLU/GELU).

Q19.7 Difference between k-NN and k-means?
Answer: k-NN is a supervised, lazy classifier/regressor (no training, $O(nd)$ per prediction); k-means is an unsupervised clustering algorithm that iteratively refines k centres. The "k" means different things (neighbours vs. clusters).

Q19.8 What does backpropagation compute, and how would you test your implementation?
Answer: The gradient of the loss with respect to every parameter by the chain rule, in one backward pass that reuses forward activations. Test with gradient checking: compare to central finite differences ($\varepsilon \approx 10^{-6}$ in double precision), relative error $< 10^{-6}$; also check that a tiny network can overfit a tiny data set.

Q19.9 Why do Transformers replace RNNs for long sequences, and what is the price?
Answer: Attention connects any two tokens directly (short gradient paths, long-range dependencies) and is parallel over positions, which suits GPUs; RNNs process tokens sequentially. The price is $O(n^2)$ time and memory in the sequence length (hence FlashAttention-style fused kernels, sparse and linear variants).

Q19.10 Why can the same model give slightly different outputs on a CPU and a GPU, and how should a test treat it?
Answer: Different operation orders (parallel reductions), fused multiply-add, different math-library implementations and reduced precision; floating-point addition is not associative. Compare with `rtol`/`atol` appropriate to the dtype and reduction length, use a higher-precision reference, and require bit-exactness only where IEEE-754 demands it.

Q19.11 What is operator fusion and why does it speed up inference?
Answer: Combining adjacent operations into one kernel so intermediates stay in registers or shared memory instead of travelling through global memory. Element-wise operations are memory-bound (Chapter 15's roofline), so removing round trips gives close to a proportional speed-up (in the toy example above: 3–8× less traffic).

Q19.12 How could machine learning help a compiler, and what must you verify?
Answer: Replace hand-tuned heuristics (inlining, register-allocation eviction, tile sizes, instruction scheduling) by learned policies or cost models. Because the learned component only *chooses* among semantics-preserving transformations, correctness is unaffected, but you must test for performance regressions across workloads, robustness to distribution shift (new code bases, new architectures), determinism of builds, and fall-back when the model is unavailable.

Q19.13 How would you test a deep-learning compiler?
Answer: Generate random valid models (operator specifications, shape and dtype constraints), choose inputs that avoid NaN/Inf, run the compiled and the eager/reference execution (and different backends or optimisation levels), compare with dtype-appropriate tolerances, reduce failing graphs automatically, and add them to a regression suite; separate performance tests with noise-aware thresholds.

20. CS Fundamentals in a Hurry: Numbers, Logic, Architecture, OS

In this chapter

Priority ★★ (the boxed "must know" lists are ★★★). Time: 60 min for everything, 20 min for the boxes and drills only. Placement papers for this role mix C/C++, Python, ML and compilers with a thin layer of computer-science fundamentals (operating systems and architecture appear in roughly one question in twenty according to the candidate reports of Chapter 2); for B.Tech. students of electrical, electronics and IT branches, digital logic and architecture are fair game. This chapter is therefore a *reference with executed drills*: each topic has the facts to remember, and wherever a fact is a computation (a speed-up, a cache miss rate, a scheduling average, a page-fault count) the number comes from a program that was run, not from memory. The C-specific material (bit tricks, endianness, struct padding, calling conventions) is in Chapter 4 and is not repeated.

20.1 Number representation and IEEE-754 ★★★

Must know

- **Two's complement, n bits:** range $-2^{n-1} \dots 2^{n-1}-1$; negate = invert all bits and add 1; the sign bit has weight -2^{n-1}; **sign extension** copies the sign bit; addition overflows when the carry *into* the top bit differs from the carry *out of* it (and signed overflow is undefined behaviour in C/C++, Chapter 4). Unsigned arithmetic wraps modulo 2^n.
- **IEEE-754 binary32:** 1 sign bit, 8 exponent bits (bias 127), 23 fraction bits; *normal* value $(-1)^s \times 1.m \times 2^{e-127}$ for $1 \le e \le 254$; **subnormal** ($e=0$, $m \neq 0$): $0.m \times 2^{-126}$, gradual underflow; $e=0, m=0$: ± 0; $e=255, m=0$: $\pm\infty$; $e=255, m \neq 0$: **NaN**. binary64: 11 exponent bits (bias 1023), 52 fraction bits. Precision: 24 significant bits $\approx$ 7 decimal digits (binary64: 53 bits, $\approx$ 16).
- **Facts that appear in questions:** ± 0 compare equal but $1/+0 = +\infty$, $1/-0 = -\infty$; `NaN != NaN`; rounding is round-to-nearest-even by default; machine epsilon $2^{-23} \approx 1.19 \times 10^{-7}$ (binary32); the gap between neighbouring floats *grows* with magnitude; 0.1 has no finite binary expansion, so `0.1+0.2 != 0.3` in binary64; floating-point addition is *not associative*; a fused multiply-add rounds once instead of twice (GPUs contract by default, Chapter 14).

```
import struct, warnings
import numpy as np
warnings.simplefilter("ignore")              # numpy warns about the deliberate overflow and divide-by-zero below

def fields(x):                               # decode a float32 into sign | exponent | mantissa
    b = format(struct.unpack(">I", struct.pack(">f", x))[0], "032b")
    s, e, m = b[0], int(b[1:9], 2), b[9:]
    if e == 255: kind = "inf" if int(m, 2) == 0 else "NaN"
    elif e == 0: kind = "zero" if int(m, 2) == 0 else "subnormal"
    else: kind = "normal"
    return "%s %s %s  %-9s" % (s, b[1:9], m, kind)

print("float32 = sign(1) | exponent(8, bias 127) | mantissa(23);  value = (-1)^s * 1.m * 2^(e-127)")
for x in (1.0, -2.5, 0.1, 0.0, -0.0, float("inf"), float("nan"), 1.1754944e-38, 1.4e-45, 3.4028235e38):
    print("%-13r %s" % (x, fields(x)))

f32 = np.float32
print("\n0.1 + 0.2 == 0.3 ?  float64:", 0.1 + 0.2 == 0.3, "| float32:", f32(0.1) + f32(0.2) == f32(0.3))
print("float32(16777217) =", int(f32(16777217)), "  (2^24 + 1 is the first integer a float32 cannot hold)")
gaps = tuple(np.spacing(f32(v)) for v in (1.0, 1e6, 1e9))
print("gap to the next float32: at 1.0 -> %.3e, at 1e6 -> %.3e, at 1e9 -> %.3e" % gaps)
print("nan == nan:", float("nan") == float("nan"), "| -0.0 == 0.0:", -0.0 == 0.0,
      "| 1/-0.0 in numpy:", np.float64(1) / np.float64(-0.0))

print("\ntwo's complement, 8 bits:")
for v in (5, -5, 127, -128):
    print("  %4d -> %s" % (v, format(v & 0xFF, "08b")))
print("  int8: 127 + 1 wraps to", int(np.int8(127) + np.int8(1)),
      "| 0xFF read as int8 =", int(np.uint8(0xFF).astype(np.int8)),
      "| sign extension of 0x80 to 32 bits = 0x%08X" % (int(np.int8(-128)) & 0xFFFFFFFF))
print("  conversions: 0xBEEF = %d = %s; 255 = 0b%s = 0o%o; -1 as uint32 = %d"
      % (0xBEEF, bin(0xBEEF), format(255, "b"), 255, (-1) & 0xFFFFFFFF))
```

Captured output

```
float32 = sign(1) | exponent(8, bias 127) | mantissa(23);  value = (-1)^s * 1.m * 2^(e-127)
1.0           0 01111111 00000000000000000000000  normal
-2.5          1 10000000 01000000000000000000000  normal
0.1           0 01111011 10011001100110011001101  normal
0.0           0 00000000 00000000000000000000000  zero
-0.0          1 00000000 00000000000000000000000  zero
inf           0 11111111 00000000000000000000000  inf
nan           0 11111111 10000000000000000000000  NaN
1.1754944e-38 0 00000001 00000000000000000000000  normal
1.4e-45       0 00000000 00000000000000000000001  subnormal
3.4028235e+38 0 11111110 11111111111111111111111  normal

0.1 + 0.2 == 0.3 ?  float64: False | float32: True
float32(16777217) = 16777216   (2^24 + 1 is the first integer a float32 cannot hold)
gap to the next float32: at 1.0 -> 1.192e-07, at 1e6 -> 6.250e-02, at 1e9 -> 6.400e+01
nan == nan: False | -0.0 == 0.0: True | 1/-0.0 in numpy: -inf

two's complement, 8 bits:
     5 -> 00000101
    -5 -> 11111011
   127 -> 01111111
  -128 -> 10000000
  int8: 127 + 1 wraps to -128 | 0xFF read as int8 = -1 | sign extension of 0x80 to 32 bits = 0xFFFFFF80
  conversions: 0xBEEF = 48879 = 0b1011111011101111; 255 = 0b11111111 = 0o377; -1 as uint32 = 4294967295
```

Reading the table: $0.1 =$ `0 01111011 10011001100110011001101`, i.e. the repeating pattern `1001` cut after 23 fraction bits and rounded up (the final `1`); $-2.5 = -1.01_2 \times 2^1$; the smallest subnormal is $2^{-149} \approx 1.4 \times 10^{-45}$; the largest finite value has exponent field 254 and an all-ones fraction. The 1-ulp gaps (`1.19e-07` at 1.0, `6.25e-02` at 10^6, 64 at 10^9) explain why accumulating small values into a large float stalls (Chapter 19).

20.2 Digital logic in ten minutes ★★

Topic	Facts
Boolean algebra	identities: $A + AB = A$, $A + \bar{A}B = A + B$, $A \oplus A = 0$; **De Morgan**: $\overline{AB} = \bar{A} + \bar{B}$, $\overline{A + B} = \bar{A}\bar{B}$; sum of products (SOP) vs product of sums (POS); K-maps group adjacent 1s in powers of two (1, 2, 4, 8, …), "don't cares" may join any group
universal gates	NAND (or NOR) alone builds every function; a gate has propagation delay, which creates *hazards* (glitches) when paths of different length reconverge
combinational blocks	multiplexer 2^n:1 has n selects; decoder $n \to 2^n$; encoder; **full adder**: $S = A \oplus B \oplus C_{in}$, $C_{out} = AB + C_{in}(A \oplus B)$; ripple-carry delay grows linearly with the width, **carry-lookahead** computes generate $g_i = a_i b_i$ and propagate $p_i = a_i \oplus b_i$ signals so that all carries are two-level logic
sequential elements	**latch** is level-sensitive, **flip-flop** edge-triggered; D copies its input at the clock edge, T toggles, JK has no forbidden input, SR forbids $S = R = 1$; *setup* and *hold* times bound when the input may change around the edge; violating them causes metamorphism/metastability
state machines	**Moore**: outputs depend on the state only (glitch-free, usually one more state); **Mealy**: outputs depend on state and input (fewer states, reacts one cycle earlier); k states need $\lceil \log_2 k \rceil$ flip-flops (binary encoding)
counters	mod-N counter: $\lceil \log_2 N \rceil$ flip-flops; *ripple* (asynchronous) counters are simple but slow because every stage waits for the previous one; *synchronous* counters share the clock; a Johnson counter with n flip-flops has $2n$ states, a ring counter n

Every claim of the kind "circuit X equals specification Y" can be **verified exhaustively** when the input space is small, the same habit as in Chapter 18: the script checks De Morgan, a NAND-only gate library, a 4-bit ripple-carry adder against a carry-lookahead adder and against integer addition on all 256 operand pairs, a finite-state machine against a regular expression on 2 000 random strings, and a 3-bit counter.

```python
import random, re

NAND = lambda a, b: 1 - (a & b)
def NOT(a): return NAND(a, a)                          # NAND is universal: everything below uses only NAND
def AND(a, b): return NOT(NAND(a, b))
def OR(a, b): return NAND(NOT(a), NOT(b))
def XOR(a, b):
    n = NAND(a, b)
```

```
    return NAND(NAND(a, n), NAND(b, n))

bits = (0, 1)
demorgan = all((1 - (a & b)) == ((1 - a) | (1 - b)) and (1 - (a | b)) == ((1 - a) & (1 - b)) for a in bits for b in bits)
print("De Morgan  ~(a&b) == ~a|~b and ~(a|b) == ~a&~b :", demorgan)
print("NAND-only AND/OR/NOT/XOR correct on all inputs:",
      all(AND(a, b) == a & b and OR(a, b) == a | b and NOT(a) == 1 - a and XOR(a, b) == a ^ b
          for a in bits for b in bits))

def full_adder(a, b, cin):                                  # sum = a^b^cin ; cout = ab + cin(a^b)
    return a ^ b ^ cin, (a & b) | (cin & (a ^ b))

def ripple_add(x, y, n=4):
    carry, out = 0, 0
    for i in range(n):
        s, carry = full_adder((x >> i) & 1, (y >> i) & 1, carry)
        out |= s << i
    return out, carry

def lookahead_add(x, y):                                    # 4-bit carry-lookahead, c0 = 0: carries are 2-level logic
    a, b = [(x >> i) & 1 for i in range(4)], [(y >> i) & 1 for i in range(4)]
    g = [a[i] & b[i] for i in range(4)]                     # generate:  this bit makes a carry by itself
    p = [a[i] ^ b[i] for i in range(4)]                     # propagate: this bit passes a carry on
    c = [0, g[0],
         g[1] | (p[1] & g[0]),
         g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0]),
         g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1]) | (p[3] & p[2] & p[1] & g[0])]
    return sum((p[i] ^ c[i]) << i for i in range(4)), c[4]

ok = all(ripple_add(x, y) == lookahead_add(x, y) == ((x + y) & 15, (x + y) >> 4) for x in range(16) for y in range(16))
print("4-bit ripple-carry == carry-lookahead == integer addition on all 256 input pairs:", ok)

def mealy_101(stream):                                      # overlapping '101' detector: output on the transition
    state, hits = 0, 0
    nxt = {(0, 0): 0, (0, 1): 1, (1, 0): 2, (1, 1): 1, (2, 0): 0, (2, 1): 1}
    for b in stream:
        if state == 2 and b == 1: hits += 1
        state = nxt[(state, b)]
    return hits

rng = random.Random(1)
streams = ["".join(rng.choice("01") for _ in range(rng.randint(0, 40))) for _ in range(2000)]
print("FSM detector for overlapping 101 vs regex on 2000 random bit strings:",
      all(mealy_101(map(int, s)) == len(re.findall(r"(?=101)", s)) for s in streams))

def counter3(clock_edges):                                  # 3 T flip-flops: each toggles when all lower bits are 1
    q, seq = [0, 0, 0], []
    for _ in range(clock_edges):
        seq.append(q[0] + 2 * q[1] + 4 * q[2])
        t = [1, q[0], q[0] & q[1]]
        q = [qi ^ ti for qi, ti in zip(q, t)]
    return seq

print("3-bit synchronous counter sequence:", counter3(10), "(wraps after 7)")
print("minimum number of flip-flops for a mod-N counter: ceil(log2 N); N = 10 ->", (10 - 1).bit_length())
```

Captured output

```
De Morgan  ~(a&b) == ~a|~b and ~(a|b) == ~a&~b : True
NAND-only AND/OR/NOT/XOR correct on all inputs: True
4-bit ripple-carry == carry-lookahead == integer addition on all 256 input pairs: True
FSM detector for overlapping 101 vs regex on 2000 random bit strings: True
3-bit synchronous counter sequence: [0, 1, 2, 3, 4, 5, 6, 7, 0, 1] (wraps after 7)
minimum number of flip-flops for a mod-N counter: ceil(log2 N); N = 10 -> 4
```

20.3 Computer architecture ★★★

Must know

- **Performance equation:** time = IC × CPI × T_{clk} (instruction count × cycles per instruction × clock period); **Amdahl's law** $S = 1/\big((1-f) + f/p\big)$ (Chapter 16); MIPS and clock rate alone are misleading.
- **RISC** (fixed-length instructions, load/store architecture, many registers: ARM, RISC-V, MIPS) versus **CISC** (variable length, memory operands: x86); modern x86 cores decode into RISC-like micro-operations. Addressing modes: immediate, register, direct, register-indirect, base+displacement, indexed, PC-relative.
- **Classic 5-stage pipeline** IF–ID–EX–MEM–WB: ideal speed-up equals the number of stages and CPI= 1; **hazards**: structural (a resource is needed twice), *data* (RAW is the real one; WAR and WAW matter

only with out-of-order execution), *control* (branches). **Forwarding** removes most RAW stalls; a **load-use** hazard still costs one stall cycle. Branch prediction (1-bit, 2-bit saturating counters, branch target buffer) hides control hazards; superscalar and out-of-order cores issue several instructions per cycle.

- **Cache:** address = *tag* | *index* | *offset*; #sets = size/(block × ways); direct-mapped = 1 way, fully associative = 1 set. **Write-through** updates memory on every store, **write-back** only when a dirty line is evicted; write-allocate loads the line on a store miss. Miss types: *compulsory*, *capacity*, *conflict* (and coherence misses on multiprocessors). **AMAT** = hit time + miss rate × miss penalty; for two levels $H_1 + m_1(H_2 + m_2 P)$ with local miss rates.
- **Virtual memory:** the page table maps virtual page numbers to frames; a **TLB** caches translations; a page fault costs milliseconds; multi-level tables keep the page table small. Physical address = frame × page size + offset.

```
from collections import OrderedDict

class Cache:                                            # set-associative, LRU replacement, write-allocate
    def __init__(self, size, block, ways):
        self.block, self.ways = block, ways
        self.sets = size // (block * ways)
        self.data = [OrderedDict() for _ in range(self.sets)]
        self.hits = self.misses = 0

    def access(self, addr):
        line = addr // self.block
        s, tag = line % self.sets, line // self.sets
        ways = self.data[s]
        if tag in ways:
            ways.move_to_end(tag)
            self.hits += 1
        else:
            self.misses += 1
            if len(ways) >= self.ways: ways.popitem(last=False)    # evict the least recently used way
            ways[tag] = True

    def miss_rate(self): return self.misses / (self.hits + self.misses)

def breakdown(addr_bits, size, block, ways):                    # how an address is split: tag | index | offset
    sets = size // (block * ways)
    off, idx = block.bit_length() - 1, sets.bit_length() - 1
    return addr_bits - idx - off, idx, off

t, i, o = breakdown(32, 32 * 1024, 64, 8)
print("32-bit address, 32 KB, 8-way, 64 B blocks: tag %d bits | index %d bits (%d sets) | offset %d bits"
      % (t, i, 2 ** i, o))

def traverse(N, order, row_ints=None, tile=None):               # sum an N x N matrix of 4-byte ints stored row by row
    row_bytes = 4 * (row_ints or N)
    c = Cache(size=4096, block=64, ways=4)
    if tile:                                                    # blocked traversal
        seq = [(i, j) for ii in range(0, N, tile) for jj in range(0, N, tile)
               for i in range(ii, ii + tile) for j in range(jj, jj + tile)]
    elif order == "row": seq = [(i, j) for i in range(N) for j in range(N)]
    else: seq = [(i, j) for j in range(N) for i in range(N)]
    for i, j in seq: c.access(i * row_bytes + 4 * j)
    return c.miss_rate()

hit, penalty = 1.0, 100.0                                       # ns
print("4 KB 4-way cache, 64 B blocks (16 ints per block); AMAT = hit time + miss rate x miss penalty (1 ns, 100 ns)")
cases = [("256x256, row-major loop order (i outer, j inner)", traverse(256, "row")),
         ("256x256, column-major loop order (j outer, i inner)", traverse(256, "col")),
         ("64x64, column-major, row length 64 ints (stride 256 B)", traverse(64, "col")),
         ("64x64, column-major, rows padded to 80 ints (stride 320 B)", traverse(64, "col", row_ints=80))]
for name, rate in cases:
    print("  %-60s miss rate %6.2f%%  AMAT %6.2f ns" % (name, 100 * rate, hit + rate * penalty))
```

Captured output

```
32-bit address, 32 KB, 8-way, 64 B blocks: tag 20 bits | index 6 bits (64 sets) | offset 6 bits
4 KB 4-way cache, 64 B blocks (16 ints per block); AMAT = hit time + miss rate x miss penalty (1 ns, 100 ns)
  256x256, row-major loop order (i outer, j inner)             miss rate   6.25%  AMAT   7.25 ns
  256x256, column-major loop order (j outer, i inner)          miss rate 100.00%  AMAT 101.00 ns
  64x64, column-major, row length 64 ints (stride 256 B)       miss rate 100.00%  AMAT 101.00 ns
  64x64, column-major, rows padded to 80 ints (stride 320 B)   miss rate   6.25%  AMAT   7.25 ns
```

The output is Chapter 11's loop interchange and padding advice in numbers: a row-major walk misses once per 16 ints (6.25%), the column-major walk misses on *every* access (the 256 lines of one column exceed the 64-line cache: *capacity* misses), and in the smaller 64×64 case the 256-byte row stride maps all rows to four sets (*conflict* misses)

until padding the rows to 80 ints spreads them over all sets. The GPU analogues are Chapter 15's coalescing and shared-memory bank conflicts: "unrelated accesses collide in a hardware structure".

```
print("virtual memory arithmetic")
va, page, pte = 32, 4096, 4
vpn_bits = va - (page.bit_length() - 1)
print("32-bit VA, 4 KB pages: offset 12 bits, VPN %d bits -> single-level table = 2^%d entries x %d B = %d MB per process"
      % (vpn_bits, vpn_bits, pte, 2 ** vpn_bits * pte // 2 ** 20))
print("two-level (10 + 10 bits): top table %d entries = %d KB; second-level tables exist only for used regions"
      % (2 ** 10, 2 ** 10 * pte // 1024))
print("x86-64: 48-bit VA = 4 levels x 9 bits + 12-bit offset; each table = 512 entries x 8 B = %d bytes = one page"
      % (512 * 8))
print("TLB: hit time 1 ns, 98%% hit rate, page walk (miss) 100 ns -> effective translation time %.1f ns"
      % (0.98 * 1 + 0.02 * (1 + 100)))
mem, fault, p = 100, 8_000_000, 1e-6                          # ns
print("page-fault service 8 ms, memory 100 ns: fault rate 1 in 10^6 slows memory access to %.0f ns (%.0f%% slower)"
      % ((1 - p) * mem + p * fault, 100 * (p * fault - p * mem) / mem))

def pipeline(instrs, forwarding):
    """Cycle-accurate 5-stage pipeline IF ID EX MEM WB. The register file is written in the first half of a
    cycle and read in the second half, so a consumer may be in ID while its producer is in WB."""
    IF, ID, EX, MEM, WB = {}, {}, {}, {}, {}
    for i, (op, dst, srcs) in enumerate(instrs):
        IF[i] = 1 if i == 0 else ID[i - 1]                    # fetched when the previous instruction leaves IF
        ready = 0
        for p in range(i):
            if instrs[p][1] is not None and instrs[p][1] in srcs:          # RAW dependence on instruction p
                if not forwarding: ready = max(ready, WB[p])               # value visible only after write-back
                else: ready = max(ready, MEM[p] if instrs[p][0] == "lw" else EX[p])    # forwarded from MEM or EX
        ID[i] = max(IF[i] + 1, ready, ID[i - 1] + 1 if i else 0)
        EX[i], MEM[i], WB[i] = ID[i] + 1, ID[i] + 2, ID[i] + 3
    return WB[len(instrs) - 1]

prog = [("lw", "r1", ["r2"]), ("add", "r3", ["r1", "r4"]), ("sub", "r5", ["r3", "r1"]), ("sw", None, ["r5", "r6"])]
print("lw r1,0(r2); add r3,r1,r4; sub r5,r3,r1; sw r5,0(r6)    (ideal: %d + 4 = %d cycles)" % (len(prog), len(prog) + 4))
for fwd in (False, True):
    cycles = pipeline(prog, fwd)
    print("  forwarding %-5s: %2d cycles in total, %d stall cycles, CPI = %.2f"
          % (fwd, cycles, cycles - len(prog) - 4, cycles / len(prog)))
```

Captured output

```
virtual memory arithmetic
32-bit VA, 4 KB pages: offset 12 bits, VPN 20 bits -> single-level table = 2^20 entries x 4 B = 4 MB per process
two-level (10 + 10 bits): top table 1024 entries = 4 KB; second-level tables exist only for used regions
x86-64: 48-bit VA = 4 levels x 9 bits + 12-bit offset; each table = 512 entries x 8 B = 4096 bytes = one page
TLB: hit time 1 ns, 98% hit rate, page walk (miss) 100 ns -> effective translation time 3.0 ns
page-fault service 8 ms, memory 100 ns: fault rate 1 in 10^6 slows memory access to 108 ns (8% slower)
lw r1,0(r2); add r3,r1,r4; sub r5,r3,r1; sw r5,0(r6)    (ideal: 4 + 4 = 8 cycles)
  forwarding False: 14 cycles in total, 6 stall cycles, CPI = 3.50
  forwarding True :  9 cycles in total, 1 stall cycles, CPI = 2.25
```

The pipeline numbers: ideal is $4 + 4 = 8$ cycles; without forwarding the dependences stall the pipeline for 6 cycles (CPI 3.5); with forwarding only the load-use hazard remains (1 stall, 9 cycles, CPI 2.25). The register file is written in the first half of a clock cycle and read in the second half, so a consumer may be in ID while its producer is in WB (the standard textbook assumption; state it in an exam).

20.4 Operating systems ★★★

Must know

- **Process** = program in execution (own address space, PCB: registers, state, page table, open files); **thread** = unit of scheduling inside a process sharing its address space; a **context switch** saves and restores CPU state (and may flush the TLB). States: new, ready, running, waiting, terminated. A **zombie** has terminated but its parent has not `wait`ed; an **orphan**'s parent died (it is adopted by `init`). `fork` duplicates the process (copy-on-write pages), `exec` replaces its image, `wait` reaps a child. IPC: pipes, message queues, shared memory, sockets, signals.
- **Scheduling criteria:** CPU utilisation, throughput, turnaround time (completion − arrival), waiting time (turnaround − burst), response time. FCFS suffers the *convoy effect*; SJF is optimal for average waiting time but needs burst prediction and can starve long jobs (fix: *aging*); Round Robin trades response time for context-switch overhead (quantum too large → FCFS, too small → overhead); multilevel feedback queues approximate SJF without knowing bursts.
- **Synchronisation:** a critical section needs mutual exclusion, progress and bounded waiting; tools: test-and-

set/compare-and-swap, mutexes, semaphores (counting, binary), monitors (Chapter 16); classic problems: producer–consumer, readers–writers, dining philosophers.

- **Deadlock** needs all four Coffman conditions (mutual exclusion, hold and wait, no preemption, circular wait); handled by prevention (break one), avoidance (**Banker's algorithm**: grant only if a safe sequence exists), detection and recovery, or ignoring it.
- **Memory:** contiguous allocation suffers external fragmentation, paging suffers internal fragmentation (the last page); *page replacement*: FIFO, LRU, OPT (optimal, unrealisable), clock; **Belady's anomaly**: more frames can mean more faults for FIFO; *thrashing*: more time paging than computing (cure: working-set model, fewer processes).

20.4.1 Scheduling: the textbook example, simulated

Captured output

```
P1(arr 0, burst 8)  P2(1, 4)  P3(2, 9)  P4(3, 5)
policy completion times                  avg wait  avg TAT
FCFS   P1:8 P2:12 P3:21 P4:26                8.75    15.25
SJF    P1:8 P2:12 P3:26 P4:17                7.75    14.25
SRTF   P1:17 P2:5 P3:26 P4:10                6.50    13.00
RR q=3 P1:23 P2:16 P3:26 P4:21              13.50    20.00
```

(Convention: a process that arrives at the instant a quantum ends enters the ready queue *before* the preempted process is re-queued.) SJF cuts the average waiting time from 8.75 to 7.75 and the preemptive variant SRTF to 6.5; Round Robin with $q = 3$ is worse on average waiting (13.5) but gives every process an early first response.

20.4.2 Page replacement and Belady's anomaly

```
def faults(refs, frames, policy):
    mem, f = [], 0
    for i, page in enumerate(refs):
        if page in mem:
            if policy == "LRU": mem.remove(page); mem.append(page)          # refresh recency
            continue
        f += 1
        if len(mem) >= frames:
            if policy in ("FIFO", "LRU"): mem.pop(0)                        # oldest / least recently used
            else:                                                           # OPT: evict the page used farthest ahead
                future = refs[i + 1:]
                mem.remove(max(mem, key=lambda p: future.index(p) if p in future else len(future) + 1))
        mem.append(page)
    return f

refs = [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]
print("reference string:", refs)
print("frames   FIFO  LRU  OPT")
for n in (3, 4):
    print("%6d   %4d %4d %4d" % (n, faults(refs, n, "FIFO"), faults(refs, n, "LRU"), faults(refs, n, "OPT")))
print("FIFO with 4 frames faults MORE than with 3: Belady's anomaly (LRU and OPT are stack algorithms: no anomaly)")
```

Captured output

```
reference string: [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]
frames   FIFO  LRU  OPT
     3      9   10    7
     4     10    8    6
FIFO with 4 frames faults MORE than with 3: Belady's anomaly (LRU and OPT are stack algorithms: no anomaly)
```

20.4.3 Deadlock avoidance: the Banker's algorithm

Three resource types, five processes; `need = max - allocation`. A request is granted only if the resulting state still has a *safe sequence* (an order in which every process can finish). The three requests below arrive one after the other:

```
avail = [3, 3, 2]
alloc = {"P0": [0, 1, 0], "P1": [2, 0, 0], "P2": [3, 0, 2], "P3": [2, 1, 1], "P4": [0, 0, 2]}
maxim = {"P0": [7, 5, 3], "P1": [3, 2, 2], "P2": [9, 0, 2], "P3": [2, 2, 2], "P4": [4, 3, 3]}

def need(p, alloc): return [m - a for m, a in zip(maxim[p], alloc[p])]

def safe_sequence(avail, alloc):
    work, left, seq = list(avail), set(alloc), []
    while left:
        for p in sorted(left):
```

```
            if all(n <= w for n, w in zip(need(p, alloc), work)):          # p can finish with what is available
                work = [w + a for w, a in zip(work, alloc[p])]
                seq.append(p); left.remove(p)
                break
        else:
            return None                                                    # nobody can finish: unsafe
    return seq

def request(p, req):                                                       # Banker's algorithm; state changes if granted
    global avail, alloc
    if any(r > n for r, n in zip(req, need(p, alloc))): return "error: exceeds declared maximum"
    if any(r > a for r, a in zip(req, avail)): return "must wait: not available"
    new_avail = [a - r for a, r in zip(avail, req)]
    new_alloc = {q: ([x + r for x, r in zip(alloc[q], req)] if q == p else alloc[q]) for q in alloc}
    seq = safe_sequence(new_avail, new_alloc)
    if not seq: return "denied: would leave the system in an UNSAFE state"
    avail, alloc = new_avail, new_alloc
    return "granted; safe sequence %s" % seq

print("available", avail, "| initial state is safe, e.g.:", safe_sequence(avail, alloc))
for p, req in (("P1", [1, 0, 2]), ("P4", [3, 3, 0]), ("P0", [0, 2, 0])):     # requests arrive one after the other
    print("%s requests %s: %s" % (p, tuple(req), request(p, req)))
```

Captured output

```
available [3, 3, 2] | initial state is safe, e.g.: ['P1', 'P3', 'P0', 'P2', 'P4']
P1 requests (1, 0, 2): granted; safe sequence ['P1', 'P3', 'P0', 'P2', 'P4']
P4 requests (3, 3, 0): must wait: not available
P0 requests (0, 2, 0): denied: would leave the system in an UNSAFE state
```

20.4.4 `fork`, buffering and zombies: counting output lines

cs_fork.c: seven variants selected by argv[1]

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char **argv) {
    int v = argc > 1 ? atoi(argv[1]) : 0;
    if (v == 1) { fork(); fork(); puts("hello"); }                          /* 2 forks: 4 processes */
    if (v == 2) { fork(); fork(); fork(); puts("hello"); }                  /* 3 forks: 8 processes */
    if (v == 3) { if (fork() && fork()) puts("both true"); puts("end"); }   /* && short-circuits in the child */
    if (v == 4) { printf("A"); fork(); }                  /* no newline: the stdio buffer is copied by fork */
    if (v == 5) { printf("A"); fflush(stdout); fork(); }  /* flushed first: printed once */
    if (v == 6) { if (fork() == 0) { puts("child"); exit(0); } wait(NULL); puts("parent"); }
    if (v == 7) { if (fork() == 0) { puts("child"); _exit(0); } wait(NULL); puts("parent"); }  /* _exit: no flush */
    while (wait(NULL) > 0) { }                                              /* reap the children: no zombies */
    return 0;
}
```

```
$ gcc cs_fork.c -o cs_fork; for v in 1..7: ./cs_fork $v | tr '\n' ' '
```

```
variant 1: hello hello hello hello
variant 2: hello hello hello hello hello hello hello hello
variant 3: end end both true end
variant 4: AA
variant 5: A
variant 6: child parent
variant 7: parent
```

n unconditional `fork` calls give 2^n processes (variants 1–2). In variant 3 the child of the first `fork` skips the second one because `&&` short-circuits (`fork()` returns 0 in a child), so *three* processes print `end`. Variants 4–5 are the buffering trap: standard output is *fully buffered* when it is a pipe, so the unflushed `A` sits in the buffer that `fork` duplicates and is printed twice; flush first and it appears once. Variants 6–7: `exit` flushes stdio buffers in the child; `_exit` does not, so "child" is lost when output goes to a pipe.

20.5 Databases in one page ★

Topic	Facts
keys	*super key* identifies a row; *candidate key* is a minimal super key; *primary key* the chosen candidate (unique, not null); *foreign key* references another table's key
normal forms	**1NF** atomic values; **2NF** no partial dependence on part of a composite key; **3NF** no transitive dependence (non-key → non-key); **BCNF** every determinant is a candidate key. Normalisation removes update anomalies, denormalisation trades them for speed
ACID	atomicity (all or nothing), consistency (constraints hold), isolation (concurrent transactions look serial), durability (committed data survives crashes; write-ahead log)
isolation levels	read uncommitted (dirty reads possible) → read committed (no dirty reads) → repeatable read (no non-repeatable reads) → serialisable (no phantoms); real systems add MVCC snapshots and differ in detail
indexes	**B+ tree**: balanced, wide nodes, leaves linked → equality and range queries in $O(\log n)$ page reads; hash index: equality only; *clustered* index = the table order; an index speeds reads and slows writes
SQL order of evaluation	`FROM/JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`; `WHERE` filters rows, `HAVING` filters groups; `NULL` is "unknown": `NULL = NULL` is not true, use `IS NULL`; `COUNT(*)` counts rows, `COUNT(col)` non-null values; `DELETE` (rows, logged), `TRUNCATE` (all rows, fast), `DROP` (the table)

Eight classic query patterns, run on a six-row example with Python's `sqlite3` (note the 3-valued logic in the last query, the anti-join for "departments without employees", and that `RANK` gives ties the same rank):

```
import sqlite3
db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE dept (id INTEGER PRIMARY KEY, name TEXT);
CREATE TABLE emp  (id INTEGER PRIMARY KEY, name TEXT, dept_id INTEGER REFERENCES dept(id),
                   salary INTEGER, boss_id INTEGER);
INSERT INTO dept VALUES (1, 'Compilers'), (2, 'Hardware'), (3, 'QA');
INSERT INTO emp VALUES (1, 'Asha', 1, 120, NULL), (2, 'Ben', 1, 100, 1), (3, 'Chen', 1, 100, 1),
                       (4, 'Dia', 2, 90, 1), (5, 'Eli', 2, 95, 4), (6, 'Fay', NULL, 70, 4);
""")

def q(title, sql):
    print("-- " + title)
    for row in db.execute(sql): print("   ", row)

q("inner join: employees with their department",
  "SELECT e.name, d.name FROM emp e JOIN dept d ON e.dept_id = d.id ORDER BY e.id")
q("left join keeps employees without a department (Fay): NULL on the right",
  "SELECT e.name, d.name FROM emp e LEFT JOIN dept d ON e.dept_id = d.id WHERE d.id IS NULL")
q("departments with no employee (QA): anti-join",
  "SELECT d.name FROM dept d WHERE NOT EXISTS (SELECT 1 FROM emp e WHERE e.dept_id = d.id)")
q("group by / having: departments with at least two employees",
  "SELECT d.name, COUNT(*), AVG(e.salary) FROM emp e JOIN dept d ON e.dept_id = d.id "
  "GROUP BY d.name HAVING COUNT(*) >= 2")
q("second highest salary", "SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp)")
q("rank inside each department (window function; ties share a rank)",
  "SELECT name, dept_id, salary, RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) "
  "FROM emp WHERE dept_id IS NOT NULL")
q("self join: employee and boss", "SELECT e.name, b.name FROM emp e JOIN emp b ON e.boss_id = b.id WHERE e.dept_id = 1")
q("NULL semantics: COUNT(*), COUNT(dept_id), COUNT(boss_id), NULL = NULL, NULL IS NULL",
  "SELECT COUNT(*), COUNT(dept_id), COUNT(boss_id), NULL = NULL, NULL IS NULL FROM emp")
```

Captured output

```
-- inner join: employees with their department
    ('Asha', 'Compilers')
    ('Ben', 'Compilers')
    ('Chen', 'Compilers')
    ('Dia', 'Hardware')
    ('Eli', 'Hardware')
-- left join keeps employees without a department (Fay): NULL on the right
    ('Fay', None)
-- departments with no employee (QA): anti-join
    ('QA',)
-- group by / having: departments with at least two employees
    ('Compilers', 3, 106.66666666666667)
    ('Hardware', 2, 92.5)
-- second highest salary
    (100,)
-- rank inside each department (window function; ties share a rank)
```

```
    ('Asha', 1, 120, 1)
    ('Ben', 1, 100, 2)
    ('Chen', 1, 100, 2)
    ('Eli', 2, 95, 1)
    ('Dia', 2, 90, 2)
-- self join: employee and boss
    ('Ben', 'Asha')
    ('Chen', 'Asha')
-- NULL semantics: COUNT(*), COUNT(dept_id), COUNT(boss_id), NULL = NULL, NULL IS NULL
    (6, 5, 5, None, 1)
```

20.6 Computer networks in one page ★

Topic	Facts
layers	OSI: physical, data link, network, transport, session, presentation, application; TCP/IP: link, internet, transport, application. Units: bit, frame, packet, segment (TCP) / datagram (UDP), data. Devices: hub (L1), switch (L2, MAC addresses), router (L3, IP addresses)
IP and subnets	IPv4 32 bits, IPv6 128 bits; CIDR `a.b.c.d/n`: n network bits, $2^{32-n} - 2$ usable hosts; private ranges 10/8, 172.16/12, 192.168/16; NAT shares one public address; ARP maps IP to MAC; ICMP carries `ping`/errors; MTU of Ethernet 1500 bytes
TCP	connection-oriented, reliable, ordered byte stream; *three-way handshake* SYN, SYN-ACK, ACK (four segments to close); sequence numbers and acknowledgements; *flow control* (receiver window) versus *congestion control* (slow start, congestion avoidance with additive increase / multiplicative decrease, fast retransmit); the sender that closes actively sits in TIME_WAIT
UDP	connectionless, unreliable datagrams, no ordering: DNS queries, streaming, games, QUIC builds reliability on top
application layer	HTTP (request/response; methods GET, POST, PUT, DELETE; status classes 2xx success, 3xx redirect, 4xx client error, 5xx server error), HTTPS = HTTP over TLS, DNS (A, AAAA, CNAME, MX records), DHCP, SSH, FTP, SMTP

```python
import ipaddress, socket, threading

net = ipaddress.ip_network("192.168.10.77/26", strict=False)
hosts = list(net.hosts())
print("192.168.10.77/26 -> network %s, mask %s, broadcast %s" % (net.network_address, net.netmask, net.broadcast_address))
print("                    first/last host %s / %s, usable hosts %d" % (hosts[0], hosts[-1], net.num_addresses - 2))
subnets = ipaddress.ip_network("10.0.0.0/24").subnets(prefixlen_diff=2)
print("split 10.0.0.0/24 into 4 subnets:", [str(s) for s in subnets])
ip = ipaddress.ip_address
print("10.1.2.3 in 10.0.0.0/8?", ip("10.1.2.3") in ipaddress.ip_network("10.0.0.0/8"),
      "| 172.32.0.1 private?", ip("172.32.0.1").is_private, "| 172.16.0.1 private?", ip("172.16.0.1").is_private)

def echo_server(srv):                                   # TCP: connection-oriented, reliable byte stream
    conn, _ = srv.accept()
    with conn:
        while data := conn.recv(1024): conn.sendall(data.upper())

srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
srv.bind(("127.0.0.1", 0))
srv.listen(1)
threading.Thread(target=echo_server, args=(srv,), daemon=True).start()
with socket.create_connection(srv.getsockname()) as c:   # connect() performs the three-way handshake
    c.sendall(b"compiler verification")
    print("TCP echo through the loopback interface:", c.recv(1024).decode())

u = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)     # UDP: no connection, no delivery guarantee
u.bind(("127.0.0.1", 0))
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.sendto(b"datagram", u.getsockname())
print("UDP datagram received:", u.recvfrom(1024)[0].decode(), "(one message = one datagram; no loss recovery)")
```

Captured output

```
192.168.10.77/26 -> network 192.168.10.64, mask 255.255.255.192, broadcast 192.168.10.127
                    first/last host 192.168.10.65 / 192.168.10.126, usable hosts 62
split 10.0.0.0/24 into 4 subnets: ['10.0.0.0/26', '10.0.0.64/26', '10.0.0.128/26', '10.0.0.192/26']
10.1.2.3 in 10.0.0.0/8? True | 172.32.0.1 private? False | 172.16.0.1 private? True
TCP echo through the loopback interface: COMPILER VERIFICATION
UDP datagram received: datagram (one message = one datagram; no loss recovery)
```

20.7 A shell-and-tools survival kit ★★

Verification teams live in the shell: results arrive as logs, and the first triage is a pipeline. The log below is a miniature of the `lit`-style output of Chapter 18; every command was run.

cs_shell_log.txt: a miniature test log

```
PASS: suite :: alpha.test (1 of 8) [0.12s]
PASS: suite :: beta.test (2 of 8) [0.40s]
FAIL: suite :: gamma.test (3 of 8) [1.90s]
PASS: suite :: delta.test (4 of 8) [0.31s]
FAIL: suite :: epsilon.test (5 of 8) [0.05s]
TIMEOUT: suite :: zeta.test (6 of 8) [30.00s]
PASS: suite :: eta.test (7 of 8) [0.22s]
FAIL: suite :: theta.test (8 of 8) [0.07s]
error: line 12: assertion 'x > 0' failed in pass InstCombine (gamma.test)
error: line 99: assertion 'x > 0' failed in pass InstCombine (theta.test)
error: line 7: use of undeclared identifier 'foo' (epsilon.test)
```

shell_kit.sh: counting, extracting, masking, parallelising

```
log=cs_shell_log.txt
echo "# counts per status"
cut -d: -f1 $log | grep -E '^(PASS|FAIL|TIMEOUT)$' | sort | uniq -c | sort -rn
echo "# names of failing tests"
awk -F' :: ' '/^FAIL/ {split($2, a, " "); print a[1]}' $log
echo "# the two slowest tests (seconds, name)"
grep -E '^(PASS|FAIL|TIMEOUT)' $log | sed -E 's/.*:: ([^ ]+) .*\[([0-9.]+)s\]/\2 \1/' | sort -rn | head -2
echo "# total time in seconds"
grep -oE '\[[0-9.]+s\]' $log | tr -d '[]s' | paste -sd+ | bc
echo "# error signatures with numbers masked"
grep '^error' $log | sed -E 's/line [0-9]+/line N/; s/\([^)]*\)$//' | sort | uniq -c | sort -rn
echo "# parallel run: 4 jobs of 1 s each with xargs -P4"
s=$(date +%s.%N)
printf '%s\n' a b c d | xargs -P4 -I{} sh -c 'sleep 1; echo done {}' | sort | tr '\n' ' '
e=$(date +%s.%N)
python3 -c "print('-> %.1f s wall time (serial would take 4 s)' % ($e - $s))"
```

```
$ bash shell_kit.sh
```

```
# counts per status
      4 PASS
      3 FAIL
      1 TIMEOUT
# names of failing tests
gamma.test
epsilon.test
theta.test
# the two slowest tests (seconds, name)
30.00 zeta.test
1.90 gamma.test
# total time in seconds
33.07
# error signatures with numbers masked
      2 error: line N: assertion 'x > 0' failed in pass InstCombine
      1 error: line N: use of undeclared identifier 'foo'
# parallel run: 4 jobs of 1 s each with xargs -P4
done a done b done c done d -> 1.0 s wall time (serial would take 4 s)
```

Tool / command	What you use it for
`grep -rn PATTERN dir`, `rg`, `find . -name '*.cu' -exec \ldots{} +`	search code and logs; `grep -E` extended regexes, `-v` invert, `-c` count
`sed -E 's/old/new/g'`, `awk -F: '{print $1}'`, `cut`, `sort | uniq -c | sort -rn`	stream editing, field extraction, the "histogram of things" idiom
`xargs -P N`, `parallel`, `timeout 10 cmd`	run commands in parallel; bound the run time; exit code 124 means timeout
`git log --oneline`, `git bisect run`, `git diff A..B`, `git stash`, `git rebase -i`, `git cherry-pick`	history, finding the bad commit (Chapter 17), shelving work, rewriting local history, porting a fix

Tool / command	What you use it for
`make -j`, `cmake -S . -B build`, `ninja`	build systems; `-j` parallel jobs; out-of-source builds
`gdb -ex run --args ./prog`, `valgrind`, `perf stat/record`, `strace`, `ltrace`	debugger, memory errors, performance counters, system and library calls
`nm`, `objdump -d`, `readelf -h -S`, `ldd`, `file`	inspect binaries: symbols, disassembly, sections, shared-library dependencies
`docker run -v $PWD:/w img cmd`	reproducible environment: a container shares the host kernel, a VM virtualises hardware
`python3 -m venv`, `pip install -r`, `pytest -x -q`	isolated Python environments; running tests

20.8 Drills, with answers computed ★★★

```
import math, ipaddress
print("1. floats in [1, 2) for float32: 2^23 = %d" % 2 ** 23)
print("2. AMAT, L1 hit 1 ns, L1 miss rate 5%%, miss penalty 100 ns: %.1f ns" % (1 + 0.05 * 100))
print("3. two-level cache: L1 1 ns / 10%% miss, L2 10 ns / 20%% local miss, memory 100 ns: AMAT = %.1f ns"
      % (1 + 0.10 * (10 + 0.20 * 100)))
print("4. 5-stage pipeline: ideal speed-up 5x; if hazards raise the average CPI to 1.5 the real speed-up is %.2fx" % (5 / 1.5))

print("5. execution time = IC x CPI x T: 2e9 instructions, CPI 1.5, 2 GHz -> %.2f s" % (2e9 * 1.5 / 2e9))
print("6. minimum flip-flops for a mod-10 counter: %d; a 4-flip-flop Johnson counter has %d states"
      % (math.ceil(math.log2(10)), 2 * 4))
sets = 8192 // (32 * 2)
print("7. cache of 8 KB, 2-way, 32 B blocks: %d sets; 32-bit address -> tag %d, index %d, offset %d bits"
      % (sets, 32 - 7 - 5, int(math.log2(sets)), 5))
print("8. 48-bit VA, 4 KB pages, 8-byte PTEs: %d PTEs per page-table page, %d levels"
      % (4096 // 8, (48 - 12) // 9))
print("9. Amdahl: 10%% serial, 8 processors -> speed-up %.2f" % (1 / (0.1 + 0.9 / 8)))
print("10. /26 subnet: %d usable hosts; /30: %d; 255.255.255.224 is /%d"
      % (ipaddress.ip_network("10.0.0.0/26").num_addresses - 2, ipaddress.ip_network("10.0.0.0/30").num_addresses - 2,
         bin(0xFFFFFFE0).count("1")))
print("11. disk: 7200 rpm -> average rotational latency %.2f ms" % (0.5 * 60000 / 7200))
print("12. 1 KB = 2^10 B; a 4 GB address space needs %d address bits; 64 KB / 16 B blocks = %d blocks"
      % (math.log2(4 * 2 ** 30), 64 * 1024 // 16))
```

Captured output

```
1. floats in [1, 2) for float32: 2^23 = 8388608
2. AMAT, L1 hit 1 ns, L1 miss rate 5%, miss penalty 100 ns: 6.0 ns
3. two-level cache: L1 1 ns / 10% miss, L2 10 ns / 20% local miss, memory 100 ns: AMAT = 4.0 ns
4. 5-stage pipeline: ideal speed-up 5x; if hazards raise the average CPI to 1.5 the real speed-up is 3.33x
5. execution time = IC x CPI x T: 2e9 instructions, CPI 1.5, 2 GHz -> 1.50 s
6. minimum flip-flops for a mod-10 counter: 4; a 4-flip-flop Johnson counter has 8 states
7. cache of 8 KB, 2-way, 32 B blocks: 128 sets; 32-bit address -> tag 20, index 7, offset 5 bits
8. 48-bit VA, 4 KB pages, 8-byte PTEs: 512 PTEs per page-table page, 4 levels
9. Amdahl: 10% serial, 8 processors -> speed-up 4.71
10. /26 subnet: 62 usable hosts; /30: 2; 255.255.255.224 is /27
11. disk: 7200 rpm -> average rotational latency 4.17 ms
12. 1 KB = 2^10 B; a 4 GB address space needs 32 address bits; 64 KB / 16 B blocks = 4096 blocks
```

20.9 Practice questions ★★

Q20.1 What is the range of an 8-bit two's-complement integer, and what is 0x80 sign-extended to 32 bits?
Answer: $-128 \ldots 127$; 0xFFFFFF80 (see the number-system output above).

Q20.2 Why is `0.1 + 0.2 == 0.3` false in double precision but true in single precision?
Answer: None of the three has an exact binary representation; in binary64 the rounding errors of 0.1 and 0.2 add up to a value one ulp away from the double nearest to 0.3; in binary32 the errors happen to cancel after rounding (the script prints `False` and `True`). The lesson: never compare floating-point results with `==`; use a tolerance.

Q20.3 A cache is 32 KB, 8-way set associative with 64-byte blocks and 32-bit addresses. How many tag, index and offset bits?
Answer: Offset 6 bits (64 B), sets $= 32768/(64 \cdot 8) = 64 \Rightarrow$ index 6 bits, tag $32 - 6 - 6 = 20$ bits.

Q20.4 Why is column-major traversal of a row-major matrix slow, and what are two fixes?
Answer: Each access touches a different cache line, so spatial locality is lost and the miss rate approaches 100% (6.25% for the row-major walk, simulated above). Fixes: loop interchange (walk rows), blocking/tiling, padding rows when the stride maps many rows to the same sets, or a transposed copy.

Q20.5 Explain forwarding and the load-use hazard.
Answer: Forwarding (bypassing) feeds an ALU result straight from the EX/MEM latch to the next instruction's ALU input, avoiding the wait for write-back. A load produces its value only at the end of MEM, so an instruction that uses it immediately must stall one cycle even with forwarding.

Q20.6 Compute the average waiting time for FCFS and SJF for processes (arrival, burst) $(0, 8), (1, 4), (2, 9), (3, 5)$.
Answer: FCFS 8.75, non-preemptive SJF 7.75 (preemptive SRTF 6.5): see the simulation above.

Q20.7 What is Belady's anomaly? Give the reference string.
Answer: Giving FIFO more frames can increase the number of page faults: for 1 2 3 4 1 2 5 1 2 3 4 5, FIFO has 9 faults with 3 frames and 10 with 4. LRU and OPT are stack algorithms and never show it.

Q20.8 State the four conditions for deadlock and how the Banker's algorithm avoids it.
Answer: Mutual exclusion, hold and wait, no preemption, circular wait. The Banker's algorithm grants a request only if, after granting, a safe sequence still exists (some order in which all processes can obtain their maximum claims and finish); otherwise the requester waits.

Q20.9 How many processes does `fork(); fork(); fork();` create? What does `if (fork() && fork())` do?
Answer: $2^3 = 8$ processes in total (7 new). In `fork() && fork()`, only the original parent executes the second `fork` (the first child sees 0 and short-circuits), so there are 3 processes, and only the parent takes the true branch.

Q20.10 Why can a `printf` before `fork` print twice?
Answer: Output to a pipe or file is fully buffered; the unflushed buffer is part of the process image that `fork` copies, so both processes flush it at exit. Use `fflush(stdout)` before `fork` or end the line when writing to a terminal.

Q20.11 A 32-bit virtual address space with 4 KB pages and 4-byte page-table entries: size of a single-level page table? Why use two levels?
Answer: 2^{20} entries $\times$ 4 B = 4 MB per process. Two levels (10 + 10 bits) need only the 4 KB top table plus second-level tables for the regions actually used.

Q20.12 Difference between a mutex and a semaphore; between a process and a thread; between TCP and UDP?
Answer: A mutex has an owner and provides mutual exclusion; a semaphore is a counter of permits that any thread may signal. A process has its own address space; threads of one process share it. TCP provides a reliable ordered byte stream with connection setup and congestion control; UDP sends independent datagrams without guarantees.

Q20.13 How many usable hosts does 192.168.10.77/26 have, and what is its broadcast address?
Answer: 62 usable hosts (network 192.168.10.64, broadcast 192.168.10.127).

Q20.14 Which normal form does a table violate if a non-key attribute determines another non-key attribute?
Answer: Third normal form (a transitive dependency); decompose into two tables.

Q20.15 `SELECT COUNT(*), COUNT(col)` on a column with NULLs: do they differ? Does `WHERE col = NULL` select the NULL rows?
Answer: They differ: `COUNT(*)` counts all rows, `COUNT(col)` only non-NULL values (the SQL output above shows 6 vs. 5). `col = NULL` is never true (it is UNKNOWN); use `col IS NULL`.

21. Aptitude and Puzzles: The Non-Technical Half of the Online Assessment

In this chapter

Priority ★★; the formula sheet and the mock test are ★★★. Time: 90 minutes (30 for the formula sheet and shortcuts, 20 for reasoning and puzzles, 40 for the timed mock test). The assessment sheet says "Mix of Aptitude and Technical (Online) on HackerRank"; the number of aptitude questions, the time limit and the negative-marking rule are **not published**. Chapter 2 reconstructs the probable shape from candidate reports: 10–15 aptitude MCQs in roughly 12 minutes (percentages, ratios, time–work, probability, permutations, series, logical reasoning, a puzzle or two). This chapter prepares that genre and a little beyond it, so that the real section feels easy. The practice items are original (no leaked questions are claimed); every formula was checked against brute force or exact arithmetic, every puzzle answer was found by a search program, and every answer of the 30-question mock test is recomputed by code (Section 21.9).

21.1 How to run the aptitude round ★★★

Ten rules for timed multiple-choice aptitude

1. **Budget:** 60–90 seconds per question on average. Mark and skip anything that gives no plan within 30 seconds; the second pass is cheaper because the easy marks are already banked.
2. **Order:** formula-plug questions first (percentages, averages, series, dice and coins), then multi-step word problems, then puzzles.
3. **Use the options.** Four options are a free hint about the size and form of the answer; for ratio, age and mixture problems *substitute the options back* instead of solving.
4. **Estimate before computing:** successive rises of 20% and 10% give a little over 30%; any option that violates the estimate is out.
5. **Units once, at the start:** km/h $\leftrightarrow$ m/s is $\times\frac{5}{18}$ or $\times\frac{18}{5}$; minutes $\leftrightarrow$ hours.
6. **Read the last line first:** what is asked (difference? percentage? *remaining* work? the smaller angle?).
7. **Complement:** "at least one" $= 1 - P(\text{none})$; "not divisible by" = total − divisible.
8. **Digits and parity eliminate:** check the last digit, the digit sum (divisibility by 3 and 9) and the parity of every option before computing anything.
9. **Draw:** a five-second sketch for direction, seating, blood-relation and Venn questions beats mental tracking.
10. **Negative marking unknown?** Assume it exists: guess only after eliminating two options.

Eight classic traps (each one is a wrong option in some test)

- **Average speed** over equal distances is the *harmonic* mean (48, not 50, for 60 and 40 km/h).
- **Percentage up and down are not symmetric:** +25% needs −20% to return; +20% followed by −20% is −4%.
- **Successive discounts do not add:** 20% and 10% make 28%, not 30%.
- **Profit is on the cost; discount is on the marked price.**
- **Compound versus simple interest:** the second year earns interest on the first year's interest; CI − SI for two years is $P(r/100)^2$.
- **Working together** adds *rates*, not times (12 and 18 days give 7.2 days, not 15).
- **Repeated letters:** divide by the factorials of the repeat counts (LEVEL has 30 arrangements, not 120).
- **Clock angles** ask for the smaller angle; direction questions ask for distance *and* direction.

On the HackerRank page

Read the instruction page before the clock starts: whether the sections are separately timed (a finished section may be locked), whether there is negative marking, and which proctoring options are on (webcam, tab-switch and copy-paste monitoring are configurable by the company, and campus drives are often supervised in a lab). Keep pen and paper ready and do not assume that an on-screen calculator exists.

21.2 Quantitative formula sheet, every row verified ★★★

Each formula below is followed by a number that appears in the checks of Section 21.2.1: the formula and a brute-force computation agree.

Topic	Formulas and rules
percentages	$x\%$ of y equals $y\%$ of x; successive changes $a\%$ and $b\%$ give a net $a+b+\frac{ab}{100}\%$ (+20%, +10% → +32%); after a rise of $x\%$ a fall of $\frac{100x}{100+x}\%$ restores the original (+25% needs −20%); percentage *points* are not percent
profit and loss	profit% $=\frac{SP-CP}{CP}\times 100$, always on the *cost* unless stated; $SP=CP(1\pm\frac{p}{100})$; a discount is on the *marked price*: $SP=MP(1-\frac{d}{100})$; markup 25% and discount 10% give profit 12.5%; successive discounts a,b give $a+b-\frac{ab}{100}$ (20% and 10% = 28%); selling 900 g as 1 kg gains $\frac{100}{900}=11.11\%$
ratio, mixture, ages	$a:b=c:d \iff ad=bc$; partnership shares $\propto$ capital × time; ages: put the ratio parts as $3x,5x$ and *substitute the options*; **alligation**: cheaper price c_1, dearer c_2, mean m gives quantity ratio $(c_2-m):(m-c_1)$ (Rs 40 and Rs 55 rice mixed to Rs 46: $9:6=3:2$); replacing x litres n times in a vessel of V litres leaves $V(1-\frac{x}{V})^n$ of the original
interest	$SI=\frac{PRT}{100}$; $A=P(1+\frac{r}{100})^n$; **CI − SI for 2 years** $=P(\frac{r}{100})^2$ (Rs 5000 at 10%: Rs 50); SI doubles money in $\frac{100}{r}$ years; **rule of 72**: compound doubling in about $72/r$ years; half-yearly compounding uses $\frac{r}{2}$ and $2n$; CAGR $=(\text{end/start})^{1/n}-1$
averages	average $=\frac{\text{sum}}{\text{count}}$; replacing one member changes the sum by the difference; weighted mean; average speed for equal distances $\frac{2xy}{x+y}$ (60 and 40 km/h: 48), for equal times $\frac{x+y}{2}$
speed, distance, time	km/h → m/s: $\times\frac{5}{18}$; relative speed *adds* for opposite directions, *subtracts* for the same direction; a train passes a pole in $\frac{L}{v}$ and a platform in $\frac{L+L_p}{v}$ (150 m train, 250 m platform, 20 s: $\frac{400}{20}\times 3.6=72$ km/h); two trains: $\frac{L_1+L_2}{v_1\pm v_2}$; boat: downstream $u+v$, upstream $u-v$ (12 km/h boat, 3 km/h stream, 45 km each way: 8 h)
work and pipes	rate $=\frac{1}{\text{days}}$ and rates *add*; two workers: $\frac{ab}{a+b}$ days (12 and 18: 7.2); **LCM method**: let the job be the LCM of the days (36 units; 3 and 2 units a day) and count units; leaks have negative rate (fill in 6 h, leak empties in 12 h: 12 h together); wages $\propto$ work done; $\frac{M_1D_1H_1}{W_1}=\frac{M_2D_2H_2}{W_2}$
divisibility	2 (last digit even), 3 and 9 (digit sum), 4 (last two digits), 5, 6 (2 and 3), 8 (last three digits), 11 (alternating digit sum), 7 (strip the last digit and subtract twice it from the rest); *abcabc* is a multiple of $1001=7\cdot 11\cdot 13$
unit digits, remainders	last digits cycle: 2: 2-4-8-6; 3: 3-9-7-1; 7: 7-9-3-1; 8: 8-4-2-6; 4 and 9 alternate; 0, 1, 5, 6 never change; reduce the exponent modulo the cycle length (7^{123} ends in 3 because $123 \bmod 4=3$); $2^{100} \bmod 7=2$ because $2^3\equiv 1$
factors and sums	$n=p^aq^b\cdots$ has $(a+1)(b+1)\cdots$ divisors ($360=2^3 3^2 5$: 24); HCF × LCM = product of the two numbers; trailing zeros of $n!=\lfloor\frac{n}{5}\rfloor+\lfloor\frac{n}{25}\rfloor+\cdots$ (100!: 24); $\sum k=\frac{n(n+1)}{2}$, $\sum k^2=\frac{n(n+1)(2n+1)}{6}$, $\sum k^3=\left(\frac{n(n+1)}{2}\right)^2$; AP: $S_n=\frac{n}{2}\big(2a+(n-1)d\big)$; GP: $S_n=a\frac{r^n-1}{r-1}$; the first n odd numbers sum to n^2
counting	$n!$ arrangements; with repeats $\frac{n!}{p!\,q!\cdots}$ (LEVEL: $\frac{5!}{2!2!}=30$); ${}^nP_r=\frac{n!}{(n-r)!}$, ${}^nC_r=\frac{n!}{r!(n-r)!}$; circular arrangements $(n-1)!$; derangements $D_n\approx\frac{n!}{e}$ ($D_4=9$); non-negative solutions of $x_1+\cdots+x_k=n$: $\binom{n+k-1}{k-1}$ (stars and bars)
probability	$P=\frac{\text{favourable}}{\text{total}}$; $P(\text{at least one})=1-P(\text{none})$; independent events multiply; $P(A\mid B)=\frac{P(A\cap B)}{P(B)}$; Bayes $P(A\mid B)=\frac{P(B\mid A)P(A)}{P(B)}$ (1% prevalence, 99% sensitivity, 5% false positives: a positive test means only $\frac{99}{594}=16.7\%$); two dice: the sum 7 is the most likely ($\frac{1}{6}$), $P(8)=\frac{5}{36}$; birthday paradox: 23 people give 50.7%; expectation is linear
clocks, calendars	hour hand 0.5°/min, minute hand 6°/min; angle $=\lvert 30h-5.5m\rvert$ (use $360-d$ if above 180; 3:40 gives 130°); the hands overlap 11 times in 12 hours; leap year: divisible by 4, but centuries only if divisible by 400; 100 years contain 5 odd days, 400 years 0
mensuration	circle πr^2, $2\pi r$; sphere $\frac{4}{3}\pi r^3$, $4\pi r^2$; cone $\frac{1}{3}\pi r^2 h$ with slant height $\sqrt{r^2+h^2}$; cylinder $\pi r^2 h$; Heron $\sqrt{s(s-a)(s-b)(s-c)}$ (13-14-15: 84); primitive Pythagorean triples below hypotenuse 30: 3-4-5, 5-12-13, 7-24-25, 8-15-17, 20-21-29; an n-gon has $\frac{n(n-3)}{2}$ diagonals and interior angles summing to $(n-2)180°$

Data interpretation (tables, bar and pie charts) in five rules

1. Read the title, the *unit* (thousands? per cent?) and the axis labels before the question.
2. **Percentage change** $=\frac{\text{new}-\text{old}}{\text{old}}$, against the *earlier* value; the largest *absolute* change is often not the

largest *percentage* change (question M15 of the mock test is built on this).
3. Compare shares as fractions by cross-multiplying; round to two significant digits unless the options are close.
4. A pie-chart sector has angle $360° \times$ share (25% is 90°); a difference of "percentage points" is not a percentage difference.
5. Combine group averages with weights (counts), never by averaging the averages.

21.2.1 The formula checks ★

The script behind the sheet pairs each closed-form formula with a brute-force or exact-fraction computation of the same quantity (divisibility rules on 1–5000, unit-digit cycles on 200 exponents, derangements by enumerating permutations, the birthday paradox with exact fractions, a clock-angle formula against geometry at all 720 minutes, the sphere and cone volumes against numerical integration, and so on) and prints one line per check. The list is also a compact index of the facts above.

48 formula checks

```
48 of 48 formula checks agree with brute force / exact arithmetic
  ok   successive +20% then +10%: a + b + ab/100
  ok   price up 25%: cut by 100x/(100+x) = 20% to restore
  ok   markup 25%, discount 10%: profit 12.5%
  ok   two discounts 20% and 10%: 20 + 10 - 20*10/100 = 28%
  ok   900 g sold as 1 kg: gain = error/(true - error) = 11.11%
  ok   alligation: Rs 40 and Rs 55 rice mixed to Rs 46 -> ratio 3:2
  ok   weighted mean of that 3:2 mixture
  ok   CI - SI after 2 years = P(r/100)^2 = 50
  ok   compound 20000 at 10% for 2 years
  ok   SI doubles money in 100/r = 12.5 years at 8%
  ok   CAGR: 100 -> 121 in 2 years is 10%
  ok   average speed for equal distances 60 and 40 km/h: 2xy/(x+y)
  ok   train 150 m crosses 250 m platform in 20 s: km/h
  ok   trains 100 m and 120 m opposite, 40 and 50 km/h: crossing time (s)
  ok   boat 12 km/h, stream 3: 45 km down + 45 km up (h)
  ok   work: A 12 days, B 18 days together: ab/(a+b) = 7.2
  ok   work: together 4 days, then B alone needs 8 more
  ok   LCM method: 36 units; A 3/day, B 2/day; together 36/5 days
  ok   pipes: fill 6 h, leak empties in 12 h -> 12 h to fill
  ok   divisibility rules for 2,3,4,5,6,8,9,11 on 1..5000
  ok   unit-digit cycles of 2,3,4,7,8,9
  ok   7^123 ends in 3
  ok   number of divisors of 360 = (3+1)(2+1)(1+1)
  ok   trailing zeros of n! = sum floor(n/5^k), n <= 300
  ok   HCF x LCM = product
  ok   sums: n(n+1)/2, n(n+1)(2n+1)/6, (n(n+1)/2)^2
  ok   sum of first 50 odd numbers = 50^2
  ok   GP: 3 + 6 + ... (10 terms) = a(r^n - 1)/(r - 1)
  ok   arrangements of LEVEL = 5!/(2!2!) = 30
  ok   nCr and nPr vs itertools (n=8, r=3)
  ok   derangements D(n) = n! sum (-1)^k/k!, n <= 7
  ok   stars and bars: x1+x2+x3 = 10 has C(12,2) = 66 solutions
  ok   circular arrangements of 5 people = 4! = 24
  ok   two dice: P(sum = 8) = 5/36
  ok   four rolls, at least one six: 1 - (5/6)^4 = 671/1296
  ok   birthday paradox: 23 people -> 50.73%
  ok   Bayes: P(disease | positive) = 99/(99+495)
  ok   expected tosses until the first head = 2 (simulation, 1e5 runs)
  ok   hands of a clock at 3:40 form 130 degrees
  ok   clock-angle formula |30h - 5.5m| vs geometry, all 720 minutes
  ok   leap year rule vs calendar.isleap, years 1..3000
  ok   15 August 1947 was a Friday
  ok   Heron: triangle 13-14-15 has area 84
  ok   sphere volume 4/3 pi r^3 = integral of the disc areas pi (r^2 - x^2)
  ok   cone volume 1/3 pi r^2 h = integral of discs of radius r x / h
  ok   circle area pi r^2 = lattice points inside the circle (r = 60, within 0.5%)
  ok   diagonals of an n-gon = n(n-3)/2 = pairs of corners minus sides
  ok   primitive Pythagorean triples with c < 30: (3,4,5) (5,12,13) (7,24,25) (8,15,17) (20,21,29)
```

21.3 Calculation shortcuts, and the proof that they work ★★★

- **Squares ending in 5:** $(10n+5)^2 = n(n+1)\,|\,25$, for example $35^2 = 3 \cdot 4\,|\,25 = 1225$.
- **Multiply by 11:** write the digits with the sum of each neighbouring pair between them and carry leftwards: $57 \times 11 = 5\,|\,12\,|\,7 = 627$.
- **Near a round number:** $97 \times 103 = (100-3)(100+3) = 9991$; $n \times 99 = 100n - n$; $n \times 101 = 100n + n$.

- **Casting out nines:** the digit root of a product equals the digit root of the product of the digit roots. It can *reject* a wrong option, never confirm a right one.
- **Divisibility by 7:** strip the last digit and subtract twice it from the rest, repeat; numbers of the form *abcabc* are multiples of $7 \cdot 11 \cdot 13 = 1001$.
- **Percentages from fractions:** memorise $\frac{1}{n}$ for $n = 2, \ldots, 12, 16$ (below) and combine: $\frac{3}{8} = 3 \times 12.5\% = 37.5\%$, $\frac{5}{7} = 71.43\%$.
- **Rule of 72:** doubling time $\approx 72/r$ years; accurate for r between about 6 and 12 per cent, a little off at the ends of the table.

Tables to memorise and the verified tricks

```
fraction -> percentage (learn 1/2 ... 1/12 and 1/16 by heart):
   1/2  = 50.00%     1/3  = 33.33%     1/4  = 25.00%     1/5  = 20.00%
   1/6  = 16.67%     1/7  = 14.29%     1/8  = 12.50%     1/9  = 11.11%
   1/10 = 10.00%     1/11 =  9.09%     1/12 =  8.33%     1/16 =  6.25%
k/7, k = 1..6: .142857  .285714  .428571  .571428  .714285  .857142  (the digits 142857, rotated)
squares 11..25: 121 144 169 196 225 256 289 324 361 400 441 484 529 576 625
cubes 1..12:    1 8 27 64 125 216 343 512 729 1000 1331 1728
powers of 2:    2 4 8 16 32 64 128 256 512 1024 2048 4096

speed tricks, each verified on every case in the stated range:
  (10n+5)^2 = n(n+1) followed by 25, n < 100                ok
  x 11: digit sums between the digits, carry left, n < 10000   ok
  (m-d)(m+d) = m^2 - d^2:  97 x 103 = 100^2 - 3^2 = 9991       ok
  n x 99 = 100n - n and n x 101 = 100n + n, n < 1000           ok
  casting out nines: root(ab) = root(root(a) root(b)), a,b<300 ok
  abcabc = abc x 7 x 11 x 13 for every 3-digit abc             ok
  divisible by 7 iff (n // 10) - 2 (n % 10) is, n < 20000      ok

rule of 72: years to double at r% compound interest
  r%            2       4       6       8      12      18      24
  72/r      36.00   18.00   12.00    9.00   6.00    4.00    3.00
  exact     35.00   17.67   11.90    9.01   6.12    4.19    3.22
```

Reading the output: the sevenths *rotate* the same six digits ($\frac{1}{7} = 0.\overline{142857}$, $\frac{2}{7} = 0.\overline{285714}$, ...), so one memorised cycle gives all six fractions; the 7-test and the casting-out-nines test were run on every number in the stated ranges, not on examples.

21.4 Logical reasoning: seven question types ★★

Type	Method that works under time pressure
number series	write the **difference table**; if the first differences are not constant look at the second; then try ratios (geometric), $n^2 \pm k$ or $n^3 \pm k$, Fibonacci type (each term the sum of the two before), differences that are themselves a pattern (powers of 2, primes), and *two interleaved series* (extend the one that the next position belongs to)
coding–decoding	positions $A = 1, \ldots, Z = 26$; constant shift; reversal then shift; opposite letter ($A \leftrightarrow Z$: the positions add to 27); find the rule on the *given* word and apply it unchanged to the question word
direction sense	draw axes and track (x, y) with north $= +y$, east $= +x$; a right turn rotates the heading clockwise; the answer is a distance (Pythagoras) *and* a direction
blood relations	draw a small tree from the speaker; maternal/paternal uncle and aunt are the siblings of the parents; "the only daughter of my mother" is the speaker if the speaker is a woman; if the sex is not determined the answer is "cannot be determined"
seating and ranking	fix the most constrained person first (the middle, the ends) and use "next to" and "right of" to split cases; position from left + position from right $= n + 1$; people between two positions $= \lvert p_1 - p_2 \rvert - 1$
syllogisms	Venn diagrams: draw every picture the premises allow and keep the conclusions that hold in *all* of them; apply the exam convention (named sets are non-empty) and the either–or rule below
sets and cubes	inclusion–exclusion $\lvert A \cup B \rvert = \lvert A \rvert + \lvert B \rvert - \lvert A \cap B \rvert$ (and its three-set form); a cube $n \times n \times n$ painted outside and cut into unit cubes has 8 corner cubes (3 painted faces), $12(n-2)$ edge cubes (2), $6(n-2)^2$ face cubes (1) and $(n-2)^3$ unpainted ones

21.4.1 Syllogisms: two conventions, one program

Placement papers give two premises and two or three conclusions and ask which *follow*. In strict logic "All A are B" does not promise that any A exists, so "Some B are A" does not follow; the usual exam convention instead treats every named set as non-empty, which makes it follow. If the worked example of your test shows "Some B are A" following from "All A are B", the exam convention is in force. The program enumerates *every model* of up to five elements (each element belongs to some subset of A, B, C) and declares a conclusion to follow only if it holds in all models of the premises; with it the whole two-premise table below is computed, not remembered.

Form	Reads	Converts to
A (universal affirmative)	All X are Y	Some Y are X (exam convention only), *not* All Y are X
E (universal negative)	No X is Y	No Y is X
I (particular affirmative)	Some X are Y	Some Y are X
O (particular negative)	Some X are not Y	nothing valid

Either–or rule. If neither of two conclusions follows alone but they are complementary ("Some X are Y" with "No X is Y", or "All X are Y" with "Some X are not Y"), then exactly one of them is true in every model, and the exam answer is "either I or II follows". The output shows this for the example of the mock test.

ap_logic.py: output

```
A = engineers, B = graduates, C = artists.  Premises: All A are B.  Some B are C.
  conclusion                      strict logic   exam convention (sets non-empty)
  Some engineers are artists      no             no
  Some artists are graduates      FOLLOWS        FOLLOWS
  No engineer is an artist        no             no
  Some graduates are engineers    no             FOLLOWS
  Some artists are not engineers  no             no
  exactly one of 'Some engineers are artists' / 'No engineer is an artist' holds in every model: True
  -> the exam answer is 'either I or II follows'

what follows about A and C from one premise about A,B and one about B,C:
  premises                        strict logic       extra under the exam convention
  All A are B; All B are C        All A are C        Some A are C
  All A are B; No B is C          No A is C          Some A are not C; Some C are not A
  No A is B; All B are C          -                  Some C are not A
  No A is B; Some B are C         Some C are not A   -
  Some A are B; All B are C       Some A are C       -
  Some A are B; No B is C         Some A are not C   -
  the other 10 of the 16 premise pairs allow no conclusion linking A and C

five people P Q R S T in a row (left to right); rules:
  1. R sits exactly in the middle
  2. P does not sit at an end
  3. Q sits somewhere to the right of R
  4. S sits next to P
  5. T does not sit at an end
arrangements that satisfy all rules: 1 -> ['SPRTQ']
   without rule 1 there would be 8 arrangements
   without rule 2 there would be 2 arrangements
   without rule 3 there would be 2 arrangements
   without rule 4 there would be 2 arrangements
   without rule 5 there would be 2 arrangements

walk 10 m north, right 5 m, right 10 m, left 3 m -> position (8, 0): 8 m east of the start
shift-by-1 code: COMPILER -> DPNQJMFS, so PTX -> QUY;  reverse + shift: NVIDIA -> BJEJWO
blood relation, all small families checked -> ['woman is his mother']
4x4x4 cube painted outside, cut into 64 unit cubes: with 0/1/2/3 painted faces = [8, 24, 24, 8]
   (formulas: (n-2)^3, 6(n-2)^2, 12(n-2), 8)

series [2, 6, 12, 20, 30]         next = 42  (constant after 2 difference row(s))
series [3, 6, 11, 18, 27]         next = 38  (constant after 2 difference row(s))
series [1, 1, 2, 3, 5, 8]         next = 13  (Fibonacci)
series [2, 3, 5, 9, 17]           next = 33  (differences are geometric)
series [3, 8, 6, 13, 9, 18, 12]   next = 23  (interleaved, constant after 1 difference row(s))
series [2, 6, 18, 54]             next = 162 (geometric)
```

Notes on the output. (1) In the table only 6 of the 16 premise pairs link A and C at all, which is why a conclusion that merely "sounds plausible" (for example "Some A are C" from "All A are B; Some B are C") is almost always wrong; the middle term must be *distributed* (covered completely) in at least one premise. (2) The seating puzzle has exactly one solution and every rule is needed: dropping any one of them leaves 2 arrangements (8 for the middle-seat rule), so none of the clues is redundant. (3) `next_term` tries the patterns in the order you should: geometric, Fibonacci, a constant difference row, geometric differences, interleaving; two equal entries in the last row are required before a polynomial pattern is trusted (four terms always fit *some* cubic, e.g. 2, 6, 18, 54 would

otherwise give 130 instead of 162).

21.5 Classic puzzles, solved by search ★★

Interviewers use puzzles to watch you *structure* a problem. Almost every classic is one of four shapes: (1) a **state graph** with a goal (jugs, river crossing, bridge and torch): model the state, then breadth-first search finds the fewest steps and Dijkstra the least cost; (2) an **information bound** (coins: w weighings have 3^w outcomes); (3) an **optimisation by dynamic programming** (egg drop); (4) an **invariant or pattern** (100 doors, Josephus). Say which shape it is *before* you start solving; that sentence is half the credit. The code is short because Chapter 7's tools (queues, heaps, DP tables) do the work.

ap_puzzles.py: the classics as search problems

```
import heapq, itertools, math
from collections import deque

def bridge(times):                          # bridge and torch: cross in pairs, somebody must carry the torch back
    everyone = frozenset(times)
    start = (everyone, 0)                   # (people on the left bank, torch side: 0 = left)
    best, heap, tick = {start: 0}, [(0, 0, start)], 0
    while heap:
        t, _, (left, side) = heapq.heappop(heap)
        if not left: return t
        if t > best.get((left, side), 1e9): continue
        group = left if side == 0 else everyone - left
        for k in (1, 2):
            for move in itertools.combinations(sorted(group), k):
                new_left = left - frozenset(move) if side == 0 else left | frozenset(move)
                nt = t + max(move)                                  # a pair walks at the slower speed
                if nt < best.get((new_left, 1 - side), 1e9):
                    best[(new_left, 1 - side)] = nt
                    tick += 1
                    heapq.heappush(heap, (nt, tick, (new_left, 1 - side)))

def jugs(ca, cb, goal):                     # BFS over (litres in jug A, litres in jug B)
    seen, queue = {(0, 0): None}, deque([(0, 0)])
    while queue:
        a, b = queue.popleft()
        if goal in (a, b):
            path, state = [], (a, b)
            while state is not None: path.append(state); state = seen[state]
            return path[::-1]
        ab, ba = min(a, cb - b), min(b, ca - a)                     # amounts that can be poured A->B and B->A
        for nxt in ((ca, b), (a, cb), (0, b), (a, 0), (a - ab, b + ab), (a + ba, b - ba)):
            if nxt not in seen: seen[nxt] = (a, b); queue.append(nxt)

def river():                                # farmer, wolf, goat, cabbage: 0 = left bank, 1 = right bank
    start, goal = (0, 0, 0, 0), (1, 1, 1, 1)
    safe = lambda s: not ((s[1] == s[2] != s[0]) or (s[2] == s[3] != s[0]))    # nobody is eaten while he is away
    seen, queue = {start: 0}, deque([start])
    while queue:
        s = queue.popleft()
        if s == goal: return seen[s]
        for cargo in (None, 1, 2, 3):                               # the farmer rows alone or with one item
            if cargo is not None and s[cargo] != s[0]: continue
            n = list(s); n[0] = 1 - s[0]
            if cargo is not None: n[cargo] = 1 - s[cargo]
            n = tuple(n)
            if safe(n) and n not in seen: seen[n] = seen[s] + 1; queue.append(n)

def egg_drop(eggs, floors):                 # worst-case number of drops, dynamic programming
    f = [[0] + [10 ** 9] * floors] + [[0] * (floors + 1) for _ in range(eggs)]    # no egg: nothing can be decided
    for e in range(1, eggs + 1):
        for fl in range(1, floors + 1):
            f[e][fl] = 1 + min(max(f[e - 1][k - 1], f[e][fl - k]) for k in range(1, fl + 1))
    return f[eggs][floors]

def open_doors(n):                          # pass k toggles every k-th door
    doors = [False] * (n + 1)
    for k in range(1, n + 1):
        for d in range(k, n + 1, k): doors[d] = not doors[d]
    return [d for d in range(1, n + 1) if doors[d]]

def josephus(n, k):                         # n people in a circle, every k-th is eliminated; returns the survivor
    people, i = list(range(1, n + 1)), 0
    while len(people) > 1:
        i = (i + k - 1) % len(people)
        people.pop(i)
    return people[0]
```

```python
def josephus_recurrence(n, k):              # J(1) = 0, J(n) = (J(n-1) + k) mod n  (0-based position of the survivor)
    j = 0
    for m in range(2, n + 1): j = (j + k) % m
    return j + 1

if __name__ == "__main__":
    print("bridge and torch, crossing times 1, 2, 5, 10 min: minimum total time =", bridge((1, 2, 5, 10)))
    path = jugs(3, 5, 4)
    print("jugs of 3 L and 5 L, measure 4 L: %d steps; states (a, b): %s" % (len(path) - 1, path))
    print("wolf, goat and cabbage: minimum number of crossings =", river())
    print("two eggs, 100 floors: worst case %d drops (14 + 13 + ... + 1 = 105 >= 100)" % egg_drop(2, 100))
    print("100 doors: open doors are", open_doors(100), "(the perfect squares)")
    print("Josephus n=41, k=3: the survivor stands at position", josephus(41, 3))
    agree = all(josephus(n, k) == josephus_recurrence(n, k) for n in range(1, 200) for k in range(2, 6))
    print("   the recurrence J(n) = (J(n-1) + k) mod n agrees with the simulation for n < 200, k = 2..5:", agree)
    closed = all(josephus(n, 2) == 2 * (n - 2 ** (n.bit_length() - 1)) + 1 for n in range(1, 200))
    print("   for k = 2 the closed form 2(n - 2^floor(log2 n)) + 1 agrees too:", closed, "(n=41 gives 19)")
    def drops(e, floors):                   # smallest t with C(t,1) + ... + C(t,e) >= floors
        t = 0
        while sum(math.comb(t, i) for i in range(1, e + 1)) < floors: t += 1
        return t
    same = all(drops(e, f) == egg_drop(e, f) for e in range(1, 5) for f in range(1, 101))
    print("   egg drop: sum of binomials C(t,i), i <= e, agrees with the DP for e <= 4 eggs, <= 100 floors:", same)
```

ap_puzzles.py: output

```
bridge and torch, crossing times 1, 2, 5, 10 min: minimum total time = 17
jugs of 3 L and 5 L, measure 4 L: 6 steps; states (a, b): [(0, 0), (0, 5), (3, 2), (0, 2), (2, 0), (2, 5), (3, 4)]
wolf, goat and cabbage: minimum number of crossings = 7
two eggs, 100 floors: worst case 14 drops (14 + 13 + ... + 1 = 105 >= 100)
100 doors: open doors are [1, 4, 9, 16, 25, 36, 49, 64, 81, 100] (the perfect squares)
Josephus n=41, k=3: the survivor stands at position 31
   the recurrence J(n) = (J(n-1) + k) mod n agrees with the simulation for n < 200, k = 2..5: True
   for k = 2 the closed form 2(n - 2^floor(log2 n)) + 1 agrees too: True (n=41 gives 19)
   egg drop: sum of binomials C(t,i), i <= e, agrees with the DP for e <= 4 eggs, <= 100 floors: True
```

Puzzle	Answer	What to say aloud
bridge and torch (1, 2, 5, 10 min)	17 min	The two slowest must cross *together* once (10 min), so do not waste a fast person on each of them: 1+2 cross (2), 1 back (1), 5+10 cross (10), 2 back (2), 1+2 cross (2). The "escort" strategy with the fastest costs 19.
jugs 3 L and 5 L, measure 4 L	6 steps	The states (a, b) form a graph; BFS gives the shortest path printed above. Because $\gcd(3, 5) = 1$ every amount from 0 to 5 litres is reachable; with jugs 4 and 6 only even amounts are.
wolf, goat, cabbage	7 crossings	Take the goat over; return; take the wolf (or cabbage); bring the goat *back*; take the cabbage (or wolf); return; take the goat. The goat is the only item that may never be left with either of the others.
two eggs, 100 floors	14 drops	Drop from floor 14; if it breaks scan floors 1–13 upwards, if not go up 13 floors and so on: $14 + 13 + \cdots + 1 = 105 \geq 100$, and 13 drops would reach only 91. For e eggs and t drops the reachable floors are $\sum_{i=1}^{e} \binom{t}{i}$.
100 doors	10 open	Door d is toggled once per divisor of d; divisors pair up as $k, d/k$, so the count is odd only when d is a perfect square.
Josephus (41, every 3rd)	position 31	Recurrence $J(1) = 0$, $J(n) = (J(n-1) + k) \bmod n$ (0-based position, add 1); for $k = 2$ the closed form is $J = 2\left(n - 2^{\lfloor \log_2 n \rfloor}\right) + 1$ (rotate the binary representation of n left by one bit), so $J(41) = 19$. Both statements are checked in the output.
12 coins, heavier or lighter	3 weighings	There are $12 \times 2 = 24$ possibilities and a weighing has three outcomes: $3^2 = 9 < 24 \leq 27 = 3^3$. The first weighing is 4 against 4; if it balances, the odd coin is among the other 4 and the remaining two weighings suffice, otherwise coins move between pans to separate suspected-heavy from suspected-light ones.
8 balls, one heavier	2 weighings	Weigh 3 against 3 (the search found exactly this). If they balance, the heavy ball is one of the other two: weigh them against each other. Otherwise take the heavier three and weigh 1 against 1; if those balance, the third is heavy.

The coin numbers come from the search whose output follows.

ap_coins.py: output

```
12 coins, one is heavier OR lighter: 24 hypotheses; 2 weighings separate at most 3^2 = 9, 3 weighings 27
search: no strategy with fewer than 3 weighings; first weighing of a 3-weighing strategy: (1, 2, 3, 4) vs (5, 6, 7, 8)
simulated on all 24 cases: always correct = True, worst case 3 weighings
8 balls, one heavier: 2 weighings suffice; first weighing (1, 2, 3) vs (4, 5, 6)
```

21.6 Probability puzzles by enumeration and simulation ★★

For a probability puzzle, **enumerate the sample space** if it is small and write down the probability of each outcome; most famous "paradoxes" disappear once the outcomes are listed with *their actual probabilities* (including the host's behaviour in Monty Hall). Use simulation only to confirm, and report its standard error.

ap_prob.py: output

```
Monty Hall: stay wins 1/3, switch wins 2/3
two children: P(both boys | at least one boy) = 1/3;  P(both boys | the elder is a boy) = 1/2
100 prisoners, loop strategy: simulated 0.318 (standard error 0.003), exact 1 - (1/51 + ... + 1/100) = 0.3118
   if everybody opens 50 boxes at random the team wins with probability 0.5^100 = 7.9e-31
expected tosses until HH: 6.0;  until HT: 4.0   (exact: 6 and 4)
expected rolls of a die until all six faces appeared: 14.7   (exact 6 x (1 + 1/2 + ... + 1/6) = 14.7)
```

Puzzle	Why
Monty Hall	Your first pick is right with probability $\frac{1}{3}$ and that does not change when the host opens a goat door: his action is *forced* by the car's position, so all the remaining $\frac{2}{3}$ moves to the other door.
two children	Ordered pairs BB, BG, GB, GG are equally likely. "At least one boy" leaves BB, BG, GB, so $\frac{1}{3}$; "the elder is a boy" leaves BB, BG, so $\frac{1}{2}$. The wording of the condition decides.
100 prisoners	Following the chain "box $\to$ number inside $\to$ that box" succeeds for everybody iff the random permutation has no cycle longer than 50, probability $1 - \sum_{k=51}^{100} \frac{1}{k} \approx 31\%$, against 2^{-100} for independent random guessing.
HH versus HT	After the first head, HT is finished by the next tail, and another head leaves you where you were: $E = 2 + 2 = 4$. HH needs a head right after a head, and a tail throws you back to the very start: $E_0 = 1 + \frac{1}{2}E_H + \frac{1}{2}E_0$ and $E_H = 1 + \frac{1}{2}E_0$ give $E_0 = 6$. A pattern whose failure loses all progress waits longer than one whose failure keeps some.
coupon collector	To see all n faces expect nH_n rolls: $6 \times 2.45 = 14.7$ for a die.
ants on a polygon	Two ants meet head-on exactly when neighbours walk in opposite senses, so nobody collides only if all walk the same way round: $\frac{2}{2^n}$.

21.7 Logic puzzles verified by model checking ★★

The same idea as the syllogism program: list all *worlds* consistent with the puzzle, test each rule in each world, and read off what is true in *all* remaining worlds. This is also exactly the thinking of Chapter 17: a claim is established by checking it on the whole (small) input space, not by examples.

```python
import itertools
from fractions import Fraction as F

# ---- knights always tell the truth, knaves always lie: try all 2^n assignments ------------------------------
def knights_and_knaves(claims):             # claims[x](world) = truth of what x says (None: x is silent)
    names, sols = list(claims), []
    for kinds in itertools.product((True, False), repeat=len(names)):          # True = knight
        world = dict(zip(names, kinds))
        if all(claims[x] is None or claims[x](world) == world[x] for x in names): sols.append(world)
    return [" ".join("%s=%s" % (k, "knight" if v else "knave") for k, v in w.items()) for w in sols]

print("A: 'B is a knave'; B: 'A and C are the same kind'; C: 'A is a knave'")
print("   ->", knights_and_knaves({"A": lambda w: not w["B"], "B": lambda w: w["A"] == w["C"],
                                   "C": lambda w: not w["A"]}))
print("A: 'B is a knave'; B: 'A and I are the same kind'")
print("   ->", knights_and_knaves({"A": lambda w: not w["B"], "B": lambda w: w["A"] == w["B"]}))
print("A: 'we are both knaves' (B says nothing)")
print("   ->", knights_and_knaves({"A": lambda w: not w["A"] and not w["B"], "B": None}))

# ---- two guards, one always lies; ask: "which door would the OTHER guard name as the way to heaven?" --------
def guard_answers_hell_door():
```

```
    verdicts = set()
    for liar, heaven in itertools.product((0, 1), repeat=2):              # which guard lies; which door is heaven
        name_heaven = lambda g: (1 - heaven) if g == liar else heaven      # what guard g says when asked directly
        for asked in (0, 1):
            reported = name_heaven(1 - asked)                              # what the other guard would say
            answer = (1 - reported) if asked == liar else reported         # reported truthfully, or the opposite
            verdicts.add(answer == 1 - heaven)                             # does the answer name the HELL door?
    return verdicts
print("two guards, 4 worlds x 2 guards asked: the answer is always the door to hell:", guard_answers_hell_door(),
      "-> take the other door")

# ---- hats in a line: the back person announces the parity of the hats he sees ------------------------------
def survivors(colours):                       # colours[0] stands at the back and sees everybody in front of him
    announced = sum(colours[1:]) % 2          # his own guess, chosen so that the others can decode the parity
    guesses = [announced]
    for i in range(1, len(colours)):          # person i heard i-1 correct answers and sees the hats in front of him
        guesses.append((announced - sum(guesses[1:i]) - sum(colours[i + 1:])) % 2)
    return sum(g == c for g, c in zip(guesses, colours))
counts = [survivors(c) for c in itertools.product((0, 1), repeat=10)]
print("10 prisoners, black/white hats, all 1024 colourings: %d to %d survive (the back person gambles)"
      % (min(counts), max(counts)))

# ---- ants on the corners of an n-gon: each walks along an edge, clockwise or anticlockwise --------------------
def no_collision(n):                          # head-on meeting: ant i goes clockwise and ant i+1 anticlockwise
    ok = sum(1 for d in itertools.product((0, 1), repeat=n)
             if not any(d[i] == 1 and d[(i + 1) % n] == 0 for i in range(n)))
    return F(ok, 2 ** n)
print("ants on an n-gon, P(nobody collides):", ", ".join("n=%d: %s" % (n, no_collision(n)) for n in (3, 4, 5, 6)))
print("   (= 2/2^n: all of them must walk the same way round)")

# ---- one poisoned bottle among 1000, test strips read after an hour: binary code ----------------------------
def poison_decoded_correctly(bottles=1000, strips=10):
    for bad in range(bottles):
        dead = [i for i in range(strips) if (bad >> i) & 1]      # strip i touches the bottles whose bit i is set
        if sum(1 << i for i in dead) != bad: return False        # the pattern of dead strips spells the bottle number
    return True
print("1000 bottles, one poisoned, 10 strips as a binary code: decoded correctly for every bottle:",
      poison_decoded_correctly())
print("   (2^10 = 1024 >= 1000)")

# ---- Cheryl's birthday: reasoning about what the other person knows is just filtering -----------------------
DATES = [("May", 15), ("May", 16), ("May", 19), ("June", 17), ("June", 18),
         ("July", 14), ("July", 16), ("Aug", 14), ("Aug", 15), ("Aug", 17)]
def same(ds, i, v): return [d for d in ds if d[i] == v]                 # dates with month (i=0) / day (i=1) v
step1 = [d for d in DATES if all(len(same(DATES, 1, e[1])) > 1 for e in same(DATES, 0, d[0]))]
step2 = [d for d in step1 if len(same(step1, 1, d[1])) == 1]
step3 = [d for d in step2 if len(same(step2, 0, d[0])) == 1]
text = lambda ds: ", ".join("%s %d" % d for d in ds)
print("Cheryl's birthday; dates left after Albert's first remark:", text(step1))
print("   after Bernard's 'now I know':", text(step2))
print("   after Albert's 'now I know too':", text(step3))
```

ap_riddles.py: output

```
A: 'B is a knave'; B: 'A and C are the same kind'; C: 'A is a knave'
   -> ['A=knight B=knave C=knave']
A: 'B is a knave'; B: 'A and I are the same kind'
   -> ['A=knight B=knave']
A: 'we are both knaves' (B says nothing)
   -> ['A=knave B=knight']
two guards, 4 worlds x 2 guards asked: the answer is always the door to hell: {True} -> take the other door
10 prisoners, black/white hats, all 1024 colourings: 9 to 10 survive (the back person gambles)
ants on an n-gon, P(nobody collides): n=3: 1/4, n=4: 1/8, n=5: 1/16, n=6: 1/32
   (= 2/2^n: all of them must walk the same way round)
1000 bottles, one poisoned, 10 strips as a binary code: decoded correctly for every bottle: True
   (2^10 = 1024 >= 1000)
Cheryl's birthday; dates left after Albert's first remark: July 14, July 16, Aug 14, Aug 15, Aug 17
   after Bernard's 'now I know': July 16, Aug 15, Aug 17
   after Albert's 'now I know too': July 16
```

Puzzle	Answer and the idea
knights and knaves	Try all 2^n assignments; a knight's statement must be true and a knave's false. "We are both knaves" forces its speaker to be a knave (a knight cannot say it) and then the other must be a knight.
two guards	Ask one guard "which door would the *other* guard name as the way to heaven?" and take the other door: the liar misreports the truth-teller's (true) answer and the truth-teller truthfully reports the liar's (false) answer, so the reply always names hell.

Puzzle	Answer and the idea
hats in a line	The back person announces the parity of the black hats he sees; everybody else computes their own hat from that parity, the answers already heard and the hats in front. Nine of ten are always right; the first is right with probability $\frac{1}{2}$.
ants on a polygon	Head-on meetings happen when neighbours walk in opposite senses, so only the two "all the same way" choices are safe: $\frac{2}{2^n}$.
poisoned bottle	Give each bottle a 10-bit number and let strip i taste every bottle whose bit i is 1; the pattern of strips that turn is the number of the bad bottle. Binary codes are the general answer to "find one defective item with few tests".
Cheryl's birthday	Each statement removes the dates that would make it false; Albert's first remark leaves July and August, Bernard's "now I know" removes 14 July and 14 August (a repeated day), Albert's "now I know too" removes August (two dates remain there). Answer: 16 July.

Puzzle (no program needed)	Answer and reasoning
two ropes, each burns in 60 min but unevenly; measure 45 min	Light rope 1 at *both* ends and rope 2 at one end. Rope 1 is gone after exactly 30 min (whatever the burn rate, the two flames together consume the 60 minutes that one flame would need). Now light the other end of rope 2, which has 30 min left on one flame and burns out in 15 more: $30 + 15 = 45$.
three switches, one bulb, one visit	Switch 1 on for ten minutes, then off; switch 2 on; walk in. Lit = switch 2; off and warm = switch 1; off and cold = switch 3. Use a second physical property (heat) to get a third state.
how often do the clock hands overlap?	The relative speed is 5.5° per minute, so every $\frac{360}{5.5} = 65.45$ minutes: 11 times in 12 hours, 22 in a day (not 24).

21.8 Verbal ability, if it appears ★

The role description asks for strong C/C++ and Python, not for English, and none of the candidate reports mentions a verbal section; if one appears it is short. Three habits are enough: **reading comprehension**: read the questions first, skim the passage for their keywords, and eliminate options with absolute words ("always", "never", "only") unless the passage says exactly that; **sentence correction**: check subject–verb agreement ("one of the engineers *is*"), tense consistency, pronoun reference, parallel structure ("reading, writing and *to code*" is wrong), articles ("an hour", "a university") and prepositions ("different *from*", "discuss" takes no *about*); **para-jumbles**: find the opening sentence (it introduces the subject without a pronoun), chain the rest by pronouns and connectors ("however", "this", "also"), and check the chronology. Confusable pairs worth ten seconds: affect/effect, principal/principle, its/it's, complement/compliment, whether/weather.

21.9 Mock test: 30 questions in 40 minutes ★★★

Rules. Set a timer for **40 minutes**, use pen and paper only, mark a guess with a star and the questions you skipped with a circle, and do not look at the key. Target: at least 23 correct (75%), at least 26 if you have the time to revise. The questions are original practice items of the genres above, deliberately a little denser than the 10–15 questions the reports suggest, so that the real section feels light.

Section A: quantitative aptitude (M1–M18)

M1. A shopkeeper marks an article 25% above its cost price and allows a discount of 10%. His profit percentage is

(A) 10% **(B)** 12.5% **(C)** 15% **(D)** 17.5%

M2. A can finish a job in 12 days and B in 18 days. They work together for 4 days, then A leaves. In how many further days does B finish the job?

(A) 6 **(B)** 7 **(C)** 8 **(D)** 9

M3. A 150 m long train crosses a 250 m long platform in 20 s. Its speed is (km/h)

(A) 54 **(B)** 60 **(C)** 66 **(D)** 72

M4. A 40-litre mixture of milk and water has milk : water = 3 : 1. How many litres of water must be added to make the ratio 3 : 2?

(A) 10 **(B)** 15 **(C)** 20 **(D)** 25

M5. The difference between compound and simple interest on Rs 5000 for 2 years at 10% per annum (compounded annually) is (Rs)

(A) 10 **(B)** 25 **(C)** 50 **(D)** 100

M6. The unit digit of 7^{123} is

(A) 1 **(B)** 3 **(C)** 7 **(D)** 9

M7. The number of trailing zeros of 100! is

(A) 20 **(B)** 22 **(C)** 24 **(D)** 25

M8. The number of positive divisors of 360 is

(A) 12 **(B)** 16 **(C)** 20 **(D)** 24

M9. A car goes from P to Q at 60 km/h and returns at 40 km/h. Its average speed for the round trip is (km/h)

(A) 48 **(B)** 50 **(C)** 52 **(D)** 54

M10. Two fair dice are rolled. The probability that the sum is 8 is

(A) $\frac{1}{6}$ **(B)** $\frac{5}{36}$ **(C)** $\frac{1}{9}$ **(D)** $\frac{1}{12}$

M11. The number of distinct arrangements of the letters of the word LEVEL is

(A) 30 **(B)** 60 **(C)** 90 **(D)** 120

M12. The angle between the hands of a clock at 3:40 is

(A) 120° **(B)** 125° **(C)** 130° **(D)** 135°

M13. The ages of A and B are in the ratio 3 : 5. After 6 years the ratio will be 3 : 4. B's present age (years) is

(A) 10 **(B)** 12 **(C)** 15 **(D)** 18

M14. The HCF of two numbers is 6 and their LCM is 180. If one number is 36, the other is

(A) 12 **(B)** 15 **(C)** 24 **(D)** 30

M15. Sales of a firm (Rs lakh): 2020: 80, 2021: 100, 2022: 90, 2023: 135, 2024: 189. In which year was the *percentage* growth over the previous year the highest, and what was it?

(A) 2021, 25% **(B)** 2024, 40% **(C)** 2022, 10% **(D)** 2023, 50%

M16. The remainder when 2^{100} is divided by 7 is

(A) 2 **(B)** 3 **(C)** 4 **(D)** 6

M17. A bag holds 5 red and 3 blue balls. Two balls are drawn without replacement. The probability that both are red is

(A) $\frac{3}{28}$ **(B)** $\frac{5}{14}$ **(C)** $\frac{25}{64}$ **(D)** $\frac{15}{28}$

M18. On a road 50% of the vehicles are cars, 30% bikes and 20% trucks. The probability of a breakdown is 2% for a car, 3% for a bike and 5% for a truck. A broken-down vehicle is seen. The probability that it is a truck is

(A) $\frac{1}{5}$ **(B)** $\frac{9}{29}$ **(C)** $\frac{10}{29}$ **(D)** $\frac{1}{2}$

Section B: logical reasoning (M19–M26)

M19. Find the next term: 2, 6, 12, 20, 30, ?

(A) 40 **(B)** 42 **(C)** 44 **(D)** 48

M20. Find the next term: 3, 8, 6, 13, 9, 18, 12, ?

(A) 15 **(B)** 21 **(C)** 22 **(D)** 23

M21. If COMPILER is written as DPNQJMFS, then PTX is written as

(A) QUY **(B)** QTX **(C)** PUX **(D)** OSW

M22. A man walks 10 m north, turns right and walks 5 m, turns right and walks 10 m, then turns left and walks 3 m. Relative to the starting point he is

(A) 3 m east **(B)** 5 m east **(C)** 8 m west **(D)** 8 m east

M23. Statements: All engineers are graduates. Some graduates are artists. Conclusions: I. Some artists are graduates. II. Some engineers are artists.

(A) Only I follows **(B)** Only II follows **(C)** Both follow **(D)** Neither follows

M24. Five people P, Q, R, S, T sit in a row. R sits exactly in the middle; P does not sit at an end; Q sits somewhere to the right of R; S sits next to P; T does not sit at an end. Who sits at the right end?

(A) P **(B)** Q **(C)** S **(D)** T

M25. Pointing to a man, a woman says, "His mother is the only daughter of my mother." How is the woman related to the man?

(A) Sister **(B)** Mother **(C)** Aunt **(D)** Daughter

M26. In a row of 31 students A is 12th from the left and B is 15th from the right. How many students stand between A and B?

(A) 2 **(B)** 3 **(C)** 4 **(D)** 5

Section C: puzzles (M27–M30)

M27. Four people must cross a bridge at night with one torch; at most two cross together at the slower person's pace. Their times are 1, 2, 5 and 10 minutes. The minimum time for all to cross is (minutes)

(A) 17 **(B)** 18 **(C)** 19 **(D)** 20

M28. With an unmarked 3-litre jug, an unmarked 5-litre jug and a tap, what is the minimum number of operations (fill, empty, pour) to leave exactly 4 litres in one jug?

(A) 4 **(B)** 5 **(C)** 6 **(D)** 7

M29. Twelve coins look identical; exactly one is counterfeit and is either heavier or lighter. What is the minimum number of weighings on a balance that guarantees finding it and telling which way it differs?

(A) 2 **(B)** 3 **(C)** 4 **(D)** 6

M30. You have two identical eggs and a 100-storey building. An egg breaks when dropped from above an unknown critical floor. The minimum number of drops that guarantees finding that floor is

(A) 10 **(B)** 13 **(C)** 14 **(D)** 20

Answer key with working

Q	Key	Working
M1	**B**	Take cost $= 100$: marked 125, sold at $125 \times 0.9 = 112.5$, profit 12.5%.
M2	**C**	LCM method: 36 units; A does 3 and B 2 units a day. Four days of both: $4 \times 5 = 20$ units; 16 remain; B needs $16/2 = 8$ days.
M3	**D**	Distance $= 150 + 250 = 400$ m in 20 s $= 20$ m/s $= 20 \times 3.6 = 72$ km/h.
M4	**A**	Milk 30 L, water 10 L. Milk stays 30, so the water must become $\frac{2}{3} \cdot 30 = 20$ L: add 10 L.
M5	**C**	CI $-$ SI for two years $= P(r/100)^2 = 5000 \times 0.01 = 50$ (interest on the first year's interest, 10% of Rs 500).
M6	**B**	The last digits of $7^1, 7^2, 7^3, 7^4$ are $7, 9, 3, 1$ (period 4); $123 \bmod 4 = 3$, so the third entry: 3.
M7	**C**	Each zero needs a factor 5 (2s are plentiful): $\lfloor 100/5 \rfloor + \lfloor 100/25 \rfloor = 20 + 4 = 24$.
M8	**D**	$360 = 2^3 \cdot 3^2 \cdot 5$, so $(3+1)(2+1)(1+1) = 24$.
M9	**A**	Equal distances: harmonic mean $\frac{2 \cdot 60 \cdot 40}{60+40} = 48$ (the arithmetic mean 50 is the trap).
M10	**B**	Favourable: $(2,6), (3,5), (4,4), (5,3), (6,2)$, that is 5 of 36 outcomes.
M11	**A**	Five letters with L twice and E twice: $\frac{5!}{2!\,2!} = 30$.
M12	**C**	Hour hand: $3 \cdot 30 + 40 \cdot 0.5 = 110°$; minute hand: $40 \cdot 6 = 240°$; difference $130°$ (already below $180°$).
M13	**A**	$\frac{3x+6}{5x+6} = \frac{3}{4} \Rightarrow 12x + 24 = 15x + 18 \Rightarrow x = 2$: B $= 10$, A $= 6$ (check: $12 : 16 = 3 : 4$).
M14	**D**	HCF $\times$ LCM $=$ product: $36y = 6 \cdot 180 \Rightarrow y = 30$ (and $\gcd(36, 30) = 6$, lcm $= 180$).
M15	**D**	Growth: 2021 +25%, 2022 -10%, 2023 +50%, 2024 +40%. The largest *absolute* rise is in 2024 (+54), the largest *percentage* rise in 2023.
M16	**A**	$2^3 = 8 \equiv 1 \pmod 7$, so $2^{100} = (2^3)^{33} \cdot 2 \equiv 2$.
M17	**B**	$\binom{5}{2} / \binom{8}{2} = 10/28 = 5/14$ (the value 25/64 is for drawing *with* replacement).
M18	**C**	Bayes: $\frac{0.2 \cdot 0.05}{0.5 \cdot 0.02 + 0.3 \cdot 0.03 + 0.2 \cdot 0.05} = \frac{0.010}{0.029} = \frac{10}{29}$ (the prior $\frac{1}{5}$ and the bike share $\frac{9}{29}$ are the traps).
M19	**B**	Differences $4, 6, 8, 10$ give the next difference 12: $30 + 12 = 42$ (the terms are $n(n+1)$).

Q	Key	Working
M20	**D**	Two interleaved series: $3, 6, 9, 12$ (+3) in the odd places and $8, 13, 18, \ldots$ (+5) in the even places; the 8th term is an even place: $18 + 5 = 23$ (15 continues the wrong series).
M21	**A**	Every letter moves one place forward (C→D, O→P, ...), so P→Q, T→U, X→Y.
M22	**D**	Positions: $(0, 10) \to (5, 10) \to (5, 0) \to (8, 0)$: he is 8 m east of the start (on the same horizontal line).
M23	**A**	I is the converse of the second premise. II fails in a model where the artists lie outside the engineers (engineers $\subset$ graduates, artists among the other graduates).
M24	**B**	R is in place 3; P and T avoid the ends, so they take places 2 and 4 and S, Q take the ends. Q is right of R, so Q is at place 5, S at place 1, hence S is next to P at place 2: S P R T Q.
M25	**B**	The only daughter of the woman's mother is the woman herself; so the man's mother is the woman.
M26	**C**	B is $31 - 15 + 1 = 17$th from the left; between places 12 and 17 stand $17 - 12 - 1 = 4$ students.
M27	**A**	1 and 2 cross (2), 1 returns (1), 5 and 10 cross (10), 2 returns (2), 1 and 2 cross (2): 17. Escorting everybody with the fastest takes 19.
M28	**C**	Fill 5, pour into 3, empty 3, pour 2 into 3, fill 5, pour into 3: states $(0, 5), (3, 2), (0, 2), (2, 0), (2, 5), (3, 4)$ — 6 steps (breadth-first search confirms nothing shorter).
M29	**B**	24 possibilities (12 coins $\times$ heavier/lighter) exceed the $3^2 = 9$ outcomes of two weighings but not the $3^3 = 27$ of three; the search found a 3-weighing strategy that starts with 4 against 4.
M30	**C**	Drop from floors 14, 27, 39, ... (steps $14, 13, 12, \ldots$): $14 + 13 + \cdots + 1 = 105 \geq 100$, while 13 drops cover only 91.

Every answer recomputed

The key above was typed by hand; the program below recomputes each answer *independently* (brute force, exact fractions, the search programs of this chapter) and checks that exactly one option equals the computed value and that it is the keyed one. Its source is listed in Appendix **??**.

ap_mock.py: output

```
M1  computed 25/2              matches option B  key B  ok
M2  computed 8                 matches option C  key C  ok
M3  computed 72                matches option D  key D  ok
M4  computed 10                matches option A  key A  ok
M5  computed 50                matches option C  key C  ok
M6  computed 3                 matches option B  key B  ok
M7  computed 24                matches option C  key C  ok
M8  computed 24                matches option D  key D  ok
M9  computed 48                matches option A  key A  ok
M10 computed 5/36              matches option B  key B  ok
M11 computed 30                matches option A  key A  ok
M12 computed 130               matches option C  key C  ok
M13 computed 10                matches option A  key A  ok
M14 computed 30                matches option D  key D  ok
M15 computed (2023, 1/2)       matches option D  key D  ok
M16 computed 2                 matches option A  key A  ok
M17 computed 5/14              matches option B  key B  ok
M18 computed 10/29             matches option C  key C  ok
M19 computed 42                matches option B  key B  ok
M20 computed 23                matches option D  key D  ok
M21 computed QUY               matches option A  key A  ok
M22 computed (8, 0)            matches option D  key D  ok
M23 computed (I)               matches option A  key A  ok
M24 computed Q                 matches option B  key B  ok
M25 computed Mother            matches option B  key B  ok
M26 computed 4                 matches option C  key C  ok
M27 computed 17                matches option A  key A  ok
M28 computed 6                 matches option C  key C  ok
M29 computed 3                 matches option B  key B  ok
M30 computed 14                matches option C  key C  ok
30 of 30 answer-key entries confirmed (exactly one option equals the computed value, and it is the keyed one)
```

Scoring and what to revise

If you missed...	Questions	Go back to
percentages, profit, ratio, mixture, interest, ages	M1, M4, M5, M13	formula sheet rows "percentages" to "interest"; the trap box
speed, trains, work, pipes	M2, M3, M9	"averages", "speed, distance, time", "work and pipes"
number system	M6, M7, M8, M14, M16	divisibility, unit digits, factors; the shortcut list
counting, probability	M10, M11, M17, M18	"counting" and "probability" rows; the probability puzzles
clocks, data interpretation	M12, M15	"clocks, calendars"; the five DI rules
series, coding, direction, ranking	M19–M22, M26	the reasoning table; run `next_term` on your own series
syllogism, seating, blood relation	M23–M25	the two-premise table and the either–or rule
puzzles	M27–M30	"Classic puzzles": name the shape of the puzzle first

After the mock test

Do not redo the questions you got right. For each miss write one line: "what I should have noticed" (a trap, a formula, a unit). Re-sit only those questions the next day; if the same *type* appears twice in your misses, that type is your next 20 minutes.